

Mindstorms\ Party\ Desktop

Generated by Doxygen 1.17.0

1 mainpage	1
2 Todo List	3
3 Directory Hierarchy	5
3.1 Directories	5
4 Namespace Index	7
4.1 Namespace List	7
5 Hierarchical Index	9
5.1 Class Hierarchy	9
6 Class Index	13
6.1 Class List	13
7 File Index	21
7.1 File List	21
8 Directory Documentation	27
8.1 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games Directory Reference	27
8.1.1 Detailed Description	27
8.1.2 Splatoon mini-game	27
8.1.3 Responsibility split	28
8.1.4 Files	28
8.1.5 The board	28
8.1.6 Network protocol	28
8.1.6.1 How phones discover the desktop IP	29
8.1.6.2 How phones discover their EV3 IP	29
8.1.6.3 How phones switch scenes automatically	29
8.1.6.4 How "Connect Unity" works	30
8.1.7 Game flow	30
8.1.8 Board game integration	31
8.1.8.1 How the real game flow works	31
8.1.8.2 How the lobby starts the game (done)	31
8.1.9 Tracking and the game are separate subsystems	32
8.1.10 Phone AR board (transparent mode)	32
8.1.11 Coordinate convention	33
8.1.12 ArUco markers (camera tracking)	33
8.1.12.1 Player <-> ArUco marker mapping	33
8.1.13 Key constants (splatoon.hpp)	33
8.1.14 Threading	34
8.1.15 Running	34
8.1.15.1 Standalone mode (testing only)	34
8.1.15.2 Real board game mode (lobby-based, current flow)	34

8.1.15.3 Testing the desktop without phones	35
9 Namespace Documentation	37
9.1 ns Namespace Reference	37
9.1.1 Detailed Description	37
9.2 TopoCategory Namespace Reference	38
9.2.1 Detailed Description	38
10 Class Documentation	39
10.1 mind::proto::ack_msg Struct Reference	39
10.1.1 Detailed Description	39
10.2 MqttReceiver.AnchorData Class Reference	40
10.3 ArUcoWorker Class Reference	40
10.4 Board Class Reference	41
10.4.1 Detailed Description	41
10.4.2 Constructor & Destructor Documentation	41
10.4.2.1 Board()	41
10.4.3 Member Function Documentation	42
10.4.3.1 addNode()	42
10.4.3.2 broadcast_board()	42
10.4.3.3 connectNodes()	42
10.4.3.4 getNode()	43
10.4.3.5 getNodes()	43
10.4.3.6 isValidNode()	43
10.4.3.7 tileTypeToString()	44
10.5 BoardLoader.BoardData Class Reference	44
10.6 BoardLoader Class Reference	44
10.6.1 Detailed Description	45
10.7 BonusAward Struct Reference	45
10.8 BonusPhase Class Reference	45
10.8.1 Detailed Description	45
10.8.2 Member Function Documentation	46
10.8.2.1 execute()	46
10.9 BonusResult Struct Reference	46
10.10 CameraCalibration Class Reference	47
10.10.1 Detailed Description	47
10.10.2 Constructor & Destructor Documentation	48
10.10.2.1 CameraCalibration()	48
10.10.3 Member Function Documentation	48
10.10.3.1 calibrate_extrinsics()	48
10.10.3.2 frame_ready	48
10.10.3.3 get_rvec()	49
10.10.3.4 get_tvec()	49

10.10.3.5 is_calibrated()	49
10.10.3.6 set_intrinsics()	49
10.10.3.7 start_camera()	50
10.11 mind::mqtt::client Class Reference	50
10.11.1 Detailed Description	51
10.11.2 Constructor & Destructor Documentation	51
10.11.2.1 client()	51
10.11.2.2 ~client()	52
10.11.3 Member Function Documentation	52
10.11.3.1 connect_to()	52
10.11.3.2 disconnect_from_broker()	52
10.11.3.3 message_received	53
10.11.3.4 publish()	53
10.11.3.5 subscribe()	53
10.12 ns::CloseShopEvent Struct Reference	54
10.12.1 Detailed Description	54
10.13 collision::collision_planner Class Reference	54
10.13.1 Detailed Description	56
10.13.2 Constructor & Destructor Documentation	56
10.13.2.1 collision_planner()	56
10.13.3 Member Function Documentation	56
10.13.3.1 clear_path()	56
10.13.3.2 debug_obstacle_grid()	56
10.13.3.3 effective_path()	57
10.13.3.4 forget()	57
10.13.3.5 is_yielding()	57
10.13.3.6 nominal_path()	57
10.13.3.7 request_path_to()	58
10.13.3.8 request_route_to()	58
10.13.3.9 route_nodes()	59
10.13.3.10 set_graph()	59
10.13.3.11 set_path_sink()	59
10.13.3.12 set_priority_rule()	59
10.13.3.13 shove_blocker()	60
10.13.3.14 update()	60
10.13.3.15 watched_pairs()	60
10.14 control_window Class Reference	61
10.14.1 Detailed Description	61
10.14.2 Constructor & Destructor Documentation	61
10.14.2.1 control_window()	61
10.14.2.2 ~control_window()	62
10.15 Dice Class Reference	62

10.15.1 Detailed Description	62
10.15.2 Member Function Documentation	62
10.15.2.1 throwDice()	62
10.16 follower::drive_cmd Struct Reference	63
10.16.1 Detailed Description	63
10.17 follower::drive_pct Struct Reference	63
10.17.1 Detailed Description	63
10.18 Effect Class Reference	64
10.18.1 Detailed Description	64
10.18.2 Constructor & Destructor Documentation	64
10.18.2.1 Effect()	64
10.18.3 Member Function Documentation	65
10.18.3.1 applyEndOfTurn()	65
10.18.3.2 applyStartOfTurn()	65
10.19 BoardLoader.EffectData Class Reference	65
10.20 ns::EndRouletteEvent Struct Reference	66
10.20.1 Detailed Description	66
10.21 fleet_panel Class Reference	66
10.21.1 Detailed Description	66
10.21.2 Constructor & Destructor Documentation	67
10.21.2.1 fleet_panel()	67
10.22 FleetDashboard Class Reference	67
10.22.1 Detailed Description	67
10.23 follower::follower_params Struct Reference	68
10.23.1 Detailed Description	69
10.23.2 Member Data Documentation	69
10.23.2.1 min_speed_pct	69
10.23.2.2 min_turn_pct	69
10.23.2.3 rotate_exit_deg	69
10.24 follower_tuning_panel Class Reference	69
10.24.1 Detailed Description	70
10.24.2 Constructor & Destructor Documentation	70
10.24.2.1 follower_tuning_panel()	70
10.24.3 Member Function Documentation	71
10.24.3.1 changed	71
10.24.3.2 params()	71
10.24.3.3 setParams()	71
10.25 Game Class Reference	72
10.25.1 Detailed Description	72
10.25.2 Constructor & Destructor Documentation	73
10.25.2.1 Game()	73
10.25.3 Member Function Documentation	73

10.25.3.1 determineTurnOrder()	73
10.25.3.2 getCurrentPlayer()	73
10.25.3.3 isLastPlayer()	73
10.25.3.4 nextRound()	74
10.26 GameManager Class Reference	74
10.26.1 Detailed Description	74
10.26.2 Constructor & Destructor Documentation	75
10.26.2.1 GameManager()	75
10.26.3 Member Function Documentation	75
10.26.3.1 notifyRobotPosition()	75
10.26.3.2 runMiniGame()	75
10.26.3.3 syncPlayers()	76
10.27 Generator Class Reference	76
10.27.1 Detailed Description	77
10.27.2 Member Function Documentation	77
10.27.2.1 get_last_number()	77
10.27.2.2 random_number()	77
10.27.2.3 roll_ball()	78
10.27.3 Member Data Documentation	78
10.27.3.1 m_numbers	78
10.28 ns::GetPlayerBetEvent Struct Reference	78
10.28.1 Detailed Description	79
10.29 collision::graph_node Struct Reference	79
10.29.1 Detailed Description	79
10.30 pathfinding::GridMap Class Reference	79
10.30.1 Detailed Description	80
10.30.2 Constructor & Destructor Documentation	80
10.30.2.1 GridMap()	80
10.30.3 Member Function Documentation	81
10.30.3.1 addObstacle()	81
10.30.3.2 blockBorder()	81
10.30.3.3 gridToWorld()	81
10.30.3.4 hasLineOfSight()	82
10.30.3.5 isInside()	82
10.30.3.6 isWalkable()	83
10.30.3.7 worldToGrid()	83
10.31 mind::proto::heartbeat_msg Struct Reference	83
10.31.1 Detailed Description	84
10.32 Inventory Class Reference	84
10.32.1 Detailed Description	84
10.32.2 Member Function Documentation	85
10.32.2.1 addItem()	85

10.32.2.2 getItem()	85
10.32.2.3 getItems()	85
10.32.2.4 getSize()	85
10.32.2.5 hasItem()	86
10.32.2.6 isFull()	86
10.32.2.7 removeItem()	86
10.33 Item Class Reference	87
10.33.1 Detailed Description	87
10.33.2 Constructor & Destructor Documentation	87
10.33.2.1 Item()	87
10.33.3 Member Function Documentation	88
10.33.3.1 applyItemEffects()	88
10.33.3.2 getDiceOverride()	88
10.33.3.3 isDiceItem()	88
10.33.3.4 requiresTarget()	89
10.33.3.5 rollSingle()	89
10.33.3.6 rollSum()	89
10.34 ItemData Struct Reference	89
10.34.1 Detailed Description	90
10.35 latency_indicator Class Reference	90
10.35.1 Detailed Description	90
10.35.2 Constructor & Destructor Documentation	91
10.35.2.1 latency_indicator()	91
10.35.3 Member Function Documentation	91
10.35.3.1 flash	91
10.36 LogEntryExit Struct Reference	91
10.36.1 Detailed Description	92
10.36.2 Constructor & Destructor Documentation	92
10.36.2.1 LogEntryExit()	92
10.37 Logger Class Reference	92
10.37.1 Detailed Description	93
10.37.2 Member Function Documentation	93
10.37.2.1 getInstance()	93
10.37.2.2 isLevelEnabled()	93
10.37.2.3 push()	94
10.37.2.4 setLoggingLevel()	94
10.38 LogStream Struct Reference	95
10.38.1 Detailed Description	95
10.38.2 Constructor & Destructor Documentation	95
10.38.2.1 LogStream()	95
10.38.2.2 ~LogStream()	96
10.38.3 Member Function Documentation	96

10.38.3.1	getStream()	96
10.38.3.2	operator<<()	96
10.39	main_window Class Reference	97
10.39.1	Detailed Description	97
10.39.2	Constructor & Destructor Documentation	98
10.39.2.1	main_window()	98
10.39.3	Member Function Documentation	98
10.39.3.1	set_fleet()	98
10.40	MapWindow Class Reference	98
10.40.1	Detailed Description	100
10.40.2	Constructor & Destructor Documentation	100
10.40.2.1	MapWindow()	100
10.40.2.2	~MapWindow()	100
10.40.3	Member Function Documentation	101
10.40.3.1	boardClicked()	101
10.40.3.2	collisionPlanner()	101
10.40.3.3	getRobotNode()	101
10.40.3.4	initializeCamera()	102
10.40.3.5	movePlayboard()	102
10.40.3.6	paintBoard()	102
10.40.3.7	planPathFor()	103
10.40.3.8	robotArrived	103
10.40.3.9	sendRobotToNode()	103
10.40.3.10	setFleet()	104
10.40.3.11	setTrackingMarkers()	104
10.40.3.12	setTrackingState()	104
10.40.3.13	startCalibration()	104
10.40.3.14	updateRobotPosition()	105
10.41	marker_tracker::marker_position Struct Reference	105
10.41.1	Detailed Description	106
10.41.2	Member Data Documentation	106
10.41.2.1	back_px	106
10.42	marker_tracker::marker_tracker Class Reference	106
10.42.1	Detailed Description	107
10.42.2	Constructor & Destructor Documentation	107
10.42.2.1	marker_tracker()	107
10.42.3	Member Function Documentation	107
10.42.3.1	detect_markers()	107
10.42.3.2	find_camera_index_by_name()	108
10.42.3.3	get_camera_index()	108
10.42.3.4	get_camera_matrix()	108
10.42.3.5	get_current_frame()	108

10.42.3.6	get_dist_coeffs()	109
10.42.3.7	get_latest_frame()	109
10.42.3.8	get_valid_robot_marker_ids()	109
10.42.3.9	initialize_camera()	110
10.42.3.10	initialize_camera_auto()	110
10.42.3.11	load_calibration()	111
10.42.3.12	reinitialize()	111
10.42.3.13	robot_slot_for_marker()	111
10.42.3.14	shutdown_camera()	112
10.43	MessageTarget Class Reference	112
10.43.1	Detailed Description	112
10.43.2	Member Function Documentation	112
10.43.2.1	handle()	112
10.44	MiniGame Class Reference	113
10.44.1	Detailed Description	113
10.44.2	Member Function Documentation	113
10.44.2.1	getName()	113
10.44.2.2	onRobotMoved()	114
10.44.2.3	play()	114
10.45	MiniGameParticipant Struct Reference	114
10.45.1	Detailed Description	115
10.46	MiniGameResult Struct Reference	115
10.46.1	Detailed Description	115
10.47	MockCamera Class Reference	116
10.47.1	Detailed Description	116
10.47.2	Constructor & Destructor Documentation	116
10.47.2.1	MockCamera()	116
10.47.3	Member Function Documentation	117
10.47.3.1	update	117
10.48	MqttReceiver Class Reference	117
10.48.1	Detailed Description	118
10.49	NetworkMonitor Class Reference	118
10.49.1	Detailed Description	118
10.50	NetworkTopologyWidget Class Reference	118
10.50.1	Detailed Description	119
10.51	Node Class Reference	119
10.51.1	Detailed Description	120
10.51.2	Constructor & Destructor Documentation	120
10.51.2.1	Node()	120
10.51.3	Member Function Documentation	121
10.51.3.1	addNeighbor()	121
10.52	pathfinding::Node Struct Reference	121

10.52.1 Detailed Description	122
10.53 pathfinding::NodeCompare Struct Reference	122
10.53.1 Detailed Description	122
10.54 BoardLoader.NodeData Class Reference	123
10.55 ns::OpenShopEvent Struct Reference	123
10.55.1 Detailed Description	123
10.56 NetworkTopologyWidget::PairingSlot Struct Reference	123
10.57 follower::path_follower Class Reference	124
10.57.1 Detailed Description	124
10.57.2 Constructor & Destructor Documentation	125
10.57.2.1 path_follower()	125
10.57.3 Member Function Documentation	125
10.57.3.1 reset()	125
10.57.3.2 step()	125
10.58 collision::path_update Struct Reference	126
10.58.1 Detailed Description	126
10.58.2 Member Data Documentation	126
10.58.2.1 waypoints	126
10.59 PathDebug Class Reference	126
10.59.1 Detailed Description	127
10.60 pathfinding::PathPlanner Class Reference	127
10.60.1 Detailed Description	127
10.60.2 Member Function Documentation	128
10.60.2.1 plan()	128
10.61 collision::playboard_graph Class Reference	128
10.61.1 Detailed Description	129
10.61.2 Member Function Documentation	129
10.61.2.1 min_edge_distance()	129
10.61.2.2 min_node_distance()	130
10.61.2.3 nearest_node()	130
10.61.2.4 node()	130
10.61.2.5 route()	131
10.61.2.6 set_nodes()	131
10.62 Player Class Reference	131
10.62.1 Detailed Description	134
10.62.2 Constructor & Destructor Documentation	134
10.62.2.1 Player()	134
10.62.3 Member Function Documentation	134
10.62.3.1 addCoins()	134
10.62.3.2 addEffect()	135
10.62.3.3 addItem()	135
10.62.3.4 canAfford()	135

10.62.3.5 checkNegativeEffects()	135
10.62.3.6 getCurrentPlayer()	136
10.62.3.7 getCurrentPlayerID()	136
10.62.3.8 getId()	136
10.62.3.9 getWorldPosition()	136
10.62.3.10 setCurrentNode()	137
10.62.3.11 setDiceMultiplier()	137
10.62.3.12 setWorldPosition()	137
10.62.3.13 startTurn()	137
10.62.3.14 stealDiceSteps()	138
10.62.3.15 swapPosition()	138
10.63 PlayerSession Struct Reference	138
10.64 PlayerStanding Struct Reference	139
10.65 PlayerStats Struct Reference	139
10.65.1 Detailed Description	139
10.66 follower::pose2d Struct Reference	139
10.66.1 Detailed Description	140
10.67 Robot Struct Reference	140
10.67.1 Detailed Description	140
10.68 mind::fleet::robot_controller Class Reference	141
10.68.1 Detailed Description	142
10.68.2 Constructor & Destructor Documentation	142
10.68.2.1 robot_controller()	142
10.68.3 Member Function Documentation	143
10.68.3.1 beep()	143
10.68.3.2 drive_for()	143
10.68.3.3 handle_status()	144
10.68.3.4 move_relative()	144
10.68.3.5 on_pose_updated	144
10.68.3.6 set_brick_address()	145
10.68.3.7 set_drive()	145
10.68.3.8 set_wheel_drive()	146
10.68.3.9 stop()	146
10.68.3.10 turn()	146
10.69 mind::fleet::robot_fleet Class Reference	147
10.69.1 Detailed Description	147
10.69.2 Constructor & Destructor Documentation	148
10.69.2.1 robot_fleet()	148
10.69.3 Member Function Documentation	148
10.69.3.1 controller()	148
10.69.3.2 robot_appeared	148
10.69.3.3 robot_disappeared	149

10.69.3.4 robot_ip_discovered	149
10.69.3.5 select	149
10.70 collision::robot_pose Struct Reference	149
10.70.1 Detailed Description	150
10.71 MqttReceiver.RobotPositionData Class Reference	150
10.72 Roulette Class Reference	150
10.72.1 Detailed Description	151
10.73 ns::SelectedItemEvent Struct Reference	151
10.73.1 Detailed Description	152
10.74 Server Class Reference	152
10.74.1 Detailed Description	153
10.74.2 Constructor & Destructor Documentation	153
10.74.2.1 Server()	153
10.74.3 Member Function Documentation	153
10.74.3.1 broadcast_json()	153
10.74.3.2 broadcast_to_all()	154
10.74.3.3 send_to_user()	154
10.74.3.4 start_accept()	154
10.75 SessionRegistry Class Reference	155
10.76 Shop Class Reference	155
10.76.1 Detailed Description	156
10.77 ShopTestWindow Class Reference	156
10.77.1 Detailed Description	156
10.77.2 Constructor & Destructor Documentation	157
10.77.2.1 ShopTestWindow()	157
10.78 RobotGame::Splatoon Class Reference	157
10.78.1 Detailed Description	158
10.78.2 Constructor & Destructor Documentation	158
10.78.2.1 Splatoon()	158
10.78.3 Member Function Documentation	159
10.78.3.1 getName()	159
10.78.3.2 onRobotMoved()	159
10.78.3.3 play()	159
10.78.3.4 updateRobotPosition()	160
10.79 SplatoonGrid Class Reference	160
10.79.1 Detailed Description	161
10.79.2 Member Function Documentation	161
10.79.2.1 cells()	161
10.79.2.2 clear()	161
10.79.2.3 countValue()	161
10.79.2.4 get()	162
10.79.2.5 set()	162

10.80 start_game Class Reference	163
10.81 ns::StartRouletteEvent Struct Reference	163
10.81.1 Detailed Description	163
10.82 TcpConnection Class Reference	163
10.82.1 Detailed Description	164
10.82.2 Constructor & Destructor Documentation	164
10.82.2.1 TcpConnection()	164
10.82.3 Member Function Documentation	165
10.82.3.1 connect_to_server()	165
10.82.3.2 send_to_server()	165
10.82.3.3 start_read()	165
10.83 test_panel Class Reference	166
10.83.1 Detailed Description	166
10.83.2 Constructor & Destructor Documentation	166
10.83.2.1 test_panel()	166
10.84 TileEffect Struct Reference	167
10.84.1 Detailed Description	167
10.85 TopoClient Struct Reference	167
10.85.1 Detailed Description	168
10.86 TopoPulse Struct Reference	168
10.86.1 Detailed Description	168
10.87 TrackedRobot Struct Reference	168
10.88 TurnManager Class Reference	169
10.88.1 Detailed Description	170
10.88.2 Member Enumeration Documentation	171
10.88.2.1 TurnState	171
10.88.3 Constructor & Destructor Documentation	171
10.88.3.1 TurnManager()	171
10.88.4 Member Function Documentation	171
10.88.4.1 applyTile()	171
10.88.4.2 currentPlayer()	172
10.88.4.3 executeSelectedItem()	172
10.88.4.4 finishRound()	172
10.88.4.5 finishTurn()	172
10.88.4.6 getState()	172
10.88.4.7 movePlayer()	173
10.88.4.8 rollDice()	173
10.88.4.9 selectItem()	173
10.88.4.10 selectPath()	174
10.88.4.11 selectTargetPlayer()	174
10.88.4.12 setFleet()	174
10.88.4.13 startMinigame()	175

10.88.4.14 startTurn()	175
10.88.4.15 update()	175
10.89 mind::net::udp_link Class Reference	176
10.89.1 Detailed Description	176
10.89.2 Constructor & Destructor Documentation	177
10.89.2.1 udp_link()	177
10.89.3 Member Function Documentation	177
10.89.3.1 datagram_received	177
10.89.3.2 send_command	177
10.89.3.3 start	178
10.90 UdpClient Class Reference	178
10.90.1 Detailed Description	178
10.90.2 Constructor & Destructor Documentation	179
10.90.2.1 UdpClient()	179
10.90.3 Member Function Documentation	179
10.90.3.1 start_receive()	179
10.91 UdpReceiver Class Reference	179
10.91.1 Detailed Description	180
10.91.2 Constructor & Destructor Documentation	180
10.91.2.1 UdpReceiver()	180
10.92 UdpSender Class Reference	180
10.92.1 Detailed Description	181
10.92.2 Constructor & Destructor Documentation	181
10.92.2.1 UdpSender() [1/2]	181
10.92.2.2 UdpSender() [2/2]	181
10.92.3 Member Function Documentation	182
10.92.3.1 send()	182
10.93 UdpServer Class Reference	182
10.93.1 Detailed Description	182
10.93.2 Constructor & Destructor Documentation	183
10.93.2.1 UdpServer()	183
10.93.3 Member Function Documentation	183
10.93.3.1 broadcast_to_all()	183
10.93.3.2 send_to_user()	183
10.93.3.3 start_receive()	184
10.94 Vec3< T > Struct Template Reference	184
10.94.1 Detailed Description	184
11 File Documentation	185
11.1 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/latency_indicator.cpp File Reference	185
11.1.1 Detailed Description	185
11.2 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/latency_indicator.hpp File Reference	185

11.2.1 Detailed Description	185
11.3 latency_indicator.hpp	186
11.4 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/main.cpp File Reference	186
11.4.1 Detailed Description	187
11.5 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/mock_camera.cpp File Reference	187
11.5.1 Detailed Description	187
11.5.2 Function Documentation	188
11.5.2.1 main()	188
11.6 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/camera_calibration.cpp File Reference	188
11.6.1 Detailed Description	188
11.7 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/camera_calibration.hpp File Reference	188
11.7.1 Detailed Description	189
11.8 camera_calibration.hpp	189
11.9 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.cpp File Reference	190
11.9.1 Detailed Description	190
11.9.2 Function Documentation	190
11.9.2.1 default_priority()	190
11.10 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.hpp File Reference	191
11.10.1 Detailed Description	192
11.10.2 Typedef Documentation	192
11.10.2.1 priority_rule	192
11.10.3 Enumeration Type Documentation	192
11.10.3.1 route_result	192
11.10.3.2 shove_result	193
11.10.4 Function Documentation	193
11.10.4.1 default_priority()	193
11.11 collision_planner.hpp	193
11.12 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/path_update.hpp File Reference	196
11.12.1 Detailed Description	196
11.12.2 Enumeration Type Documentation	197
11.12.2.1 follow_state	197
11.13 path_update.hpp	197
11.14 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.cpp File Reference	197
11.14.1 Detailed Description	198
11.15 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.hpp File Reference	198
11.15.1 Detailed Description	198
11.16 playboard_graph.hpp	198

11.17 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/handler_template.cpp File Reference	199
11.17.1 Detailed Description	199
11.17.2 Function Documentation	199
11.17.2.1 broadcast_update()	199
11.17.2.2 register_handlers()	200
11.18 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/handler_template.hpp File Reference	200
11.18.1 Detailed Description	201
11.18.2 Function Documentation	201
11.18.2.1 broadcast_update()	201
11.18.2.2 register_handlers()	201
11.19 handler_template.hpp	202
11.20 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.cpp File Reference	202
11.20.1 Detailed Description	202
11.20.2 Function Documentation	203
11.20.2.1 process()	203
11.20.2.2 register_message_handler()	203
11.21 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.hpp File Reference	203
11.21.1 Detailed Description	204
11.21.2 Typedef Documentation	204
11.21.2.1 message_handler	204
11.21.3 Function Documentation	205
11.21.3.1 process()	205
11.21.3.2 register_message_handler()	205
11.22 message_processing.hpp	205
11.23 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp File Reference	206
11.23.1 Detailed Description	206
11.24 network_messages.hpp	207
11.25 session_registry.hpp	207
11.26 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.hpp File Reference	208
11.26.1 Detailed Description	209
11.27 TCP_Server.hpp	209
11.28 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.cpp File Reference	210
11.28.1 Detailed Description	210
11.29 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.hpp File Reference	211
11.29.1 Detailed Description	211
11.30 UDP_Server.hpp	211

11.31	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus.cpp File Reference	212
11.31.1	Detailed Description	212
11.32	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus.hpp File Reference	213
11.32.1	Detailed Description	213
11.33	bonus.hpp	213
11.34	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.cpp File Reference	214
11.34.1	Detailed Description	214
11.34.2	Function Documentation	214
11.34.2.1	broadcastGameResults()	214
11.34.2.2	makeGameResultsMessage()	215
11.35	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.hpp File Reference	215
11.35.1	Detailed Description	215
11.35.2	Function Documentation	215
11.35.2.1	broadcastGameResults()	215
11.35.2.2	makeGameResultsMessage()	215
11.36	bonus_json.hpp	216
11.37	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.cpp File Reference	216
11.37.1	Detailed Description	216
11.38	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.hpp File Reference	216
11.38.1	Detailed Description	217
11.39	game.hpp	217
11.40	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.cpp File Reference	217
11.40.1	Detailed Description	218
11.41	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.hpp File Reference	218
11.41.1	Detailed Description	218
11.42	game_manager.hpp	218
11.43	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/lobby.cpp File Reference	219
11.43.1	Detailed Description	219
11.44	network_utils.hpp	219
11.45	start_game.hpp	220
11.46	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.cpp File Reference	220
11.46.1	Detailed Description	221
11.46.2	Function Documentation	221
11.46.2.1	register_handlers_turnmanager() [1/2]	221
11.46.2.2	register_handlers_turnmanager() [2/2]	221
11.47	/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.hpp File Reference	221
11.47.1	Detailed Description	222
11.47.2	Function Documentation	222
11.47.2.1	register_handlers_turnmanager() [1/2]	222
11.47.2.2	register_handlers_turnmanager() [2/2]	222
11.48	turnManager.hpp	223

11.49 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.cpp File Reference	224
11.49.1 Detailed Description	224
11.50 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.hpp File Reference	224
11.50.1 Detailed Description	225
11.51 item.hpp	225
11.52 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.cpp File Reference	226
11.52.1 Detailed Description	227
11.53 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.hpp File Reference	227
11.53.1 Detailed Description	227
11.54 shop.hpp	228
11.55 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mainpage.md File Reference	228
11.55.1 Detailed Description	228
11.56 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.cpp File Reference	229
11.56.1 Detailed Description	229
11.56.2 Function Documentation	230
11.56.2.1 register_handlers_board()	230
11.57 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.hpp File Reference	230
11.57.1 Detailed Description	231
11.57.2 Function Documentation	231
11.57.2.1 register_handlers_board()	231
11.58 board.hpp	231
11.59 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.cpp File Reference	232
11.59.1 Detailed Description	232
11.60 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.hpp File Reference	233
11.60.1 Detailed Description	233
11.61 follower_tuning_panel.hpp	233
11.62 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/mapwindow.cpp File Reference	234
11.62.1 Detailed Description	234
11.63 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/mapwindow.hpp File Reference	234
11.63.1 Detailed Description	235
11.64 mapwindow.hpp	235
11.65 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.cpp File Reference	238
11.65.1 Detailed Description	239
11.66 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.hpp File Reference	239
11.66.1 Detailed Description	239
11.67 rng.hpp	239
11.68 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/roulette.cpp File Reference	240
11.68.1 Detailed Description	240
11.68.2 Function Documentation	240
11.68.2.1 broadcast_update_roulette()	240
11.68.2.2 register_handlers_roulette()	241

11.69 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/roulette.hpp File Reference	241
11.69.1 Detailed Description	242
11.69.2 Function Documentation	242
11.69.2.1 broadcast_update_roulette()	242
11.69.2.2 register_handlers_roulette()	242
11.70 roulette.hpp	242
11.71 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame.hpp File Reference	243
11.71.1 Detailed Description	243
11.72 minigame.hpp	244
11.73 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_participant.hpp File Reference	244
11.73.1 Detailed Description	244
11.74 minigame_participant.hpp	244
11.75 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_result.hpp File Reference	245
11.75.1 Detailed Description	245
11.76 minigame_result.hpp	245
11.77 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon.cpp File Reference	245
11.77.1 Detailed Description	246
11.78 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon.hpp File Reference	246
11.78.1 Detailed Description	246
11.79 splatoon.hpp	246
11.80 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon_grid.cpp File Refer- ence	248
11.80.1 Detailed Description	248
11.81 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon_grid.hpp File Refer- ence	248
11.81.1 Detailed Description	248
11.82 splatoon_grid.hpp	249
11.83 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/udp_reciever.hpp File Refer- ence	249
11.83.1 Detailed Description	250
11.84 udp_reciever.hpp	250
11.85 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/udp_sender.hpp File Reference	251
11.85.1 Detailed Description	251
11.86 udp_sender.hpp	252
11.87 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.cpp File Reference . . .	253
11.87.1 Detailed Description	253
11.88 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.hpp File Reference . . .	253
11.88.1 Detailed Description	253
11.89 client.hpp	254
11.90 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.cpp File Ref- erence	254
11.90.1 Detailed Description	255

11.90.2 Function Documentation	255
11.90.2.1 to_percent()	255
11.91 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.hpp File Reference	255
11.91.1 Detailed Description	256
11.91.2 Function Documentation	256
11.91.2.1 to_percent()	256
11.92 path_follower.hpp	257
11.93 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/gridmap.cpp File Reference	258
11.93.1 Detailed Description	258
11.94 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/gridmap.hpp File Reference	258
11.94.1 Detailed Description	259
11.95 gridmap.hpp	259
11.96 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.cpp File Reference	259
11.96.1 Detailed Description	260
11.97 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.hpp File Reference	260
11.97.1 Detailed Description	260
11.98 path_planner.hpp	260
11.99 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/boardEffects.cpp File Reference	261
11.99.1 Detailed Description	261
11.99.2 Function Documentation	261
11.99.2.1 applyTileEffect()	261
11.100 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/boardEffects.hpp File Reference	262
11.100.1 Detailed Description	262
11.100.2 Function Documentation	262
11.100.2.1 applyTileEffect()	262
11.101 boardEffects.hpp	262
11.102 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.cpp File Reference	263
11.102.1 Detailed Description	263
11.103 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.hpp File Reference	263
11.103.1 Detailed Description	263
11.104 dice.hpp	263
11.105 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.cpp File Reference	264
11.105.1 Detailed Description	264
11.106 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.hpp File Reference	264
11.106.1 Detailed Description	264
11.107 inventory.hpp	265
11.108 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.cpp File Reference	265
11.108.1 Detailed Description	265
11.109 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.hpp File Reference	266
11.109.1 Detailed Description	266
11.110 player.hpp	267

11.111 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.cpp File Reference . . .	269
11.111.1 Detailed Description	270
11.111.2 Function Documentation	270
11.111.2.1 decode_ack()	270
11.111.2.2 decode_heartbeat()	270
11.111.2.3 encode_beep()	271
11.111.2.4 encode_drive()	271
11.111.2.5 encode_drive_for()	271
11.111.2.6 encode_ping()	272
11.111.2.7 encode_stop()	272
11.111.2.8 encode_turn()	273
11.111.2.9 encode_wheel_drive()	273
11.111.2.10 peek_status_type()	273
11.112 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.hpp File Reference . . .	274
11.112.1 Detailed Description	275
11.112.2 Enumeration Type Documentation	275
11.112.2.1 opcode	275
11.112.2.2 robot_state	276
11.112.2.3 status_type	276
11.112.3 Function Documentation	276
11.112.3.1 decode_ack()	276
11.112.3.2 decode_heartbeat()	277
11.112.3.3 encode_beep()	277
11.112.3.4 encode_drive()	278
11.112.3.5 encode_drive_for()	278
11.112.3.6 encode_ping()	278
11.112.3.7 encode_stop()	279
11.112.3.8 encode_turn()	279
11.112.3.9 encode_wheel_drive()	280
11.112.3.10 peek_status_type()	280
11.113 codec.hpp	280
11.114 background_thread.hpp	281
11.115 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.cpp File Refer- ence	282
11.115.1 Detailed Description	282
11.116 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.hpp File Refer- ence	283
11.116.1 Detailed Description	283
11.117 control_window.hpp	283
11.118 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_dashboard.cpp File Refer- ence	284
11.118.1 Detailed Description	284

11.119 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_dashboard.hpp File Reference	284
11.119.1 Detailed Description	284
11.120 fleet_dashboard.hpp	285
11.121 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.cpp File Reference	285
11.121.1 Detailed Description	286
11.122 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.hpp File Reference	286
11.122.1 Detailed Description	286
11.123 fleet_panel.hpp	286
11.124 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.cpp File Reference	287
11.124.1 Detailed Description	287
11.125 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.hpp File Reference	287
11.125.1 Detailed Description	288
11.126 main_window.hpp	288
11.127 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.cpp File Reference	289
11.127.1 Detailed Description	289
11.128 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.hpp File Reference	290
11.128.1 Detailed Description	290
11.129 network_monitor.hpp	290
11.130 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.cpp File Reference	291
11.130.1 Detailed Description	291
11.131 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.hpp File Reference	291
11.131.1 Detailed Description	292
11.132 network_topology_widget.hpp	293
11.133 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/path_debug.cpp File Reference	300
11.133.1 Detailed Description	300
11.134 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/path_debug.hpp File Reference	300
11.134.1 Detailed Description	300
11.135 path_debug.hpp	301
11.136 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.cpp File Reference	301
11.136.1 Detailed Description	301
11.137 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.hpp File Reference	302
11.137.1 Detailed Description	302
11.138 shop_test_window.hpp	302
11.139 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.cpp File Reference	303
11.139.1 Detailed Description	303
11.140 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.hpp File Reference	303
11.140.1 Detailed Description	304

11.141 test_panel.hpp	304
11.142 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.cpp File Reference	304
11.142.1 Detailed Description	305
11.143 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.hpp File Reference	305
11.143.1 Detailed Description	305
11.144 robot_controller.hpp	305
11.145 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.cpp File Reference	307
11.145.1 Detailed Description	307
11.146 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.hpp File Reference	307
11.146.1 Detailed Description	308
11.147 robot_fleet.hpp	308
11.148 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.cpp File Reference	308
11.148.1 Detailed Description	309
11.149 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.hpp File Reference	309
11.149.1 Detailed Description	310
11.150 tracker.hpp	310
11.151 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/udp_link.cpp File Reference	311
11.151.1 Detailed Description	311
11.152 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/udp_link.hpp File Reference	311
11.152.1 Detailed Description	312
11.153 udp_link.hpp	312
11.154 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/mapLogic.cpp File Reference	312
11.154.1 Detailed Description	313
11.154.2 Function Documentation	313
11.154.2.1 main()	313
11.154.2.2 nodeTypeToString()	314
11.155 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/enum_tools.hpp File Reference	314
11.155.1 Detailed Description	314
11.155.2 Enumeration Type Documentation	314
11.155.2.1 LogLevel	314
11.155.3 Function Documentation	315
11.155.3.1 EnumToString() [1/2]	315
11.155.3.2 EnumToString() [2/2]	315
11.156 enum_tools.hpp	316
11.157 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/Logger.cpp File Reference	316
11.157.1 Detailed Description	316
11.158 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/Logger.hpp File Reference	316
11.158.1 Detailed Description	317
11.158.2 Macro Definition Documentation	318

11.158.2.1 EXPECT_EQ	318
11.158.2.2 EXPECT_NEQ	318
11.158.2.3 LOG	319
11.158.2.4 LOG_ENTRY_EXIT	319
11.158.2.5 LOG_ENTRY_EXIT_THROTTLED	319
11.158.2.6 SET_LOG_LEVELS	320
11.159 Logger.hpp	320
Index	323

Chapter 1

mainpage

/**

•

Chapter 2

Todo List

Member `mind::mqtt::client::connect_to` (QString host, int port)

host and port could be set in the constructor.

Chapter 3

Directory Hierarchy

3.1 Directories

mini_games	27
minigame.hpp	243
minigame_participant.hpp	244
minigame_result.hpp	245
splatoon.cpp	245
splatoon.hpp	246
splatoon_grid.cpp	248
splatoon_grid.hpp	248
udp_reciever.hpp	249
udp_sender.hpp	251

Chapter 4

Namespace Index

4.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

ns	Network message payload types shared between the desktop app and the AR/Unity client. Each struct uses NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE to generate to_json/from_json, so a handler registered via register_message_handler() can deserialize the incoming JSON with message.get<ns::SomeEvent>()	37
TopoCategory	Traffic category identifiers used throughout the topology	38

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

mind::proto::ack_msg	39
MqttReceiver.AnchorData	40
Board	41
BoardLoader.BoardData	44
BonusAward	45
BonusPhase	45
BonusResult	46
ns::CloseShopEvent	54
collision::collision_planner	54
Dice	62
follower::drive_cmd	63
follower::drive_pct	63
Effect	64
BoardLoader.EffectData	65
std::enable_shared_from_this	
TcpConnection	163
ns::EndRouletteEvent	66
follower::follower_params	68
Game	72
GameManager	74
Generator	76
ns::GetPlayerBetEvent	78
collision::graph_node	79
pathfinding::GridMap	79
mind::proto::heartbeat_msg	83
Inventory	84
Item	87
ItemData	89
LogEntryExit	91
Logger	92
LogStream	95
marker_tracker::marker_position	105
marker_tracker::marker_tracker	106
MessageTarget	112
MiniGame	113

RobotGame::Splatoon	157
MiniGameParticipant	114
MiniGameResult	115
MonoBehaviour	
BoardLoader	44
MqttReceiver	117
Node	119
pathfinding::Node	121
pathfinding::NodeCompare	122
BoardLoader.NodeData	123
ns::OpenShopEvent	123
NetworkTopologyWidget::PairingSlot	123
follower::path_follower	124
collision::path_update	126
pathfinding::PathPlanner	127
collision::playboard_graph	128
Player	131
PlayerSession	138
PlayerStanding	139
PlayerStats	139
follower::pose2d	139
QMainWindow	
MapWindow	98
control_window	61
main_window	97
QObject	
ArUcoWorker	40
CameraCalibration	47
MockCamera	116
mind::fleet::robot_controller	141
mind::fleet::robot_fleet	147
mind::mqtt::client	50
mind::net::udp_link	176
QWidget	
FleetDashboard	67
NetworkMonitor	118
NetworkTopologyWidget	118
PathDebug	126
ShopTestWindow	156
fleet_panel	66
follower_tuning_panel	69
latency_indicator	90
test_panel	166
Robot	140
collision::robot_pose	149
MqttReceiver.RobotPositionData	150
Roulette	150
ns::SelectedItemEvent	151
Server	152
SessionRegistry	155
Shop	155
SplatoonGrid	160
start_game	163
ns::StartRouletteEvent	163
TileEffect	167
TopoClient	167
TopoPulse	168
TrackedRobot	168

TurnManager	169
UdpClient	178
UdpReceiver	179
UdpSender	180
UdpServer	182
Vec3< T >	184

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

mind::proto::ack_msg	Decoded <code>ack</code> status payload (EV3 -> desktop, MQTT). Echoes the command that triggered it (or <code>no_cmd_id</code>) plus the robot's current state, battery level, and error detail	39
MqttReceiver.AnchorData		40
ArUcoWorker		40
Board	Owns the static node graph for the game board and can broadcast it to clients over TCP. The graph is hard-coded in the constructor; nothing in this class mutates it afterward	41
BoardLoader.BoardData		44
BoardLoader	Loads the <code>board.json</code> file exported by the Desktop App	44
BonusAward		45
BonusPhase	Stateless end-of-game scoring phase. Scans all players in a Game , awards bonus stars, and returns the complete result needed by presentation and networking layers	45
BonusResult		46
CameraCalibration	Captures frames from a webcam and computes the camera's extrinsic pose relative to a known 3D field layout. Owns a QTimer-driven capture loop that emits each frame as a QImage for display, and wraps OpenCV's <code>solvePnP</code> to derive rotation/translation vectors once intrinsic parameters and a set of matching 3D/2D point correspondences (e.g. detected field markers) are supplied	47
mind::mqtt::client	Asynchronous MQTT client, wrapping <code>libmosquitto</code> behind Qt signals	50
ns::CloseShopEvent	Payload of the "close_shop" message: identifies which player closed the shop. Parsed in shop.cpp 's "close_shop" handler and used to build a temporary Player before calling Shop::close_shop()	54
collision::collision_planner	Central, single-instance collision-avoidance coordinator (plain C++/STL + OpenCV, no Qt). Owns the board frame, the stateless <code>PathPlanner</code> , every robot's nominal + effective path, and per-pair proximity state; driven by update() , publishes versioned <code>path_updates</code> through a sink. Implements CTFS (Component-Token Freeze-Senior Step-Off): on every tick, connected components of currently conflicting robots are resolved by letting the lowest-id member of each component keep moving while every other member is frozen, stepped off the lane, or held	54

control_window	Standalone window for robot control. Owns the MQTT client, the robot_fleet auto-discovery, and a fleet_panel that shows one row per discovered robot. Connects to the broker on construction (host + port from env, defaults to localhost:1883). main.cpp just opens this	61
Dice	Represents a die that produces a random roll in [1, 10]. Instances are typically constructed as throwaway objects purely to invoke throwDice() ; the "dice" parameter accepted by the member functions below is currently unused	62
follower::drive_cmd	Controller output: a velocity setpoint for the drive base	63
follower::drive_pct	Brick tank-drive percentages, each clamped to [-100, 100]	63
Effect	Base class for timed status effects attached to a Player . Concrete subclasses implement applyStartOfTurn()/applyEndOfTurn() to modify player state; the effect expires once its duration counts down to zero (see tick()/isExpired())	64
BoardLoader.EffectData		65
ns::EndRouletteEvent	Payload of the "end_roulette" message: identifies which player's roulette round is ending. Parsed in roulette.cpp 's "end_roulette" handler and used to build a temporary Player before calling Roulette::end_roulette()	66
fleet_panel	Vertical stack of one row per discovered robot. Listens to robot_fleet for appear events and to each robot_controller for state, battery, and connection updates. Rows stay around when a robot goes offline (the state label flips to "offline") - the controller itself stays alive for state continuity if the brick returns	66
FleetDashboard	Casino-themed Fleet Dashboard showing all 4 robot slots with live connection, battery, EV3 state, follower mode, target node, cross-track error, and calibration scales. Refreshes at 4 Hz via an internal QTimer	67
follower::follower_params	All tunable gains and limits for the path follower, in one plain, copyable, JSON-friendly struct. A Qt tuning panel can read/write these live and save/load them. The path_follower re-reads this struct fresh on every step(), so set_params() takes effect on the very next tick	68
follower_tuning_panel	Live tuning panel for the path-following driver. A thin Qt view over the plain follower_params struct: editing any gain emits changed() so the owner can push it to every running follower (taking effect on the next 33 ms tick). Save/Load persist a tune to JSON so it survives a restart; PANIC STOP halts all robots	69
Game	Top-level game session state: the board, the participating players, and whose turn/round it currently is. Turn order and round advancement are driven externally (e.g. by a turn manager) via nextPlayer()/nextRound()/determineTurnOrder() ; Game itself does not run a game loop	72
GameManager	Owns all Players and orchestrates running mini-games. Converts Players to MiniGameParticipant DTOs before a mini-game starts, runs the mini-game to completion, and reconciles the returned MiniGameResult back into the real Player objects (currently: coin balance only; stars are awarded elsewhere)	74
Generator	Simulates drawing a number from a European roulette wheel. Picks a uniformly random position around the wheel and maps it to the pocket number printed at that position, so results follow the physical wheel layout rather than a plain sequential 0-36 order	76
ns::GetPlayerBetEvent	Payload of the "make_bet" message: the player's roulette wager. Parsed in roulette.cpp 's "make_bet" handler and forwarded to Roulette::setBet() , which wakes any thread blocked in Roulette::choose_bet_type()	78

collision::graph_node	One node of the playboard graph in the planner's board frame (centimetres). This is the Qt-free, unit-consistent mirror of map/board.hpp 's Node : the caller (MapWindow) converts each Node 's screen pixels to cm once and injects the result via collision_planner::set_graph	79
pathfinding::GridMap	2D occupancy grid over the physical play area, used for path planning. Stores one walkable/↔ blocked flag per cell and provides obstacle marking, world↔grid coordinate conversion, and a line-of-sight test consumed by the pathfinder	79
mind::proto::heartbeat_msg	Decoded heartbeat status payload (EV3 -> desktop, UDP). Published periodically regardless of state; the datagram's source address doubles as the desktop's discovery mechanism for the robot	83
Inventory	Holds a bounded collection of items owned by a player. Enforces a maximum capacity (MAX_↔ INVENTORY_SIZE) and provides index-based lookup, addition, and removal of items in insertion order	84
Item	A single item instance identified by its ItemType . Depending on the type, using it either overrides the current player's dice roll (getDiceOverride()) or applies an effect to the current and/or a target player (applyItemEffects())	87
ItemData	Static catalog entry describing a purchasable item: display name, effect type, catalog id, and shop price	89
latency_indicator	Visual indicator for measuring round-trip latency	90
LogEntryExit	RAII helper that records a start timestamp on construction and emits a Trace-level "EXIT - <seconds>s" log message (via a temporary LogStream) on destruction	91
Logger	Singleton that owns a background thread which asynchronously drains queued log messages	92
LogStream	RAII helper that accumulates a log message via operator<< and forwards the accumulated text to the Logger when it goes out of scope	95
main_window	Main race-monitor window: owns the camera marker_tracker , runs the ArUco detection loop on a timer, performs extrinsic calibration via solvePnP, and publishes anchor + robot positions to MQTT and to any connected TCP clients	97
MapWindow	Live tracking + driving Qt window for up to 4 robots. Owns the board node graph, the ArUco marker_tracker , the collision_planner , and one path_follower per robot slot. Runs in either an offline kinematic simulation (default, no camera required) or LIVE camera mode (toggled via the toolbar); a separate ARM DRIVE toggle independently gates whether commands reach real EV3 bricks over the injected robot_fleet . Ticks every 33 ms via an internal QTimer (see onTrackerTick)	98
marker_tracker::marker_position	Result of one ArUco marker detection: identifier plus its pose in the current camera frame. Position and angle are derived directly from the raw detected corners in camera pixel space; converting to real-world floor coordinates (e.g. via homography/extrinsics) is left to the caller	105
marker_tracker::marker_tracker	Owns one camera device and the ArUco detection pipeline for that camera. Wraps a cv::Video_↔ Capture plus a custom 12-marker ArUco dictionary/detector, and exposes the camera intrinsic calibration (shared via static members) used to interpret detected marker poses	106
MessageTarget	Placeholder for the class that owns the logic a message should trigger. In real use this is something like GameManager or Board . Passed into register_handlers() by reference and must outlive the server	112

MiniGame	Abstract base class for all mini-games. Operates only on MiniGameParticipant - no dependency on Player , Board , or inventory. Results are returned via MiniGameResult and applied to real Players by GameManager	113
MiniGameParticipant	Data-transfer object representing a player during a mini-game. Created from a real Player before the mini-game starts. Mini-games read and mutate this copy in isolation from Player/Board/↔ inventory; GameManager reconciles the (possibly changed) fields back into the real Player once play() returns	114
MiniGameResult	Returned by every mini-game after play() finishes. The board-game layer (GameManager) reads this and applies rewards to the real Player objects	115
MockCamera	Fakes the tracking camera pipeline: on a timer, publishes a fixed calibration anchor plus procedurally-animated robot and board-marker positions to MQTT, in the same JSON shape and on the same topics the real camera (main_window) uses	116
MqttReceiver	Receives anchor and robot position data from the Desktop App via MQTT	117
NetworkMonitor	Casino-themed Network Monitor showing TCP server sessions, tracking camera state, homography state, and per-robot fleet connectivity. Refreshes at 4 Hz via an internal QTimer	118
NetworkTopologyWidget	Network topology widget with 3-column layout: Unity TCP clients (left) SERVER (center) Robot UDP clients (right)	118
Node	One position ("tile") in the board's node graph. neighbors form an undirected adjacency list used for robot movement/pathfinding	119
pathfinding::Node	A single cell visited during A* search, linked to its predecessor for path retracing. Coordinates (x, y) are grid cell indices (not world/mm coordinates); g/h/f follow the standard A* convention where g is the accumulated cost from the start, h is the heuristic estimate to the goal, and f = g + h is the priority used to order the open list	121
pathfinding::NodeCompare	Min-heap comparator for the A* open list, ordering Node pointers so that std::priority_queue pops the node with the lowest f-cost first	122
BoardLoader.NodeData		123
ns::OpenShopEvent	Payload of the "open_shop" message: identifies which player opened the shop. Parsed in shop.cpp 's "open_shop" handler and used to build a temporary Player before calling Shop::openShop()	123
NetworkTopologyWidget::PairingSlot		123
follower::path_follower	Per-robot path-following controller: pure pursuit + heading PID. Combines lookahead pursuit on the planner's waypoint polyline with a heading PID, curvature/goal speed-scaling, a rotate-in-place phase for large heading errors, and arrival/stop detection. Consumes the planner's already-rounded polyline, so it never cuts corners itself. Plain C++/STL + OpenCV only, no Qt, no network - unit-testable in a kinematic sim	124
collision::path_update	One immutable snapshot of a single robot's effective path. The planner publishes a fresh snapshot (never edits one in place) whenever the path changes; consumers keep only the highest version seen and follow it	126
PathDebug	Casino-themed Path Debug panel showing per-robot collision planner state (effective path, nominal path, route nodes, yielding status, watched pairs) and follower phase/cross-track data. Refreshes at 4 Hz	126

pathfinding::PathPlanner	
Grid-based A* path planner that finds a walkable route between two world-space points. Queries a GridMap for walkability and coordinate conversion, then post-processes the raw grid-aligned path with line-of-sight smoothing before returning it	127
collision::playboard_graph	
The playboard's node graph, used for edge-following routes (move_playboard) and for placing a shoved robot off the lanes. Plain C++/STL + OpenCV, no Qt. Routing is exact shortest path (Dijkstra by Euclidean edge length); the graph is small (~21 nodes) so this is trivially cheap . .	128
Player	
Represents one game participant: identity, board position, coins/stars, active status effects, inventory, and per-turn state. A player is paired with a physical robot (robotID); moving the player's logical node (setCurrentNode) also requests the collision planner to route that robot to the new node	131
PlayerSession	138
PlayerStanding	139
PlayerStats	
Aggregate per-player session statistics (turns played, items used, effects received, fields moved, coins earned)	139
follower::pose2d	
Robot pose in the board frame. Position in centimetres, heading in radians using atan2(dy, dx) (0 = +x), the SAME convention as the planner and the kinematic sim. The board cm frame is y-down (left-handed); that is fine as long as everything (planner, follower, sim) shares it - the only place the real-world chirality matters is the hardware steering sign (params.flip_turn)	139
Robot	
Live state of one tracked/simulated robot slot (0-3), used for rendering and hit-testing. The authoritative pose for driving is follower::pose2d (see MapWindow::poseOf); x/y/heading↔_rad here are the on-screen projection of that pose, refreshed every tick	140
mind::fleet::robot_controller	
Per-robot facade owned by robot_fleet : translates method calls into MQTT/UDP commands and translates incoming EV3 status into Qt signals. Owns its own timers and runs the move_relative turn -> drive_for state machine internally; one instance exists per tracked robot	141
mind::fleet::robot_fleet	
Auto-discovers EV3 robots and owns one robot_controller per robot. Robots are registered on their first UDP heartbeat (see on_datagram_received); MQTT status acks on mind/+/status are then routed to the matching already-discovered controller (see on_status_message). Also tracks which robot is currently "active" (see select() /active())	147
collision::robot_pose	
Live pose of one robot in the board frame (centimetres), fed to the planner each tick. moving distinguishes an actively-driving robot (a collision partner, watched pairwise) from a parked one (a static obstacle for others)	149
MqttReceiver.RobotPositionData	150
Roulette	
Runs one roulette betting round for a single player, driven by network messages from the AR/Unity client. Owns the roll-to-result logic (via Generator) and blocks the calling thread in choose_bet_type() until the client's bet arrives over the network, synchronizing between the asio message-handler thread (setBet()) and whichever thread drives the game turn (give_take_coins())	150
ns::SelectedItemEvent	
Payload describing a player's item selection from the shop UI	151
Server	
Verwaltet den TCP-Netzwerk-Server und die Verteilung von Nachrichten an verbundene Clients	152
SessionRegistry	155

Shop	
The in-game item shop: a fixed catalog of purchasable items, opened/closed via the "open_↔shop"/"close_shop" network messages (see openShop() , close_shop()). An item selection made on the AR/Unity client is intended to arrive asynchronously via setSelectedItem() and be consumed by selectItem() , which blocks the calling thread until a selection is made, after which purchaseItem() charges the player; neither pathway is currently wired to a network message handler in this codebase	155
ShopTestWindow	
Manual test/debug panel with buttons to trigger a shop-open or roulette-start broadcast for a chosen player id, without going through the actual game flow. Constructs (and reuses) a throwaway Player instance purely to satisfy the Shop/Roulette APIs, which take a Player& but only read its current player id	156
RobotGame::Splatoon	
Mini-game where each robot paints cells of a 9x15 grid by driving over them. Converts camera-tracked robot positions into grid cells, mirrors the board state and countdown to every player's phone over UDP, and scores by painted-cell count once the fixed-length round (DURATION_↔SECONDS) ends	157
SplatoonGrid	
9x15 grid modeling the physical board area painted during the Splatoon mini-game. Each cell holds 0 (neutral/unpainted) or 1-4 (painted by that player). Row 0 is the top row, column 0 is the left column, and cells are stored row-major in a flat array (index = row * COLS + col). Pure data class - no Qt, no rendering	160
start_game	163
ns::StartRouletteEvent	
Payload of the "roulette_start" message: identifies which player is starting a roulette round. Parsed in roulette.cpp 's "roulette_start" handler and used to build a temporary Player before calling Roulette::start_roulette()	163
TcpConnection	
Repräsentiert eine einzelne aktive TCP-Verbindung zu einem Client (Unity / AR-App / Gerät)	163
test_panel	
Six-button panel for manually exercising the active robot's drive API. Listens to robot_fleet::↔active_changed to retarget itself. Buttons cover the Goal D behaviours: continuous drive (hot path + brick brake stop), precision drive_for / turn (calibration verify), stop interrupting a move mid-flight (drive_for + stop, turn + stop), and an emergency stop. Per-test QTimer's are stop()'d before each press so back-to-back clicks do not chop each other	166
TileEffect	
One effect entry attached to a board node. The actual gameplay logic for each type (coin gain/↔loss, opening the shop) is implemented in applyTileEffect() (player/boardEffects.cpp), not here - this struct only carries the data	167
TopoClient	
One client entry for the topology diagram	167
TopoPulse	
A single animated pulse traveling along a link arrow	168
TrackedRobot	168
TurnManager	
Per-game state machine that drives a single player through one turn: item usage, dice roll, movement, tile effects, and (at the end of a round) the minigame phase. Owns no gameplay data itself; each handler mutates the associated Game/Player and advances state. Progression through most states is driven by update() , but the WaitingFor* states pause until the matching select/skip method is called (e.g. from network/UI input)	169
mind::net::udp_link	
Thin UDP transport for loss-tolerant streams: sends command datagrams to a brick and receives heartbeats on a bound port. Decoding is the caller's job (mind::proto); this layer only moves raw bytes. Runs on its own QThread; send_command is self-marshalling, so callers on any thread queue onto this object's thread via QMetaObject::invokeMethod	176
UdpClient	
Provides a UDP client interface for communicating with the server	178

[UdpReceiver](#)

Listens for UDP datagrams on a fixed port and reports each one via a callback. Binds to INADDR_ANY on construction and spins up a dedicated background thread that blocks on `recvfrom()` for the lifetime of the object 179

[UdpSender](#)

Fire-and-forget UDP sender for one fixed destination (ip:port). Create one instance per target and reuse it for every [send\(\)](#) call; each instance owns a single UDP socket for its lifetime . . . 180

[UdpServer](#)

Manages a UDP server that tracks multiple connected clients 182

[Vec3< T >](#)

Generic 3-component vector; used here to store a player's world-space (x, y, z) position 184

Chapter 7

File Index

7.1 File List

Here is a list of all documented files with brief descriptions:

/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/ latency_indicator.cpp	
Implementation of the latency measurement indicator widget	185
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/ latency_indicator.hpp	
Declares a small widget that visually flashes on each command sent, for latency measurement	185
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/ main.cpp	
Application entry point: builds the Qt windows and game-logic objects, wires the fleet into the map, and runs the TCP/JSON server for the AR app on its own thread alongside the Qt event loop	186
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/ mock_camera.cpp	
Standalone tool that simulates the tracking camera by publishing synthetic anchor/robot/board-marker positions over MQTT, so downstream consumers (e.g. the Unity AR app) can be developed and tested without real camera hardware	187
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/ camera_calibration.cpp	
Implementation of CameraCalibration : camera capture loop and solvePnP-based extrinsic calibration	188
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/ camera_calibration.hpp	
Declares CameraCalibration , a live camera capture loop plus OpenCV solvePnP-based extrinsic pose calibration	188
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/ collision_planner.cpp	
Implementation of collision_planner : free-space and graph path planning, corner rounding, per-pair proximity hysteresis, and CTFS (Component-Token Freeze-Senior Step-Off) conflict resolution	190
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/ collision_planner.hpp	
Declares collision_planner , the multi-robot collision-avoidance and routing coordinator (CTFS: Component-Token Freeze-Senior Step-Off) for the playboard	191
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/ path_update.hpp	
Output contract (path_update, follow_state, path_sink) between the collision planner and the path follower / PID driver	196
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/ playboard_graph.cpp	
Implementation of the playboard node graph: lookups, nearest-node search, Dijkstra routing, and edge/node distance queries	197
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/ playboard_graph.hpp	
Board-frame node graph used for edge-following routes and park-spot placement, with Dijkstra shortest-path routing	198
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/ handler_template.cpp	
Implementation of the AR app -> desktop TCP/JSON message handling template	199

/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/handler_template.hpp	
Template for AR app -> desktop TCP/JSON message handling	200
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.cpp	
Implementation of the JSON message dispatch registry declared in message_processing.hpp	202
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.hpp	
JSON message dispatch for the AR app <-> desktop TCP channel: socket.cpp hands each received frame to process() , which parses it and routes it by its "type" field to the handler registered for that type via register_message_handler() . Add a new message kind with a single register_message_handler() call. EV3 traffic is a separate binary protocol, not this	203
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp	
JSON-serializable payload structs (DTOs) for the shop and roulette network events exchanged with the AR/Unity client, matched by message_processing.cpp 's "type" field dispatch	206
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/session_registry.hpp	207
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.hpp	
Definiert die TCP-Netzwerkschnittstellen für Client- und Server-Komponenten	208
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.cpp	
Implementation of UDP communication interfaces	210
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.hpp	
Defines UDP networking interfaces for server and client nodes	211
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus.cpp	
Implementation of the end-of-game bonus-star scoring phase	212
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus.hpp	
Declares BonusPhase , which awards end-of-game bonus stars based on per-player session statistics	213
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.cpp	
JSON serialization and TCP broadcast for the end-of-game result	214
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.hpp	
Converts end-of-game bonus results into the JSON message consumed by Unity	215
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.cpp	
Implementation of the Game class: turn/round bookkeeping and dice-roll-based turn order determination	216
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.hpp	
Declares the Game class, which owns the board and player list for a match and tracks whose turn it is, the current round, and match completion	216
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.cpp	
Implementation of GameManager : player ownership and mini-game orchestration	217
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.hpp	
Defines the GameManager class, which owns all players and orchestrates running mini-games between them	218
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/lobby.cpp	
Handles lobby related UDP and TCP messages	219
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/network_utils.hpp	219
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/start_game.hpp	220
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.cpp	
Implementation of TurnManager , the per-turn state machine	220
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.hpp	
Declares TurnManager , the per-turn state machine driving a player through item selection, dice rolling, movement, tile effects, and minigames	221
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.cpp	
Implementation of the Item class: dice-roll overrides and targeted effects applied when a player uses an item	224
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.hpp	
Declares ItemType , ItemData , and the Item class: items that can override a player's dice roll and/or apply a targeted or self effect (position swap, control inversion, dice theft, coin bonus) during a turn	224
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.cpp	
Implementation of Shop 's purchase flow and the network handlers that open/close the shop and broadcast its inventory to connected clients	226

/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/ shop.hpp	
Declares the Shop class (fixed item catalog and purchase flow) and the free functions that register its network message handlers and broadcast shop state to connected clients	227
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ board.cpp	
Implementation of Board : the hard-coded node graph, node lookups, and TCP broadcast of board data	229
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ board.hpp	
Declares the static game-board node graph (Board , Node , TileEffect) and the TCP registration for board-load requests	230
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ follower_tuning_panel.cpp	
Implementation of the path-follower tuning panel: builds the spin-box grid and handles JSON persistence	232
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ follower_tuning_panel.hpp	
Qt widget for live-tuning the path-follower's motion/PID gains, with JSON save/load and a panic-stop button	233
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ mapwindow.cpp	
Implementation of MapWindow : rendering, camera tracking, collision-aware robot driving (real + offline sim), arrival detection, and auto-calibration. Node graph data lives in Board (board.cpp)	234
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ mapwindow.hpp	
Declares MapWindow , the live tracking + driving Qt window: renders the board and robots, consumes the shared camera/offline-sim tick loop, and drives each robot via a path_follower	234
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ rng.cpp	
Implementation of the Generator roulette-wheel random number generator	238
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ rng.hpp	
Declares the Generator class, a roulette-wheel style random number generator	239
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ roulette.cpp	
Implementation of the Roulette betting game and its network message handlers	240
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/ roulette.hpp	
Declares the Roulette betting game: bet types, round state, and its network message wiring	241
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ minigame.hpp	
Defines the abstract MiniGame interface implemented by every playable mini-game	243
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ minigame_participant.hpp	
Defines the plain data-transfer object mini-games use to represent players	244
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ minigame_result.hpp	
Defines the outcome type returned by MiniGame::play()	245
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ splatoon.cpp	
Implementation of the Splatoon mini-game: camera-driven grid painting, UDP sync with each player's phone, and end-of-round scoring	245
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ splatoon.hpp	
Splatoon mini-game: robots paint cells of a grid by driving over them, tracked via camera position and mirrored to each player's phone over UDP	246
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ splatoon_grid.cpp	
Implementation of SplatoonGrid , the flat paint-ownership grid used by the Splatoon mini-game	248
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ splatoon_grid.hpp	
Fixed-size grid of per-cell paint ownership used by the Splatoon mini-game	248
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ udp_reciever.hpp	
Cross-platform (Windows/POSIX) UDP receiver that dispatches each datagram to a callback on a background thread	249
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/ udp_sender.hpp	
Cross-platform (Windows/POSIX) UDP datagram sender bound to a single destination	251
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/ client.cpp	
Implementation of the Qt/libmosquitto MQTT client wrapper	253
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/ client.hpp	
Declares the Qt wrapper around libmosquitto for async MQTT connectivity	253
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/ path_follower.cpp	
Implementation of the pure-pursuit + heading-PID path follower: polyline projection, lookahead, rotate-in-place turn dead-reckoning with live turn-rate learning, and the drive_cmd -> brick percentage mapping	254

/Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.hpp	
Per-robot path-following controller (pure pursuit + heading PID) that turns the collision planner's waypoint polyline and the robot's current pose into per-tick velocity/turn-rate drive commands	255
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/gridmap.cpp	
Implementation of the 2D occupancy grid map used for path planning	258
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/gridmap.hpp	
Defines the 2D occupancy grid map used for path planning	258
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.cpp	
Implementation of the A* path planner declared in path_planner.hpp	259
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.hpp	
Declares the A* grid path planner used to route between two world-space points on a GridMap	260
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/boardEffects.cpp	
Implementation of tile-effect application (coin changes, shop, hazards) for the board	261
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/boardEffects.hpp	
Declares the function that applies a board tile's effects to a player	262
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.cpp	
Implementation of the Dice class's random-roll generation	263
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.hpp	
Declares the Dice class used to roll random dice values for a player's turn	263
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.cpp	
Implementation of the Inventory class for managing a player's held items	264
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.hpp	
Defines the Inventory class, a bounded collection of items held by a player	264
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.cpp	
Implementation of the Player and Effect classes: turn state, board position, status effects, and coin/star/inventory bookkeeping	265
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.hpp	
Defines the Player class (turn state, board position, coins/stars, inventory, and status effects) and the Effect base class used to model timed buffs/debuffs applied to a player	266
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.cpp	
Implementation of the binary wire codec for the desktop <-> EV3 contract	269
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.hpp	
Binary wire codec for the desktop <-> EV3 contract (shared/protocol.md, Goal C)	274
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/background_thread.hpp	281
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.cpp	
Implementation of control_window: MQTT/UDP setup, robot fleet wiring, and UI layout for the desktop control window	282
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.hpp	
Top-level Qt window that wires up MQTT, UDP, and robot fleet discovery for the desktop control UI	283
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_dashboard.cpp	
Casino-themed Fleet Dashboard: live per-robot status overview	284
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_dashboard.hpp	
Standalone debug window: per-robot fleet overview with live battery, connection, follower state, and calibration data in the casino theme	284
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.cpp	
Implementation of the fleet panel's row creation and live-update wiring	285
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.hpp	
Widget that lists all discovered robots with live status and quick-test controls	286
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.cpp	
Implementation of main_window: widget setup, the ArUco tracking loop, camera extrinsic calibration, MQTT/TCP position publishing, and the Splatoon test harness	287
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.hpp	
Declares main_window, the race-monitor window that runs ArUco marker tracking, camera extrinsic calibration, and MQTT/TCP position publishing, plus a standalone Splatoon mini-game test harness	287
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.cpp	
Casino-themed Network Monitor: live connectivity and session overview	289

/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ network_monitor.hpp	
Standalone debug window: network connectivity, TCP sessions, and tracking status in the casino theme	290
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ network_topology_widget.cpp	
Compiles the MOC output for the header-only NetworkTopologyWidget	291
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ network_topology_widget.hpp	
Custom QPainter-drawn network topology diagram: 3-column layout (Unity clients Server Robot clients) with animated traffic pulses color-coded by category and pairing links between matched Unity-Robot pairs	291
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ path_debug.cpp	
Casino-themed Path Debug: collision planner and follower state overview	300
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ path_debug.hpp	
Standalone debug window: collision planner paths, routes, and follower state in the casino theme	300
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ shop_test_window.cpp	
Implementation of ShopTestWindow : builds the test panel UI and forwards button clicks to Shop::openShop() / Roulette::start_roulette() for a synthetic test player	301
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ shop_test_window.hpp	
Declares ShopTestWindow , a manual test panel for triggering shop and roulette network broadcasts to the AR/Unity client	302
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ test_panel.cpp	
Implementation of the manual robot-drive test panel	303
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/ test_panel.hpp	
Defines a manual test panel widget for exercising the active robot's drive API	303
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/ robot_controller.cpp	
Implementation of robot_controller : MQTT/UDP command publishing, the continuous-drive republish loop, and the move_relative state machine	304
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/ robot_controller.hpp	
Declares robot_controller , the per-robot facade that turns method calls into MQTT/UDP commands and turns EV3 status messages into Qt signals	305
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/ robot_fleet.cpp	
Implementation of robot_fleet : UDP-heartbeat-driven robot discovery and MQTT status ack routing	307
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/ robot_fleet.hpp	
Declares robot_fleet , which discovers EV3 robots via UDP heartbeats, routes MQTT status acks to them, and owns their robot_controller instances	307
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/ tracker.cpp	
Implementation of the ArUco marker detection camera pipeline (marker_tracker::marker_tracker)	308
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/ tracker.hpp	
Declares the ArUco marker detection camera pipeline (marker_tracker::marker_tracker) and the per-marker detection result (marker_tracker::marker_position)	309
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/ udp_link.cpp	
Implementation of the UDP transport for command and heartbeat datagrams	311
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/ udp_link.hpp	
Defines a thin, self-marshalling UDP transport for exchanging command and heartbeat datagrams with a brick	311
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/ mapLogic.cpp	
Standalone prototype of the board graph model (nodes, edges, node types) for the Mario Party style map	312
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/ enum_tools.hpp	
Utilities for converting enums to string representations. Also defines the LogLevel enum type used elsewhere in the codebase	314
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/ Logger.cpp	
Implementation of the Logger and RAII helper classes	316
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/ Logger.hpp	
Thread-safe asynchronous logging system	316

Chapter 8

Directory Documentation

8.1 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games Directory Reference

Files

- file [minigame.hpp](#)
Defines the abstract [MiniGame](#) interface implemented by every playable mini-game.
- file [minigame_participant.hpp](#)
Defines the plain data-transfer object mini-games use to represent players.
- file [minigame_result.hpp](#)
Defines the outcome type returned by [MiniGame::play\(\)](#).
- file [splatoon.cpp](#)
Implementation of the Splatoon mini-game: camera-driven grid painting, UDP sync with each player's phone, and end-of-round scoring.
- file [splatoon.hpp](#)
Splatoon mini-game: robots paint cells of a grid by driving over them, tracked via camera position and mirrored to each player's phone over UDP.
- file [splatoon_grid.cpp](#)
Implementation of [SplatoonGrid](#), the flat paint-ownership grid used by the Splatoon mini-game.
- file [splatoon_grid.hpp](#)
Fixed-size grid of per-cell paint ownership used by the Splatoon mini-game.
- file [udp_reciever.hpp](#)
Cross-platform (Windows/POSIX) UDP receiver that dispatches each datagram to a callback on a background thread.
- file [udp_sender.hpp](#)
Cross-platform (Windows/POSIX) UDP datagram sender bound to a single destination.

8.1.1 Detailed Description

8.1.2 Splatoon mini-game

A territory-painting race. Each player drives one EV3 robot around a physical 9×15 board for 60 seconds; the cells a robot drives over are painted in that player's colour, and the player owning the most cells at the end wins.

This folder holds the **desktop (Qt/C++) half** of the game. The desktop is a referee only — it keeps the clock, holds the authoritative board, tracks where each robot is, and broadcasts the board to every phone. It does **not** drive any robot.

8.1.3 Responsibility split

Component	Role
Desktop (this folder)	Clock, authoritative grid, scoring, continuous robot tracking, board broadcast
Phone (Unity ar-app)	Reads the joystick, drives its EV3 brick directly, paints cells from camera data, reports claimed cells
EV3 brick	Executes drive commands received straight from the phone
Ceiling camera (desktop tracking)	Tracks each robot continuously throughout the game; desktop converts position to grid cell and forwards to the phone

8.1.4 Files

File	Purpose
splatoon.hpp / .cpp	The game: 60 s loop, claim handling, scoring, all UDP to/from phones
splatoon_grid.hpp / .cpp	Pure 9×15 board data (0 = neutral, 1 . . 4 = player). No Qt, no rendering
minigame.hpp	MiniGame interface (play, onRobotMoved, getName)
minigame_participant.hpp	One player in a game: playerId, coins
minigame_result.hpp	participants + winnerId, returned by play()
udp_sender.hpp	Thin UDP send wrapper (one per phone, per destination port)
udp_reciever.hpp	UDP listen wrapper; hands each datagram + source IP to a callback
network_utils.hpp / .cpp	<code>RobotGame::broadcastUdpMessage(port, msg)</code> — one-shot UDP broadcast helper used by TurnManager to send MINIGAME_START/END from the background thread

8.1.5 The board

- 9 rows × 15 cols = 135 cells, stored row-major (`index = row * COLS + col`).
- Cell value: 0 neutral, 1 . . 4 owner. Row 0 is the **top** of the board.
- Maps to a physical field of **3.00 m (15 cols) × 1.85 m (9 rows)**; `CELL_SIZE_CM = 20`, with a `Y_OFFSET_CM = 2.5` margin on the short axis.

8.1.6 Network protocol

Direction	Port	Message
Phone → Desktop	12346	<code>register, playerId</code> and <code>CLAIM, playerId, row, col</code>
Desktop → Phone	12347	<code>ROBOT, playerId, row, col</code> (current robot cell, sent on every cell change)
Desktop → Phone	12345	136 numbers: <code>secondsLeft</code> then 135 cell values (board + timer); also <code>WINNER, playerId</code> once at game end
Phone → EV3 brick	7778	4 bytes [<code>nn</code> , <code>0x01</code> , <code>speed</code> , <code>turn</code>] (not handled here)
EV3 → Desktop	7779	5-byte heartbeat [<code>0x11</code> , <code>nn</code> , <code>state</code> , <code>battery_lo</code> , <code>battery_hi</code>] broadcast to <code>255.255.255.255</code> every 1 s

Direction	Port	Message
Desktop → Phone	9999	Unity (desktop IP discovery, broadcast every 3 s) · Mindstorm↔IP, <nn>, <ev3_ip> (EV3 IP per player, sent on every heartbeat) · MINIGAME_START, <count> · MINIGAME_END (scene switching)
Desktop → EV3	7780	SERVEREV3 (only needed with old EV3 binary so the brick learns the desktop IP for MQTT)

`playerId` is 1..4; the participant index used internally is `playerId - 1`. Phone IP addresses are never hardcoded — they are learned at runtime from the source IP of each `register/CLAIM` datagram, so the board and starting cell can be sent back.

8.1.6.1 How phones discover the desktop IP

The phone no longer needs the desktop IP hardcoded. When "Connect Unity" is clicked on the desktop, it broadcasts "Unity" on port 9999 to the whole LAN. The phone's `UdpListener` receives it and reads the sender address — that becomes the desktop IP stored in `NetworkRegistry.ServerIp`. `CellClaimSender` subscribes to `NetworkRegistry.OnServerIpUpdated` (requires `useDynamicIp = true` in the Inspector) and automatically switches its endpoint.

8.1.6.2 How phones discover their EV3 IP

EV3 bricks broadcast a heartbeat to `255.255.255.255:7779` every second. The desktop's `robot_fleet` receives each heartbeat, reads `nn` (robot number) from byte 1 and the sender IP from the UDP packet source, then broadcasts "MindstormIP, <nn>, <ev3_ip>" on port 9999.

`UdpListener` on each phone receives this broadcast every second. It parses `nn` and compares it to `Ev3UdpClient.Instance.PlayerId`. Only the phone whose player ID matches `nn` calls `Ev3UdpClient.SetIp(ip)`, which creates the UDP socket and starts sending drive commands to that robot. No IP is ever hardcoded on the phone — `Ev3UdpClient` starts with no IP and waits for the first matching `MindstormIP` broadcast before it can send anything.

Self-healing: because the broadcast repeats every second, DHCP IP changes are corrected automatically within 1 second.

Note (old EV3 binary): If the prebuilt binary predates the broadcast heartbeat change, the robot sends heartbeats to a specific desktop IP read from `MIND_BROKER_HOST` in `start_mind.sh` instead of `255.255.255.255`. In that case update `start_mind.sh` on the robot with the current desktop IP, or ask the teammate who owns the EV3 code to rebuild and push the binary.

8.1.6.3 How phones switch scenes automatically

The desktop also uses port 9999 to tell phones when to switch scenes. All messages are sent as LAN broadcasts (`255.255.255.255`) so every phone receives them simultaneously — no per-phone IP needed.

Message	When sent	Phone action
MINIGAME_START, <count>	Start of <code>TurnManager::startMinigame()</code> thread	UdpListener → MinigameBridge.StartMinigame() → loads SplattoonScene
MINIGAME_END	After the 60 s game loop finishes	UdpListener → SceneManager.LoadScene(returnSceneName)

UdpListener is marked DontDestroyOnLoad so it survives the scene transition and is still listening while SplattoonScene is active. MinigameBridge is a static C# class (no GameObject) that carries returnSceneName and activePlayerCount across scene loads.

For standalone testing without a running desktop, MinigameTrigger.cs listens on port 12348 for "MINIGAME, <count>" and triggers the same MinigameBridge.StartMinigame() path:

```
python3 -c "import socket; s=socket.socket(socket.AF_INET,socket.SOCK_DGRAM);
s.sendto(b'MINIGAME,2', ('<PHONE_IP>', 12348))"
```

See ar-app/Assets/Scripts/Splattoon/Scene_switching.md for full Unity-side documentation of these scripts.

8.1.6.4 How "Connect Unity" works

The "Connect Unity" button on the desktop starts a repeating timer that broadcasts "Unity" on port 9999 every 3 seconds. The button toggles the timer on/off and its label changes to "Stop Unity" while active. Any phone that joins the network late will receive the broadcast within 3 seconds and auto-discover the desktop IP — no button re-click needed.

8.1.7 Game flow

```
register (phone -> desktop)      : desktop learns the phone's address
camera sees robot                : desktop -> phone  ROBOT,id,row,col (continuously on cell change)
phone receives ROBOT packet      : joystick unlocks; phone paints the cell; CLAIM -> desktop
CLAIM (phone -> desktop)         : desktop stores the cell, rebroadcasts the board
board + timer (desktop)          : desktop -> all phones, 136 numbers on :12345
60 s elapsed                     : desktop scores, joystick stops on the phone
```

- Discovery.** Each phone announces itself with `register, playerId`. The desktop records the source IP and builds two senders for that player: the board sender (:12345) and the robot-position sender (:12347).
- Start.** `Splattoon::play()` initialises the grid, opens the claim receiver on :12346, and runs the 60 s loop (re-broadcasting the board once per second so the phone countdown keeps ticking).
- Tracking (continuous).** Camera tracking feeds robot positions into `updateRobotPosition()` ~30x/s throughout the game. Every time a robot crosses into a new grid cell, the desktop calls `claimCell()` to record the new owner in its authoritative grid and immediately broadcast the updated board, then calls `sendStartCell()` to send `ROBOT, playerId, row, col` to the phone on port 12347. The phone's joystick is locked until the first ROBOT packet arrives; after that it stays unlocked for the rest of the round.
Late-registration fix: if the camera already tracked the robot before the phone sent its first `register` packet, the desktop replays the last known cell immediately on registration so the phone never misses the spawn signal.
- Painting.** The desktop is the sole painting authority — it writes to the grid in `claimCell()` as the camera tracks each robot. The phone also paints cells locally when it receives ROBOT packets, and sends CLAIM back to the desktop, but the desktop **ignores** CLAIM content — `handleCellClaim()` only reads the source IP to keep the phone's sender address up to date.
- End.** After 60 s the loop stops, `calculateRanking()` counts each player's cells, and `applyRewards()` grants coins (100 for 1st, 50 for 2nd). The winning `playerId` is sent once as `WINNER, playerId` (before the final board push, so the phone has it before its timer reaches zero). The result carries the ranked participants and `winnerId`.

8.1.8 Board game integration

Splatoon can be launched in two ways:

Mode	How it starts	Coins go to
Standalone test	Click "Start Splatoon" button in <code>main_window</code>	Nobody (placeholder players only)
Real board game	<code>TurnManager</code> reaches end-of-round Minigame state	Real board game players

8.1.8.1 How the real game flow works

At the end of every board game round, `TurnManager::startMinigame()` fires. If a `GameManager*` has been wired in via `setGameManager()`, it:

1. Calls `gameManager->syncPlayers(game._players)` — copies the real board game players (correct IDs, names, coin balances) into `GameManager`, replacing the hardcoded placeholders used by the standalone button.
2. Launches `gameManager->runMiniGame(splatoon)` on a background thread so the Qt UI and camera timer keep running.
3. After the 60 s round, diffs the coin changes from `GameManager::players_` back into `game._players` so Splatoon rewards (100 / 50 coins) carry into the board game.
4. Sets `state = EndRound` — the board game continues.

Camera tracking works identically in both modes: `update_tracking()` always calls `gameManager->notifyRobotPosition()`, which forwards to whichever Splatoon instance is currently active inside `GameManager::activeGame_`.

8.1.8.2 How the lobby starts the game (done)

`lobby.cpp` handles the `start_game` message sent by the host phone. It calls:

```
start_game::startGame(configured_player_count, configured_round_count);
```

`GameManager*` is registered at desktop startup via `start_game::setGameManager(gameManager->.get())` in `main_window.cpp`, so `startGame()` picks it up automatically even though `lobby.cpp` doesn't have direct access to `gameManager_`. No manual wiring needed.

`start_game::update()` is called every tick inside `update_tracking()` in `main_window.cpp`, so the `TurnManager` state machine advances automatically as long as tracking is running.

8.1.9 Tracking and the game are separate subsystems

Camera tracking and the game are controlled by two independent buttons that meet at a single point: `GameManager::notifyRobotPosition`.

- **"Start Tracking"** turns on the camera. `update_tracking()` then runs $\sim 30\times/s$ and calls `notifyRobotPosition(marker_id, x_cm, y_cm)` for every detected robot marker — continuously, whether or not a game is running.
- **"Start Splatoon"** runs the game. `GameManager::runMiniGame()` sets `activeGame_` for the duration of `play()` and clears it at the end.

`notifyRobotPosition` only forwards positions while a game is active:

```
void GameManager::notifyRobotPosition(int index, double x_cm, double y_cm) {
    if (activeGame_) // the gate
        activeGame_>onRobotMoved(index, x_cm, y_cm); // -> Splatoon::updateRobotPosition
}
```

Moment	Where the camera feed goes
Tracking on, before Start Splatoon	<code>activeGame_</code> is null — positions are dropped (still sent to MQTT and the map UI)
During the 60 s round	positions reach <code>updateRobotPosition()</code> — every cell change sends <code>ROBOT, id, row, col</code> to the phone
After the round ends	<code>activeGame_</code> is null again — positions dropped

Tracking is a general camera subsystem (it also feeds the map and MQTT robot positions); the game does not start the camera, it only hooks into the stream the camera already produces while it is the active game. `positions_` is reset to `-1` at the start of every `play()`, so the first cell seen per robot also serves as the spawn signal for the phone's joystick lock. The robot must be on the board and visible — if tracking is off or uncalibrated when Start Splatoon is pressed, no positions flow and the phone's joystick never unlocks.

8.1.10 Phone AR board (transparent mode)

There is no image robot in the AR scene. The board (`FieldGrid.cs`) is rendered as a semi-transparent overlay (`boardAlpha = 0.15`) so the player sees the real physical robot through the AR grid. Painted cells appear as ink-splat sprites at 70% opacity (`paintedCellAlpha = 0.7`) so the robot remains visible even when driving over painted territory.

`RobotController.cs` waits for the first `ROBOT` packet on port 12347 before unlocking the joystick ("Waiting for robot tracking..." $\rightarrow$ "Robot found! Go!"). After that, every `ROBOT` packet paints the cell in the local grid and triggers a `CLAIM` back to the desktop (desktop ignores the `CLAIM` content — source IP only). The authoritative board state arrives separately on port 12345 from the desktop's own `claimCell()` calls and overwrites the local grid each broadcast.

`InklingFollower.cs` reads the same `RobotPositionReceiver` data (via `GetRobotRow / GetRobotCol`, without consuming `WasUpdated`) and smoothly moves a 3-D character to the robot's current cell with bounce animation. The character is parented to `FieldGrid` so it stays anchored to the AR board. It spawns on the first valid `ROBOT` packet and follows continuously; `robotId` in the Inspector must match the phone's `playerId`.

`ScorePanel_Splatoon.cs` reads `FieldGrid.CountCells(playerId)` every 0.2 s and displays a coloured chip per player with the live cell count. The leading player's chip is scaled up and marked with `*`. The panel builds its UI at runtime and requires a font asset (LiberationSans SDF) assigned in the Inspector.

8.1.11 Coordinate convention

The phone treats row 0 as the top of the board. The desktop computes the cell from camera centimetres ($col = x_{cm} / CELL_SIZE$, $row = (y_{cm} - Y_OFFSET) / CELL_SIZE$) and then flips the row, $row = (ROWS-1) - row$, so both sides agree. CLAIM messages are stored verbatim (already in the phone's convention); the flip is applied only to the desktop-computed starting cell. Column direction and which physical edge is row 0 depend on camera calibration and the AR board anchor, not on this code.

8.1.12 ArUco markers (camera tracking)

- **Robots:** marker IDs defined in `tracker.cpp` — see table below.
- **Field calibration:** marker IDs 5–10 at fixed floor positions (board corners and mid-edges); at least 4 must be visible to calibrate.
- Custom dictionary: `cv::aruco::extendDictionary(12, 3)` (12 markers, 3×3 bits) — standard ArUco dictionaries will not be detected.

8.1.12.1 Player <-> ArUco marker mapping

The robot slot is determined by position in `robot_marker_ids` array in `desktop-app/src/robot_tracker/tracker.c`

```
constexpr std::array<int, 4> robot_marker_ids = {0, 1, 2, 3};
```

Player	ArUco marker	Robot ID	Slot	Phone playerId
1	0	ev3_01	0	1
2	1	ev3_02	1	2
3	2	ev3_03	2	3
4	3	ev3_04	3	4

The camera feeds the array position (slot) as `participantIndex` into `notifyRobotPosition`, which sends `ROBOT, playerId, row, col` where `playerId = participantIndex + 1`.

Note: marker IDs 4 and 11 are unassigned. Marker IDs 5–10 are field calibration markers. `robot_slot_for(_marker())` returns -1 for all of them, so the robot tracking pipeline ignores them.

8.1.13 Key constants (`splatoon.hpp`)

Constant	Value	Meaning
<code>DURATION_SECONDS</code>	60	Round length
<code>COINS_FIRST / COINS_SECOND</code>	100 / 50	Rewards for 1st / 2nd
<code>CELL_SIZE_CM</code>	20	Physical cell size
<code>Y_OFFSET_CM</code>	2.↵ 5	Margin on the short axis
<code>SplatoonGrid::ROWS / COLS</code>	9 / 15	Board dimensions

8.1.14 Threading

The camera tracking thread, the UDP claim-receiver thread, and the game-loop thread all touch shared state. `gameMutex_` guards the grid, the position trackers, and the per-phone sender maps.

8.1.15 Running

8.1.15.1 Standalone mode (testing only)

1. Build and launch the desktop app (`desktop-app/build.sh`).
2. Click **Start Tracking** — the camera starts and markers are detected.
3. Make sure at least 4 of the board calibration markers (IDs 5–10) are visible, then click **Calibrate Camera**. This computes the camera pose (`has_extrinsics_`). Without this step robot pixel positions cannot be converted to real-world centimetres, so `notifyRobotPosition` is never called, no `ROBOT` packets are sent, and the phone's joystick never unlocks — even if the timer is already ticking on the phone.
4. Click **Start Splatoon**. Now the full chain is active: camera detects robot → desktop converts to grid cell → `ROBOT, id, row, col` sent to phone on port 12347 → phone shows "Robot found! Go!" and the joystick unlocks.

8.1.15.2 Real board game mode (lobby-based, current flow)

Desktop setup (do this before starting the game):

1. Click **Start Tracking** — camera opens, ArUco detection begins.
2. Click **Calibrate Camera** — at least 4 calibration markers (IDs 5–10) must be visible. Without this, robot positions are never converted to grid cells and `ROBOT` packets are never sent to phones.
3. Click **Connect Unity** — starts repeating "Unity" broadcast every 3 s so phones auto-discover the desktop IP. Leave it running.
4. Click **Connect Robot** — broadcasts "SERVEREV3" so EV3 bricks learn the desktop IP.

Phone setup:

- `Player` IDs are assigned automatically by the lobby — no Inspector hardcoding needed. The host becomes `Player 1`; joiners are assigned 2, 3, 4 in join order.
- No EV3 IPs need to be set anywhere. `Ev3UdpClient` starts with no IP and discovers the robot automatically once the desktop receives the first heartbeat from the matching EV3 brick (within 1 second of the robot program starting).

Starting the game:

1. One phone opens the app → **Create Game** (becomes `Player 1 / host`).
2. Other phones → **Join Game** (assigned `Player 2, 3, 4` in order).
3. Host sets player count and round count, then taps **Start Game**.

4. All phones load the board scene; desktop calls `start_game::startGame(playerCount, roundCount)` with `GameManager` wired.
5. `update_tracking()` ticks `TurnManager` every frame — board game advances.
6. At the end of each round Splatoon launches automatically. Phones switch to `SplatoonScene` on `MINIGAME_START` and back to `GameScene` on `MINIGAME_END`.

Physical robot <-> player mapping: The robot with marker **0** is `ev3_01` / `Player 1`, marker **1** is `ev3_02` / `Player 2`, marker **2** is `ev3_03` / `Player 3`, and marker **3** is `ev3_04` / `Player 4`. Each player must use the phone whose lobby slot matches the robot they are physically driving (first to join = `Player 1`, etc.).

Why the timer arrives but the joystick doesn't unlock: `publishGrid()` (timer + board on port 12345) runs as soon as the phone registers and Splatoon starts — it needs no calibration. `sendStartCell()` (ROBOT on port 12347) requires calibration because it first converts camera pixels to world centimetres. If calibration was skipped, the timer ticks on the phone but `has_extrinsics_` is false so the robot cell is never computed and no ROBOT packet is ever sent.

Why injecting a fake ROBOT packet from the terminal unlocks the joystick but cells still don't get painted: A manual `printf 'ROBOT,2,4,7' | nc -u -w1 <PHONE_IP> 12347` goes directly to the phone, bypassing the desktop. The phone receives it, `RobotPositionReceiver` stores the position, and `RobotController` shows "Robot found!" and unlocks the joystick. But all subsequent ROBOT packets must come from the desktop, and those are gated behind `has_extrinsics_`. Without calibration the desktop never calls `notifyRobotPosition`, so no further ROBOT messages are sent and no new cells are painted as the robot moves. The fake packet only proves the phone-side code is correct; the root cause is always the missing calibration step.

8.1.15.3 Testing the desktop without phones

```
printf 'register,1' | nc -u -w1 <DESKTOP_IP> 12346 # desktop learns the sender IP
```

After the `register`, the desktop sends the board back to the `nc` source IP. Note: `CLAIM` packets are now ignored by the desktop (camera is the sole painting authority), so sending a `CLAIM` from the terminal will not paint any cell. Grid painting only happens when the camera tracks a robot to a new cell.

Chapter 9

Namespace Documentation

9.1 ns Namespace Reference

Network message payload types shared between the desktop app and the AR/Unity client. Each struct uses `NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE` to generate `to_json/from_json`, so a handler registered via `register_message_handler()` can deserialize the incoming JSON with `message.get<ns::SomeEvent>()`.

Classes

- struct [OpenShopEvent](#)
Payload of the "open_shop" message: identifies which player opened the shop. Parsed in [shop.cpp](#)'s "open_shop" handler and used to build a temporary [Player](#) before calling [Shop::openShop\(\)](#).
- struct [SelectedItemEvent](#)
Payload describing a player's item selection from the shop UI.
- struct [CloseShopEvent](#)
Payload of the "close_shop" message: identifies which player closed the shop. Parsed in [shop.cpp](#)'s "close_shop" handler and used to build a temporary [Player](#) before calling [Shop::close_shop\(\)](#).
- struct [StartRouletteEvent](#)
Payload of the "roulette_start" message: identifies which player is starting a roulette round. Parsed in [roulette.cpp](#)'s "roulette_start" handler and used to build a temporary [Player](#) before calling [Roulette::start_roulette\(\)](#).
- struct [GetPlayerBetEvent](#)
Payload of the "make_bet" message: the player's roulette wager. Parsed in [roulette.cpp](#)'s "make_bet" handler and forwarded to [Roulette::setBet\(\)](#), which wakes any thread blocked in [Roulette::choose_bet_type\(\)](#).
- struct [EndRouletteEvent](#)
Payload of the "end_roulette" message: identifies which player's roulette round is ending. Parsed in [roulette.cpp](#)'s "end_roulette" handler and used to build a temporary [Player](#) before calling [Roulette::end_roulette\(\)](#).

9.1.1 Detailed Description

Network message payload types shared between the desktop app and the AR/Unity client. Each struct uses `NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE` to generate `to_json/from_json`, so a handler registered via `register_message_handler()` can deserialize the incoming JSON with `message.get<ns::SomeEvent>()`.

9.2 TopoCategory Namespace Reference

Traffic category identifiers used throughout the topology.

Variables

- constexpr char **Game** [] = "game"
- constexpr char **Heartbeat** [] = "heartbeat"
- constexpr char **Response** [] = "response"
- constexpr char **Robot** [] = "robot"
- constexpr char **Pairing** [] = "pairing"

9.2.1 Detailed Description

Traffic category identifiers used throughout the topology.

Chapter 10

Class Documentation

10.1 mind::proto::ack_msg Struct Reference

Decoded `ack` status payload (EV3 -> desktop, MQTT). Echoes the command that triggered it (or `no_cmd_id`) plus the robot's current state, battery level, and error detail.

```
#include <codec.hpp>
```

Public Attributes

- `std::uint16_t cmd_id`
Echo of the triggering command's id, or `no_cmd_id` if none.
- `robot_state state`
Current robot state.
- `std::uint16_t battery_mv`
Battery voltage, in millivolts.
- `err_code err`
Meaningful only when state == `robot_state::error`.

10.1.1 Detailed Description

Decoded `ack` status payload (EV3 -> desktop, MQTT). Echoes the command that triggered it (or `no_cmd_id`) plus the robot's current state, battery level, and error detail.

The documentation for this struct was generated from the following file:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.hpp`

10.2 MqttReceiver.AnchorData Class Reference

Public Attributes

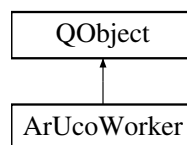
- string **type**
- double **tx**
- double **ty**
- double **tz**
- double **rx**
- double **ry**
- double **rz**

The documentation for this class was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/MqttReceiver.cs

10.3 ArUcoWorker Class Reference

Inheritance diagram for ArUcoWorker:



Public Slots

- void **start_tracking** ()
- void **stop_tracking** ()
- void **calibrate_camera** ()
- void **process_frame** ()

Signals

- void **frame_ready** (const QImage &frame)
- void **robots_updated** (const std::vector< [TrackedRobot](#) > &robots)
- void **status_message** (const QString &message)
- void **anchor_calibrated** (double tx, double ty, double tz, double rx, double ry, double rz)

Public Member Functions

- **ArUcoWorker** (QObject *parent=nullptr)

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/background_thread.hpp
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/background_thread.cpp

10.4 Board Class Reference

Owns the static node graph for the game board and can broadcast it to clients over TCP. The graph is hard-coded in the constructor; nothing in this class mutates it afterward.

```
#include <board.hpp>
```

Public Member Functions

- [Board](#) ()
Constructs the board with its fixed 21-node graph. [Node](#) positions are given in screen pixels for a 1400x860 window; the physical layout is a 4-column x 5-row grid (columns at 25/100/175/250 cm, rows at 15/55/95/135/175 cm) with all connected nodes at least 40 cm apart. Each entry is {id, x, y, neighbors, effects}.
- const [Node](#) & [getNode](#) (int id) const
Returns a single node by id.
- const std::vector< [Node](#) > & [getNodes](#) () const
Returns all nodes in the board graph.
- void [broadcast_board](#) ([Server](#) &server)
Serializes the full node graph to JSON and broadcasts it to all connected clients. Sent as a "board_load" message; each node's effect types are converted to their string names via [tileTypeToString\(\)](#) before serialization.
- std::string [tileTypeToString](#) ([TileType](#) type)
Converts a [TileType](#) to its human-readable / wire string name.
- bool [isValidNode](#) (int id) const
Returns true if a node with this id exists in the graph.
- void [addNode](#) (int id, float x, float y, [NodeType](#) type)
Creates a new node and registers it on the board under the given id.
- void [connectNodes](#) (int id1, int id2)
Creates a bidirectional edge between two existing nodes.
- void [printGraph](#) ()
Prints the full board graph to stdout: every node's id, type, position, and connected neighbor ids.

10.4.1 Detailed Description

Owns the static node graph for the game board and can broadcast it to clients over TCP. The graph is hard-coded in the constructor; nothing in this class mutates it afterward.

Owns the full set of board nodes and their connections, forming the map graph. Nodes are addressed by integer id and heap-allocated on insertion.

10.4.2 Constructor & Destructor Documentation

10.4.2.1 Board()

```
Board::Board ()
```

Constructs the board with its fixed 21-node graph. [Node](#) positions are given in screen pixels for a 1400x860 window; the physical layout is a 4-column x 5-row grid (columns at 25/100/175/250 cm, rows at 15/55/95/135/175 cm) with all connected nodes at least 40 cm apart. Each entry is {id, x, y, neighbors, effects}.

Constructs the board with its fixed 21-node graph.

10.4.3 Member Function Documentation

10.4.3.1 addNode()

```
void Board::addNode (
    int id,
    float x,
    float y,
    NodeType type) [inline]
```

Creates a new node and registers it on the board under the given id.

Parameters

<i>id</i>	Unique identifier for the node; overwrites any existing node already stored under this id (without freeing it).
<i>x</i>	X position of the node.
<i>y</i>	Y position of the node.
<i>type</i>	Gameplay role of the node.

10.4.3.2 broadcast_board()

```
void Board::broadcast_board (
    Server & server)
```

Serializes the full node graph to JSON and broadcasts it to all connected clients. Sent as a "board_load" message; each node's effect types are converted to their string names via [tileTypeToString\(\)](#) before serialization.

Serializes the full node graph to JSON and broadcasts it to all connected clients.

Parameters

<i>server</i>	The TCP server used to broadcast the message.
---------------	---

10.4.3.3 connectNodes()

```
void Board::connectNodes (
    int id1,
    int id2) [inline]
```

Creates a bidirectional edge between two existing nodes.

Parameters

<i>id1</i>	Id of the first node.
<i>id2</i>	Id of the second node.

Note

If either id is not present in the map, operator[] default-constructs a null Node* entry for it (leaving a stray null entry in the map) and the connection is silently skipped.

10.4.3.4 getNode()

```
const Node & Board::getNode (
    int id) const
```

Returns a single node by id.

Returns a single node by id; throws `std::runtime_error` if not found.

Parameters

<i>id</i>	The node's id.
-----------	----------------

Returns

The matching node.

Note

Throws `std::runtime_error` if no node with this id exists.

10.4.3.5 getNodes()

```
const std::vector< Node > & Board::getNodes () const
```

Returns all nodes in the board graph.

Returns

All nodes in the board graph.

10.4.3.6 isValidNode()

```
bool Board::isValidNode (
    int id) const
```

Returns true if a node with this id exists in the graph.

Parameters

<i>id</i>	The node id to check.
-----------	-----------------------

Returns

True if a node with this id exists, false otherwise.

10.4.3.7 tileTypeToString()

```
std::string Board::tileTypeToString (
    TileType type)
```

Converts a [TileType](#) to its human-readable / wire string name.

Parameters

<i>type</i>	The tile effect type.
-------------	-----------------------

Returns

"Plus", "Minus", "Shop", "Hazard", or "Unknown" if the type is unrecognized.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/mapLogic.cpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.cpp](#)

10.5 BoardLoader.BoardData Class Reference

Public Attributes

- List< [NodeData](#) > **nodes**

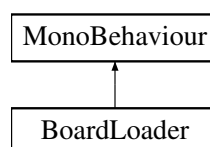
The documentation for this class was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/BoardLoader.cs](#)

10.6 BoardLoader Class Reference

Loads the board.json file exported by the Desktop App.

Inheritance diagram for BoardLoader:



Classes

- class [NodeData](#)
- class [EffectData](#)
- class [BoardData](#)

Public Member Functions

- void **LoadBoardFromFile** (string filePath)
- string **GetAndroidPath** ()

Properties

- [BoardData](#) **LoadedBoard** [get]

10.6.1 Detailed Description

Loads the board.json file exported by the Desktop App.

The documentation for this class was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/BoardLoader.cs

10.7 BonusAward Struct Reference

Public Attributes

- BonusCategory **category**
- int **value**
- std::vector< int > **winnerIds**

The documentation for this struct was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/[bonus.hpp](#)

10.8 BonusPhase Class Reference

Stateless end-of-game scoring phase. Scans all players in a [Game](#), awards bonus stars, and returns the complete result needed by presentation and networking layers.

```
#include <bonus.hpp>
```

Static Public Member Functions

- static [BonusResult](#) **execute** ([Game](#) &game)
Awards bonus stars for movement, item usage, and coin earnings. Finds every player tied for the highest positive value in movement, item usage, and earned coins, and awards each winner one star.

10.8.1 Detailed Description

Stateless end-of-game scoring phase. Scans all players in a [Game](#), awards bonus stars, and returns the complete result needed by presentation and networking layers.

10.8.2 Member Function Documentation

10.8.2.1 execute()

```
BonusResult BonusPhase::execute (  
    Game & game) [static]
```

Awards bonus stars for movement, item usage, and coin earnings. Finds every player tied for the highest positive value in movement, item usage, and earned coins, and awards each winner one star.

Awards bonus stars for movement, item usage, and coin earnings.

Parameters

<i>game</i>	The game whose players are scored.
-------------	------------------------------------

Returns

Bonus awards, final standings, and every overall winner.

Note

A player can win multiple categories. Categories whose best value is zero award no star. Final standings rank by stars, then current coins.

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/[bonus.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/[bonus.cpp](#)

10.9 BonusResult Struct Reference

Public Member Functions

- bool **isTie** () const

Public Attributes

- std::vector< [BonusAward](#) > **awards**
- std::vector< [PlayerStanding](#) > **standings**
- std::vector< int > **winnerIds**

The documentation for this struct was generated from the following file:

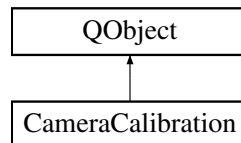
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/[bonus.hpp](#)

10.10 CameraCalibration Class Reference

Captures frames from a webcam and computes the camera's extrinsic pose relative to a known 3D field layout. Owns a QTimer-driven capture loop that emits each frame as a QImage for display, and wraps OpenCV's solvePnP to derive rotation/translation vectors once intrinsic parameters and a set of matching 3D/2D point correspondences (e.g. detected field markers) are supplied.

```
#include <camera_calibration.hpp>
```

Inheritance diagram for CameraCalibration:



Signals

- void [frame_ready](#) (const QImage &image)
Emitted after each captured frame has been converted to QImage and is ready for display.

Public Member Functions

- [CameraCalibration](#) (QObject *parent=nullptr)
Constructs the calibration helper and wires the internal timer to the frame-update slot.
- [~CameraCalibration](#) ()
Stops the camera stream and releases the capture device.
- bool [start_camera](#) (int device_id=0)
Opens the given camera device and starts the ~33 FPS frame capture timer.
- void [stop_camera](#) ()
Stops the capture timer (if running) and releases the camera device (if open).
- void [set_intrinsics](#) (const cv::Mat &camera_matrix, const cv::Mat &dist_coeffs)
Stores a copy of the camera's intrinsic parameters for later use by [calibrate_extrinsics\(\)](#).
- bool [calibrate_extrinsics](#) (const std::vector< cv::Point3f > &object_points, const std::vector< cv::Point2f > &image_points)
Computes the camera's extrinsic pose (rotation and translation) from 3D-to-2D point correspondences via solvePnP.
- bool [is_calibrated](#) () const
Reports whether the most recent [calibrate_extrinsics\(\)](#) call succeeded.
- const cv::Mat & [get_rvec](#) () const
Returns the Rodrigues rotation vector from the last successful extrinsic calibration.
- const cv::Mat & [get_tvec](#) () const
Returns the translation vector from the last successful extrinsic calibration.

10.10.1 Detailed Description

Captures frames from a webcam and computes the camera's extrinsic pose relative to a known 3D field layout. Owns a QTimer-driven capture loop that emits each frame as a QImage for display, and wraps OpenCV's solvePnP to derive rotation/translation vectors once intrinsic parameters and a set of matching 3D/2D point correspondences (e.g. detected field markers) are supplied.

10.10.2 Constructor & Destructor Documentation

10.10.2.1 CameraCalibration()

```
CameraCalibration::CameraCalibration (
    QObject * parent = nullptr) [explicit]
```

Constructs the calibration helper and wires the internal timer to the frame-update slot.

Parameters

<i>parent</i>	Optional Qt parent object for ownership/lifetime management.
---------------	--

10.10.3 Member Function Documentation

10.10.3.1 calibrate_extrinsics()

```
bool CameraCalibration::calibrate_extrinsics (
    const std::vector< cv::Point3f > & object_points,
    const std::vector< cv::Point2f > & image_points)
```

Computes the camera's extrinsic pose (rotation and translation) from 3D-to-2D point correspondences via solvePnP.

Computes the camera's extrinsic pose via solvePnP from 3D-to-2D point correspondences.

Requires [set_intrinsics\(\)](#) to have been called first with a non-empty camera matrix and distortion coefficients, and at least 4 matching point pairs (uses cv::SOLVEPNP_IPPE).

Parameters

<i>object_points</i>	Reference 3D points in the world/field coordinate frame.
<i>image_points</i>	Corresponding 2D pixel coordinates observed in the camera image; must be the same size as <i>object_points</i> .

Returns

true if solvePnP succeeded and *rvec_*/*tvec_* were updated; false if the inputs are invalid or the solve failed.

10.10.3.2 frame_ready

```
void CameraCalibration::frame_ready (
    const QImage & image) [signal]
```

Emitted after each captured frame has been converted to QImage and is ready for display.

Parameters

<i>image</i>	The latest camera frame, converted via <code>cv_mat_to_qimage()</code> .
--------------	--

10.10.3.3 get_rvec()

```
const cv::Mat & CameraCalibration::get_rvec () const [inline]
```

Returns the Rodrigues rotation vector from the last successful extrinsic calibration.

Returns

Reference to the rotation vector produced by solvePnP.

10.10.3.4 get_tvec()

```
const cv::Mat & CameraCalibration::get_tvec () const [inline]
```

Returns the translation vector from the last successful extrinsic calibration.

Returns

Reference to the translation vector produced by solvePnP.

10.10.3.5 is_calibrated()

```
bool CameraCalibration::is_calibrated () const
```

Reports whether the most recent [calibrate_extrinsics\(\)](#) call succeeded.

Returns

true if the last [calibrate_extrinsics\(\)](#) call succeeded; false if it has not been called yet or the last attempt failed.

10.10.3.6 set_intrinsics()

```
void CameraCalibration::set_intrinsics (  
    const cv::Mat & camera_matrix,  
    const cv::Mat & dist_coeffs)
```

Stores a copy of the camera's intrinsic parameters for later use by [calibrate_extrinsics\(\)](#).

Parameters

<i>camera_matrix</i>	3x3 OpenCV camera intrinsic matrix.
----------------------	-------------------------------------

<code>dist_coeffs</code>	OpenCV lens distortion coefficients.
--------------------------	--------------------------------------

10.10.3.7 start_camera()

```
bool CameraCalibration::start_camera (
    int device_id = 0)
```

Opens the given camera device and starts the ~33 FPS frame capture timer.

Parameters

<code>device_id</code>	OpenCV device index to open (default ID 0 is usually the primary webcam).
------------------------	---

Returns

true if the device was opened successfully, false otherwise.

The documentation for this class was generated from the following files:

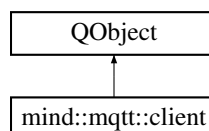
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/camera_calibration.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/camera_calibration.cpp](#)

10.11 mind::mqtt::client Class Reference

Asynchronous MQTT client, wrapping libmosquitto behind Qt signals.

```
#include <client.hpp>
```

Inheritance diagram for mind::mqtt::client:



Signals

- void **connected** ()
Emitted on the Qt thread after a successful (re)connect, once queued subscriptions have been replayed.
- void **disconnected** ()
Emitted on the Qt thread after the broker connection is lost or explicitly closed.
- void **message_received** (QString topic, QByteArray payload)
Emitted on the Qt thread for each message received on a subscribed topic.

Public Member Functions

- [client](#) (QObject *parent=nullptr)
Initializes the mosquitto library and creates the connection handle. Increments the process-wide mosquitto library refcount (calling `mosquitto_lib_init` only for the first live instance), allocates the `mosquitto` handle with a host+PID-derived client id, and registers the static callback trampolines. Opens no network connection; call [connect_to\(\)](#) for that.
- [~client](#) () override
Stops the network thread (if running), destroys the mosquitto handle, and decrements the library refcount, calling `mosquitto_lib_cleanup` once the last instance is destroyed.
- **client** (const client &)=delete
- **client & operator=** (const [client](#) &)=delete
- void [connect_to](#) (QString host, int port)
Starts an asynchronous connection to the broker and the network thread. Returns immediately; success is signaled later via [connected\(\)](#). The network thread (started here via `mosquitto_loop_start`) auto-reconnects on connection loss, using the delay/backoff configured in this call.
- void [disconnect_from_broker](#) ()
Gracefully disconnects from the broker and stops the network thread. Counterpart of [connect_to\(\)](#). No-op if never connected.
- bool [publish](#) (QString topic, const QByteArray &payload, int qos=1)
Publishes a payload to a topic.
- bool [subscribe](#) (QString topic, int qos=1)
Subscribes to a topic and remembers it for automatic re-subscription. The topic/qos pair is stored in `subscriptions_` regardless of whether the immediate subscribe call succeeds, so it is replayed automatically after every (re)connect.

10.11.1 Detailed Description

Asynchronous MQTT client, wrapping libmosquitto behind Qt signals.

Owns a single `mosquitto` handle and runs libmosquitto's own network thread (via `mosquitto_loop_start`) for connecting, reconnecting, and I/O. The C library's callbacks fire on that network thread; they are marshaled onto the Qt (main) thread via `QMetaObject::invokeMethod` with `Qt::QueuedConnection` before the corresponding [connected\(\)](#), [disconnected\(\)](#), or [message_received\(\)](#) signal is emitted, so consumers of those signals never need to worry about cross-thread access.

10.11.2 Constructor & Destructor Documentation

10.11.2.1 client()

```
mind::mqtt::client::client (
    QObject * parent = nullptr) [explicit]
```

Initializes the mosquitto library and creates the connection handle. Increments the process-wide mosquitto library refcount (calling `mosquitto_lib_init` only for the first live instance), allocates the `mosquitto` handle with a host+PID-derived client id, and registers the static callback trampolines. Opens no network connection; call [connect_to\(\)](#) for that.

Initializes the mosquitto library and creates the connection handle.

Parameters

<i>parent</i>	Optional Qt parent for ownership/lifetime management.
---------------	---

10.11.2.2 ~client()

```
mind::mqtt::client::~~client () [override]
```

Stops the network thread (if running), destroys the mosquitto handle, and decrements the library refcount, calling `mosquitto_lib_cleanup` once the last instance is destroyed.

Stops the network thread (if running), destroys the mosquitto handle, and releases the library refcount.

10.11.3 Member Function Documentation**10.11.3.1 connect_to()**

```
void mind::mqtt::client::connect_to (
    QString host,
    int port)
```

Starts an asynchronous connection to the broker and the network thread. Returns immediately; success is signaled later via [connected\(\)](#). The network thread (started here via `mosquitto_loop_start`) auto-reconnects on connection loss, using the delay/backoff configured in this call.

Starts an asynchronous connection to the broker and the network thread.

Parameters

<i>host</i>	Broker hostname or IP address.
<i>port</i>	Broker TCP port.

Note

Must not be called while the network thread is already running (`loop_started_ true`), asserted via `EXPECT↵_NEQ`.

Todo host and port could be set in the constructor.

10.11.3.2 disconnect_from_broker()

```
void mind::mqtt::client::disconnect_from_broker ()
```

Gracefully disconnects from the broker and stops the network thread. Counterpart of [connect_to\(\)](#). No-op if never connected.

Gracefully disconnects from the broker and stops the network thread.

10.11.3.3 message_received

```
void mind::mqtt::client::message_received (
    QString topic,
    QByteArray payload) [signal]
```

Emitted on the Qt thread for each message received on a subscribed topic.

Parameters

<i>topic</i>	Topic the message was published to.
<i>payload</i>	Raw message bytes, forwarded unchanged; decoding is left to the caller.

10.11.3.4 publish()

```
bool mind::mqtt::client::publish (
    QString topic,
    const QByteArray & payload,
    int qos = 1)
```

Publishes a payload to a topic.

Parameters

<i>topic</i>	MQTT topic to publish to.
<i>payload</i>	Raw message bytes; sent unchanged.
<i>qos</i>	MQTT quality-of-service level (0, 1, or 2).

Returns

true if libmosquitto accepted the publish request, false if the client handle is unavailable or the call failed.

10.11.3.5 subscribe()

```
bool mind::mqtt::client::subscribe (
    QString topic,
    int qos = 1)
```

Subscribes to a topic and remembers it for automatic re-subscription. The topic/qos pair is stored in subscriptions← regardless of whether the immediate subscribe call succeeds, so it is replayed automatically after every (re)connect.

Subscribes to a topic and remembers it for automatic re-subscription.

Parameters

<i>topic</i>	MQTT topic filter to subscribe to.
<i>qos</i>	MQTT quality-of-service level (0, 1, or 2).

Returns

true if libmosquitto accepted the subscribe request, false if the client handle is unavailable or the call failed.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.cpp](#)

10.12 ns::CloseShopEvent Struct Reference

Payload of the "close_shop" message: identifies which player closed the shop. Parsed in [shop.cpp](#)'s "close_shop" handler and used to build a temporary [Player](#) before calling [Shop::close_shop\(\)](#).

```
#include <network_messages.hpp>
```

Public Attributes

- int **playerId**

10.12.1 Detailed Description

Payload of the "close_shop" message: identifies which player closed the shop. Parsed in [shop.cpp](#)'s "close_shop" handler and used to build a temporary [Player](#) before calling [Shop::close_shop\(\)](#).

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp](#)

10.13 collision::collision_planner Class Reference

Central, single-instance collision-avoidance coordinator (plain C++/STL + OpenCV, no Qt). Owns the board frame, the stateless PathPlanner, every robot's nominal + effective path, and per-pair proximity state; driven by [update\(\)](#), publishes versioned path_updates through a sink. Implements CTFS (Component-Token Freeze-Senior Step-↔ Off): on every tick, connected components of currently conflicting robots are resolved by letting the lowest-id member of each component keep moving while every other member is frozen, stepped off the lane, or held.

```
#include <collision_planner.hpp>
```


Public Member Functions

- [collision_planner](#) (float board_width_cm=300.0f, float board_height_cm=185.0f, float resolution_cm=2.0f)
Constructs the planner and sizes its internal obstacle grid.
- void [set_path_sink](#) ([path_sink](#) sink)
Registers the push sink (the follower's update function). Replaces any previously registered sink.
- void [set_priority_rule](#) ([priority_rule](#) rule)
Overrides the "who yields" rule. Defaults to default_priority.
- void [set_graph](#) ([playboard_graph](#) graph)
Injects the playboard node graph (board-frame centimetres). Enables request_route_to and shove_blocker. Safe to call again to rebuild. No graph => those calls report no_route / no_spot.
- bool [request_path_to](#) (int robot_id, cv::Point2f goal_cm)
Plans and publishes a straight free-space nominal path from the robot's last-known pose to goal_cm, avoiding parked robots. Publishes with state=nominal on success, or state=holding with an empty path if no path could be planned.
- [route_result](#) [request_route_to](#) (int robot_id, int target_node)
Moves a robot to target_node along the graph edges (not a free-space line): computes the shortest node route, builds an A-checked edge-following polyline, rounds corners, and publishes.*
- [shove_result](#) [shove_blocker](#) (int blocker_id, int clear_node)
Nudges a parked robot off clear_node to a valid park spot (off every edge, node- and robot-clear, A-reachable) and publishes a short move (state=nominal). The planner self-clears the robot (deactivates it) once it arrives at the spot, or once a no-progress watchdog fires - no external driver completion signal is assumed. Plans only, never commands hardware. Public for independent driving/testing.*
- void [clear_path](#) (int robot_id)
Turn ended: drops the robot's active path so it stops being a collision partner and leaves any watched pairs. Its pose is intentionally kept - it is still on the board and must remain a static obstacle. Use forget() for a robot that is gone (lost tracking / never tracked). Resumes any partner that was frozen or stepped off for this robot.
- void [forget](#) (int robot_id)
Drops a robot entirely: erases its pose, path record and every pair, so it is no longer an obstacle, collision partner, shove target, or routing blocker. Call once a robot has gone stale/untracked so a phantom can't sit on a tile and deadlock a real robot. Publishes a terminal empty snapshot if it had a live path. Idempotent (no-op for an unknown id).
- void [update](#) (const std::vector< [robot_pose](#) > &poses)
Feeds the current live poses of all known robots; the heartbeat driving every other state transition. Stores the poses, then each tick: releases self-shoved blockers that arrived or stalled, progresses/trims each graph robot's route, revalidates paths gone stale against newly-parked robots, updates the per-pair avoidance hysteresis, resolves every conflicting component (CTFS), and applies the hard-stop overlap guard. May publish new path_updates (avoidance, hold, or resume) for any robot as a side effect.
- [path_update](#) [effective_path](#) (int robot_id) const
Pull alternative to the sink: a copy of the robot's latest published snapshot.
- bool [is_yielding](#) (int robot_id) const
True while a robot is temporarily stepping off the lane to yield to a higher-priority mover and will resume to its target afterwards. A driver must NOT treat its current stopping point as an arrival while this is true. Returns false for a same-target loser parking aside for good (a permanent, not temporary, stop). Thread-safe.
- std::vector< cv::Point2f > [nominal_path](#) (int robot_id) const
The robot's plan-once nominal path (empty if none). Thread-safe.
- std::vector< std::pair< int, int > > [watched_pairs](#) () const
Currently-conflicting path pairs as (low_id, high_id). Thread-safe.
- std::vector< int > [route_nodes](#) (int robot_id) const
Remaining node-id route for a graph-mode robot (empty in free mode). Thread-safe.
- [pathfinding::GridMap](#) [debug_obstacle_grid](#) (int self_id, int block_id=-1) const
Debug/visualization: a copy of the obstacle grid the planner would use for self_id right now. Blocked cells are parked/holding discs; passing block_id also stamps that holder's disc plus its swept lookahead region, i.e. exactly what a yielder's A sees when planning around it. Thread-safe.*

10.13.1 Detailed Description

Central, single-instance collision-avoidance coordinator (plain C++/STL + OpenCV, no Qt). Owns the board frame, the stateless PathPlanner, every robot's nominal + effective path, and per-pair proximity state; driven by [update\(\)](#), publishes versioned path_updates through a sink. Implements CTFS (Component-Token Freeze-Senior Step-Off): on every tick, connected components of currently conflicting robots are resolved by letting the lowest-id member of each component keep moving while every other member is frozen, stepped off the lane, or held.

Note

Threading: call the mutating methods from a single thread (camera-tick / GUI); [effective_path\(\)](#) is mutex-guarded so a follower on another thread can pull safely; the sink is always invoked without the lock held.

10.13.2 Constructor & Destructor Documentation

10.13.2.1 collision_planner()

```
collision::collision_planner::collision_planner (
    float board_width_cm = 300.0f,
    float board_height_cm = 185.0f,
    float resolution_cm = 2.0f) [explicit]
```

Constructs the planner and sizes its internal obstacle grid.

Parameters

<i>board_width_cm</i>	Physical board width, centimetres.
<i>board_height_cm</i>	Physical board height, centimetres.
<i>resolution_cm</i>	Grid cell size for the A* obstacle grid, centimetres.

10.13.3 Member Function Documentation

10.13.3.1 clear_path()

```
void collision::collision_planner::clear_path (
    int robot_id)
```

Turn ended: drops the robot's active path so it stops being a collision partner and leaves any watched pairs. Its pose is intentionally kept - it is still on the board and must remain a static obstacle. Use [forget\(\)](#) for a robot that is gone (lost tracking / never tracked). Resumes any partner that was frozen or stepped off for this robot.

Turn ended: drops the robot's active path so it stops being a collision partner.

10.13.3.2 debug_obstacle_grid()

```
pathfinding::GridMap collision::collision_planner::debug_obstacle_grid (
    int self_id,
    int block_id = -1) const
```

Debug/visualization: a copy of the obstacle grid the planner would use for *self_id* right now. Blocked cells are parked/holding discs; passing *block_id* also stamps that holder's disc plus its swept lookahead region, i.e. exactly what a yielder's A* sees when planning around it. Thread-safe.

Debug/visualization: a copy of the obstacle grid the planner would use for *self_id* right now.

10.13.3.3 effective_path()

```
path_update collision::collision_planner::effective_path (
    int robot_id) const
```

Pull alternative to the sink: a copy of the robot's latest published snapshot.

Returns

The robot's current [path_update](#), or version 0 with empty waypoints if it has none. Thread-safe.

10.13.3.4 forget()

```
void collision::collision_planner::forget (
    int robot_id)
```

Drops a robot entirely: erases its pose, path record and every pair, so it is no longer an obstacle, collision partner, shove target, or routing blocker. Call once a robot has gone stale/untracked so a phantom can't sit on a tile and deadlock a real robot. Publishes a terminal empty snapshot if it had a live path. Idempotent (no-op for an unknown id).

Drops a robot entirely: erases its pose, path record and every pair it was in.

10.13.3.5 is_yielding()

```
bool collision::collision_planner::is_yielding (
    int robot_id) const
```

True while a robot is temporarily stepping off the lane to yield to a higher-priority mover and will resume to its target afterwards. A driver must NOT treat its current stopping point as an arrival while this is true. Returns false for a same-target loser parking aside for good (a permanent, not temporary, stop). Thread-safe.

True while a robot is temporarily stepping off the lane to yield and will resume afterwards.

10.13.3.6 nominal_path()

```
std::vector< cv::Point2f > collision::collision_planner::nominal_path (
    int robot_id) const
```

The robot's plan-once nominal path (empty if none). Thread-safe.

The robot's plan-once nominal path (empty if none).

10.13.3.7 request_path_to()

```
bool collision::collision_planner::request_path_to (
    int robot_id,
    cv::Point2f goal_cm)
```

Plans and publishes a straight free-space nominal path from the robot's last-known pose to goal_cm, avoiding parked robots. Publishes with state=nominal on success, or state=holding with an empty path if no path could be planned.

Plans and publishes a straight free-space nominal path from the robot's last-known pose to goal_cm.

Parameters

<i>robot_id</i>	Robot to move.
<i>goal_cm</i>	Target point, board-frame centimetres.

Returns

True if a drivable path was produced; false if the robot's pose is unknown (requires a prior pose via [update\(\)](#)) or no path could be planned (the robot is published holding instead).

10.13.3.8 request_route_to()

```
route_result collision::collision_planner::request_route_to (
    int robot_id,
    int target_node)
```

Moves a robot to target_node along the graph edges (not a free-space line): computes the shortest node route, builds an A*-checked edge-following polyline, rounds corners, and publishes.

Moves a robot to target_node along the graph edges (edge-following route, not a free-space line).

Parameters

<i>robot_id</i>	Robot to move.
<i>target_node</i>	Destination node id in the injected playboard_graph .

Returns

[route_result::routed](#) on success; [route_result::no_route](#) if the graph is empty, target_node is unknown, or no drivable route exists; [route_result::unknown_pose](#) if the robot's pose has not been fed via [update\(\)](#) yet.

Note

If target_node is already occupied by a parked robot, this call does NOT shove the occupant - it parks the arriver cleanly beside the node instead (first-come-first-served). Requires a prior pose via [update\(\)](#) and an injected graph.

10.13.3.9 route_nodes()

```
std::vector< int > collision::collision_planner::route_nodes (
    int robot_id) const
```

Remaining node-id route for a graph-mode robot (empty in free mode). Thread-safe.

Remaining node-id route for a graph-mode robot (empty in free mode).

10.13.3.10 set_graph()

```
void collision::collision_planner::set_graph (
    playboard_graph graph)
```

Injects the playboard node graph (board-frame centimetres). Enables request_route_to and shove_blocker. Safe to call again to rebuild. No graph => those calls report no_route / no_spot.

Injects the playboard node graph (board-frame centimetres).

10.13.3.11 set_path_sink()

```
void collision::collision_planner::set_path_sink (
    path_sink sink)
```

Registers the push sink (the follower's update function). Replaces any previously registered sink.

Registers the push sink (the follower's update function).

10.13.3.12 set_priority_rule()

```
void collision::collision_planner::set_priority_rule (
    priority_rule rule)
```

Overrides the "who yields" rule. Defaults to default_priority.

Note

The stored rule is currently never invoked elsewhere in this file - resolve_components decides priority directly (lowest robot id in a conflicting component moves), not through this callback.

10.13.3.13 shove_blocker()

```
shove_result collision::collision_planner::shove_blocker (
    int blocker_id,
    int clear_node)
```

Nudges a parked robot off `clear_node` to a valid park spot (off every edge, node- and robot-clear, A*-reachable) and publishes a short move (state=nominal). The planner self-clears the robot (deactivates it) once it arrives at the spot, or once a no-progress watchdog fires - no external driver completion signal is assumed. Plans only, never commands hardware. Public for independent driving/testing.

Nudges a parked robot off `clear_node` to a valid park spot and publishes a short move.

Parameters

<code>blocker_id</code>	Robot to nudge aside.
<code>clear_node</code>	Node id the blocker should vacate.

Returns

shoved on success; no_spot if no valid park spot exists near the node; unreachable if a spot was found but the blocker cannot reach it collision-free; unknown_pose if the blocker's pose is not yet known.

10.13.3.14 update()

```
void collision::collision_planner::update (
    const std::vector< robot_pose > & poses)
```

Feeds the current live poses of all known robots; the heartbeat driving every other state transition. Stores the poses, then each tick: releases self-shoved blockers that arrived or stalled, progresses/trims each graph robot's route, revalidates paths gone stale against newly-parked robots, updates the per-pair avoidance hysteresis, resolves every conflicting component (CTFS), and applies the hard-stop overlap guard. May publish new path_updates (avoidance, hold, or resume) for any robot as a side effect.

Feeds the current live poses of all known robots; the heartbeat driving every other state transition.

Parameters

<code>poses</code>	Latest pose of every currently-tracked robot.
--------------------	---

10.13.3.15 watched_pairs()

```
std::vector< std::pair< int, int > > collision::collision_planner::watched_pairs () const
```

Currently-conflicting path pairs as (low_id, high_id). Thread-safe.

Currently-conflicting path pairs as (low_id, high_id).

The documentation for this class was generated from the following files:

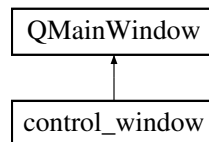
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.hpp
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.cpp

10.14 control_window Class Reference

Standalone window for robot control. Owns the MQTT client, the robot_fleet auto-discovery, and a [fleet_panel](#) that shows one row per discovered robot. Connects to the broker on construction (host + port from env, defaults to localhost:1883). [main.cpp](#) just opens this.

```
#include <control_window.hpp>
```

Inheritance diagram for control_window:



Public Member Functions

- [control_window](#) (QWidget *parent=nullptr)
Builds the window and starts up the MQTT/UDP/fleet plumbing.
- [~control_window](#) () override
Stops the udp-worker thread before member destruction so udp_link's QUdpSocket is never touched concurrently while its destructor runs.
- [mind::fleet::robot_fleet * getFleet](#) () const
Returns the owned robot_fleet instance (non-owning raw pointer).

10.14.1 Detailed Description

Standalone window for robot control. Owns the MQTT client, the robot_fleet auto-discovery, and a [fleet_panel](#) that shows one row per discovered robot. Connects to the broker on construction (host + port from env, defaults to localhost:1883). [main.cpp](#) just opens this.

10.14.2 Constructor & Destructor Documentation

10.14.2.1 control_window()

```
control_window::control_window (
    QWidget * parent = nullptr) [explicit]
```

Builds the window and starts up the MQTT/UDP/fleet plumbing.

Reads the MQTT broker host/port from the MIND_BROKER_HOST / MIND_BROKER_PORT environment variables (defaulting to 127.0.0.1:1883), moves the udp_link onto a dedicated worker thread, builds the robot_fleet, and lays out the [fleet_panel](#), [test_panel](#) and [latency_indicator](#) widgets. The broker connection attempt itself is asynchronous and does not block construction.

Parameters

<i>parent</i>	Optional parent widget, passed through to QMainWindow.
---------------	--

10.14.2.2 ~control_window()

```
control_window::~~control_window () [override]
```

Stops the udp-worker thread before member destruction so udp_link's QUdpSocket is never touched concurrently while its destructor runs.

Stops the udp-worker thread before member destruction.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.cpp](#)

10.15 Dice Class Reference

Represents a die that produces a random roll in [1, 10]. Instances are typically constructed as throwaway objects purely to invoke [throwDice\(\)](#); the "dice" parameter accepted by the member functions below is currently unused.

```
#include <dice.hpp>
```

Public Member Functions

- [int throwDice \(\)](#)

Rolls the die and returns a uniformly random value. A new `std::random_device` / `std::mt19937` generator is created and seeded on every call, rather than reusing a persistent generator across rolls.

10.15.1 Detailed Description

Represents a die that produces a random roll in [1, 10]. Instances are typically constructed as throwaway objects purely to invoke [throwDice\(\)](#); the "dice" parameter accepted by the member functions below is currently unused.

10.15.2 Member Function Documentation

10.15.2.1 throwDice()

```
int Dice::throwDice ()
```

Rolls the die and returns a uniformly random value. A new `std::random_device` / `std::mt19937` generator is created and seeded on every call, rather than reusing a persistent generator across rolls.

Returns

A random integer in the inclusive range [1, 10].

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.cpp](#)

10.16 follower::drive_cmd Struct Reference

Controller output: a velocity setpoint for the drive base.

```
#include <path_follower.hpp>
```

Public Attributes

- float **v_mm_s** = 0.0f
forward speed, mm/s
- float **omega_dps** = 0.0f
turn rate, degrees/s
- bool **stop** = true

10.16.1 Detailed Description

Controller output: a velocity setpoint for the drive base.

Note

stop == true means "command (0, 0)" - the actuator's watchdog/ release then halts the brick. Values are pre-clamp; [to_percent\(\)](#) maps them onto the brick's [-100, 100] percentage range.

The documentation for this struct was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/[path_follower.hpp](#)

10.17 follower::drive_pct Struct Reference

Brick tank-drive percentages, each clamped to [-100, 100].

```
#include <path_follower.hpp>
```

Public Attributes

- int **speed_pct** = 0
forward/back duty, percent of brick_max_mm_s
- int **turn_pct** = 0
steering duty, percent of brick_max_dps

10.17.1 Detailed Description

Brick tank-drive percentages, each clamped to [-100, 100].

The documentation for this struct was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/[path_follower.hpp](#)

10.18 Effect Class Reference

Base class for timed status effects attached to a [Player](#). Concrete subclasses implement [applyStartOfTurn\(\)](#)/[applyEndOfTurn\(\)](#) to modify player state; the effect expires once its duration counts down to zero (see [tick\(\)](#)/[isExpired\(\)](#)).

```
#include <player.hpp>
```

Public Member Functions

- [Effect](#) (int duration, [EffectType](#) type)
Constructs an effect with a given lifetime and polarity.
- virtual std::string **getName** () const =0
Returns the display name of the effect.
- virtual void [applyStartOfTurn](#) ([Player](#) &player)=0
Applies the effect's logic at the start of the affected player's turn.
- virtual void [applyEndOfTurn](#) ([Player](#) &player)=0
Applies the effect's logic at the end of the affected player's turn.
- bool **isExpired** () const
Checks whether the effect's duration has counted down to zero or below.
- void **tick** ()
Decrements the remaining duration by one turn.
- [EffectType](#) **getType** () const
Returns whether this effect is classified as positive or negative.

Protected Attributes

- int **duration** = 0
Remaining lifetime in turns; decremented by [tick\(\)](#).
- [EffectType](#) **type** = [EffectType::Positive](#)
Whether this effect is classified as positive or negative.

10.18.1 Detailed Description

Base class for timed status effects attached to a [Player](#). Concrete subclasses implement [applyStartOfTurn\(\)](#)/[applyEndOfTurn\(\)](#) to modify player state; the effect expires once its duration counts down to zero (see [tick\(\)](#)/[isExpired\(\)](#)).

10.18.2 Constructor & Destructor Documentation

10.18.2.1 Effect()

```
Effect::Effect (
    int duration,
    EffectType type)
```

Constructs an effect with a given lifetime and polarity.

Parameters

<i>duration</i>	Number of turns the effect remains active before isExpired() returns true.
<i>type</i>	Whether the effect is positive or negative.

10.18.3 Member Function Documentation**10.18.3.1 applyEndOfTurn()**

```
virtual void Effect::applyEndOfTurn (  
    Player & player) [pure virtual]
```

Applies the effect's logic at the end of the affected player's turn.

Parameters

<i>player</i>	The player whose turn is ending.
---------------	----------------------------------

10.18.3.2 applyStartOfTurn()

```
virtual void Effect::applyStartOfTurn (  
    Player & player) [pure virtual]
```

Applies the effect's logic at the start of the affected player's turn.

Parameters

<i>player</i>	The player whose turn is starting.
---------------	------------------------------------

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.cpp](#)

10.19 BoardLoader.EffectData Class Reference**Public Attributes**

- string **type**
- int **power**

The documentation for this class was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/BoardLoader.cs](#)

10.20 ns::EndRouletteEvent Struct Reference

Payload of the "end_roulette" message: identifies which player's roulette round is ending. Parsed in [roulette.cpp](#)'s "end_roulette" handler and used to build a temporary [Player](#) before calling [Roulette::end_roulette\(\)](#).

```
#include <network_messages.hpp>
```

Public Attributes

- int **playerId**

10.20.1 Detailed Description

Payload of the "end_roulette" message: identifies which player's roulette round is ending. Parsed in [roulette.cpp](#)'s "end_roulette" handler and used to build a temporary [Player](#) before calling [Roulette::end_roulette\(\)](#).

The documentation for this struct was generated from the following file:

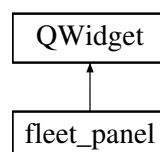
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp](#)

10.21 fleet_panel Class Reference

Vertical stack of one row per discovered robot. Listens to robot_fleet for appear events and to each robot_controller for state, battery, and connection updates. Rows stay around when a robot goes offline (the state label flips to "offline") - the controller itself stays alive for state continuity if the brick returns.

```
#include <fleet_panel.hpp>
```

Inheritance diagram for fleet_panel:



Public Member Functions

- [fleet_panel](#) ([mind::fleet::robot_fleet](#) *fleet, QWidget *parent=nullptr)
Builds the panel's layout (title + stretch) and subscribes to fleet appearance events.

10.21.1 Detailed Description

Vertical stack of one row per discovered robot. Listens to robot_fleet for appear events and to each robot_controller for state, battery, and connection updates. Rows stay around when a robot goes offline (the state label flips to "offline") - the controller itself stays alive for state continuity if the brick returns.

10.21.2 Constructor & Destructor Documentation

10.21.2.1 fleet_panel()

```
fleet_panel::fleet_panel (
    mind::fleet::robot_fleet * fleet,
    QWidget * parent = nullptr) [explicit]
```

Builds the panel's layout (title + stretch) and subscribes to fleet appearance events.

Parameters

<i>fleet</i>	Fleet whose robot_appeared signal populates the rows; not owned by this widget.
<i>parent</i>	Optional Qt parent widget.

The documentation for this class was generated from the following files:

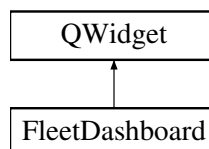
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanel/fleet_panel.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanel/fleet_panel.cpp](#)

10.22 FleetDashboard Class Reference

Casino-themed Fleet Dashboard showing all 4 robot slots with live connection, battery, EV3 state, follower mode, target node, cross-track error, and calibration scales. Refreshes at 4 Hz via an internal QTimer.

```
#include <fleet_dashboard.hpp>
```

Inheritance diagram for FleetDashboard:



Public Member Functions

- **FleetDashboard** ([MapWindow](#) &mapwindow, QWidget *parent=nullptr)

10.22.1 Detailed Description

Casino-themed Fleet Dashboard showing all 4 robot slots with live connection, battery, EV3 state, follower mode, target node, cross-track error, and calibration scales. Refreshes at 4 Hz via an internal QTimer.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanel/fleet_dashboard.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanel/fleet_dashboard.cpp](#)

10.23 follower::follower_params Struct Reference

All tunable gains and limits for the path follower, in one plain, copyable, JSON-friendly struct. A Qt tuning panel can read/write these live and save/load them. The [path_follower](#) re-reads this struct fresh on every `step()`, so `set_params()` takes effect on the very next tick.

```
#include <path_follower.hpp>
```

Public Attributes

- float **v_max_mm_s** = 150.0f
top forward speed, mm/s
- float **v_min_mm_s** = 40.0f
floor while pursuing in open driving, mm/s
- float **lookahead_cm** = 20.0f
pure-pursuit lookahead distance, cm
- float **omega_cap_dps** = 120.0f
max turn rate, deg/s
- float **omega_rotate_dps** = 90.0f
spin rate in rotate-in-place, deg/s
- float **min_turn_pct** = 40.0f
- float **min_speed_pct** = 0.0f
- float **kp** = 1.6f
heading PID on lookahead bearing (rad/s per rad)
- float **ki** = 0.0f
integral trim (per-brick wheel bias)
- float **kd** = 0.20f
derivative damp (latency overshoot)
- float **i_max_dps** = 30.0f
anti-windup clamp on the integral term
- float **rotate_enter_deg** = 40.0f
|bearing| above this -> rotate in place
- float **rotate_exit_deg** = 25.0f
- float **decel_radius_cm** = 30.0f
start ramping speed down within this of the goal, cm
- float **arrive_radius_cm** = 1.5f
within this of the final waypoint -> stop (on the node), cm
- float **creep_mm_s** = 25.0f
gentle approach speed inside decel_radius, mm/s
- float **brick_max_mm_s** = 450.0f
100% speed_pct on the (Rust) brick, mm/s
- float **brick_max_dps** = 250.0f
100% turn_pct on the (Rust) brick, deg/s
- bool **flip_turn** = false
hardware steering-sign flip (set at bring-up)
- float **drive_scale** = 1.0f
learned forward-speed calibration multiplier
- float **turn_scale** = 1.0f
learned turn-rate calibration multiplier

10.23.1 Detailed Description

All tunable gains and limits for the path follower, in one plain, copyable, JSON-friendly struct. A Qt tuning panel can read/write these live and save/load them. The [path_follower](#) re-reads this struct fresh on every `step()`, so `set_params()` takes effect on the very next tick.

10.23.2 Member Data Documentation

10.23.2.1 min_speed_pct

```
float follower::follower_params::min_speed_pct = 0.0f
```

forward deadband floor: a non-zero `speed_pct` below the EV3's drive breakaway duty stalls the robot (it never reaches the goal -> the creep near a node hangs). Floored to this magnitude so a creep always rolls. 0 disables (default: off; the Auto-tune sets it for HW).

10.23.2.2 min_turn_pct

```
float follower::follower_params::min_turn_pct = 40.0f
```

in-place-pivot deadband floor: a `turn_pct` smaller than this can't overcome the EV3's static friction (the robot twitches without yawing), so an in-place turn (`speed_pct==0`) is floored to this magnitude. 0 disables.

10.23.2.3 rotate_exit_deg

```
float follower::follower_params::rotate_exit_deg = 25.0f
```

hysteresis: leave rotate-in-place below this. Wide enough that a moderate open-loop OVER-shoot (e.g. an uncalibrated sim turning 15% too far) hands off to pursue - which CURVES onto the line - instead of re-turning the residual and visibly correcting back.

The documentation for this struct was generated from the following file:

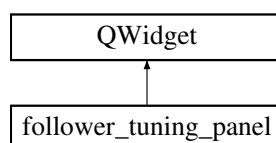
- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.hpp`

10.24 follower_tuning_panel Class Reference

Live tuning panel for the path-following driver. A thin Qt view over the plain `follower_params` struct: editing any gain emits `changed()` so the owner can push it to every running follower (taking effect on the next 33 ms tick). Save/Load persist a tune to JSON so it survives a restart; PANIC STOP halts all robots.

```
#include <follower_tuning_panel.hpp>
```

Inheritance diagram for `follower_tuning_panel`:



Signals

- void `changed` (`follower::follower_params` p)
Emitted whenever the user edits any gain or the flip-turn checkbox.
- void `panicStop` ()
Emitted when the PANIC STOP button is pressed; the owner is expected to halt all robots.

Public Member Functions

- `follower_tuning_panel` (`follower::follower_params` init, `QWidget *parent=nullptr`)
Builds one spin-box row per tunable gain (see `specs()` in the `.cpp`), plus the flip-turn checkbox and the Save/Load/↔ Reset/PANIC STOP buttons.
- `follower::follower_params` `params` () const
Returns the parameter set currently held by the panel's controls.
- void `setParams` (const `follower::follower_params` &p)
Overwrites every control with the given parameter set.

10.24.1 Detailed Description

Live tuning panel for the path-following driver. A thin Qt view over the plain `follower_params` struct: editing any gain emits `changed()` so the owner can push it to every running follower (taking effect on the next 33 ms tick). Save/Load persist a tune to JSON so it survives a restart; PANIC STOP halts all robots.

10.24.2 Constructor & Destructor Documentation

10.24.2.1 `follower_tuning_panel()`

```
follower_tuning_panel::follower_tuning_panel (
    follower::follower_params init,
    QWidget * parent = nullptr) [explicit]
```

Builds one spin-box row per tunable gain (see `specs()` in the `.cpp`), plus the flip-turn checkbox and the Save/Load/↔ Reset/PANIC STOP buttons.

Builds one spin-box row per tunable gain plus the flip-turn checkbox and the Save/Load/Reset/PANIC STOP buttons, seeded from `init`.

Parameters

<i>init</i>	Initial parameter values used to populate every control.
<i>parent</i>	Optional owning widget.

10.24.3 Member Function Documentation

10.24.3.1 changed

```
void follower_tuning_panel::changed (  
    follower::follower_params p) [signal]
```

Emitted whenever the user edits any gain or the flip-turn checkbox.

Parameters

<i>p</i>	The updated parameter set.
----------	----------------------------

10.24.3.2 params()

```
follower::follower_params follower_tuning_panel::params () const [inline]
```

Returns the parameter set currently held by the panel's controls.

Returns

The current parameter set.

10.24.3.3 setParams()

```
void follower_tuning_panel::setParams (  
    const follower::follower_params & p)
```

Overwrites every control with the given parameter set.

Overwrites every control with the given parameter set without re-emitting [changed\(\)](#).

Parameters

<i>p</i>	The parameter values to display.
----------	----------------------------------

Note

Blocks each control's signals while updating it, so this does not emit [changed\(\)](#).

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.cpp](#)

10.25 Game Class Reference

Top-level game session state: the board, the participating players, and whose turn/round it currently is. Turn order and round advancement are driven externally (e.g. by a turn manager) via `nextPlayer()/nextRound()/determineTurnOrder()`; `Game` itself does not run a game loop.

```
#include <game.hpp>
```

Public Member Functions

- `Game (std::vector< std::shared_ptr< Player > > players, int maxRounds=10)`
Constructs a game for the given players and round limit.
- `Player & getCurrentPlayer ()`
Returns the player whose turn it currently is.
- `void nextPlayer ()`
Advances to the next player's turn, wrapping back to index 0 after the last player.
- `void nextRound ()`
Advances to the next round and resets the turn to the first player. Sets `_finished` to true once `_currentRound` exceeds `_maxRounds`.
- `bool isLastPlayer () const`
Checks whether the current player is the last one in the turn order.
- `void determineTurnOrder ()`
Determines player turn order for the match by rolling dice. Each player rolls once; players are ranked by descending roll. Players tied on their first roll are grouped and re-roll amongst themselves (discarding ties again if needed) until all members of the group have distinct values, which are then used to order that group. Reorders `_players` in place and resets `_currentPlayerIndex` to 0.

Public Attributes

- `Board _board`
- `std::vector< std::shared_ptr< Player > > _players`
Participating players, reordered in place by `determineTurnOrder()`.
- `int _currentPlayerIndex = 0`
Index into `_players` of the player whose turn it currently is.
- `int _currentRound = 1`
- `int _maxRounds = 10`
- `bool _finished = false`
Set once `_currentRound` exceeds `_maxRounds`; the match should end when true.

10.25.1 Detailed Description

Top-level game session state: the board, the participating players, and whose turn/round it currently is. Turn order and round advancement are driven externally (e.g. by a turn manager) via `nextPlayer()/nextRound()/determineTurnOrder()`; `Game` itself does not run a game loop.

10.25.2 Constructor & Destructor Documentation

10.25.2.1 Game()

```
Game::Game (
    std::vector< std::shared_ptr< Player > > players,
    int maxRounds = 10)
```

Constructs a game for the given players and round limit.

Parameters

<i>players</i>	The participating players, in their initial (pre-turn-order) sequence.
<i>maxRounds</i>	Number of rounds after which the match is considered finished.

10.25.3 Member Function Documentation

10.25.3.1 determineTurnOrder()

```
void Game::determineTurnOrder ()
```

Determines player turn order for the match by rolling dice. Each player rolls once; players are ranked by descending roll. Players tied on their first roll are grouped and re-roll amongst themselves (discarding ties again if needed) until all members of the group have distinct values, which are then used to order that group. Reorders `_players` in place and resets `_currentPlayerIndex` to 0.

Determines player turn order (Zugreihenfolge) for the match by rolling dice.

10.25.3.2 getCurrentPlayer()

```
Player & Game::getCurrentPlayer ()
```

Returns the player whose turn it currently is.

Returns

Reference to the player at `_currentPlayerIndex` in `_players`.

Note

Throws `std::runtime_error` if `_players` is empty.

10.25.3.3 isLastPlayer()

```
bool Game::isLastPlayer () const
```

Checks whether the current player is the last one in the turn order.

Returns

true if `_currentPlayerIndex` refers to the last entry in `_players`.

10.25.3.4 nextRound()

```
void Game::nextRound ()
```

Advances to the next round and resets the turn to the first player. Sets `_finished` to true once `_currentRound` exceeds `_maxRounds`.

Advances to the next round and resets the turn to the first player.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.cpp](#)

10.26 GameManager Class Reference

Owns all Players and orchestrates running mini-games. Converts Players to [MiniGameParticipant](#) DTOs before a mini-game starts, runs the mini-game to completion, and reconciles the returned [MiniGameResult](#) back into the real [Player](#) objects (currently: coin balance only; stars are awarded elsewhere).

```
#include <game_manager.hpp>
```

Public Member Functions

- [GameManager](#) (const std::vector< [Player](#) > &players)
Constructs the manager with an initial copy of the session's players.
- void [runMiniGame](#) ([MiniGame](#) &game)
Runs one mini-game end-to-end and applies its result to the real Players. Converts all players to [MiniGameParticipant](#) DTOs, marks `game` as the active mini-game, blocks on [MiniGame::play\(\)](#), then clears the active-game pointer and reconciles the returned [MiniGameResult](#) back into `players_`.
- void [notifyRobotPosition](#) (int index, double x_cm, double y_cm)
Forwards a tracked robot position update to the currently running mini-game. Called by the tracker on every camera frame; silently ignored if no mini-game is currently active (i.e. outside of [runMiniGame\(\)](#)).
- const std::vector< [Player](#) > &[getPlayers](#) () const
Returns a read-only reference to the current list of players.
- void [syncPlayers](#) (const std::vector< std::shared_ptr< [Player](#) > > &players)
Replaces the internal player list with a snapshot of the board-game players. Called by [TurnManager](#) just before launching a minigame so that [GameManager](#)'s participants start with the correct coin balance from the real board game.

10.26.1 Detailed Description

Owns all Players and orchestrates running mini-games. Converts Players to [MiniGameParticipant](#) DTOs before a mini-game starts, runs the mini-game to completion, and reconciles the returned [MiniGameResult](#) back into the real [Player](#) objects (currently: coin balance only; stars are awarded elsewhere).

10.26.2 Constructor & Destructor Documentation

10.26.2.1 GameManager()

```
GameManager::GameManager (
    const std::vector< Player > & players)
```

Constructs the manager with an initial copy of the session's players.

Parameters

<i>players</i>	The players taking part in the game session; stored by value in <code>players_</code> .
----------------	---

10.26.3 Member Function Documentation

10.26.3.1 notifyRobotPosition()

```
void GameManager::notifyRobotPosition (
    int index,
    double x_cm,
    double y_cm)
```

Forwards a tracked robot position update to the currently running mini-game. Called by the tracker on every camera frame; silently ignored if no mini-game is currently active (i.e. outside of [runMiniGame\(\)](#)).

Forwards a tracked robot position update to the currently running mini-game.

Parameters

<i>index</i>	Identifies which robot/participant moved (see MiniGame::onRobotMoved).
<i>x_cm</i>	Robot 's tracked X position, in centimeters.
<i>y_cm</i>	Robot 's tracked Y position, in centimeters.

Note

Thread-safe: takes `activeGameMutex_`, since this is expected to be called from the tracker thread while [runMiniGame\(\)](#) concurrently starts/stops a game on another thread.

10.26.3.2 runMiniGame()

```
void GameManager::runMiniGame (
    MiniGame & game)
```

Runs one mini-game end-to-end and applies its result to the real Players. Converts all players to [MiniGameParticipant](#) DTOs, marks `game` as the active mini-game, blocks on [MiniGame::play\(\)](#), then clears the active-game pointer and reconciles the returned [MiniGameResult](#) back into `players_`.

Runs one mini-game end-to-end and applies its result to the real Players.

Parameters

<i>game</i>	The mini-game to run. While play() is in progress it is reachable via notifyRobotPosition() so tracker updates can reach it.
-------------	--

Note

Blocks the calling thread until [MiniGame::play\(\)](#) returns.

10.26.3.3 syncPlayers()

```
void GameManager::syncPlayers (
    const std::vector< std::shared_ptr< Player > > & players)
```

Replaces the internal player list with a snapshot of the board-game players. Called by [TurnManager](#) just before launching a minigame so that [GameManager](#)'s participants start with the correct coin balance from the real board game.

Replaces the internal player list with a snapshot of the board-game players.

Parameters

<i>players</i>	The board-game's live player list (shared_ptr vector from Game::_players).
----------------	---

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.cpp](#)

10.27 Generator Class Reference

Simulates drawing a number from a European roulette wheel. Picks a uniformly random position around the wheel and maps it to the pocket number printed at that position, so results follow the physical wheel layout rather than a plain sequential 0-36 order.

```
#include <rng.hpp>
```

Public Member Functions

- [int roll_ball \(\)](#)
Simulates a single spin of the roulette wheel. Draws a new random pocket number and stores it so it can be re-queried later via [get_last_number\(\)](#).
- [int get_last_number \(\)](#)
Retrieves the result of the most recent [roll_ball\(\)](#) call.

Protected Member Functions

- `int random_number ()`

Draws a uniformly random wheel position and looks up its pocket number. Constructs a new `std::random_device` and `std::mt19937` engine on every call.

Protected Attributes

- `int m_last_number = 0`

Pocket number from the most recent `roll_ball()` call; 0 before the first roll.

- `int m_numbers [37]`

Pocket numbers in physical order around a European roulette wheel, indexed by wheel position (not by number value).

10.27.1 Detailed Description

Simulates drawing a number from a European roulette wheel. Picks a uniformly random position around the wheel and maps it to the pocket number printed at that position, so results follow the physical wheel layout rather than a plain sequential 0-36 order.

10.27.2 Member Function Documentation

10.27.2.1 `get_last_number()`

```
int Generator::get_last_number ()
```

Retrieves the result of the most recent `roll_ball()` call.

Returns

The last rolled pocket number, or 0 if `roll_ball()` has not been called yet.

10.27.2.2 `random_number()`

```
int Generator::random_number () [protected]
```

Draws a uniformly random wheel position and looks up its pocket number. Constructs a new `std::random_device` and `std::mt19937` engine on every call.

Draws a uniformly random wheel position and looks up its pocket number.

Returns

The pocket number found at the randomly chosen wheel position in `m_numbers`.

10.27.2.3 roll_ball()

```
int Generator::roll_ball ()
```

Simulates a single spin of the roulette wheel. Draws a new random pocket number and stores it so it can be re-queried later via [get_last_number\(\)](#).

Simulates a single spin of the roulette wheel.

Returns

The pocket number landed on (0-36, European wheel numbering).

10.27.3 Member Data Documentation

10.27.3.1 m_numbers

```
int Generator::m_numbers[37] [protected]
```

Initial value:

```
=
{
    0, 32, 15, 19, 4, 21, 2, 25, 17, 34, 6, 27, 13,
    36, 11, 30, 8, 23, 10, 5, 24, 16, 33, 1, 20, 14,
    31, 9, 22, 18, 29, 7, 28, 12, 35, 3, 26
}
```

Pocket numbers in physical order around a European roulette wheel, indexed by wheel position (not by number value).

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.cpp](#)

10.28 ns::GetPlayerBetEvent Struct Reference

Payload of the "make_bet" message: the player's roulette wager. Parsed in [roulette.cpp](#)'s "make_bet" handler and forwarded to [Roulette::setBet\(\)](#), which wakes any thread blocked in [Roulette::choose_bet_type\(\)](#).

```
#include <network_messages.hpp>
```

Public Attributes

- int **playerId**
- int **amount**
- int **choice**

amount: coins wagered. choice: player's guessed outcome; encoding depends on bet_type (see [Roulette::roulette_odd_even\(\)](#)/[roulette_red_black\(\)](#)).

- [Bet_type](#) **bet_type**

Category of the bet; serialized as the underlying int of [Bet_type](#) (no NLOHMANN_JSON_SERIALIZE_ENUM mapping is defined for it).

10.28.1 Detailed Description

Payload of the "make_bet" message: the player's roulette wager. Parsed in [roulette.cpp](#)'s "make_bet" handler and forwarded to [Roulette::setBet\(\)](#), which wakes any thread blocked in [Roulette::choose_bet_type\(\)](#).

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp](#)

10.29 collision::graph_node Struct Reference

One node of the playboard graph in the planner's board frame (centimetres). This is the Qt-free, unit-consistent mirror of [map/board.hpp](#)'s [Node](#): the caller ([MapWindow](#)) converts each [Node](#)'s screen pixels to cm once and injects the result via [collision_planner::set_graph](#).

```
#include <playboard_graph.hpp>
```

Public Attributes

- **int id**
node identifier, referenced by [neighbors](#) and by [route\(\)](#)
- **cv::Point2f pos_cm**
node centre, board-frame centimetres
- **std::vector< int > neighbors**
ids of adjacent nodes (graph edges are undirected in practice)

10.29.1 Detailed Description

One node of the playboard graph in the planner's board frame (centimetres). This is the Qt-free, unit-consistent mirror of [map/board.hpp](#)'s [Node](#): the caller ([MapWindow](#)) converts each [Node](#)'s screen pixels to cm once and injects the result via [collision_planner::set_graph](#).

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.hpp](#)

10.30 pathfinding::GridMap Class Reference

2D occupancy grid over the physical play area, used for path planning. Stores one walkable/blocked flag per cell and provides obstacle marking, world<->grid coordinate conversion, and a line-of-sight test consumed by the pathfinder.

```
#include <gridmap.hpp>
```

Public Member Functions

- [GridMap](#) (int width, int height, float resolution)
Constructs an empty grid map with every cell initially walkable.
- void [addObstacle](#) (cv::Point2f world_pos, float radius_mm)
Marks a circular area of cells around a world-space position as blocked. Converts the center to grid coordinates and clears (sets non-walkable) every cell whose center falls within radius_mm of it.
- void [blockBorder](#) (int thickness)
Marks a band of thickness cells around all four edges as non-walkable. Keeps a planned path off the physical board walls by forbidding cells within thickness cells of any edge.
- void **clear** ()
Resets every cell in the grid to walkable, removing all obstacles.
- bool [isInside](#) (int x, int y) const
Checks whether a grid coordinate lies within the map bounds.
- bool [isWalkable](#) (int x, int y) const
Checks whether a grid cell is both in bounds and not blocked.
- bool [hasLineOfSight](#) (cv::Point2i p1, cv::Point2i p2) const
Tests whether a straight line between two grid cells passes only through walkable cells. Walks the line with a Bresenham-style supercover algorithm; on diagonal steps it also requires both corner-adjacent cells to be walkable so the ray cannot slip between two touching obstacle corners, matching the no-corner-cutting rule used by the pathfinder (e.g. A).*
- cv::Point2i [worldToGrid](#) (cv::Point2f world_pos) const
Converts a world-space position to its containing grid cell.
- cv::Point2f [gridToWorld](#) (cv::Point2i grid_pos) const
Converts a grid cell index to the world-space position of its center.
- int **getWidth** () const noexcept
Returns the grid width, in cells.
- int **getHeight** () const noexcept
Returns the grid height, in cells.
- float **getResolution** () const noexcept
Returns the cell size, in millimeters per cell.

10.30.1 Detailed Description

2D occupancy grid over the physical play area, used for path planning. Stores one walkable/blocked flag per cell and provides obstacle marking, world $\leftrightarrow$ grid coordinate conversion, and a line-of-sight test consumed by the pathfinder.

10.30.2 Constructor & Destructor Documentation

10.30.2.1 GridMap()

```
pathfinding::GridMap::GridMap (
    int width,
    int height,
    float resolution)
```

Constructs an empty grid map with every cell initially walkable.

Parameters

<i>width</i>	Grid width, in cells.
<i>height</i>	Grid height, in cells.
<i>resolution</i>	Cell size, in millimeters per cell (world units per cell).

10.30.3 Member Function Documentation**10.30.3.1 addObstacle()**

```
void pathfinding::GridMap::addObstacle (
    cv::Point2f world_pos,
    float radius_mm)
```

Marks a circular area of cells around a world-space position as blocked. Converts the center to grid coordinates and clears (sets non-walkable) every cell whose center falls within `radius_mm` of it.

Marks a circular area of cells around a world-space position as blocked.

Parameters

<i>world_pos</i>	Obstacle center, in world coordinates (millimeters).
<i>radius_mm</i>	Obstacle radius, in millimeters.

10.30.3.2 blockBorder()

```
void pathfinding::GridMap::blockBorder (
    int thickness)
```

Marks a band of `thickness` cells around all four edges as non-walkable. Keeps a planned path off the physical board walls by forbidding cells within `thickness` cells of any edge.

Marks a band of `thickness` cells around all four edges as non-walkable.

Parameters

<i>thickness</i>	Width of the blocked border, in cells. No-op if ≤ 0 .
------------------	--

10.30.3.3 gridToWorld()

```
cv::Point2f pathfinding::GridMap::gridToWorld (
    cv::Point2i grid_pos) const [nodiscard]
```

Converts a grid cell index to the world-space position of its center.

Parameters

<i>grid_pos</i>	Grid cell index.
-----------------	------------------

Returns

World-space coordinates (millimeters) of the cell's center.

10.30.3.4 hasLineOfSight()

```
bool pathfinding::GridMap::hasLineOfSight (
    cv::Point2i p1,
    cv::Point2i p2) const [nodiscard]
```

Tests whether a straight line between two grid cells passes only through walkable cells. Walks the line with a Bresenham-style supercover algorithm; on diagonal steps it also requires both corner-adjacent cells to be walkable so the ray cannot slip between two touching obstacle corners, matching the no-corner-cutting rule used by the pathfinder (e.g. A*).

Tests whether a straight line between two grid cells passes only through walkable cells.

Parameters

<i>p1</i>	Start cell, in grid coordinates.
<i>p2</i>	End cell, in grid coordinates.

Returns

true if every cell along the line (including the corner checks) is walkable.

10.30.3.5 isInside()

```
bool pathfinding::GridMap::isInside (
    int x,
    int y) const [nodiscard]
```

Checks whether a grid coordinate lies within the map bounds.

Parameters

<i>x</i>	Grid column index.
<i>y</i>	Grid row index.

Returns

true if (x, y) is inside [0, width) x [0, height).

10.30.3.6 isWalkable()

```
bool pathfinding::GridMap::isWalkable (
    int x,
    int y) const [nodiscard]
```

Checks whether a grid cell is both in bounds and not blocked.

Parameters

<i>x</i>	Grid column index.
<i>y</i>	Grid row index.

Returns

true if the cell is inside the map and marked walkable.

10.30.3.7 worldToGrid()

```
cv::Point2i pathfinding::GridMap::worldToGrid (
    cv::Point2f world_pos) const [nodiscard]
```

Converts a world-space position to its containing grid cell.

Parameters

<i>world_pos</i>	Position in world coordinates (millimeters).
------------------	--

Returns

Grid cell index containing *world_pos* (each axis truncated toward zero).

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/[gridmap.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/[gridmap.cpp](#)

10.31 mind::proto::heartbeat_msg Struct Reference

Decoded `heartbeat` status payload (EV3 -> desktop, UDP). Published periodically regardless of state; the datagram's source address doubles as the desktop's discovery mechanism for the robot.

```
#include <codec.hpp>
```

Public Attributes

- `std::uint8_t nn`
Robot index (the NN of ev3_NN), not the full robot_id string.
- `robot_state state`
Current robot state.
- `std::uint16_t battery_mv`
Battery voltage, in millivolts.

10.31.1 Detailed Description

Decoded `heartbeat` status payload (EV3 -> desktop, UDP). Published periodically regardless of state; the datagram's source address doubles as the desktop's discovery mechanism for the robot.

The documentation for this struct was generated from the following file:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.hpp`

10.32 Inventory Class Reference

Holds a bounded collection of items owned by a player. Enforces a maximum capacity (`MAX_INVENTORY_SIZE`) and provides index-based lookup, addition, and removal of items in insertion order.

```
#include <inventory.hpp>
```

Public Member Functions

- `bool addItem (const Item &item)`
Adds an item to the inventory if space is available.
- `bool removeItem (int index)`
Removes the item at the given position.
- `bool hasItem (ItemType type) const`
Checks whether the inventory currently holds an item of the given type.
- `const std::vector< Item > &getItems () const`
Provides read-only access to all held items.
- `int getSize () const`
Returns the number of items currently held.
- `bool isFull () const`
Checks whether the inventory has reached MAX_INVENTORY_SIZE.
- `Item * getItem (int index)`
Retrieves a mutable pointer to the item at the given position.

10.32.1 Detailed Description

Holds a bounded collection of items owned by a player. Enforces a maximum capacity (`MAX_INVENTORY_SIZE`) and provides index-based lookup, addition, and removal of items in insertion order.

10.32.2 Member Function Documentation

10.32.2.1 addItem()

```
bool Inventory::addItem (
    const Item & item)
```

Adds an item to the inventory if space is available.

Parameters

<i>item</i>	The item to copy and append to the collection.
-------------	--

Returns

true if the item was added; false if the inventory is already full (see isFull).

10.32.2.2 getItem()

```
Item * Inventory::getItem (
    int index)
```

Retrieves a mutable pointer to the item at the given position.

Parameters

<i>index</i>	Zero-based position of the item to retrieve.
--------------	--

Returns

Pointer to the item, or nullptr if index is out of range.

10.32.2.3 getItems()

```
const std::vector< Item > & Inventory::getItems () const
```

Provides read-only access to all held items.

Returns

Const reference to the underlying item collection.

10.32.2.4 getSize()

```
int Inventory::getSize () const
```

Returns the number of items currently held.

Returns

Current item count.

10.32.2.5 hasItem()

```
bool Inventory::hasItem (  
    ItemType type) const
```

Checks whether the inventory currently holds an item of the given type.

Parameters

<i>type</i>	The item type to search for.
-------------	------------------------------

Returns

true if at least one held item matches type.

10.32.2.6 isFull()

```
bool Inventory::isFull () const
```

Checks whether the inventory has reached MAX_INVENTORY_SIZE.

Returns

true if no more items can be added via addItem.

10.32.2.7 removeItem()

```
bool Inventory::removeItem (  
    int index)
```

Removes the item at the given position.

Parameters

<i>index</i>	Zero-based position of the item to remove.
--------------	--

Returns

true if the item was removed; false if index is out of range.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.cpp](#)

10.33 Item Class Reference

A single item instance identified by its [ItemType](#). Depending on the type, using it either overrides the current player's dice roll ([getDiceOverride\(\)](#)) or applies an effect to the current and/or a target player ([applyItemEffects\(\)](#)).

```
#include <item.hpp>
```

Public Member Functions

- **Item** ()=default
Constructs an item with a default-initialized (indeterminate) type.
- **Item** ([ItemType](#) type)
Constructs an item of the given type.
- **ItemType** [getItemType](#) () const
Returns this item's type.
- bool [isDiceItem](#) () const
Returns whether this item overrides the dice roll instead of a targeted effect (true for DoubleDice, TripleDice, and CasinoDice).
- bool [requiresTarget](#) () const
Returns whether this item's effect needs a target player to be chosen (true for SwapPosition, InvertedControls, and StealDice).
- int [rollSingle](#) ()
Rolls a single die.
- int [rollSum](#) (int count)
Rolls count dice and sums the results.
- std::optional< int > [getDiceOverride](#) ([Player](#) ¤tPlayer)
Computes the dice-roll value this item substitutes for a normal roll.
- void [applyItemEffects](#) ([Player](#) ¤tPlayer, [Player](#) *targetPlayer, const [Board](#) &board)
Applies this item's effect to the current and/or target player.

10.33.1 Detailed Description

A single item instance identified by its [ItemType](#). Depending on the type, using it either overrides the current player's dice roll ([getDiceOverride\(\)](#)) or applies an effect to the current and/or a target player ([applyItemEffects\(\)](#)).

10.33.2 Constructor & Destructor Documentation

10.33.2.1 Item()

```
Item::Item (  
    ItemType type) [inline]
```

Constructs an item of the given type.

Parameters

<i>type</i>	The effect this item represents.
-------------	----------------------------------

10.33.3 Member Function Documentation

10.33.3.1 applyItemEffects()

```
void Item::applyItemEffects (
    Player & currentPlayer,
    Player * targetPlayer,
    const Board & board)
```

Applies this item's effect to the current and/or target player.

Parameters

<i>currentPlayer</i>	The player who used the item.
<i>targetPlayer</i>	The chosen target player, or nullptr if none was selected; required for SwapPosition, InvertedControls, and StealDice (the effect is skipped if null for those types).
<i>board</i>	Forwarded to swapPosition(); currently unused there.

10.33.3.2 getDiceOverride()

```
std::optional< int > Item::getDiceOverride (
    Player & currentPlayer) [nodiscard]
```

Computes the dice-roll value this item substitutes for a normal roll.

Parameters

<i>currentPlayer</i>	The player about to roll; currently unused by the implementation.
----------------------	---

Returns

For DoubleDice/TripleDice, the sum of 2/3 rolls; for StealDice, a single roll; for CasinoDice, a scaled roll (see the .cpp for the exact ranges); for all other item types, std::nullopt (no override).

Note

CasinoDice scales a single roll: values above 5 are doubled, values from 2 to 5 are halved (integer division), and a value of 1 is returned unchanged; together these three conditions cover the Dice class's full [1, 10] output range. The final `casino == 6` branch is unreachable dead code, since `casino > 5` already matches 6. This case block has no break, so it relies on one of its branches always returning; if none did, it would fall through into the StealDice case below.

10.33.3.3 isDiceItem()

```
bool Item::isDiceItem () const
```

Returns whether this item overrides the dice roll instead of a targeted effect (true for DoubleDice, TripleDice, and CasinoDice).

Returns whether this item overrides the dice roll instead of a targeted effect.

10.33.3.4 requiresTarget()

```
bool Item::requiresTarget () const
```

Returns whether this item's effect needs a target player to be chosen (true for SwapPosition, InvertedControls, and StealDice).

Returns whether this item's effect needs a target player to be chosen.

10.33.3.5 rollSingle()

```
int Item::rollSingle ()
```

Rolls a single die.

Returns

A random value from [Dice::throwDice\(\)](#) (range [1, 10]).

10.33.3.6 rollSum()

```
int Item::rollSum (  
    int count)
```

Rolls `count` dice and sums the results.

Parameters

<code>count</code>	Number of dice to roll.
--------------------	-------------------------

Returns

The sum of all individual rolls.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.cpp](#)

10.34 ItemData Struct Reference

Static catalog entry describing a purchasable item: display name, effect type, catalog id, and shop price.

```
#include <item.hpp>
```

Public Attributes

- `std::string name`
- `ItemType type {}`
- `int id = 0`
- `int price = 0`

10.34.1 Detailed Description

Static catalog entry describing a purchasable item: display name, effect type, catalog id, and shop price.

The documentation for this struct was generated from the following file:

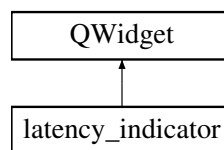
- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.hpp`

10.35 latency_indicator Class Reference

Visual indicator for measuring round-trip latency.

```
#include <latency_indicator.hpp>
```

Inheritance diagram for `latency_indicator`:



Public Slots

- `void flash ()`
Triggers the visual flash. Sets the square to white (if not already lit) and (re)arms the revert timer, so a burst of rapid calls keeps the square lit continuously rather than flickering. Intended to be connected to the `mind::net::udp_link::command_sent` signal; Qt's signal/slot mechanism ensures this runs on the GUI thread even when the signal is emitted from a non-GUI thread.

Public Member Functions

- `latency_indicator (QWidget *parent=nullptr)`
Constructs the latency indicator widget.

10.35.1 Detailed Description

Visual indicator for measuring round-trip latency.

Renders a square (200x200 px minimum size) that flashes white whenever a packet is sent from the desktop to the robot via UDP, then reverts to black after a short idle window. Designed to be visible in a high-FPS screen recording so the flash timing can be used to measure system latency externally.

10.35.2 Constructor & Destructor Documentation

10.35.2.1 latency_indicator()

```
latency_indicator::latency_indicator (
    QWidget * parent = nullptr) [explicit]
```

Constructs the latency indicator widget.

Parameters

<i>parent</i>	The parent widget, usually the main application window.
---------------	---

10.35.3 Member Function Documentation

10.35.3.1 flash

```
void latency_indicator::flash () [slot]
```

Triggers the visual flash. Sets the square to white (if not already lit) and (re)arms the revert timer, so a burst of rapid calls keeps the square lit continuously rather than flickering. Intended to be connected to the [mind::net::udp_link::command_sent](#) signal; Qt's signal/slot mechanism ensures this runs on the GUI thread even when the signal is emitted from a non-GUI thread.

Triggers the visual flash.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/latency_indicator.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/latency_indicator.cpp](#)

10.36 LogEntryExit Struct Reference

RAII helper that records a start timestamp on construction and emits a Trace-level "EXIT - <seconds>s" log message (via a temporary [LogStream](#)) on destruction.

```
#include <Logger.hpp>
```

Public Member Functions

- [LogEntryExit](#) (const char *function_name, bool should_log=true)
Records the current time so the destructor can compute how long the traced function ran.
- [~LogEntryExit](#) ()
Computes the elapsed time since construction and emits a Trace-level "EXIT - <seconds>s" log entry.

10.36.1 Detailed Description

RAII helper that records a start timestamp on construction and emits a Trace-level "EXIT - <seconds>s" log message (via a temporary [LogStream](#)) on destruction.

Note

Used by LOG_ENTRY_EXIT to measure the wall-clock duration of the enclosing scope.

10.36.2 Constructor & Destructor Documentation

10.36.2.1 LogEntryExit()

```
LogEntryExit::LogEntryExit (
    const char * function_name,
    bool should_log = true) [explicit]
```

Records the current time so the destructor can compute how long the traced function ran.

Parameters

<i>function_name</i>	Name of the function being traced (typically CURRENT_FUNCTION), reused for the exit log message.
----------------------	--

The documentation for this struct was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/[Logger.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/[Logger.cpp](#)

10.37 Logger Class Reference

Singleton that owns a background thread which asynchronously drains queued log messages.

```
#include <Logger.hpp>
```

Public Member Functions

- **Logger** (const [Logger](#) &)=delete
- [Logger](#) & **operator=** (const [Logger](#) &)=delete
- **Logger** ([Logger](#) &&)=delete
- [Logger](#) & **operator=** ([Logger](#) &&)=delete
- template<typename... Levels>
requires (std::is_same_v<Levels, [LogLevel](#)> && ...)
void [setLoggingLevel](#) (Levels... to_log)
Configures which logging severity levels are enabled.
- bool [isLevelEnabled](#) ([LogLevel](#) level) const
Checks whether a specific log level is currently permitted to log.
- void [push](#) (std::string &&msg)
Adds a message to the logging queue.
- ~**Logger** ()
Signals the worker thread to stop, wakes it, and joins it before destruction completes.

Static Public Member Functions

- static [Logger](#) & [getInstance](#) ()
Access the singleton instance.

Public Attributes

- std::atomic< bool > **is_logging_enabled_** {true}
flag indicating whether the [Logger](#) is enabled

10.37.1 Detailed Description

Singleton that owns a background thread which asynchronously drains queued log messages.

Note

Every message is appended to "logs.log" (path resolved relative to the process's current working directory at the time the worker thread opens it). ENTRY/EXIT trace markers produced by LOG_ENTRY_EXIT are written to that file but filtered out of stdout to keep the console readable.

10.37.2 Member Function Documentation

10.37.2.1 [getInstance\(\)](#)

```
Logger & Logger::getInstance () [inline], [static]
```

Access the singleton instance.

Returns

[Logger](#)& Reference to the global logger.

10.37.2.2 [isLevelEnabled\(\)](#)

```
bool Logger::isLevelEnabled (  
    LogLevel level) const
```

Checks whether a specific log level is currently permitted to log.

Parameters

<i>level</i>	The LogLevel severity to check.
--------------	---

Returns

true If the given level is explicitly enabled, or if no filters have been set (meaning all levels are enabled).
false If filters are active and the specified level is not among them.

•

Note

Thread-safe: Acquires a lock on the internal mutex `m_` to safely inspect the underlying filter set.

10.37.2.3 push()

```
void Logger::push (
    std::string && msg)
```

Adds a message to the logging queue.

Adds a message to the logging queue and wakes the worker thread.

Parameters

<i>msg</i>	The formatted log string to record.
------------	-------------------------------------

Note

Thread-safe: locks internally and wakes the worker thread via the condition variable.

10.37.2.4 setLoggingLevel()

```
template<typename... Levels>
requires (std::is_same_v<Levels, LogLevel> && ...)
void Logger::setLoggingLevel (
    Levels... to_log) [inline]
```

Configures which logging severity levels are enabled.

Clears any previously tracked logging levels and populates the filter with the provided arguments. If the resulting set is empty, all logging levels are treated as enabled by default.

•

Template Parameters

<i>Levels</i>	Variadic template parameter pack enforcing that all passed arguments are exactly of type LogLevel .
---------------	---

Parameters

<i>to_log</i>	A parameter pack of LogLevel enums to explicitly enable.
---------------	--

Note

- Thread-safe: Acquires a lock on the internal mutex `m_` before modifying the active log level filter set.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/Logger.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/Logger.cpp](#)

10.38 LogStream Struct Reference

RAII helper that accumulates a log message via operator<< and forwards the accumulated text to the [Logger](#) when it goes out of scope.

```
#include <Logger.hpp>
```

Public Member Functions

- `std::stringstream & getStream ()`
Access the internal stream buffer.
- `LogStream (LogLevel log_level, std::thread::id thread_id, const char *function_name)`
Captures the severity, thread id, and originating function for this log entry.
- `~LogStream ()`
Forwards the accumulated stream text to [Logger::push\(\)](#) so it is queued for the worker thread.
- `template<typename T>
LogStream & operator<< (const T &value)`
Operator to append data to the log stream.

10.38.1 Detailed Description

RAII helper that accumulates a log message via operator<< and forwards the accumulated text to the [Logger](#) when it goes out of scope.

Note

The severity, thread id, and function name passed to the constructor are stored on the instance but are not currently read anywhere afterwards; only the streamed text ends up in the message handed to [Logger::push\(\)](#).

10.38.2 Constructor & Destructor Documentation

10.38.2.1 [LogStream\(\)](#)

```
LogStream::LogStream (  
    LogLevel log_level,  
    std::thread::id thread_id,  
    const char * function_name)
```

Captures the severity, thread id, and originating function for this log entry.

Parameters

<i>log_level</i>	Severity tag for this message (stored only; see class
------------------	---

Note

above).

Parameters

<i>thread_id</i>	ID of the calling thread.
<i>function_name</i>	Name of the function that issued the log call (typically CURRENT_FUNCTION).

10.38.2.2 ~LogStream()

```
LogStream::~~LogStream ()
```

Forwards the accumulated stream text to [Logger::push\(\)](#) so it is queued for the worker thread.

Forwards the accumulated stream text to [Logger::push\(\)](#).

10.38.3 Member Function Documentation**10.38.3.1 getStream()**

```
std::stringstream & LogStream::getStream ()
```

Access the internal stream buffer.

Returns

std::stringstream& Reference to the stream.

10.38.3.2 operator<<()

```
template<typename T>
LogStream & LogStream::operator<< (
    const T & value) [inline]
```

Operator to append data to the log stream.

Template Parameters

<i>T</i>	Type of the data to append.
----------	-----------------------------

Parameters

<i>value</i>	The value to log.
--------------	-------------------

Returns

[LogStream](#)& Reference to this instance for chaining.

The documentation for this struct was generated from the following files:

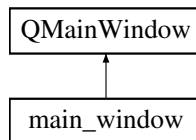
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/[Logger.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/[Logger.cpp](#)

10.39 main_window Class Reference

Main race-monitor window: owns the camera `marker_tracker`, runs the ArUco detection loop on a timer, performs extrinsic calibration via `solvePnP`, and publishes anchor + robot positions to MQTT and to any connected TCP clients.

```
#include <main_window.hpp>
```

Inheritance diagram for `main_window`:



Signals

- void **marker_positions_updated** (const std::vector< [marker_tracker::marker_position](#) > &markers)
Publishes the latest raw marker batch to other GUI consumers.
- void **tracking_state_changed** (bool active)
Reports whether the shared camera feed is currently active.
- void **frame_updated** (const QImage &img)
Publishes the latest camera frame as a QImage.

Public Member Functions

- [main_window](#) ([Server](#) &server, QWidget *parent=nullptr)
Constructs the race-monitor window and connects to the local MQTT broker. Builds the widget tree (status label, video view, start/calibrate/Splatton buttons), wires up the update timer and marker tracker, and opens a mosquitto connection to 127.0.0.1:1883 (the broker is expected to run on this same desktop).
- virtual ~**main_window** () override
Disconnects from the MQTT broker and releases the mosquitto client/library.
- void [set_fleet](#) ([mind::fleet::robot_fleet](#) *fleet)
Borrows the robot fleet owned by [control_window](#) so the Splatton drive lambda can reach `robot_controller::set_↔drive()`.

10.39.1 Detailed Description

Main race-monitor window: owns the camera `marker_tracker`, runs the ArUco detection loop on a timer, performs extrinsic calibration via `solvePnP`, and publishes anchor + robot positions to MQTT and to any connected TCP clients.

Note

Connects to the MQTT broker on construction (127.0.0.1:1883 - mosquitto runs on this same desktop).

10.39.2 Constructor & Destructor Documentation

10.39.2.1 main_window()

```
main_window::main_window (
    Server & server,
    QWidget * parent = nullptr) [explicit]
```

Constructs the race-monitor window and connects to the local MQTT broker. Builds the widget tree (status label, video view, start/calibrate/Splatton buttons), wires up the update timer and marker tracker, and opens a mosquitto connection to 127.0.0.1:1883 (the broker is expected to run on this same desktop).

Constructs the race-monitor window and connects to the local MQTT broker.

Parameters

<i>parent</i>	Optional Qt parent widget; ownership follows normal Qt rules.
---------------	---

10.39.3 Member Function Documentation

10.39.3.1 set_fleet()

```
void main_window::set_fleet (
    mind::fleet::robot\_fleet * fleet)
```

Borrows the robot fleet owned by [control_window](#) so the Splatton drive lambda can reach `robot_controller::set_↔drive()`.

Borrows the robot fleet owned by [control_window](#).

Parameters

<i>fleet</i>	Non-owning: control_window keeps ownership; both windows live for the whole app (see app/main.cpp).
--------------	--

The documentation for this class was generated from the following files:

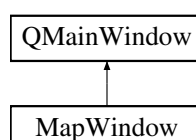
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.cpp](#)

10.40 MapWindow Class Reference

Live tracking + driving Qt window for up to 4 robots. Owns the board node graph, the ArUco `marker_tracker`, the `collision_planner`, and one `path_follower` per robot slot. Runs in either an offline kinematic simulation (default, no camera required) or LIVE camera mode (toggled via the toolbar); a separate ARM DRIVE toggle independently gates whether commands reach real EV3 bricks over the injected `robot_fleet`. Ticks every 33 ms via an internal `QTimer` (see `onTrackerTick`).

```
#include <mapwindow.hpp>
```

Inheritance diagram for MapWindow:



Signals

- void [robotArrived](#) (int robot_id, int node_id)
Fired when a robot physically arrives at its target node. [Game](#) logic connects to this to continue the turn.

Public Member Functions

- [MapWindow](#) ([UdpServer](#) &udp_server, [Server](#) &tcp_server, QWidget *parent=nullptr)
Builds the window, spawns the 4 robots at well-separated corner nodes, restores any persisted follower tune, wires up the toolbar (LIVE MODE / Tune / Auto-tune / ARM DRIVE / PANIC STOP / R1 ONLY), installs the central canvas, and starts the 33 ms tick timer.
- virtual [~MapWindow](#) () override
Stops the tick timer; does not otherwise release any real robot (see panicStopAll).
- bool [initializeCamera](#) ()
Reports whether the shared tracking camera is active.
- void [updateRobotPosition](#) (int robot_id, cv::Point2f camera_pixel, float marker_angle)
Converts a camera pixel to a screen position and updates the given robot's rendering state.
- void [sendRobotToNode](#) (int robot_id, int target_node_id)
Computes the distance and turn angle from the robot to a target node and logs them.
- const [Board](#) & [getBoard](#) () const
Provides read-only access to the board graph for game logic and player classes.
- int [getRobotNode](#) (int robot_id) const
Returns the node id the robot is currently near.
- void [planPathFor](#) (int robot_id, int target_node_id)
Builds (and then sustains) a collision-free straight-line path for a robot to a target node. Computes the path only via collision_planner::request_path_to - it does NOT command the robot; a driver consumes the path (see [collision::path_update](#)).
- void [movePlayboard](#) (int robot_id, int target_node_id)
Moves a robot to target_node ALONG the playboard graph edges (not a straight free-space line). The planner routes node-to-node, detours parked robots, and shoves a robot aside if it is squatting the target. Computes and sustains the path only via collision_planner::request_route_to - it does NOT command the robot.
- [collision::collision_planner](#) & [collisionPlanner](#) ()
Access to the collision planner (e.g. to register a path sink / pull effective paths from game logic).
- void [setFleet](#) ([mind::fleet::robot_fleet](#) *fleet)
Injects the robot fleet so the driver can command real robots via robot_controller::set_drive.
- void [invalidateBoardCache](#) ()
Invalidates the board cache so it is rebuilt on next paint.
- void [setTrackingMarkers](#) (const std::vector< [marker_tracker::marker_position](#) > &markers)
Accepts the latest marker batch from [main_window](#)'s shared camera feed.
- void [setTrackingState](#) (bool active)
Enables or disables the shared live tracking feed.
- void [paintBoard](#) (QPainter &painter)
Draws the full board (background, grid, node graph, nodes, robots, collision overlay).
- void [boardClicked](#) (QPoint pos, Qt::MouseButton button)
Handles a click on the board canvas: first click selects the nearest robot, a subsequent click on (near) a node drives the selected robot there via movePlayboard.
- void [startCalibration](#) ()
Auto-calibrates the selected robot: commands a known turn + drive, measures the actual result via the tracked/↔ simulated pose, and learns drive_scale/turn_scale so future commands hit their target. Runs many alternating-direction passes per axis (see runCalibrationStep) and persists the result via saveFollowerParams on completion.
- [follower::follower_params](#) [followerParams](#) () const noexcept
The current shared follower gains (for inspection / tests).

- bool **isSimMode** () const noexcept
- bool **isDrivingEnabled** () const noexcept
- bool **isCameraReady** () const noexcept
- bool **isHomographyReady** () const noexcept
- int **getTargetNode** (int id) const noexcept
- const std::vector< [Robot](#) > & **getRobots** () const noexcept
- const std::array< [follower::path_follower](#), 4 > & **getFollowers** () const noexcept
- [mind::fleet::robot_fleet](#) * **getFleet** () const noexcept

10.40.1 Detailed Description

Live tracking + driving Qt window for up to 4 robots. Owns the board node graph, the ArUco marker_tracker, the collision_planner, and one path_follower per robot slot. Runs in either an offline kinematic simulation (default, no camera required) or LIVE camera mode (toggled via the toolbar); a separate ARM DRIVE toggle independently gates whether commands reach real EV3 bricks over the injected robot_fleet. Ticks every 33 ms via an internal QTimer (see onTrackerTick).

Note

Coordinate pipeline: camera pixels -> physical centimetres (homography_) -> screen pixels (scale_x_/↔ scale_y_). The collision planner and path followers work entirely in board-frame centimetres; only rendering/↔ hit-testing uses screen pixels.

10.40.2 Constructor & Destructor Documentation

10.40.2.1 MapWindow()

```
MapWindow::MapWindow (
    UdpServer & udp_server,
    Server & tcp_server,
    QWidget * parent = nullptr) [explicit]
```

Builds the window, spawns the 4 robots at well-separated corner nodes, restores any persisted follower tune, wires up the toolbar (LIVE MODE / Tune / Auto-tune / ARM DRIVE / PANIC STOP / R1 ONLY), installs the central canvas, and starts the 33 ms tick timer.

Builds the window, spawns the robots, restores the follower tune, wires up the toolbar, and starts the tick timer.

Parameters

<i>parent</i>	Optional owning widget.
---------------	-------------------------

10.40.2.2 ~MapWindow()

```
MapWindow::~~MapWindow () [override], [virtual]
```

Stops the tick timer; does not otherwise release any real robot (see panicStopAll).

Stops the tick timer.

10.40.3 Member Function Documentation

10.40.3.1 boardClicked()

```
void MapWindow::boardClicked (
    QPoint pos,
    Qt::MouseButton button)
```

Handles a click on the board canvas: first click selects the nearest robot, a subsequent click on (near) a node drives the selected robot there via movePlayboard.

Handles a click on the board canvas: first click selects the nearest robot, a subsequent click on (near) a node drives it there.

Parameters

<i>pos</i>	Click position in board pixels.
<i>button</i>	Mouse button pressed; only Qt::LeftButton is handled.

10.40.3.2 collisionPlanner()

```
collision::collision_planner & MapWindow::collisionPlanner () [inline]
```

Access to the collision planner (e.g. to register a path sink / pull effective paths from game logic).

Note

Lives here because [MapWindow](#) owns the board frame + every robot's live position.

10.40.3.3 getRobotNode()

```
int MapWindow::getRobotNode (
    int robot_id) const
```

Returns the node id the robot is currently near.

Parameters

<i>robot↔ _id</i>	Robot slot (0-3).
-----------------------	-----------------------------------

Returns

The node id, or -1 if the robot is not near any node or robot_id is out of range.

10.40.3.4 initializeCamera()

```
bool MapWindow::initializeCamera ()
```

Reports whether the shared tracking camera is active.

Reports whether [main_window](#)'s shared tracking camera is active.

Returns

True after [main_window](#) has successfully started the camera.

10.40.3.5 movePlayboard()

```
void MapWindow::movePlayboard (
    int robot_id,
    int target_node_id)
```

Moves a robot to `target_node` ALONG the playboard graph edges (not a straight free-space line). The planner routes node-to-node, detours parked robots, and shoves a robot aside if it is squatting the target. Computes and sustains the path only via `collision_planner::request_route_to` - it does NOT command the robot.

Moves a robot to `target_node` ALONG the playboard graph edges (not a straight free-space line).

Parameters

<i>robot_id</i>	Robot slot (0-3).
<i>target_node</i> ↔ <i>_id</i>	Destination node id.

10.40.3.6 paintBoard()

```
void MapWindow::paintBoard (
    QPainter & painter)
```

Draws the full board (background, grid, node graph, nodes, robots, collision overlay).

Parameters

<i>painter</i>	Painter bound to the canvas; coordinates drawn are board pixels (1400x860).
----------------	---

Note

Rendering + input live on a central canvas widget (a `QMainWindow` with a toolbar but no central widget has an unreliable paint/mouse frame); `MapCanvas` forwards its `paintEvent` here.

10.40.3.7 planPathFor()

```
void MapWindow::planPathFor (
    int robot_id,
    int target_node_id)
```

Builds (and then sustains) a collision-free straight-line path for a robot to a target node. Computes the path only via `collision_planner::request_path_to` - it does NOT command the robot; a driver consumes the path (see [collision::path_update](#)).

Builds (and then sustains) a collision-free straight-line path for a robot to a target node.

Parameters

<i>robot_id</i>	Robot slot (0-3).
<i>target_node_id</i>	Destination node id.

10.40.3.8 robotArrived

```
void MapWindow::robotArrived (
    int robot_id,
    int node_id) [signal]
```

Fired when a robot physically arrives at its target node. [Game](#) logic connects to this to continue the turn.

Parameters

<i>robot_id</i>	Robot slot (0-3) that arrived.
<i>node_id</i>	The node it arrived at.

10.40.3.9 sendRobotToNode()

```
void MapWindow::sendRobotToNode (
    int robot_id,
    int target_node_id)
```

Computes the distance and turn angle from the robot to a target node and logs them.

Parameters

<i>robot_id</i>	Robot slot (0-3).
<i>target_node_id</i>	Destination node id.

Note

Latches `target_node_[robot_id]` so `checkArrival` knows what to wait for. Does NOT currently publish anything to the robot (the MQTT dispatch is a TODO in the implementation); real driving goes through `movePlayboard/driveRobots` instead.

10.40.3.10 setFleet()

```
void MapWindow::setFleet (
    mind::fleet::robot_fleet * fleet) [inline]
```

Injects the robot fleet so the driver can command real robots via robot_controller::set_drive.

Parameters

<i>fleet</i>	Non-owning pointer to the fleet; without it (or before the camera is up) MapWindow runs the offline sim only.
--------------	---

10.40.3.11 setTrackingMarkers()

```
void MapWindow::setTrackingMarkers (
    const std::vector< marker_tracker::marker_position > & markers)
```

Accepts the latest marker batch from [main_window](#)'s shared camera feed.

Stores the latest marker batch from [main_window](#)'s shared camera feed.

Parameters

<i>markers</i>	Raw marker detections from the current camera frame.
----------------	--

10.40.3.12 setTrackingState()

```
void MapWindow::setTrackingState (
    bool active)
```

Enables or disables the shared live tracking feed.

Parameters

<i>active</i>	True while main_window owns an active camera capture.
---------------	---

10.40.3.13 startCalibration()

```
void MapWindow::startCalibration ()
```

Auto-calibrates the selected robot: commands a known turn + drive, measures the actual result via the tracked/↔ simulated pose, and learns drive_scale/turn_scale so future commands hit their target. Runs many alternating-direction passes per axis (see runCalibrationStep) and persists the result via saveFollowerParams on completion.

Auto-calibrates the selected robot: commands a known turn + drive, measures the actual result, and learns drive↔_scale/turn_scale.

10.40.3.14 updateRobotPosition()

```
void MapWindow::updateRobotPosition (
    int robot_id,
    cv::Point2f camera_pixel,
    float marker_angle)
```

Converts a camera pixel to a screen position and updates the given robot's rendering state.

Parameters

<i>robot_id</i>	Robot slot (0-3) to update.
<i>camera_pixel</i>	Marker position in raw camera pixel coordinates.
<i>marker_angle</i>	Robot heading in radians from ArUco detection (image-space; stored in Robot::angle).

Note

No-op if homography_ready_ is false or robot_id is out of range. Also snaps [Robot::current_node](#) to the nearest board node and requests a repaint.

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/[mapwindow.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/[mapwindow.cpp](#)

10.41 marker_tracker::marker_position Struct Reference

Result of one ArUco marker detection: identifier plus its pose in the current camera frame. Position and angle are derived directly from the raw detected corners in camera pixel space; converting to real-world floor coordinates (e.g. via homography/extrinsics) is left to the caller.

```
#include <tracker.hpp>
```

Public Attributes

- int **marker_id**
ArUco marker identifier.
- bool **detected**
Detection status flag.
- cv::Point2f **world_position**
Marker centre, in raw CAMERA pixel coordinates (despite the name, not converted to world units here).
- double **angle**
Orientation in radians (0 = up, clockwise positive).
- cv::Point2f **front_px** {0.f, 0.f}
Marker front-edge centre in CAMERA px (top_center).
- cv::Point2f **back_px** {0.f, 0.f}

10.41.1 Detailed Description

Result of one ArUco marker detection: identifier plus its pose in the current camera frame. Position and angle are derived directly from the raw detected corners in camera pixel space; converting to real-world floor coordinates (e.g. via homography/extrinsics) is left to the caller.

10.41.2 Member Data Documentation

10.41.2.1 back_px

```
cv::Point2f marker_tracker::marker_position::back_px {0.f, 0.f}
```

Marker back-edge centre in CAMERA px (bottom_center) front_px/back_px give a board-frame heading via the homography (atan2 in cm), robust to camera skew unlike the image-space [angle](#).

The documentation for this struct was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.hpp

10.42 marker_tracker::marker_tracker Class Reference

Owns one camera device and the ArUco detection pipeline for that camera. Wraps a cv::VideoCapture plus a custom 12-marker ArUco dictionary/detector, and exposes the camera intrinsic calibration (shared via static members) used to interpret detected marker poses.

```
#include <tracker.hpp>
```

Public Member Functions

- [marker_tracker](#) ()
Constructs the tracker without opening a camera. Sets up the ArUco dictionary/detector immediately; a camera must still be opened separately via [initialize_camera\(\)](#) (or one of its variants) before [detect_markers\(\)](#)/[get_latest_frame\(\)](#) can be used.
- [~marker_tracker](#) ()
Destroys the tracker, releasing the camera device if open.
- bool [initialize_camera](#) (int camera_index=0)
Opens the camera at the given device index and configures it for tracking. Opens via V4L2 on Linux / DirectShow (CAP_DSHOW) on Windows for reliable manual control, then requests a fixed resolution/FPS and forces manual exposure, brightness and focus. Property `set()` failures are logged but not fatal, since some devices (e.g. v4l2loopback) have fixed formats and report EBUSY on set.
- bool [initialize_camera_auto](#) (const std::string &name_substr="EOS Webcam Utility", int fallback_index=0)
Auto-detect the tracking camera by its device name and initialize it.
- bool [reinitialize](#) (int camera_index)
Reinitialize with new camera, releasing previous.
- void [shutdown_camera](#) ()
Releases the camera device, if currently open, and clears the initialized flag. Safe to call when no camera is open.
- std::vector< [marker_position](#) > [detect_markers](#) ()
Detects ArUco markers in the frame last captured by [get_latest_frame\(\)](#).
- cv::Mat [get_latest_frame](#) ()
Blocks on the camera device for the next frame and caches it.
- void [load_calibration](#) (const cv::Mat &camera_matrix, const cv::Mat &dist_coeffs)
Overwrites the shared camera intrinsic calibration parameters.
- int [get_camera_index](#) () const
Get currently active camera index.
- const cv::Mat & [get_current_frame](#) () const
Get most recent captured frame.

Static Public Member Functions

- static std::vector< int > [get_valid_robot_marker_ids](#) ()
Returns the marker IDs reserved for robots (as opposed to board corner markers).
- static int [robot_slot_for_marker](#) (int marker_id)
Maps a physical ArUco marker ID to the logical robot slot.
- static int [find_camera_index_by_name](#) (const std::string &name_substr)
Finds the camera device index whose name contains name_substr.
- static const cv::Mat & [get_camera_matrix](#) ()
Get the shared camera intrinsic matrix used to interpret marker detections.
- static const cv::Mat & [get_dist_coeffs](#) ()
Get the shared lens distortion coefficients paired with [get_camera_matrix\(\)](#).

10.42.1 Detailed Description

Owns one camera device and the ArUco detection pipeline for that camera. Wraps a cv::VideoCapture plus a custom 12-marker ArUco dictionary/detector, and exposes the camera intrinsic calibration (shared via static members) used to interpret detected marker poses.

10.42.2 Constructor & Destructor Documentation

10.42.2.1 marker_tracker()

```
marker_tracker::marker_tracker::marker_tracker ()
```

Constructs the tracker without opening a camera. Sets up the ArUco dictionary/detector immediately; a camera must still be opened separately via [initialize_camera\(\)](#) (or one of its variants) before [detect_markers\(\)](#)/[get_latest_frame\(\)](#) can be used.

Constructs the tracker without opening a camera.

10.42.3 Member Function Documentation

10.42.3.1 detect_markers()

```
std::vector< marker_position > marker_tracker::marker_tracker::detect_markers ()
```

Detects ArUco markers in the frame last captured by [get_latest_frame\(\)](#).

Returns

Detected marker positions; empty if no frame has been captured yet (current_frame_ is empty) or the detector was not created.

Note

Side effect: draws the detected marker outlines/IDs onto current_frame_ for visualization (via cv::aruco::drawDetectedMarkers).

10.42.3.2 find_camera_index_by_name()

```
int marker_tracker::marker_tracker::find_camera_index_by_name (
    const std::string & name_substr) [static]
```

Finds the camera device index whose name contains name_substr.

Returns

Lowest matching device index, or -1 if none found

Note

On Linux, scans /sys/class/video4linux/videoN/name. On Windows, enumerates DirectShow video capture devices via ICreateDevEnum and matches FriendlyName. On other platforms, always returns -1.

10.42.3.3 get_camera_index()

```
int marker_tracker::marker_tracker::get_camera_index () const [inline]
```

Get currently active camera index.

Returns

Index passed to the most recent successful [initialize_camera\(\)](#)/[reinitialize\(\)](#) call

10.42.3.4 get_camera_matrix()

```
const cv::Mat & marker_tracker::marker_tracker::get_camera_matrix () [inline], [static]
```

Get the shared camera intrinsic matrix used to interpret marker detections.

Returns

3x3 camera matrix; the hard-coded default (see [tracker.cpp](#)) unless [load_calibration\(\)](#) has overwritten it.

10.42.3.5 get_current_frame()

```
const cv::Mat & marker_tracker::marker_tracker::get_current_frame () const [inline]
```

Get most recent captured frame.

Returns

Reference to the frame last produced by [get_latest_frame\(\)](#); may be empty if no frame has been captured yet.

10.42.3.6 get_dist_coeffs()

```
const cv::Mat & marker_tracker::marker_tracker::get_dist_coeffs () [inline], [static]
```

Get the shared lens distortion coefficients paired with [get_camera_matrix\(\)](#).

Returns

Distortion coefficients (k1, k2, p1, p2, k3); the hard-coded default (see [tracker.cpp](#)) unless [load_calibration\(\)](#) has overwritten it.

10.42.3.7 get_latest_frame()

```
cv::Mat marker_tracker::marker_tracker::get_latest_frame ()
```

Blocks on the camera device for the next frame and caches it.

Returns

A clone of the newly captured frame; an empty cv::Mat if the camera is not initialized/open or the capture failed (logged as an error).

Note

Deliberately does not undistort per frame: the board homography used downstream is fit in distorted pixel space and already absorbs lens distortion, so undistorting here would only cost time on the ~30 Hz loop.

10.42.3.8 get_valid_robot_marker_ids()

```
std::vector< int > marker_tracker::marker_tracker::get_valid_robot_marker_ids () [static]
```

Returns the marker IDs reserved for robots (as opposed to board corner markers).

Returns

The robot marker IDs in [MapWindow](#) slot order: {0, 1, 2, 3}.

10.42.3.9 initialize_camera()

```
bool marker_tracker::marker_tracker::initialize_camera (
    int camera_index = 0)
```

Opens the camera at the given device index and configures it for tracking. Opens via V4L2 on Linux / DirectShow (CAP_DSHOW) on Windows for reliable manual control, then requests a fixed resolution/FPS and forces manual exposure, brightness and focus. Property `set()` failures are logged but not fatal, since some devices (e.g. v4l2loopback) have fixed formats and report EBUSY on set.

Opens the camera at the given device index and configures it for tracking.

Parameters

<i>camera_index</i>	Camera device index (default: 2)
---------------------	----------------------------------

Returns

True if the device opened successfully; false if it could not be opened at all

Note

On the very first successful open, if the camera's actual resolution differs from the resolution the static intrinsics were calibrated at, the shared `camera_matrix_` is rescaled in place to match (once only, across all instances).

10.42.3.10 initialize_camera_auto()

```
bool marker_tracker::marker_tracker::initialize_camera_auto (
    const std::string & name_substr = "EOS Webcam Utility",
    int fallback_index = 0)
```

Auto-detect the tracking camera by its device name and initialize it.

Parameters

<i>name_substr</i>	Substring matched against camera device names (V4L2 name on Linux, DirectShow friendly name on Windows)
<i>fallback_index</i>	Index used if no match found; -1 means fail rather than grab an unrelated camera

Returns

True if initialization successful

10.42.3.11 load_calibration()

```
void marker_tracker::marker_tracker::load_calibration (
    const cv::Mat & camera_matrix,
    const cv::Mat & dist_coeffs)
```

Overwrites the shared camera intrinsic calibration parameters.

Parameters

<i>camera_matrix</i>	3x3 camera matrix
<i>dist_coeffs</i>	Distortion coefficients

Note

camera_matrix_/*dist_coeffs_* are static, so this replaces the default hard-coded calibration for every [marker_tracker](#) instance, not just this one.

10.42.3.12 reinitialize()

```
bool marker_tracker::marker_tracker::reinitialize (
    int camera_index)
```

Reinitialize with new camera, releasing previous.

Parameters

<i>camera_index</i>	New camera device index
---------------------	-------------------------

Returns

True if reinitialization successful

10.42.3.13 robot_slot_for_marker()

```
int marker_tracker::marker_tracker::robot_slot_for_marker (
    int marker_id) [static]
```

Maps a physical ArUco marker ID to the logical robot slot.

Parameters

<i>marker_id</i>	Detected ArUco marker ID.
------------------	---------------------------

Returns

The matching slot for robot marker IDs 0-3, or -1 when the marker is not assigned to a robot.

10.42.3.14 shutdown_camera()

```
void marker_tracker::marker_tracker::shutdown_camera ()
```

Releases the camera device, if currently open, and clears the initialized flag. Safe to call when no camera is open.

Releases the camera device, if currently open, and clears the initialized flag.

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.hpp
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.cpp

10.43 MessageTarget Class Reference

Placeholder for the class that owns the logic a message should trigger. In real use this is something like [GameManager](#) or [Board](#). Passed into [register_handlers\(\)](#) by reference and must outlive the server.

```
#include <handler_template.hpp>
```

Public Member Functions

- void [handle](#) (std::uint8_t player_index, int value)
Placeholder method an inbound message ends up calling. Give it whatever signature the feature needs; player_index and value stand in for the real message fields in this template.

10.43.1 Detailed Description

Placeholder for the class that owns the logic a message should trigger. In real use this is something like [GameManager](#) or [Board](#). Passed into [register_handlers\(\)](#) by reference and must outlive the server.

10.43.2 Member Function Documentation

10.43.2.1 handle()

```
void MessageTarget::handle (
    std::uint8_t player_index,
    int value)
```

Placeholder method an inbound message ends up calling. Give it whatever signature the feature needs; player_index and value stand in for the real message fields in this template.

Placeholder method an inbound message ends up calling.

Parameters

<i>player_index</i>	Index of the player the message concerns.
<i>value</i>	Placeholder payload value.

The documentation for this class was generated from the following files:

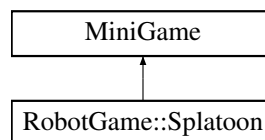
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/[handler_template.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/[handler_template.cpp](#)

10.44 MiniGame Class Reference

Abstract base class for all mini-games. Operates only on [MiniGameParticipant](#) - no dependency on [Player](#), [Board](#), or inventory. Results are returned via [MiniGameResult](#) and applied to real Players by [GameManager](#).

```
#include <minigame.hpp>
```

Inheritance diagram for MiniGame:

**Public Member Functions**

- virtual `~MiniGame()`=default
Virtual destructor; allows [GameManager](#) to own mini-games polymorphically.
- virtual `MiniGameResult play (std::vector< MiniGameParticipant > &participants)=0`
Runs the mini-game with all participants and returns the result.
- virtual `std::string getName () const =0`
Returns the display name of this mini-game.
- virtual `void onRobotMoved (int index, double x_cm, double y_cm)`
Called by [GameManager](#) whenever the tracker detects a robot has moved.

10.44.1 Detailed Description

Abstract base class for all mini-games. Operates only on [MiniGameParticipant](#) - no dependency on [Player](#), [Board](#), or inventory. Results are returned via [MiniGameResult](#) and applied to real Players by [GameManager](#).

10.44.2 Member Function Documentation

10.44.2.1 getName()

```
virtual std::string MiniGame::getName () const [pure virtual]
```

Returns the display name of this mini-game.

Implemented in [RobotGame::Splatoon](#).

10.44.2.2 onRobotMoved()

```
virtual void MiniGame::onRobotMoved (
    int index,
    double x_cm,
    double y_cm) [inline], [virtual]
```

Called by [GameManager](#) whenever the tracker detects a robot has moved.

Parameters

<i>index</i>	Identifies which robot/participant moved; the exact meaning (participant index vs. marker/robot ID) is defined by the calling context and interpreted by the overriding mini-game.
<i>x_cm</i>	Robot 's tracked X position, in centimeters.
<i>y_cm</i>	Robot 's tracked Y position, in centimeters.

Note

Default implementation is a no-op; mini-games that don't need position data can ignore this by not overriding it.

Reimplemented in [RobotGame::Splatoon](#).

10.44.2.3 play()

```
virtual MiniGameResult MiniGame::play (
    std::vector< MiniGameParticipant > & participants) [pure virtual]
```

Runs the mini-game with all participants and returns the result.

Parameters

<i>participants</i>	The players taking part; implementations read and may update per-participant state (e.g. coins) as the mini-game progresses.
---------------------	--

Returns

The outcome of the mini-game, including the (possibly updated) participants and the winning playerId.

Implemented in [RobotGame::Splatoon](#).

The documentation for this class was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame.hpp](#)

10.45 MiniGameParticipant Struct Reference

Data-transfer object representing a player during a mini-game. Created from a real [Player](#) before the mini-game starts. Mini-games read and mutate this copy in isolation from Player/Board/inventory; [GameManager](#) reconciles the (possibly changed) fields back into the real [Player](#) once play() returns.

```
#include <minigame_participant.hpp>
```

Public Attributes

- int **playerId** = -1
ID of the originating [Player](#); used by [GameManager](#) to match this participant back to its real [Player](#).
- int **robotId** = -1
ID of the physical robot associated with this player (currently mirrors playerId, see [GameManager::toParticipant](#)).
- std::string **name**
- int **coins** = 0
[Player](#)'s coin count, seeded from the real [Player](#) and diffed against it after play() to award/deduct coins.

10.45.1 Detailed Description

Data-transfer object representing a player during a mini-game. Created from a real [Player](#) before the mini-game starts. Mini-games read and mutate this copy in isolation from Player/Board/inventory; [GameManager](#) reconciles the (possibly changed) fields back into the real [Player](#) once play() returns.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_participant.hpp](#)

10.46 MiniGameResult Struct Reference

Returned by every mini-game after play() finishes. The board-game layer ([GameManager](#)) reads this and applies rewards to the real [Player](#) objects.

```
#include <minigame_result.hpp>
```

Public Attributes

- std::vector< [MiniGameParticipant](#) > **participants**
Updated state of all participants after the mini-game finished.
- int **winnerId** = -1
playerId of the winner, -1 if draw.

10.46.1 Detailed Description

Returned by every mini-game after play() finishes. The board-game layer ([GameManager](#)) reads this and applies rewards to the real [Player](#) objects.

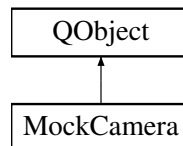
The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_result.hpp](#)

10.47 MockCamera Class Reference

Fakes the tracking camera pipeline: on a timer, publishes a fixed calibration anchor plus procedurally-animated robot and board-marker positions to MQTT, in the same JSON shape and on the same topics the real camera ([main_window](#)) uses.

Inheritance diagram for MockCamera:



Public Slots

- void [update](#) ()

Timer-driven tick: publishes one fixed anchor pose, 4 animated robot positions, and 6 fixed board-marker positions, mirroring the topics/payload shape of the real camera pipeline.

Public Member Functions

- [MockCamera](#) (const std::string &broker_address, int port)

Initializes the mosquitto library and connects to the given MQTT broker. Also starts the internal elapsed-time clock used to animate the fake robot positions in [update\(\)](#).

- [~MockCamera](#) ()

Disconnects from the broker and releases the mosquitto client and library.

10.47.1 Detailed Description

Fakes the tracking camera pipeline: on a timer, publishes a fixed calibration anchor plus procedurally-animated robot and board-marker positions to MQTT, in the same JSON shape and on the same topics the real camera ([main_window](#)) uses.

10.47.2 Constructor & Destructor Documentation

10.47.2.1 MockCamera()

```

MockCamera::MockCamera (
    const std::string & broker_address,
    int port) [inline]
  
```

Initializes the mosquitto library and connects to the given MQTT broker. Also starts the internal elapsed-time clock used to animate the fake robot positions in [update\(\)](#).

Parameters

<i>broker_address</i>	Hostname or IP address of the MQTT broker.
<i>port</i>	TCP port of the MQTT broker.

Note

Connection failures are only logged to stderr; construction still succeeds and [update\(\)](#) will keep calling `mosquitto_publish()` regardless.

10.47.3 Member Function Documentation**10.47.3.1 update**

```
void MockCamera::update () [inline], [slot]
```

Timer-driven tick: publishes one fixed anchor pose, 4 animated robot positions, and 6 fixed board-marker positions, mirroring the topics/payload shape of the real camera pipeline.

[Robot](#) positions (logical IDs 0-3, physical marker IDs 0-3) move on circles of increasing radius and speed around a common center, parameterized by elapsed wall-clock time since construction, so consumers see continuously moving targets.

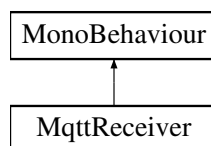
The documentation for this class was generated from the following file:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/mock_camera.cpp`

10.48 MqttReceiver Class Reference

Receives anchor and robot position data from the Desktop App via MQTT.

Inheritance diagram for MqttReceiver:

**Classes**

- class [AnchorData](#)
- class [RobotPositionData](#)

Events

- Action< [AnchorData](#) > **OnAnchorReceived**
- Action< [RobotPositionData](#) > **OnRobotPositionReceived**

10.48.1 Detailed Description

Receives anchor and robot position data from the Desktop App via MQTT.

The documentation for this class was generated from the following file:

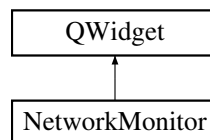
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/MqttReceiver.cs

10.49 NetworkMonitor Class Reference

Casino-themed Network Monitor showing TCP server sessions, tracking camera state, homography state, and per-robot fleet connectivity. Refreshes at 4 Hz via an internal QTimer.

```
#include <network_monitor.hpp>
```

Inheritance diagram for NetworkMonitor:



Public Member Functions

- **NetworkMonitor** ([MapWindow](#) &mapwindow, [UdpServer](#) &udp_server, std::shared_ptr< [SessionRegistry](#) > registry, QWidget *parent=nullptr)

10.49.1 Detailed Description

Casino-themed Network Monitor showing TCP server sessions, tracking camera state, homography state, and per-robot fleet connectivity. Refreshes at 4 Hz via an internal QTimer.

The documentation for this class was generated from the following files:

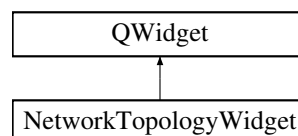
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/[network_monitor.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/[network_monitor.cpp](#)

10.50 NetworkTopologyWidget Class Reference

Network topology widget with 3-column layout: Unity TCP clients (left) | SERVER (center) | [Robot](#) UDP clients (right).

```
#include <network_topology_widget.hpp>
```

Inheritance diagram for NetworkTopologyWidget:



Classes

- struct [PairingSlot](#)

Public Member Functions

- **NetworkTopologyWidget** (QWidget *parent=nullptr)
- void **setClients** (const std::vector< [TopoClient](#) > &clients)
Replace the current client list and resize pulse arrays.
- void **setPairingBps** (int client_idx, int bps)
Set pairing traffic rate (bytes/sec) for a client index.
- void **setCategoryVisible** (const char *category, bool visible)
Show or hide a traffic category.
- bool **isCategoryVisible** (const char *category) const
- const std::vector< [TopoClient](#) > & **clients** () const
Read-only accessors for testing.
- const std::vector< int > & **unityIndices** () const
- const std::vector< int > & **robotIndices** () const
- const std::unordered_map< int, [PairingSlot](#) > & **pairingMap** () const
- QSize **sizeHint** () const override

Protected Member Functions

- void **paintEvent** (QPaintEvent *) override

10.50.1 Detailed Description

Network topology widget with 3-column layout: Unity TCP clients (left) | SERVER (center) | [Robot](#) UDP clients (right).

Each connection has TWO arrows (bidirectional):

- OUT (server->client): game data, commands, heartbeat pings - offset -7
- IN (client->server): responses, telemetry, heartbeat pongs - offset +7

Pairing links: curved gold lines connecting matched Unity-Robot pairs. Supports up to 4 Unity + 4 [Robot](#) clients.

The documentation for this class was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/[network_topology_widget.hpp](#)

10.51 Node Class Reference

One position ("tile") in the board's node graph. neighbors form an undirected adjacency list used for robot movement/pathfinding.

```
#include <board.hpp>
```

Public Member Functions

- [Node](#) (int id, float x, float y, [NodeType](#) type)
Constructs a node with its identity, position, and gameplay type.
- void [addNeighbor](#) ([Node](#) *node)
Adds a node to this node's adjacency list, forming a one-directional edge.

Public Attributes

- int **id**
- int **x**
Screen-pixel x coordinate for the 1400x860 board window, not board-frame cm.
- int **y**
Screen-pixel y coordinate for the 1400x860 board window, not board-frame cm.
- std::vector< int > **neighbors**
IDs of the nodes directly connected to this one.
- std::vector< [TileEffect](#) > **effects**
Effects applied to a player that lands on this node.
- float **x**
Position on the board, x axis (unit/frame not yet defined).
- float **y**
Position on the board, y axis (unit/frame not yet defined).
- [NodeType](#) **type**
- vector< [Node](#) * > **neighbors**
Directly connected nodes (non-owning pointers; the [Board](#) owns the underlying instances).

10.51.1 Detailed Description

One position ("tile") in the board's node graph. neighbors form an undirected adjacency list used for robot movement/pathfinding.

A single tile on the board graph, holding its position, gameplay type, and adjacency list.

10.51.2 Constructor & Destructor Documentation

10.51.2.1 Node()

```
Node::Node (
    int id,
    float x,
    float y,
    NodeType type) [inline]
```

Constructs a node with its identity, position, and gameplay type.

Parameters

<i>id</i>	Unique identifier used to look the node up in the Board 's node map.
-----------	--

<i>x</i>	X position of the node.
<i>y</i>	Y position of the node.
<i>type</i>	Gameplay role of the node.

10.51.3 Member Function Documentation

10.51.3.1 addNeighbor()

```
void Node::addNeighbor (
    Node * node) [inline]
```

Adds a node to this node's adjacency list, forming a one-directional edge.

Parameters

<i>node</i>	The neighboring node to link to.
-------------	----------------------------------

Note

Does not add the reciprocal edge; use [Board::connectNodes](#) for a bidirectional connection.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/mapLogic.cpp](#)

10.52 pathfinding::Node Struct Reference

A single cell visited during A* search, linked to its predecessor for path retracing. Coordinates (x, y) are grid cell indices (not world/mm coordinates); g/h/f follow the standard A* convention where g is the accumulated cost from the start, h is the heuristic estimate to the goal, and f = g + h is the priority used to order the open list.

```
#include <path_planner.hpp>
```

Public Member Functions

- **Node** (int x, int y)
Constructs a node at the given grid cell with all A costs initialized to zero.*
- bool **operator**> (const [Node](#) &other) const
Compares two nodes by f-cost, so that a std::priority_queue (a max-heap) using this operator would surface the node with the lowest f-cost first.

Public Attributes

- int **x**
- int **y**

Grid cell coordinates (column, row), not world coordinates.

- float **g**
- float **h**
- float **f**

A costs: g = cost from start, h = heuristic estimate to goal, $f = g + h$.*

- [Node](#) * **parent** = nullptr

Predecessor on the best known path so far; nullptr for the start node. Not owned by [Node](#) - lifetime is managed by the node pool in [PathPlanner::plan](#).

10.52.1 Detailed Description

A single cell visited during A* search, linked to its predecessor for path retracing. Coordinates (x, y) are grid cell indices (not world/mm coordinates); g/h/f follow the standard A* convention where g is the accumulated cost from the start, h is the heuristic estimate to the goal, and $f = g + h$ is the priority used to order the open list.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.hpp](#)

10.53 pathfinding::NodeCompare Struct Reference

Min-heap comparator for the A* open list, ordering [Node](#) pointers so that `std::priority_queue` pops the node with the lowest f-cost first.

Public Member Functions

- bool **operator()** (const [Node](#) *a, const [Node](#) *b) const

10.53.1 Detailed Description

Min-heap comparator for the A* open list, ordering [Node](#) pointers so that `std::priority_queue` pops the node with the lowest f-cost first.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.cpp](#)

10.54 BoardLoader.NodeData Class Reference

Public Attributes

- int **id**
- float **x**
- float **y**
- List< int > **neighbors**
- List< [EffectData](#) > **effects**

The documentation for this class was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/BoardLoader.cs](#)

10.55 ns::OpenShopEvent Struct Reference

Payload of the "open_shop" message: identifies which player opened the shop. Parsed in [shop.cpp](#)'s "open_shop" handler and used to build a temporary [Player](#) before calling [Shop::openShop\(\)](#).

```
#include <network_messages.hpp>
```

Public Attributes

- int **playerId**

10.55.1 Detailed Description

Payload of the "open_shop" message: identifies which player opened the shop. Parsed in [shop.cpp](#)'s "open_shop" handler and used to build a temporary [Player](#) before calling [Shop::openShop\(\)](#).

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp](#)

10.56 NetworkTopologyWidget::PairingSlot Struct Reference

Public Attributes

- int **unity** = -1
- int **robot** = -1

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.hpp](#)

10.57 follower::path_follower Class Reference

Per-robot path-following controller: pure pursuit + heading PID. Combines lookahead pursuit on the planner's waypoint polyline with a heading PID, curvature/goal speed-scaling, a rotate-in-place phase for large heading errors, and arrival/stop detection. Consumes the planner's already-rounded polyline, so it never cuts corners itself. Plain C++/STL + OpenCV only, no Qt, no network - unit-testable in a kinematic sim.

```
#include <path_follower.hpp>
```

Public Types

- enum class [mode](#) {
idle , **rotate_in_place** , **pursue** , **arrive_creep** ,
stopped }
Current control-loop phase, for the overlay/HUD and sim assertions. idle = not yet stepped / just reset; rotate_↔ in_place = an open-loop turn episode is in progress; pursue = driving the lookahead line outside decel_radius_cm; arrive_creep = slow final approach inside decel_radius_cm; stopped = arrived at the goal, or the path is holding.

Public Member Functions

- [path_follower](#) ([follower_params](#) params={})
Constructs the controller with the given tunable parameters.
- [drive_cmd_step](#) (const [pose2d](#) &pose, const [collision::path_update](#) &path, float dt_s)
Runs one control step and returns the drive command to apply. Projects the robot onto the planner's waypoint polyline, computes a pure-pursuit lookahead point, and then either dead-reckons an open-loop rotate-in-place turn (large heading error) or drives the line with the heading PID and curvature/goal speed-scaling, stopping at the final waypoint or while the path is holding.
- void [reset](#) ()
Clears PID and progress state; call on (re)start or a fresh path version.
- void [set_params](#) (const [follower_params](#) &p)
Replaces the tunable parameters; takes effect on the next [step\(\)](#).
- const [follower_params](#) & [params](#) () const noexcept
Returns the currently active tunable parameters.
- cv::Point2f [lookahead_cm](#) () const noexcept
Current pure-pursuit lookahead point, board-frame centimetres.
- [mode](#) [current_mode](#) () const noexcept
Current control-loop phase.
- float [cross_track_cm](#) () const noexcept
Perpendicular distance from the robot to the nearest polyline segment, cm.
- float [turn_corr](#) () const noexcept
Learned open-loop turn-rate calibration multiplier (1.0 = perfect). Converges over successive rotate-in-place episodes; see the live turn-rate learning in [step\(\)](#).

10.57.1 Detailed Description

Per-robot path-following controller: pure pursuit + heading PID. Combines lookahead pursuit on the planner's waypoint polyline with a heading PID, curvature/goal speed-scaling, a rotate-in-place phase for large heading errors, and arrival/stop detection. Consumes the planner's already-rounded polyline, so it never cuts corners itself. Plain C++/STL + OpenCV only, no Qt, no network - unit-testable in a kinematic sim.

10.57.2 Constructor & Destructor Documentation

10.57.2.1 path_follower()

```
follower::path_follower::path_follower (
    follower_params params = {}) [inline], [explicit]
```

Constructs the controller with the given tunable parameters.

Parameters

<i>params</i>	Initial gains/limits; defaults to follower_params {}. Can be changed later via set_params() without recreating the controller.
---------------	--

10.57.3 Member Function Documentation

10.57.3.1 reset()

```
void follower::path_follower::reset ()
```

Clears PID and progress state; call on (re)start or a fresh path version.

Note

Does NOT reset the learned turn_corr_ calibration - that persists across resets so the turn-rate learning survives a restart.

10.57.3.2 step()

```
drive_cmd follower::path_follower::step (
    const pose2d & pose,
    const collision::path_update & path,
    float dt_s)
```

Runs one control step and returns the drive command to apply. Projects the robot onto the planner's waypoint polyline, computes a pure-pursuit lookahead point, and then either dead-reckons an open-loop rotate-in-place turn (large heading error) or drives the line with the heading PID and curvature/goal speed-scaling, stopping at the final waypoint or while the path is holding.

Runs one control step and returns the drive command to apply.

Parameters

<i>pose</i>	Current robot pose in the board frame (see pose2d).
<i>path</i>	Latest planner output for this robot; a version bump resets PID/projection state but preserves an in-progress rotate episode.
<i>dt_s</i>	Wall time since the previous call, in seconds; clamped internally to a sane band so a dropped frame or the first call can't explode the PID or the dead-reckoned turn.

Returns

The velocity setpoint (or stop) to send to the actuator this tick.

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/[path_follower.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/[path_follower.cpp](#)

10.58 collision::path_update Struct Reference

One immutable snapshot of a single robot's effective path. The planner publishes a fresh snapshot (never edits one in place) whenever the path changes; consumers keep only the highest version seen and follow it.

```
#include <path_update.hpp>
```

Public Attributes

- int **robot_id**
logical robot slot (0-3), not the raw ArUco marker id
- std::uint64_t **version**
monotonic per robot; latest wins
- std::vector< cv::Point2f > **waypoints**
- **follow_state** state
see [follow_state](#)

10.58.1 Detailed Description

One immutable snapshot of a single robot's effective path. The planner publishes a fresh snapshot (never edits one in place) whenever the path changes; consumers keep only the highest version seen and follow it.

10.58.2 Member Data Documentation

10.58.2.1 waypoints

```
std::vector<cv::Point2f> collision::path_update::waypoints
```

ordered drive polyline, board-frame centimetres (physical 300x185 cm board). Empty = hold/stop.

The documentation for this struct was generated from the following file:

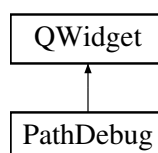
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/[path_update.hpp](#)

10.59 PathDebug Class Reference

Casino-themed Path Debug panel showing per-robot collision planner state (effective path, nominal path, route nodes, yielding status, watched pairs) and follower phase/cross-track data. Refreshes at 4 Hz.

```
#include <path_debug.hpp>
```

Inheritance diagram for PathDebug:



Public Member Functions

- **PathDebug** ([MapWindow](#) &mapwindow, QWidget *parent=nullptr)

10.59.1 Detailed Description

Casino-themed Path Debug panel showing per-robot collision planner state (effective path, nominal path, route nodes, yielding status, watched pairs) and follower phase/cross-track data. Refreshes at 4 Hz.

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/[path_debug.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/[path_debug.cpp](#)

10.60 pathfinding::PathPlanner Class Reference

Grid-based A* path planner that finds a walkable route between two world-space points. Queries a [GridMap](#) for walkability and coordinate conversion, then post-processes the raw grid-aligned path with line-of-sight smoothing before returning it.

```
#include <path_planner.hpp>
```

Public Member Functions

- **PathPlanner** ()=default
Constructs a planner with no persistent state; all working data lives inside [plan\(\)](#).
- `std::vector< cv::Point2f > plan` (const [GridMap](#) &map, cv::Point2f start, cv::Point2f goal)
Finds a path from start to goal on the given map using A search over 8-connected grid cells.*

10.60.1 Detailed Description

Grid-based A* path planner that finds a walkable route between two world-space points. Queries a [GridMap](#) for walkability and coordinate conversion, then post-processes the raw grid-aligned path with line-of-sight smoothing before returning it.

10.60.2 Member Function Documentation

10.60.2.1 plan()

```
std::vector< cv::Point2f > pathfinding::PathPlanner::plan (
    const GridMap & map,
    cv::Point2f start,
    cv::Point2f goal)
```

Finds a path from start to goal on the given map using A* search over 8-connected grid cells.

Parameters

<i>map</i>	Occupancy grid to search; also used to convert between world (mm) and grid (cell) coordinates.
<i>start</i>	Start position in world coordinates (mm).
<i>goal</i>	Goal position in world coordinates (mm).

Returns

Ordered list of waypoints in world coordinates from start to goal, reduced by line-of-sight smoothing; empty if the start or goal cell is not walkable, or if no path exists.

Note

Diagonal moves cost $\sqrt{2}$ and are disallowed when they would cut through the corner of an obstacle (i.e. when either orthogonal neighbor forming the corner is not walkable).

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.cpp](#)

10.61 collision::playboard_graph Class Reference

The playboard's node graph, used for edge-following routes (`move_playboard`) and for placing a shoved robot off the lanes. Plain C++/STL + OpenCV, no Qt. Routing is exact shortest path (Dijkstra by Euclidean edge length); the graph is small (~21 nodes) so this is trivially cheap.

```
#include <playboard_graph.hpp>
```

Public Member Functions

- void **set_nodes** (std::vector< [graph_node](#) > nodes)
Replace the whole node set. Safe to call again to rebuild from scratch.
- bool **empty** () const noexcept
True if no nodes have been set (or set_nodes was given an empty list).
- bool **has_node** (int id) const noexcept
True if a node with this id exists in the graph.
- const [graph_node](#) * **node** (int id) const
Look up a node by id.
- const std::vector< [graph_node](#) > & **nodes** () const noexcept
All nodes currently in the graph, in the order passed to [set_nodes\(\)](#).
- int **nearest_node** (cv::Point2f p) const
Nearest node to a board-frame point. Ties resolve to the lowest id so route starts are deterministic.
- std::vector< int > **route** (int from, int to, const std::function< bool(int, int)> &edge_blocked={}) const
*Exact shortest route (Euclidean edge weights) from *from* to *to*.*
- float **min_edge_distance** (cv::Point2f p) const
Minimum distance from p to ANY graph edge segment. Used to keep a park spot off the lanes.
- float **min_node_distance** (cv::Point2f p, int except_id=-1) const
*Minimum distance from p to any node centre, excluding *except_id*. Used to keep a park spot clear of other nodes.*

10.61.1 Detailed Description

The playboard's node graph, used for edge-following routes (`move_playboard`) and for placing a shoved robot off the lanes. Plain C++/STL + OpenCV, no Qt. Routing is exact shortest path (Dijkstra by Euclidean edge length); the graph is small (~21 nodes) so this is trivially cheap.

10.61.2 Member Function Documentation

10.61.2.1 min_edge_distance()

```
float collision::playboard_graph::min_edge_distance (
    cv::Point2f p) const
```

Minimum distance from p to ANY graph edge segment. Used to keep a park spot off the lanes.

Minimum distance from p to ANY graph edge segment.

Parameters

<i>p</i>	Query point, board-frame centimetres.
----------	---------------------------------------

Returns

Distance in centimetres, or a large sentinel value if there are no edges.

10.61.2.2 min_node_distance()

```
float collision::playboard_graph::min_node_distance (
    cv::Point2f p,
    int except_id = -1) const
```

Minimum distance from p to any node centre, excluding `except_id`. Used to keep a park spot clear of other nodes.

Minimum distance from p to any node centre, excluding `except_id`.

Parameters

<code>p</code>	Query point, board-frame centimetres.
<code>except_id</code>	Node id to skip (e.g. the node being placed near), or -1 to consider all nodes.

Returns

Distance in centimetres, or a large sentinel value if no node qualifies.

10.61.2.3 nearest_node()

```
int collision::playboard_graph::nearest_node (
    cv::Point2f p) const [nodiscard]
```

Nearest node to a board-frame point. Ties resolve to the lowest id so route starts are deterministic.

Nearest node to a board-frame point.

Parameters

<code>p</code>	Query point, board-frame centimetres.
----------------	---------------------------------------

Returns

[Node](#) id of the nearest node, or -1 if the graph is empty.

10.61.2.4 node()

```
const graph\_node * collision::playboard_graph::node (
    int id) const [nodiscard]
```

Look up a node by id.

Parameters

<code>id</code>	Node identifier to search for.
-----------------	--

Returns

Pointer to the node, or nullptr if absent.

10.61.2.5 route()

```
std::vector< int > collision::playboard_graph::route (
    int from,
    int to,
    const std::function< bool(int, int)> & edge_blocked = {}) const [nodiscard]
```

Exact shortest route (Euclidean edge weights) from `from` to `to`.

Parameters

<i>from</i>	Start node id.
<i>to</i>	Destination node id.
<i>edge_blocked</i>	Optional predicate; when set and returning true for a pair (a, b), that edge is removed from consideration for this call only.

Returns

Ordered node ids INCLUDING both endpoints, or {} if `to` is unreachable (or either id is unknown). Pure: no hidden fallback.

10.61.2.6 set_nodes()

```
void collision::playboard_graph::set_nodes (
    std::vector< graph_node > nodes)
```

Replace the whole node set. Safe to call again to rebuild from scratch.

Replace the whole node set.

Parameters

<i>nodes</i>	New complete node list, moved in.
--------------	-----------------------------------

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.cpp](#)

10.62 Player Class Reference

Represents one game participant: identity, board position, coins/stars, active status effects, inventory, and per-turn state. A player is paired with a physical robot (`robotID`); moving the player's logical node (`setCurrentNode`) also requests the collision planner to route that robot to the new node.

```
#include <player.hpp>
```

Public Member Functions

- **Player** (int id, int robotID, const std::string &name, int startNode)
Constructs a player with identity, associated robot, and starting board position.
- bool **canAfford** (int price) const noexcept
Checks whether the player has enough coins to pay a given price.
- int **getRobotId** () const noexcept
Returns the identifier of the physical robot associated with this player.
- void **startTurn** ()
Marks this player as the active player and runs start-of-turn bookkeeping. Increments the turns-played stat and applies each active effect's start-of-turn logic (via [checkNegativeEffects\(\)](#)).
- void **endTurn** ()
Ends this player's turn, clearing the active-player flag.
- bool **getCurrentPlayer** () const noexcept
Returns whether this player is the one currently taking their turn.
- int **getCurrentPlayerID** () const noexcept
Returns this player's unique identifier.
- void **applyEndOfTurnEffects** ()
Applies each active effect's end-of-turn logic, ticks down their durations, and removes any that have expired.
- void **checkNegativeEffects** ()
Applies each active effect's start-of-turn logic to this player.
- void **removeExpiredEffects** ()
Removes all effects whose `isExpired()` returns true from the active list.
- int **getCurrentNode** () const noexcept
Returns the board graph node id the player currently occupies.
- const **Vec3**< float > & **getWorldPosition** () const
Returns the player's cached world-space position.
- void **setWorldPosition** (const **Vec3**< float > &pos)
Updates the player's cached world-space position.
- void **setCurrentNode** (int nodeId, int robotId)
Updates the player's logical board node and requests the collision planner route the given robot there.
- const std::string & **getName** () const noexcept
Returns the player's display name.
- int **getId** () const noexcept
Returns this player's unique identifier.
- void **addEffect** (std::shared_ptr< **Effect** > effect)
Attaches a status effect to this player and increments the effects-received stat.
- void **addCoins** (int amount)
Adds (or, with a negative amount, subtracts) coins from the player's balance.
- void **addStar** ()
Awards the player one star.
- int **getStars** () const noexcept
Returns the number of stars the player has collected.
- int **getCoins** () const noexcept
Returns the player's current coin balance.
- **Inventory** & **getInventory** () noexcept
Returns a mutable reference to the player's inventory.
- const **Inventory** & **getInventory** () const noexcept
Returns a read-only reference to the player's inventory.
- void **addItem** (const **Item** &item)
Adds an item to the player's inventory.

- void **setDiceMultiplier** (int mult)
Sets the multiplier to apply to the player's next dice roll.
- int **getDiceMultiplier** () const noexcept
Returns the multiplier currently queued for the player's next dice roll.
- void **resetDiceMultiplier** ()
Resets the next-roll dice multiplier back to 1 (no bonus).
- void **activateDoubleCoins** ()
Enables the double-coins bonus for this player.
- bool **hasDoubleCoins** () const noexcept
Returns whether the double-coins bonus is currently active.
- void **clearDoubleCoins** ()
Disables the double-coins bonus.
- void **activateInvertedControls** ()
Enables the inverted-controls effect for this player.
- bool **hasInvertedControls** () const noexcept
Returns whether the inverted-controls effect is currently active.
- void **clearInvertedControls** ()
Disables the inverted-controls effect.
- void **swapPosition** (Player &other, const Board &board)
Exchanges this player's and other's current board node.
- void **stealDiceSteps** (Player &other, int amount)
Adds bonus dice steps to this player.
- const PlayerStats & **getStats** () const noexcept
Returns the player's aggregate session statistics.
- void **addMovedField** ()
Increments the total-fields-moved stat by one.
- void **addUsedItem** ()
Increments the items-used stat by one.

Protected Attributes

- int **playerId** = 0
- std::string **name**
- int **stars** = 0
- int **coins** = 0
- int **currentNodeId** = 0
Current position: board graph node id.
- bool **isCurrentPlayer** = false
- PlayerStats **stats**
- Inventory **inventory**
- std::vector< std::shared_ptr< Effect > > **effects**
Status effects currently active on this player.
- bool **doubleCoinsActive** = false
True while an item-granted coin-doubling bonus is active.
- bool **invertedControlsActive** = false
True while an item-granted control-inversion effect is active.
- int **nextDiceMultiplier** = 1
Multiplier queued for the player's next dice roll; reset to 1 via [resetDiceMultiplier\(\)](#).
- int **diceBonusSteps** = 0
Bonus dice steps accumulated via [stealDiceSteps\(\)](#); not yet consumed by a turn manager.
- int **robotID** = 0
Identifier of the physical robot associated with this player, used when routing via the collision planner.
- Vec3< float > **worldPosition** {}
Cached world-space position; see [getWorldPosition\(\)/setWorldPosition\(\)](#).

10.62.1 Detailed Description

Represents one game participant: identity, board position, coins/stars, active status effects, inventory, and per-turn state. A player is paired with a physical robot (`robotID`); moving the player's logical node (`setCurrentNode`) also requests the collision planner to route that robot to the new node.

10.62.2 Constructor & Destructor Documentation

10.62.2.1 `Player()`

```
Player::Player (
    int id,
    int robotID,
    const std::string & name,
    int startNode)
```

Constructs a player with identity, associated robot, and starting board position.

Parameters

<i>id</i>	Unique player identifier.
<i>robotID</i>	Identifier of the physical robot controlled by this player.
<i>name</i>	Display name.
<i>startNode</i>	Board graph node id the player starts on.

10.62.3 Member Function Documentation

10.62.3.1 `addCoins()`

```
void Player::addCoins (
    int amount)
```

Adds (or, with a negative amount, subtracts) coins from the player's balance.

Parameters

<i>amount</i>	Signed coin delta; positive amounts are also added to the <code>totalCoinsEarned</code> stat.
---------------	---

Note

The resulting balance is clamped to a minimum of 0.

10.62.3.2 addEffect()

```
void Player::addEffect (
    std::shared_ptr< Effect > effect)
```

Attaches a status effect to this player and increments the effects-received stat.

Parameters

<i>effect</i>	The effect to attach; ownership is shared with the caller.
---------------	--

10.62.3.3 addItem()

```
void Player::addItem (
    const Item & item)
```

Adds an item to the player's inventory.

Parameters

<i>item</i>	The item to add.
-------------	------------------

Note

Declared but not defined anywhere in [player.cpp](#) and not called from any current caller; calling this will fail to link. [Item](#) additions currently go through [Inventory::addItem\(\)](#) directly.

10.62.3.4 canAfford()

```
bool Player::canAfford (
    int price) const [noexcept]
```

Checks whether the player has enough coins to pay a given price.

Parameters

<i>price</i>	Cost to compare against the player's coin balance.
--------------	--

Returns

true if coins $\geq$ price.

10.62.3.5 checkNegativeEffects()

```
void Player::checkNegativeEffects ()
```

Applies each active effect's start-of-turn logic to this player.

Note

Despite the name, this applies ALL active effects regardless of [EffectType](#) (not just Negative ones). Called from [startTurn\(\)](#).

10.62.3.6 `getCurrentPlayer()`

```
bool Player::getCurrentPlayer () const [noexcept]
```

Returns whether this player is the one currently taking their turn.

Returns

true if set by [startTurn\(\)](#) and not yet cleared by [endTurn\(\)](#).

10.62.3.7 `getCurrentPlayerID()`

```
int Player::getCurrentPlayerID () const [nodiscard], [noexcept]
```

Returns this player's unique identifier.

Note

Equivalent to [getId\(\)](#); used by turn-management logic.

10.62.3.8 `getId()`

```
int Player::getId () const [nodiscard], [noexcept]
```

Returns this player's unique identifier.

Note

Equivalent to [getCurrentPlayerID\(\)](#).

10.62.3.9 `getWorldPosition()`

```
const Vec3< float > & Player::getWorldPosition () const
```

Returns the player's cached world-space position.

Note

Declared but not currently defined anywhere (the implementation in [player.cpp](#) is commented out); calling this will fail to link.

10.62.3.10 setCurrentNode()

```
void Player::setCurrentNode (
    int nodeId,
    int robotId)
```

Updates the player's logical board node and requests the collision planner route the given robot there.

Parameters

<i>nodeId</i>	Target board graph node id to move to.
<i>robotId</i>	Identifier of the physical robot to route to <i>nodeId</i> (passed through to <code>collision_planner::request_route_to()</code>).

10.62.3.11 setDiceMultiplier()

```
void Player::setDiceMultiplier (
    int mult)
```

Sets the multiplier to apply to the player's next dice roll.

Parameters

<i>mult</i>	Multiplier value (e.g. 2 for a double-dice item).
-------------	---

10.62.3.12 setWorldPosition()

```
void Player::setWorldPosition (
    const Vec3< float > & pos)
```

Updates the player's cached world-space position.

Parameters

<i>pos</i>	New world-space position to store.
------------	------------------------------------

Note

Declared but not currently defined anywhere (the implementation in `player.cpp` is commented out); calling this will fail to link.

10.62.3.13 startTurn()

```
void Player::startTurn ()
```

Marks this player as the active player and runs start-of-turn bookkeeping. Increments the turns-played stat and applies each active effect's start-of-turn logic (via `checkNegativeEffects()`).

Marks this player as the active player and runs start-of-turn bookkeeping.

Note

The player does not roll the dice itself; that is handled elsewhere.

10.62.3.14 stealDiceSteps()

```
void Player::stealDiceSteps (
    Player & other,
    int amount)
```

Adds bonus dice steps to this player.

Parameters

<i>other</i>	Unused by the current implementation.
<i>amount</i>	Number of bonus steps to add; ignored if not positive.

10.62.3.15 swapPosition()

```
void Player::swapPosition (
    Player & other,
    const Board & board)
```

Exchanges this player's and *other*'s current board node.

Exchanges this player's and other's current board node.

Parameters

<i>other</i>	The player to swap positions with.
<i>board</i>	Unused by the current implementation.

Note

Both resulting [setCurrentNode\(\)](#) calls pass `other.getRobotId()` as the routed robot id (see [setCurrentNode\(\)](#)).

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.cpp](#)

10.63 PlayerSession Struct Reference

Public Attributes

- `ConnectionID id {}`
- `std::string device_name {}`
- `std::shared_ptr< TcpConnection > tcp_conn {}`
- `boost::asio::ip::udp::endpoint udp_endpoint {}`
- `std::chrono::steady_clock::time_point last_seen {}`
- `std::uint64_t bytes_tx = 0`
- `std::uint64_t bytes_rx = 0`
- `std::unordered_map< std::string, std::pair< std::uint64_t, std::uint64_t > > typed_traffic {}`
Per-category (tx, rx) byte counters. Keys: "game", "heartbeat", "response", "register".

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/session_registry.hpp](#)

10.64 PlayerStanding Struct Reference

Public Attributes

- int **rank**
- int **playerId**
- std::string **playerName**
- int **starsBeforeBonus**
- int **starsAfterBonus**
- int **coins**

The documentation for this struct was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/[bonus.hpp](#)

10.65 PlayerStats Struct Reference

Aggregate per-player session statistics (turns played, items used, effects received, fields moved, coins earned).

```
#include <player.hpp>
```

Public Attributes

- int **totalFieldsMoved** = 0
- int **turnsPlayed** = 0
- int **itemsUsed** = 0
- int **effectsReceived** = 0
- int **totalCoinsEarned** = 0

Sum of positive coin gains only; addCoins() does not add negative amounts here.

10.65.1 Detailed Description

Aggregate per-player session statistics (turns played, items used, effects received, fields moved, coins earned).

The documentation for this struct was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/[player.hpp](#)

10.66 follower::pose2d Struct Reference

[Robot](#) pose in the board frame. Position in centimetres, heading in radians using atan2(dy, dx) (0 = +x), the SAME convention as the planner and the kinematic sim. The board cm frame is y-down (left-handed); that is fine as long as everything (planner, follower, sim) shares it - the only place the real-world chirality matters is the hardware steering sign (params.flip_turn).

```
#include <path_follower.hpp>
```

Public Attributes

- float **x_cm** = 0.0f
- float **y_cm** = 0.0f
- float **heading_rad** = 0.0f

10.66.1 Detailed Description

Robot pose in the board frame. Position in centimetres, heading in radians using $\text{atan2}(\text{dy}, \text{dx})$ ($0 = +x$), the SAME convention as the planner and the kinematic sim. The board cm frame is y-down (left-handed); that is fine as long as everything (planner, follower, sim) shares it - the only place the real-world chirality matters is the hardware steering sign (`params.flip_turn`).

The documentation for this struct was generated from the following file:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.hpp`

10.67 Robot Struct Reference

Live state of one tracked/simulated robot slot (0-3), used for rendering and hit-testing. The authoritative pose for driving is `follower::pose2d` (see `MapWindow::poseOf`); `x/y/heading_rad` here are the on-screen projection of that pose, refreshed every tick.

```
#include <mapwindow.hpp>
```

Public Attributes

- int **id**
***Robot** slot (0-3); also the index into per-robot arrays elsewhere in `MapWindow`.*
- int **x**
Current on-screen board position in pixels (not board-frame cm).
- int **y**
Current on-screen board position in pixels (not board-frame cm).
- int **current_node** = -1
***Node** the robot is currently near (-1 if none).*
- QColor **color**
Rendering color used to distinguish this robot on the board.
- float **angle** = 0.0f
Tracker image-space heading in degrees (legacy, unused by the driver).
- float **heading_rad** = 0.0f
Board-frame heading ($\text{atan2}(\text{dy}, \text{dx})$, radians) for the driver/planner.

10.67.1 Detailed Description

Live state of one tracked/simulated robot slot (0-3), used for rendering and hit-testing. The authoritative pose for driving is `follower::pose2d` (see `MapWindow::poseOf`); `x/y/heading_rad` here are the on-screen projection of that pose, refreshed every tick.

The documentation for this struct was generated from the following file:

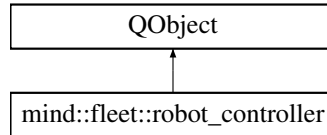
- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/mapwindow.hpp`

10.68 mind::fleet::robot_controller Class Reference

Per-robot facade owned by [robot_fleet](#): translates method calls into MQTT/UDP commands and translates incoming EV3 status into Qt signals. Owns its own timers and runs the move_relative turn -> drive_for state machine internally; one instance exists per tracked robot.

```
#include <robot_controller.hpp>
```

Inheritance diagram for mind::fleet::robot_controller:



Public Slots

- void [on_pose_updated](#) (double x_mm, double y_mm, double theta_rad, qint64 ts_ms)
Records the robot's latest tracked pose. Forward-looking seam for closed-loop control (plan A.9): today this only stores the pose in last_pose_; nothing consumes it yet.*

Signals

- void **connection_changed** (bool online)
Emitted when [is_online\(\)](#) changes, e.g. first status received or offline timeout fired.
- void **state_changed** (QString state)
Emitted when the decoded EV3 state string changes.
- void **battery_changed** (int millivolts)
Emitted when the decoded battery voltage (in millivolts) changes.

Public Member Functions

- [robot_controller](#) (QString robot_id, [mind::mqtt::client](#) *mqtt, [mind::net::udp_link](#) *udp, QObject *parent=nullptr)
Constructs the controller for one robot and wires up its internal timers. Derives the UDP routing byte (nn_) from the trailing digits of robot_id and builds the MQTT command topic ("mind/<robot_id>/cmd"). Does not connect to MQTT/UDP itself; mqtt and udp are shared, externally-owned links used for all robots.
- **robot_controller** (const robot_controller &)=delete
- [robot_controller](#) & **operator=** (const [robot_controller](#) &)=delete
- QString **robot_id** () const noexcept
Returns this controller's robot id (e.g. "ev3_03"), used as the MQTT topic segment.
- bool **is_online** () const noexcept
Returns whether a status message has been seen within the offline timeout window.
- QString **state** () const noexcept
Returns the last known EV3 state string ("idle", "driving", "moving", or "error").
- int **battery_mv** () const noexcept
Returns the last known battery voltage, in millivolts.
- void [set_drive](#) (int speed_pct, int turn_pct)

Sets the continuous tank-drive command (speed + turn rate). Stores the percents, switches `drive_mode` to tank, and ensures the 50 Hz republish loop is running. Callable at any input rate; the same values are simply re-sent on the next tick. Also sends one publish immediately so the wire latency is bound by this call rather than the next republish tick.

- void `set_wheel_drive` (int left_wheel, int right_wheel)
Sets the continuous independent-wheel-drive command. Same republish-loop and release-to-stop semantics as `set_drive`; the controller switches between tank and wheel mode depending on whichever setter (`set_drive` or `set_wheel_drive`) was called most recently.
- void `stop` ()
Immediately halts the robot and cancels any in-flight motion. Publishes one stop command, halts the republish loop, and aborts any in-flight `move_relative` (resets its state machine without publishing anything extra, since the stop covers it). Safe to call even when nothing is in flight.
- void `beep` (int duration_ms, int frequency_hz)
Requests a one-shot beep on the brick's speaker.
- void `move_relative` (double forward_cm, double right_cm)
Starts a robot-relative move: turn to face (forward_cm, right_cm), then drive that distance. Converts the relative offset to a heading (atan2) and a distance (hypot), then fires the turn leg; `handle_status` advances to the `drive_for` leg once the matching ack reports the robot idle again.
- void `drive_for` (int distance_mm, int speed_mm_s)
One-shot straight-line move that bypasses the `move_relative` state machine (no sequencing, no ack tracking).
- void `turn` (int angle_deg, int rate_dps)
One-shot in-place rotation that bypasses the `move_relative` state machine (no sequencing, no ack tracking).
- void `handle_status` (const QByteArray &status)
Feeds one raw status payload (ack or heartbeat) addressed to this robot. Restarts the offline timer and flags online on any status. Decodes the payload to update state/battery (emitting the corresponding signals on change), and, if the state machine is waiting on this exact ack's `cmd_id` and the robot has gone idle, advances `move_relative`.
- void `set_brick_address` (QHostAddress addr)
Records the brick's UDP command address, learned from its heartbeat source. Set/refreshed by `robot_fleet` on every heartbeat; until called at least once, `publish_drive/publish_wheel_drive` drop their datagrams.

10.68.1 Detailed Description

Per-robot facade owned by `robot_fleet`: translates method calls into MQTT/UDP commands and translates incoming EV3 status into Qt signals. Owns its own timers and runs the `move_relative` turn -> `drive_for` state machine internally; one instance exists per tracked robot.

10.68.2 Constructor & Destructor Documentation

10.68.2.1 robot_controller()

```
mind::fleet::robot_controller::robot_controller (
    QString robot_id,
    mind::mqtt::client * mqtt,
    mind::net::udp_link * udp,
    QObject * parent = nullptr)
```

Constructs the controller for one robot and wires up its internal timers. Derives the UDP routing byte (`nn_`) from the trailing digits of `robot_id` and builds the MQTT command topic ("`mind/<robot_id>/cmd`"). Does not connect to MQTT/UDP itself; `mqtt` and `udp` are shared, externally-owned links used for all robots.

Constructs the controller for one robot and wires up its internal timers.

Parameters

<i>robot↔ _id</i>	Robot identifier in "ev3_NN" form.
<i>mqtt</i>	Non-owning pointer to the shared MQTT client used to publish commands.
<i>udp</i>	Non-owning pointer to the shared UDP link used for continuous-drive datagrams.
<i>parent</i>	Optional QObject parent (also owns the internal QTimer).

10.68.3 Member Function Documentation**10.68.3.1 beep()**

```
void mind::fleet::robot_controller::beep (
    int duration_ms,
    int frequency_hz)
```

Requests a one-shot beep on the brick's speaker.

Parameters

<i>duration_ms</i>	Tone duration, in milliseconds.
<i>frequency_hz</i>	Tone frequency, in hertz.

10.68.3.2 drive_for()

```
void mind::fleet::robot_controller::drive_for (
    int distance_mm,
    int speed_mm_s)
```

One-shot straight-line move that bypasses the move_relative state machine (no sequencing, no ack tracking).

One-shot straight-line move that bypasses the move_relative state machine.

Parameters

<i>distance_mm</i>	Distance to travel, in millimeters.
<i>speed_mm↔ _s</i>	Target speed, in millimeters per second.

Note

Caller must call [stop\(\)](#) to abort a move already in progress.

10.68.3.3 `handle_status()`

```
void mind::fleet::robot_controller::handle_status (
    const QByteArray & status)
```

Feeds one raw status payload (ack or heartbeat) addressed to this robot. Restarts the offline timer and flags online on any status. Decodes the payload to update state/battery (emitting the corresponding signals on change), and, if the state machine is waiting on this exact ack's `cmd_id` and the robot has gone idle, advances `move_relative`.

Feeds one raw status payload (ack or heartbeat) addressed to this robot.

Parameters

<i>status</i>	Raw status payload as received from MQTT/UDP, still wire-encoded.
---------------	---

10.68.3.4 `move_relative()`

```
void mind::fleet::robot_controller::move_relative (
    double forward_cm,
    double right_cm)
```

Starts a robot-relative move: turn to face (`forward_cm`, `right_cm`), then drive that distance. Converts the relative offset to a heading (`atan2`) and a distance (`hypot`), then fires the turn leg; `handle_status` advances to the drive_for leg once the matching ack reports the robot idle again.

Starts a robot-relative move: turn to face (`forward_cm`, `right_cm`), then drive that distance.

Parameters

<i>forward_cm</i>	Offset along the robot's current forward axis, in centimeters.
<i>right_cm</i>	Offset along the robot's current right axis, in centimeters.

Note

Pre-condition: no continuous drive active (shares EV3 state with `drive/wheel_drive`, they fight over the brick). A move already in flight silently drops the new call; call [stop\(\)](#) first.

10.68.3.5 `on_pose_updated`

```
void mind::fleet::robot_controller::on_pose_updated (
    double x_mm,
    double y_mm,
    double theta_rad,
    qint64 ts_ms) [slot]
```

Records the robot's latest tracked pose. Forward-looking seam for closed-loop control (plan A.9): today this only stores the pose in `last_pose_*`; nothing consumes it yet.

Records the robot's latest tracked pose.

Parameters

<i>x_mm</i>	Pose X position, in millimeters.
<i>y_mm</i>	Pose Y position, in millimeters.
<i>theta_rad</i>	Pose heading, in radians.
<i>ts_ms</i>	Timestamp of the pose sample, in milliseconds.

10.68.3.6 set_brick_address()

```
void mind::fleet::robot_controller::set_brick_address (
    QHostAddress addr)
```

Records the brick's UDP command address, learned from its heartbeat source. Set/refreshed by [robot_fleet](#) on every heartbeat; until called at least once, `publish_drive/publish_wheel_drive` drop their datagrams.

Records the brick's UDP command address, learned from its heartbeat source.

Parameters

<i>addr</i>	Source address of the brick's most recent heartbeat.
-------------	--

10.68.3.7 set_drive()

```
void mind::fleet::robot_controller::set_drive (
    int speed_pct,
    int turn_pct)
```

Sets the continuous tank-drive command (speed + turn rate). Stores the percents, switches `drive_mode` to tank, and ensures the 50 Hz republish loop is running. Callable at any input rate; the same values are simply re-sent on the next tick. Also sends one publish immediately so the wire latency is bound by this call rather than the next republish tick.

Sets the continuous tank-drive command (speed + turn rate).

Parameters

<i>speed_pct</i>	Forward/back speed as a percent of <code>WHEEL_MAX_MM_S</code> , -100..100.
<i>turn_pct</i>	Turn rate as a percent of <code>DRIVE_MAX_TURN_DPS</code> , -100..100.

Note

(0, 0) held for `RELEASE_IDLE_MS` auto-publishes a stop and halts the republish loop.

10.68.3.8 set_wheel_drive()

```
void mind::fleet::robot_controller::set_wheel_drive (
    int left_wheel,
    int right_wheel)
```

Sets the continuous independent-wheel-drive command. Same republish-loop and release-to-stop semantics as `set_drive`; the controller switches between tank and wheel mode depending on whichever setter (`set_drive` or `set_wheel_drive`) was called most recently.

Sets the continuous independent-wheel-drive command.

Parameters

<i>left_wheel</i>	Left wheel speed as a percent of WHEEL_MAX_MM_S, -100..100.
<i>right_wheel</i>	Right wheel speed as a percent of WHEEL_MAX_MM_S, -100..100.

10.68.3.9 stop()

```
void mind::fleet::robot_controller::stop ()
```

Immediately halts the robot and cancels any in-flight motion. Publishes one stop command, halts the republish loop, and aborts any in-flight `move_relative` (resets its state machine without publishing anything extra, since the stop covers it). Safe to call even when nothing is in flight.

Immediately halts the robot and cancels any in-flight motion.

10.68.3.10 turn()

```
void mind::fleet::robot_controller::turn (
    int angle_deg,
    int rate_dps)
```

One-shot in-place rotation that bypasses the `move_relative` state machine (no sequencing, no ack tracking).

One-shot in-place rotation that bypasses the `move_relative` state machine.

Parameters

<i>angle_deg</i>	Angle to rotate, in degrees.
<i>rate_dps</i>	Rotation rate, in degrees per second.

Note

Caller must call [stop\(\)](#) to abort a turn already in progress.

The documentation for this class was generated from the following files:

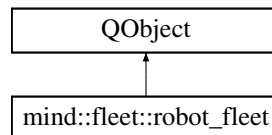
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.cpp](#)

10.69 mind::fleet::robot_fleet Class Reference

Auto-discovers EV3 robots and owns one [robot_controller](#) per robot. Robots are registered on their first UDP heartbeat (see `on_datagram_received`); MQTT status acks on `mind/+ /status` are then routed to the matching already-discovered controller (see `on_status_message`). Also tracks which robot is currently "active" (see `select()`/`← active()`).

```
#include <robot_fleet.hpp>
```

Inheritance diagram for `mind::fleet::robot_fleet`:



Public Slots

- void [select](#) (QString robot_id)
Makes robot_id the active robot and beeps it for audible confirmation. Logs a warning and does nothing if robot_id has not been discovered yet; does nothing (no beep, no signal) if robot_id is already active.

Signals

- void [robot_appeared](#) (QString robot_id, [robot_controller](#) *controller)
- void [robot_disappeared](#) (QString robot_id)
- void [active_changed](#) ([robot_controller](#) *active)
Emitted after [select\(\)](#) changes which robot is active.
- void [robot_ip_discovered](#) (quint8 nn, QString ip_address)

Public Member Functions

- [robot_fleet](#) ([mind::mqtt::client](#) *mqtt, [mind::net::udp_link](#) *udp, QObject *parent=nullptr)
Wires up MQTT/UDP signal routing for robot discovery and status handling. Subscribes to `mind/+ /status` for command acks (mqtt_client replays the subscription automatically on every reconnect) and connects to the UDP link's datagram signal so heartbeats drive discovery.
- [robot_fleet](#) (const [robot_fleet](#) &)=delete
- [robot_fleet](#) & [operator=](#) (const [robot_fleet](#) &)=delete
- [robot_controller](#) * [active](#) () const
Returns the currently active (selected) robot's controller, or nullptr if none has been selected yet.
- [robot_controller](#) * [controller](#) (QString robot_id) const
Looks up the controller for a given robot_id. The seam used by the joystick adapter to route incoming TCP/JSON messages to the right robot by their robot_id field.

10.69.1 Detailed Description

Auto-discovers EV3 robots and owns one [robot_controller](#) per robot. Robots are registered on their first UDP heartbeat (see `on_datagram_received`); MQTT status acks on `mind/+ /status` are then routed to the matching already-discovered controller (see `on_status_message`). Also tracks which robot is currently "active" (see `select()`/`← active()`).

10.69.2 Constructor & Destructor Documentation

10.69.2.1 robot_fleet()

```
mind::fleet::robot_fleet::robot_fleet (
    mind::mqtt::client * mqtt,
    mind::net::udp_link * udp,
    QObject * parent = nullptr)
```

Wires up MQTT/UDP signal routing for robot discovery and status handling. Subscribes to `mind/+/status` for command acks (`mqtt_client` replays the subscription automatically on every reconnect) and connects to the UDP link's datagram signal so heartbeats drive discovery.

Wires up MQTT/UDP signal routing for robot discovery and status handling.

Parameters

<i>mqtt</i>	MQTT client used to receive status acks; not owned.
<i>udp</i>	UDP link used to receive heartbeat datagrams; not owned.
<i>parent</i>	Optional QObject parent for ownership/lifetime.

10.69.3 Member Function Documentation

10.69.3.1 controller()

```
robot_controller * mind::fleet::robot_fleet::controller (
    QString robot_id) const
```

Looks up the controller for a given `robot_id`. The seam used by the joystick adapter to route incoming TCP/JSON messages to the right robot by their `robot_id` field.

Looks up the controller for a given `robot_id`.

Parameters

<i>robot_id</i>	Robot identifier, e.g. "ev3_00".
-----------------	----------------------------------

Returns

The matching `robot_controller`, or `nullptr` if no robot with that id has been discovered yet.

10.69.3.2 robot_appeared

```
void mind::fleet::robot_fleet::robot_appeared (
    QString robot_id,
    robot_controller * controller) [signal]
```

Emitted the first time a robot's UDP heartbeat is seen; its `robot_controller` has just been created and registered.

10.69.3.3 robot_disappeared

```
void mind::fleet::robot_fleet::robot_disappeared (
    QString robot_id) [signal]
```

Emitted when a previously discovered robot's controller reports going offline.

10.69.3.4 robot_ip_discovered

```
void mind::fleet::robot_fleet::robot_ip_discovered (
    quint8 nn,
    QString ip_address) [signal]
```

Emitted on every heartbeat with the robot number (1-4) and its source IP. Consumers can forward this to phones so they learn which EV3 IP to drive.

10.69.3.5 select

```
void mind::fleet::robot_fleet::select (
    QString robot_id) [slot]
```

Makes robot_id the active robot and beeps it for audible confirmation. Logs a warning and does nothing if robot_id has not been discovered yet; does nothing (no beep, no signal) if robot_id is already active.

Makes robot_id the active robot and beeps it for audible confirmation.

Parameters

<i>robot_id</i>	Robot identifier to select as active.
-----------------	---------------------------------------

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.hpp
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.cpp

10.70 collision::robot_pose Struct Reference

Live pose of one robot in the board frame (centimetres), fed to the planner each tick. `moving` distinguishes an actively-driving robot (a collision partner, watched pairwise) from a parked one (a static obstacle for others).

```
#include <collision_planner.hpp>
```

Public Attributes

- int **robot_id**
Logical robot slot (0-3), not the raw ArUco marker id.
- cv::Point2f **position_cm**
Board-frame position, centimetres.
- float **heading_deg**
0 = +x axis, CCW positive (matches `std::atan2(dy, dx)`).
- bool **moving**
True while actively driving (a live collision partner); false when parked (a static obstacle to others).

10.70.1 Detailed Description

Live pose of one robot in the board frame (centimetres), fed to the planner each tick. `moving` distinguishes an actively-driving robot (a collision partner, watched pairwise) from a parked one (a static obstacle for others).

The documentation for this struct was generated from the following file:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.hpp`

10.71 MqttReceiver.RobotPositionData Class Reference

Public Attributes

- int **id**
- float **x**
- float **y**

The documentation for this class was generated from the following file:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/MqttReceiver.cs`

10.72 Roulette Class Reference

Runs one roulette betting round for a single player, driven by network messages from the AR/Unity client. Owns the roll-to-result logic (via `Generator`) and blocks the calling thread in `choose_bet_type()` until the client's bet arrives over the network, synchronizing between the asio message-handler thread (`setBet()`) and whichever thread drives the game turn (`give_take_coins()`).

```
#include <roulette.hpp>
```


Public Member Functions

- **Roulette** ([Server](#) &server)
- void **start_roulette** ([Player](#) &player)

Announces the start of a roulette round for the given player to all connected clients.
- [Bet_type](#) **choose_bet_type** ([Player](#) &player)

Blocks until the client's bet for this round has arrived, then returns its bet type.
- int **make_bet** ([Player](#) &player)

Returns the wagered coin amount from the most recently received bet.
- bool **roulette_odd_even** ()

Checks the current player's odd/even guess against the last table result.
- bool **roulette_red_black** ()

Checks the current player's red/black guess against the last table result.
- bool **calculate_results** ([Bet_type](#) type, [Player](#) &player, int player_choice)

Rolls the ball and evaluates the given bet against the outcome.
- void **give_take_coins** ([Player](#) &player)

Runs one full betting round end-to-end and credits or debits the player's coins.
- void **end_roulette** ([Player](#) &player)

Announces the end of the roulette round for the given player to all connected clients.
- void **setBet** ([Bet_type](#) type, int amount, int choice)

Records a bet from the network and wakes up any thread blocked in [choose_bet_type\(\)](#).

Public Attributes

- [Bet_type](#) **bet_type**
- int **player_choice**
- int **bet_amount**

10.72.1 Detailed Description

Runs one roulette betting round for a single player, driven by network messages from the AR/Unity client. Owns the roll-to-result logic (via [Generator](#)) and blocks the calling thread in [choose_bet_type\(\)](#) until the client's bet arrives over the network, synchronizing between the asio message-handler thread ([setBet\(\)](#)) and whichever thread drives the game turn ([give_take_coins\(\)](#)).

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/roulette.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/roulette.cpp](#)

10.73 ns::SelectedItemEvent Struct Reference

Payload describing a player's item selection from the shop UI.

```
#include <network_messages.hpp>
```

Public Attributes

- int **playerId**
- int **itemID**

10.73.1 Detailed Description

Payload describing a player's item selection from the shop UI.

Note

No handler currently deserializes this event in [shop.cpp](#); item selection is instead delivered to [Shop::setSelectedItem\(int\)](#) directly. itemID is expected to match the "id" field of a [Shop::getShopInventory\(\)](#) entry.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp](#)

10.74 Server Class Reference

Verwaltet den TCP-Netzwerk-Server und die Verteilung von Nachrichten an verbundene Clients.

```
#include <TCP_Server.hpp>
```

Public Member Functions

- **Server** (const [Server](#) &)=delete
- **Server** & **operator=** (const [Server](#) &)=delete
- **Server** ([Server](#) &&)=delete
- **Server** & **operator=** ([Server](#) &&)=delete
- **Server** (net::io_context &ioc, int port, std::shared_ptr< [SessionRegistry](#) > registry)
Initialisiert den TCP-Server auf einem bestimmten Port.
- auto **start_accept** () -> void
Startet das asynchrone Annehmen neuer Client-Verbindungen.
- auto **send_to_user** (ConnectionID user_id, std::string_view message, std::optional< [LogLevel](#)[LogLevel](#) > [LogLevel](#)[LogLevel](#)=[LogLevel](#)::Info, std::string_view category="game") const -> void
Sends a raw string message to a specific connected user. Passes the data payload down to the underlying TCP connection socket.
- auto **broadcast_to_all** (std::string_view message, std::optional< [LogLevel](#)[LogLevel](#) > [LogLevel](#)[LogLevel](#)=[LogLevel](#)::Info, std::string_view category="game") const -> void
Broadcasts a raw string message to all currently connected users. Iterates through the active connections list and invokes send_to_user on each.
- void **start_heartbeat** ()
Starts the periodic heartbeat timer. Sends a {"type":"ping"} to all connected clients every 5 seconds so they can keep their sessions alive by responding with {"type":"pong"}.
- auto **send_json** (ConnectionID user_id, const nlohmann::json &message, std::optional< [LogLevel](#)[LogLevel](#) > [LogLevel](#)[LogLevel](#)=[LogLevel](#)::Info, std::string_view category="game") const -> void
Serializes a JSON object and sends it to a specific user.
- auto **broadcast_json** (const nlohmann::json &message, std::optional< [LogLevel](#)[LogLevel](#) > [LogLevel](#)[LogLevel](#)=[LogLevel](#)::Info, std::string_view category="game") const -> void
Serializes a JSON object and broadcasts it to all connected users. Formats the JSON payload with a delimiter and distributes it across the entire network.
- std::shared_ptr< [SessionRegistry](#) > **get_registry** () const
Returns the shared session registry used by this server.

Friends

- class **TcpConnection**

10.74.1 Detailed Description

Verwaltet den TCP-Netzwerk-Server und die Verteilung von Nachrichten an verbundene Clients.

10.74.2 Constructor & Destructor Documentation

10.74.2.1 Server()

```
Server::Server (
    net::io_context & ioc,
    int port,
    std::shared_ptr< SessionRegistry > registry)
```

Initialisiert den TCP-Server auf einem bestimmten Port.

Initializes the server, setting up the acceptor.

Parameters

<i>ioc</i>	Boost.Asio I/O Context.
<i>port</i>	Portnummer (z. B. 1234).
<i>registry</i>	Shared Pointer zum SessionRegistry .

10.74.3 Member Function Documentation

10.74.3.1 broadcast_json()

```
void Server::broadcast_json (
    const nlohmann::json & message,
    std::optional< LogLevel > LogLevel = LogLevel::Info,
    std::string_view category = "game") const -> void
```

Serializes a JSON object and broadcasts it to all connected users. Formats the JSON payload with a delimiter and distributes it across the entire network.

Serializes a JSON object and broadcasts it to all users.

Parameters

<i>message</i>	The JSON object payload to broadcast.
----------------	---------------------------------------

10.74.3.2 broadcast_to_all()

```
void Server::broadcast_to_all (
    std::string_view message,
    std::optional< LogLevel > LogLevel = LogLevel::Info,
    std::string_view category = "game") const -> void
```

Broadcasts a raw string message to all currently connected users. Iterates through the active connections list and invokes `send_to_user` on each.

Broadcasts a raw string to all connected users.

Parameters

<i>message</i>	The raw string data to transmit to everyone.
----------------	--

10.74.3.3 send_to_user()

```
void Server::send_to_user (
    ConnectionID user_id,
    std::string_view message,
    std::optional< LogLevel > LogLevel = LogLevel::Info,
    std::string_view category = "game") const -> void
```

Sends a raw string message to a specific connected user. Passes the data payload down to the underlying TCP connection socket.

Sends a message to a specific connected user.

Parameters

<i>user_id</i>	Connection ID of the target client.
<i>message</i>	The raw string data to be transmitted.
<i>user_socket</i>	Shared pointer to the target connection.
<i>message</i>	The string message to send.

10.74.3.4 start_accept()

```
void Server::start_accept () -> void
```

Startet das asynchrone Annehmen neuer Client-Verbindungen.

Accepts new incoming connections asynchronously.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.cpp](#)

10.75 SessionRegistry Class Reference

Public Member Functions

- ConnectionID **create_session** (std::string_view name, std::shared_ptr< [TcpConnection](#) > conn)
- void **update_udp_endpoint** (ConnectionID id, const boost::asio::ip::udp::endpoint &ep)
- boost::asio::ip::udp::endpoint **get_udp_endpoint** (ConnectionID id) const
- void **remove_session** (ConnectionID id)
- void **touch_session** (ConnectionID id)
- void **record_traffic** (ConnectionID id, std::uint64_t tx, std::uint64_t rx)
- void **record_traffic** (ConnectionID id, std::uint64_t tx, std::uint64_t rx, std::string_view category)
- void **sweep_stale_sessions** (std::chrono::seconds timeout)
- ConnectionID **get_id_by_name** (std::string_view name) const
- [PlayerSession](#) **get_session** (ConnectionID id) const
- std::unordered_map< ConnectionID, boost::asio::ip::udp::endpoint > **get_all_udp_endpoints** () const
- std::unordered_map< ConnectionID, [PlayerSession](#) > **get_all_sessions** () const

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/session_registry.hpp
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/session_registry.cpp

10.76 Shop Class Reference

The in-game item shop: a fixed catalog of purchasable items, opened/closed via the "open_shop"/"close_shop" network messages (see [openShop\(\)](#), [close_shop\(\)](#)). An item selection made on the AR/Unity client is intended to arrive asynchronously via [setSelectedItem\(\)](#) and be consumed by [selectItem\(\)](#), which blocks the calling thread until a selection is made, after which [purchaseItem\(\)](#) charges the player; neither pathway is currently wired to a network message handler in this codebase.

```
#include <shop.hpp>
```

Public Member Functions

- **Shop** ([Server](#) &server)
- void **openShop** ([Player](#) &player)
Opens the shop for the given player and broadcasts its inventory to all clients.
- int **getItemPrice** ([ItemType](#) type)
Looks up the price of an item type in the shop's inventory.
- void **close_shop** ([Player](#) &player)
Closes the shop for the given player and broadcasts a "shop_close" message.
- [Item](#) **selectItem** ()
Blocks the calling thread until a shop item selection is delivered.
- void **purchaseItem** ([Player](#) &player)
Runs the full purchase flow: waits for a selection, then charges the player.
- void **setSelectedItem** (int item_id)
Delivers an item selection to a thread blocked in [selectItem\(\)](#).
- const std::vector< [ItemData](#) > & **getShopInventory** () const

10.76.1 Detailed Description

The in-game item shop: a fixed catalog of purchasable items, opened/closed via the "open_shop"/"close_shop" network messages (see [openShop\(\)](#), [close_shop\(\)](#)). An item selection made on the AR/Unity client is intended to arrive asynchronously via [setSelectedItem\(\)](#) and be consumed by [selectItem\(\)](#), which blocks the calling thread until a selection is made, after which [purchaseItem\(\)](#) charges the player; neither pathway is currently wired to a network message handler in this codebase.

The documentation for this class was generated from the following files:

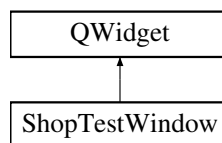
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.cpp](#)

10.77 ShopTestWindow Class Reference

Manual test/debug panel with buttons to trigger a shop-open or roulette-start broadcast for a chosen player id, without going through the actual game flow. Constructs (and reuses) a throwaway [Player](#) instance purely to satisfy the Shop/Roulette APIs, which take a [Player&](#) but only read its current player id.

```
#include <shop_test_window.hpp>
```

Inheritance diagram for ShopTestWindow:



Public Member Functions

- [ShopTestWindow](#) ([Shop](#) &shop, [Roulette](#) &roulette, QWidget *parent=nullptr)
Builds the test panel UI (player-id spin box and open-shop/start-roulette buttons) and wires their clicks to the corresponding handlers.
- [~ShopTestWindow](#) () override=default
Destroys the window; child widgets are cleaned up via Qt's parent-child ownership.

10.77.1 Detailed Description

Manual test/debug panel with buttons to trigger a shop-open or roulette-start broadcast for a chosen player id, without going through the actual game flow. Constructs (and reuses) a throwaway [Player](#) instance purely to satisfy the Shop/Roulette APIs, which take a [Player&](#) but only read its current player id.

10.77.2 Constructor & Destructor Documentation

10.77.2.1 ShopTestWindow()

```
ShopTestWindow::ShopTestWindow (
    Shop & shop,
    Roulette & roulette,
    QWidget * parent = nullptr) [explicit]
```

Builds the test panel UI (player-id spin box and open-shop/start-roulette buttons) and wires their clicks to the corresponding handlers.

Builds the test panel UI and wires button clicks to their handlers.

Parameters

<i>shop</i>	Shop instance to trigger via the "Open Shop" button.
<i>roulette</i>	Roulette instance to trigger via the "Start Roulette" button.
<i>parent</i>	Optional owning widget.

The documentation for this class was generated from the following files:

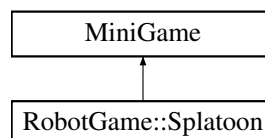
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.cpp](#)

10.78 RobotGame::Splatoon Class Reference

Mini-game where each robot paints cells of a 9x15 grid by driving over them. Converts camera-tracked robot positions into grid cells, mirrors the board state and countdown to every player's phone over UDP, and scores by painted-cell count once the fixed-length round (DURATION_SECONDS) ends.

```
#include <splatoon.hpp>
```

Inheritance diagram for RobotGame::Splatoon:



Public Member Functions

- [Splatoon](#) ()
Default-constructs the mini-game. Phone addresses are discovered later, at runtime, from the source IP of each phone's packets in `handleCellClaim()`, so there is nothing to set up here.
- [MiniGameResult play](#) (std::vector< [MiniGameParticipant](#) > &participants) override
Runs one round of [Splatoon](#) to completion and returns the outcome. Starts a UDP receiver for phone packets on port 12346, resets the grid and per-participant positions, then blocks the calling thread in a passive loop - republishing the grid and countdown to all known phones at least once per second - until `DURATION_SECONDS` have elapsed. [Robot](#) cell tracking and phone notification happen asynchronously as the tracker reports new robot positions via `updateRobotPosition()`. After the timer expires, stops the UDP receiver, tallies the final ranking, applies coin rewards, and broadcasts the winner before returning.
- std::string [getName](#) () const override
Returns the mini-game's display name ("Splatoon").
- void [updateRobotPosition](#) (int participantIndex, double x_cm, double y_cm)
Updates a robot's position from continuous camera-tracker data and notifies its phone of the new cell. Converts the tracked centimeter coordinates into a grid cell (removing the board's Y-axis margin offset, inverting the row to match the phone's top-down convention, and clipping to the grid bounds). If the robot has moved into a new cell since the last update, records the new cell and notifies that player's phone via `sendStartCell()` so it can paint the cell locally and report it back for scoring.
- void [onRobotMoved](#) (int index, double x_cm, double y_cm) override
[MiniGame](#) interface hook: forwards the tracker's robot position to `updateRobotPosition()`. Lets [GameManager](#) stay generic - it calls `onRobotMoved()` on whichever mini-game is active without needing to know that it is [Splatoon](#).

Public Member Functions inherited from [MiniGame](#)

- virtual ~[MiniGame](#) ()=default
Virtual destructor; allows [GameManager](#) to own mini-games polymorphically.

10.78.1 Detailed Description

Mini-game where each robot paints cells of a 9x15 grid by driving over them. Converts camera-tracked robot positions into grid cells, mirrors the board state and countdown to every player's phone over UDP, and scores by painted-cell count once the fixed-length round (`DURATION_SECONDS`) ends.

10.78.2 Constructor & Destructor Documentation

10.78.2.1 [Splatoon](#)()

```
RobotGame::Splatoon::Splatoon ()
```

Default-constructs the mini-game. Phone addresses are discovered later, at runtime, from the source IP of each phone's packets in `handleCellClaim()`, so there is nothing to set up here.

Default-constructs the mini-game; phone addresses are discovered later.

10.78.3 Member Function Documentation

10.78.3.1 getName()

```
std::string RobotGame::Splatoon::getName () const [override], [virtual]
```

Returns the mini-game's display name ("Splatoon").

Implements [MiniGame](#).

10.78.3.2 onRobotMoved()

```
void RobotGame::Splatoon::onRobotMoved (
    int index,
    double x_cm,
    double y_cm) [override], [virtual]
```

[MiniGame](#) interface hook: forwards the tracker's robot position to [updateRobotPosition\(\)](#). Lets [GameManager](#) stay generic - it calls [onRobotMoved\(\)](#) on whichever mini-game is active without needing to know that it is [Splatoon](#).

[MiniGame](#) interface hook: forwards the tracker's robot position to [updateRobotPosition\(\)](#).

Parameters

<i>index</i>	Participant index, forwarded unchanged to updateRobotPosition() .
<i>x_cm</i>	Tracked X position in centimeters.
<i>y_cm</i>	Tracked Y position in centimeters.

Reimplemented from [MiniGame](#).

10.78.3.3 play()

```
MiniGameResult RobotGame::Splatoon::play (
    std::vector< MiniGameParticipant > & participants) [override], [virtual]
```

Runs one round of [Splatoon](#) to completion and returns the outcome. Starts a UDP receiver for phone packets on port 12346, resets the grid and per-participant positions, then blocks the calling thread in a passive loop - republishing the grid and countdown to all known phones at least once per second - until `DURATION_SECONDS` have elapsed. [Robot](#) cell tracking and phone notification happen asynchronously as the tracker reports new robot positions via [updateRobotPosition\(\)](#). After the timer expires, stops the UDP receiver, tallies the final ranking, applies coin rewards, and broadcasts the winner before returning.

Runs one round of [Splatoon](#) to completion and returns the outcome.

Parameters

<i>participants</i>	Participants entering the round; their coins are updated in place and the same data is copied into the returned result.
---------------------	---

Returns

The final participant states plus the winning playerId (-1 if there is no ranking, e.g. zero participants).

Note

Blocks the calling thread for the full duration of the round.

Implements [MiniGame](#).

10.78.3.4 updateRobotPosition()

```
void RobotGame::Splatoon::updateRobotPosition (
    int participantIndex,
    double x_cm,
    double y_cm)
```

Updates a robot's position from continuous camera-tracker data and notifies its phone of the new cell. Converts the tracked centimeter coordinates into a grid cell (removing the board's Y-axis margin offset, inverting the row to match the phone's top-down convention, and clipping to the grid bounds). If the robot has moved into a new cell since the last update, records the new cell and notifies that player's phone via `sendStartCell()` so it can paint the cell locally and report it back for scoring.

Updates a robot's position from continuous camera-tracker data and notifies its phone of the new cell.

Parameters

<i>participantIndex</i>	Index into <code>positions_</code> (and the participants vector passed to <code>play()</code>).
<i>x_cm</i>	Tracked X position in centimeters, board frame (long axis, unchanged).
<i>y_cm</i>	Tracked Y position in centimeters, board frame, before the margin offset.

Note

Thread-safe: acquires `gameMutex_` for the duration of the update.

The documentation for this class was generated from the following files:

- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon.hpp`
- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon.cpp`

10.79 SplatoonGrid Class Reference

9x15 grid modeling the physical board area painted during the Splatoon mini-game. Each cell holds 0 (neutral/↔ unpainted) or 1-4 (painted by that player). Row 0 is the top row, column 0 is the left column, and cells are stored row-major in a flat array (index = row * COLS + col). Pure data class - no Qt, no rendering.

```
#include <splatoon_grid.hpp>
```

Public Member Functions

- **SplatoonGrid ()**
Constructs the grid and clears every cell to neutral (0).
- `int get (int row, int col) const`
Returns the value stored at (row, col).
- `void set (int row, int col, int value)`
Overwrites the value at (row, col).
- `void clear (int value=0)`
Sets every cell to the same value.
- `int countValue (int value) const noexcept`
Counts how many cells currently hold the given value.
- `const std::array< int, SIZE > & cells () const noexcept`
Read-only access to the underlying flat storage.

Static Public Attributes

- static constexpr int **ROWS** = 9
- static constexpr int **COLS** = 15
- static constexpr int **SIZE** = ROWS * COLS

10.79.1 Detailed Description

9x15 grid modeling the physical board area painted during the Splatoon mini-game. Each cell holds 0 (neutral/↔ unpainted) or 1-4 (painted by that player). Row 0 is the top row, column 0 is the left column, and cells are stored row-major in a flat array (index = row * COLS + col). Pure data class - no Qt, no rendering.

10.79.2 Member Function Documentation**10.79.2.1 cells()**

```
const std::array< int, SplatoonGrid::SIZE > & SplatoonGrid::cells () const [nodiscard], [noexcept]
```

Read-only access to the underlying flat storage.

Returns

Row-major array of SIZE cells (index = row * COLS + col).

10.79.2.2 clear()

```
void SplatoonGrid::clear (
    int value = 0)
```

Sets every cell to the same value.

Parameters

<i>value</i>	Value to fill the whole grid with (default: 0 = neutral).
--------------	---

10.79.2.3 countValue()

```
int SplatoonGrid::countValue (
    int value) const [nodiscard], [noexcept]
```

Counts how many cells currently hold the given value.

Parameters

<i>value</i>	The cell value to count (0 = neutral, 1-4 = a player).
--------------	--

Returns

Number of matching cells; used for end-of-round scoring.

10.79.2.4 get()

```
int SplatoonGrid::get (
    int row,
    int col) const [nodiscard]
```

Returns the value stored at (row, col).

Parameters

<i>row</i>	Row index, 0 = top.
<i>col</i>	Column index, 0 = left.

Returns

0 for neutral, or 1-4 for the owning player.

Note

Throws `std::out_of_range` if row/col are outside the grid bounds.

10.79.2.5 set()

```
void SplatoonGrid::set (
    int row,
    int col,
    int value)
```

Overwrites the value at (row, col).

Parameters

<i>row</i>	Row index, 0 = top.
<i>col</i>	Column index, 0 = left.
<i>value</i>	0 for neutral, or 1-4 to mark the cell as painted by that player.

Note

Throws `std::out_of_range` if row/col are outside the grid bounds.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon_grid.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatoon_grid.cpp](#)

10.80 start_game Class Reference

Static Public Member Functions

- static void **startGame** (int playerCount, int roundCount, [GameManager](#) *gm, [Server](#) &server, [Shop](#) &shop)
- static void **setPlayerNames** (std::vector< std::string > names)
- static void **setGameManager** ([GameManager](#) *gm)
- static void **setFleet** ([mind::fleet::robot_fleet](#) *fleet)
- static void **update** ()
- static void **selectItem** (int itemIndex)
- static void **skipItemSelection** ()
- static void **selectPath** (int nodeId)
- static void **selectTargetPlayer** (int playerId)
- static [TurnManager](#) * **getTurnManager** ()

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/start_game.hpp
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/start_game.cpp

10.81 ns::StartRouletteEvent Struct Reference

Payload of the "roulette_start" message: identifies which player is starting a roulette round. Parsed in [roulette.cpp](#)'s "roulette_start" handler and used to build a temporary [Player](#) before calling [Roulette::start_roulette\(\)](#).

```
#include <network_messages.hpp>
```

Public Attributes

- int **playerId**

10.81.1 Detailed Description

Payload of the "roulette_start" message: identifies which player is starting a roulette round. Parsed in [roulette.cpp](#)'s "roulette_start" handler and used to build a temporary [Player](#) before calling [Roulette::start_roulette\(\)](#).

The documentation for this struct was generated from the following file:

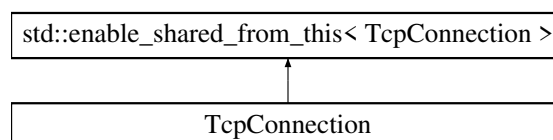
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/[network_messages.hpp](#)

10.82 TcpConnection Class Reference

Repräsentiert eine einzelne aktive TCP-Verbindung zu einem Client (Unity / AR-App / Gerät).

```
#include <TCP_Server.hpp>
```

Inheritance diagram for TcpConnection:



Public Member Functions

- [TcpConnection](#) (net::io_context &ioc, ConnectionID id, std::shared_ptr< [SessionRegistry](#) > registry)
Initializes the [TcpConnection](#) and logs the entry.
- auto [connect_to_server](#) (net::io_context &ioc, const std::string &host, const std::string &port) -> void
Verbindet sich asynchron mit einem Remote-Server (Client-Modus).
- auto [send_to_server](#) (const std::string &message) -> void
Sendet eine Rohdaten-Zeichenkette asynchron an den [Server](#).
- auto [start_read](#) () -> void
Startet die asynchrone Leseschleife für eingehende TCP-Pakete.

Public Attributes

- ConnectionID **ID** {1}
Eindeutige Verbindungs-ID (vom [SessionRegistry](#) zugewiesen).
- tcp::socket **socket**
Der unterliegende Boost.Asio TCP Socket.
- tcp::resolver **resolver**
- std::shared_ptr< [SessionRegistry](#) > **session_registry_**
- net::streambuf **read_buffer_** {}

10.82.1 Detailed Description

Repräsentiert eine einzelne aktive TCP-Verbindung zu einem Client (Unity / AR-App / Gerät).

10.82.2 Constructor & Destructor Documentation

10.82.2.1 TcpConnection()

```
TcpConnection::TcpConnection (
    net::io_context & ioc,
    ConnectionID id,
    std::shared_ptr< SessionRegistry > registry)
```

Initializes the [TcpConnection](#) and logs the entry.

Parameters

<i>ioc</i>	The io_context used for async operations.
<i>id</i>	Unique identifier for this connection.

10.82.3 Member Function Documentation

10.82.3.1 connect_to_server()

```
void TcpConnection::connect_to_server (
    net::io_context & ioc,
    const std::string & host,
    const std::string & port) -> void
```

Verbindet sich asynchron mit einem Remote-Server (Client-Modus).

Connects to a remote server asynchronously.

Parameters

<i>ioc</i>	The io_context.
<i>host</i>	The remote hostname or IP.
<i>port</i>	The port to connect to.

Note

Uses `shared_from_this()` to ensure the object persists during the async sequence.

10.82.3.2 send_to_server()

```
void TcpConnection::send_to_server (
    const std::string & message) -> void
```

Sendet eine Rohdaten-Zeichenkette asynchron an den [Server](#).

Sends a string message to the server asynchronously.

Parameters

<i>message</i>	The raw message to send.
----------------	--------------------------

10.82.3.3 start_read()

```
void TcpConnection::start_read () -> void
```

Startet die asynchrone Leseschleife für eingehende TCP-Pakete.

Starts the asynchronous read loop, listening for a delimiter.

Reads until `']'` is found, then processes the buffer.

The documentation for this class was generated from the following files:

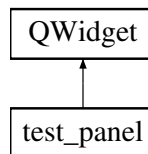
- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.hpp`
- `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.cpp`

10.83 test_panel Class Reference

Six-button panel for manually exercising the active robot's drive API. Listens to `robot_fleet::active_changed` to retarget itself. Buttons cover the Goal D behaviours: continuous drive (hot path + brick brake stop), precision drive_for / turn (calibration verify), stop interrupting a move mid-flight (drive_for + stop, turn + stop), and an emergency stop. Per-test QTimer's are stop()'d before each press so back-to-back clicks do not chop each other.

```
#include <test_panel.hpp>
```

Inheritance diagram for test_panel:



Public Member Functions

- [test_panel](#) ([mind::fleet::robot_fleet](#) *fleet, QWidget *parent=nullptr)

Builds the test panel UI and wires it to the given fleet. All test buttons start disabled; they are enabled once fleet reports an active robot.

10.83.1 Detailed Description

Six-button panel for manually exercising the active robot's drive API. Listens to `robot_fleet::active_changed` to retarget itself. Buttons cover the Goal D behaviours: continuous drive (hot path + brick brake stop), precision drive_for / turn (calibration verify), stop interrupting a move mid-flight (drive_for + stop, turn + stop), and an emergency stop. Per-test QTimer's are stop()'d before each press so back-to-back clicks do not chop each other.

10.83.2 Constructor & Destructor Documentation

10.83.2.1 test_panel()

```
test_panel::test_panel (
    mind::fleet::robot_fleet * fleet,
    QWidget * parent = nullptr) [explicit]
```

Builds the test panel UI and wires it to the given fleet. All test buttons start disabled; they are enabled once fleet reports an active robot.

Builds the test panel UI and wires it to the given fleet.

Parameters

<i>fleet</i>	Robot fleet supplying the active robot and its active_changed signal; not owned.
<i>parent</i>	Optional Qt parent widget for ownership/layout.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.cpp](#)

10.84 TileEffect Struct Reference

One effect entry attached to a board node. The actual gameplay logic for each type (coin gain/loss, opening the shop) is implemented in [applyTileEffect\(\)](#) ([player/boardEffects.cpp](#)), not here - this struct only carries the data.

```
#include <board.hpp>
```

Public Attributes

- [TileType](#) **type**
- int **power**

Magnitude selector, not a raw amount: for Plus/Minus, 0 = normal +-3 coins and non-zero = boosted +-6 coins ("last turns" bonus). Unused for Shop/Hazard.

10.84.1 Detailed Description

One effect entry attached to a board node. The actual gameplay logic for each type (coin gain/loss, opening the shop) is implemented in [applyTileEffect\(\)](#) ([player/boardEffects.cpp](#)), not here - this struct only carries the data.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.hpp](#)

10.85 TopoClient Struct Reference

One client entry for the topology diagram.

```
#include <network_topology_widget.hpp>
```

Public Attributes

- QString **label**
e.g. "AR_01" or "ev3_00"
- bool **tcp_online** = false
- bool **udp_online** = false
- int **game_bps** = 0
game data bytes/sec (server->client)
- int **response_bps** = 0
response bytes/sec (client->server)
- int **heartbeat_bps** = 0
ping/pong bytes/sec
- int **robot_bps** = 0
UDP robot telemetry bytes/sec.
- bool **is_robot** = false
true = robot (UDP), false = Unity (TCP)
- int **pairing_id** = -1
-1 = no pairing; same ID = paired Unity<->[Robot](#)

10.85.1 Detailed Description

One client entry for the topology diagram.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.hpp](#)

10.86 TopoPulse Struct Reference

A single animated pulse traveling along a link arrow.

```
#include <network_topology_widget.hpp>
```

Public Attributes

- float **pos** {0.0f}
0.0 = source end, 1.0 = destination end
- float **speed** {0.0f}
position units per tick (~30 Hz)
- float **alpha** {0.0f}
base brightness [0..1]
- float **radius** {0.0f}
glow radius in pixels

10.86.1 Detailed Description

A single animated pulse traveling along a link arrow.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.hpp](#)

10.87 TrackedRobot Struct Reference

Public Attributes

- int **slot_id**
- int **marker_id**
- double **x_m**
- double **y_m**

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/background_thread.hpp](#)

10.88 TurnManager Class Reference

Per-game state machine that drives a single player through one turn: item usage, dice roll, movement, tile effects, and (at the end of a round) the minigame phase. Owns no gameplay data itself; each handler mutates the associated Game/Player and advances state. Progression through most states is driven by [update\(\)](#), but the WaitingFor* states pause until the matching select/skip method is called (e.g. from network/UI input).

```
#include <turnManager.hpp>
```

Public Types

- enum [TurnState](#) {
[StartRound](#) , [StartTurn](#) , [WaitingForItemChoice](#) , [WaitingForTargetChoice](#) ,
[Rolling](#) , [Moving](#) , [WaitingForPathChoice](#) , [ApplyingTile](#) ,
[Minigame](#) , [EndTurn](#) , [EndRound](#) , [EndGame](#) }

Phases of a single player's turn, also used to drive round/game lifecycle transitions. [update\(\)](#) dispatches on this value to the matching handler; the WaitingFor states have no case in [update\(\)](#) and instead only advance once the corresponding select/skip method is called.*

Public Member Functions

- [Player](#) & [currentPlayer](#) ()
Returns the player whose turn is currently active.
- [TurnManager](#) ([Game](#) &game, [Server](#) &server, [Shop](#) &shop)
Constructs a turn manager bound to the given game session. Initializes all selection/step counters to their "nothing selected" values and sets the initial state to StartTurn.
- [~TurnManager](#) ()
Joins the minigame thread if one was launched.
- void [setGameManager](#) ([GameManager](#) *gm)
Wires an optional [GameManager](#) so [startMinigame\(\)](#) can launch Splatoon.
- void [setFleet](#) ([mind::fleet::robot_fleet](#) *fleet)
Wires the robot fleet so [movePlayer\(\)](#) can issue physical drive commands.
- [TurnState](#) [getState](#) () const
this was private. on purpose?
- int [lastRoll](#) () const
Returns the dice value from the most recent roll.
- void [update](#) ()
Advances the state machine by one step according to the current state. Dispatches to the handler for the current [TurnState](#) ([startTurn\(\)](#), [rollDice\(\)](#), [movePlayer\(\)](#), [applyTile\(\)](#), [finishTurn\(\)](#), [finishRound\(\)](#), [startMinigame\(\)](#)) or, for [EndGame](#), marks [Game::_finished](#). The WaitingFor states have no case here and instead advance only via the corresponding select/skip method.*
- void [startTurn](#) ()
Begins the current player's turn and moves to item selection. Calls [Player::startTurn\(\)](#) (start-of-turn effect bookkeeping) then transitions to [WaitingForItemChoice](#).
- void [selectItem](#) (int inventoryIndex)
Records the player's chosen inventory item and immediately resolves it.
- void [skipItemSelection](#) ()
Skips item usage for this turn and proceeds directly to the dice roll.
- void [executeSelectedItem](#) ()
Resolves the item selected via [selectItem\(\)](#). If the item is missing or is a dice item, falls through to [Rolling](#) without consuming it (dice items are instead applied inside [rollDice\(\)](#)). If it requires a target, transitions to [WaitingForTargetChoice](#) instead of applying it here. Otherwise applies its effects immediately (with no target), removes it from the inventory, records it as used, and transitions to [Rolling](#).

- void [selectTargetPlayer](#) (int playerId, int robotId)
Resolves an item that requires a target player, then proceeds to the dice roll. Looks up the target by playerId in [Game::_players](#), applies the selected item's effects against it, removes the item from the current player's inventory, and transitions to Rolling.
- void [rollDice](#) ()
Rolls the dice for the current player's movement and transitions to Moving. If an item is selected and its [getDiceOverride\(\)](#) returns a value (e.g. a fixed/boosted roll), that override is used instead of an actual dice throw, and the item is consumed. Otherwise rolls a fresh [Dice](#). Resets the player's dice multiplier and clears the item selection.
- void [movePlayer](#) ()
Advances the player one board node per call while dice steps remain. When `_remainingSteps` reaches zero, stops movement and transitions to ApplyingTile. Otherwise inspects the current node's neighbors: more than one transitions to WaitingForPathChoice and waits for [selectPath\(\)](#); none transitions directly to ApplyingTile; exactly one moves the player onto it, increments the moved-fields stat, and decrements `_remainingSteps`.
- void [selectPath](#) (int nodeId)
Resolves a branching-node movement choice and continues moving. Moves the current player onto `nodeId`, increments the moved-fields stat, decrements `_remainingSteps`, and transitions back to Moving so [movePlayer\(\)](#) continues consuming any remaining steps.
- void [applyTile](#) ()
Applies the current player's landed-on tile effects and ends the turn. Looks up the current player's node and delegates to [applyTileEffect\(\)](#), then transitions to EndTurn.
- void [finishTurn](#) ()
Finalizes the current player's turn and advances to the next player or the minigame. Applies end-of-turn effects, clears the active-player flag, and either transitions to Minigame (if this was the last player in turn order) or advances [Game](#) to the next player and returns to StartTurn.
- void [finishRound](#) ()
Advances the round counter and ends the game once the round limit is reached. Delegates to [Game::nextRound\(\)](#); if that marks the game finished, runs [BonusPhase::execute\(\)](#) and transitions to EndGame, otherwise transitions back to StartRound.
- void [startMinigame](#) ()
Placeholder for running the end-of-round minigame, then proceeds to round cleanup. Currently copies all players into a local vector but does not run any minigame ([GameManager/minigame](#) creation are commented out); the copy is discarded and the state unconditionally transitions to EndRound.

Public Attributes

- [Game](#) & [game](#)
The game session this turn manager drives.
- [TurnState](#) [state](#)
Current phase of the state machine; see [TurnState](#).

10.88.1 Detailed Description

Per-game state machine that drives a single player through one turn: item usage, dice roll, movement, tile effects, and (at the end of a round) the minigame phase. Owns no gameplay data itself; each handler mutates the associated Game/Player and advances state. Progression through most states is driven by [update\(\)](#), but the WaitingFor* states pause until the matching select/skip method is called (e.g. from network/UI input).

10.88.2 Member Enumeration Documentation

10.88.2.1 TurnState

```
enum TurnManager::TurnState
```

Phases of a single player's turn, also used to drive round/game lifecycle transitions. `update()` dispatches on this value to the matching handler; the `WaitingFor*` states have no case in `update()` and instead only advance once the corresponding select/skip method is called.

Enumerator

StartRound	Beginning of a new round; <code>update()</code> immediately advances to StartTurn.
StartTurn	Beginning of a player's turn; handled by <code>startTurn()</code> .
WaitingForItemChoice	Waiting for <code>selectItem()</code> or <code>skipItemSelection()</code> .
WaitingForTargetChoice	Waiting for <code>selectTargetPlayer()</code> to resolve a targeted item.
Rolling	<code>Dice</code> roll phase; handled by <code>rollDice()</code> .
Moving	Step-by-step movement phase; handled by <code>movePlayer()</code> .
WaitingForPathChoice	Waiting for <code>selectPath()</code> when the current node has multiple neighbors.
ApplyingTile	Applying the landed-on tile's effects; handled by <code>applyTile()</code> .
Minigame	End-of-round minigame phase; handled by <code>startMinigame()</code> .
EndTurn	Turn cleanup phase; handled by <code>finishTurn()</code> .
EndRound	Round cleanup phase; handled by <code>finishRound()</code> .
EndGame	Terminal state; <code>update()</code> sets <code>Game::_finished</code> to true.

10.88.3 Constructor & Destructor Documentation

10.88.3.1 TurnManager()

```
TurnManager::TurnManager (
    Game & game,
    Server & server,
    Shop & shop)
```

Constructs a turn manager bound to the given game session. Initializes all selection/step counters to their "nothing selected" values and sets the initial state to StartTurn.

Constructs a turn manager bound to the given game session.

Parameters

<code>game</code>	The game session whose turns this instance will drive.
-------------------	--

10.88.4 Member Function Documentation

10.88.4.1 applyTile()

```
void TurnManager::applyTile ()
```

Applies the current player's landed-on tile effects and ends the turn. Looks up the current player's node and delegates to `applyTileEffect()`, then transitions to EndTurn.

Applies the current player's landed-on tile effects and ends the turn.

10.88.4.2 currentPlayer()

```
Player & TurnManager::currentPlayer ()
```

Returns the player whose turn is currently active.

Returns

Reference obtained via [Game::getCurrentPlayer\(\)](#).

10.88.4.3 executeSelectedItem()

```
void TurnManager::executeSelectedItem ()
```

Resolves the item selected via [selectItem\(\)](#). If the item is missing or is a dice item, falls through to Rolling without consuming it (dice items are instead applied inside [rollDice\(\)](#)). If it requires a target, transitions to WaitingFor↔TargetChoice instead of applying it here. Otherwise applies its effects immediately (with no target), removes it from the inventory, records it as used, and transitions to Rolling.

Resolves the item selected via [selectItem\(\)](#).

10.88.4.4 finishRound()

```
void TurnManager::finishRound ()
```

Advances the round counter and ends the game once the round limit is reached. Delegates to [Game::nextRound\(\)](#); if that marks the game finished, runs [BonusPhase::execute\(\)](#) and transitions to EndGame, otherwise transitions back to StartRound.

Advances the round counter and ends the game once the round limit is reached.

10.88.4.5 finishTurn()

```
void TurnManager::finishTurn ()
```

Finalizes the current player's turn and advances to the next player or the minigame. Applies end-of-turn effects, clears the active-player flag, and either transitions to Minigame (if this was the last player in turn order) or advances [Game](#) to the next player and returns to StartTurn.

Finalizes the current player's turn and advances to the next player or the minigame.

10.88.4.6 getState()

```
TurnManager::TurnState TurnManager::getState () const
```

this was private. on purpose?

Returns the current turn state.

Returns the current turn state.

Returns

The current [TurnState](#) (same value as the public state member).

10.88.4.7 movePlayer()

```
void TurnManager::movePlayer ()
```

Advances the player one board node per call while dice steps remain. When `_remainingSteps` reaches zero, stops movement and transitions to `ApplyingTile`. Otherwise inspects the current node's neighbors: more than one transitions to `WaitingForPathChoice` and waits for [selectPath\(\)](#); none transitions directly to `ApplyingTile`; exactly one moves the player onto it, increments the moved-fields stat, and decrements `_remainingSteps`.

Advances the player one board node per call while dice steps remain.

Note

[update\(\)](#) calls this once per invocation, so covering multiple steps requires [update\(\)](#) to be called repeatedly.

10.88.4.8 rollDice()

```
void TurnManager::rollDice ()
```

Rolls the dice for the current player's movement and transitions to `Moving`. If an item is selected and its `getDiceOverride()` returns a value (e.g. a fixed/boosted roll), that override is used instead of an actual dice throw, and the item is consumed. Otherwise rolls a fresh [Dice](#). Resets the player's dice multiplier and clears the item selection.

Rolls the dice for the current player's movement and transitions to `Moving`.

Note

The rolled/overridden value is not multiplied by [Player::getDiceMultiplier\(\)](#); `resetDiceMultiplier()` clears the multiplier without it ever having been applied to the roll.

10.88.4.9 selectItem()

```
void TurnManager::selectItem (  
    int inventoryIndex)
```

Records the player's chosen inventory item and immediately resolves it.

Parameters

<i>inventoryIndex</i>	Index into the current player's inventory (see Inventory::getItem).
-----------------------	--

10.88.4.10 selectPath()

```
void TurnManager::selectPath (
    int nodeId)
```

Resolves a branching-node movement choice and continues moving. Moves the current player onto `nodeId`, increments the moved-fields stat, decrements `_remainingSteps`, and transitions back to Moving so [movePlayer\(\)](#) continues consuming any remaining steps.

Resolves a branching-node movement choice and continues moving.

Parameters

<i>node↔ Id</i>	Board graph node id the player chose to move to.
---------------------	--

10.88.4.11 selectTargetPlayer()

```
void TurnManager::selectTargetPlayer (
    int playerId,
    int robotId)
```

Resolves an item that requires a target player, then proceeds to the dice roll. Looks up the target by `playerId` in [Game::_players](#), applies the selected item's effects against it, removes the item from the current player's inventory, and transitions to Rolling.

Resolves an item that requires a target player, then proceeds to the dice roll.

Parameters

<i>player↔ Id</i>	Id of the chosen target player.
<i>robotId</i>	Unused by this function.

Note

If the current player has no item at `_selectedItemIndex`, returns without changing state, leaving the state machine stuck outside Rolling.

10.88.4.12 setFleet()

```
void TurnManager::setFleet (
    mind::fleet::robot_fleet * fleet)
```

Wires the robot fleet so [movePlayer\(\)](#) can issue physical drive commands.

Parameters

<i>fleet</i>	Non-owning pointer to robot_fleet.
--------------	------------------------------------

10.88.4.13 startMinigame()

```
void TurnManager::startMinigame ()
```

Placeholder for running the end-of-round minigame, then proceeds to round cleanup. Currently copies all players into a local vector but does not run any minigame (GameManager/minigame creation are commented out); the copy is discarded and the state unconditionally transitions to EndRound.

Launches Splatton via [GameManager](#) on a background thread, or skips to EndRound if no [GameManager](#) is wired.

10.88.4.14 startTurn()

```
void TurnManager::startTurn ()
```

Begins the current player's turn and moves to item selection. Calls [Player::startTurn\(\)](#) (start-of-turn effect bookkeeping) then transitions to WaitingForItemChoice.

Begins the current player's turn and moves to item selection.

10.88.4.15 update()

```
void TurnManager::update ()
```

Advances the state machine by one step according to the current state. Dispatches to the handler for the current [TurnState](#) ([startTurn\(\)](#), [rollDice\(\)](#), [movePlayer\(\)](#), [applyTile\(\)](#), [finishTurn\(\)](#), [finishRound\(\)](#), [startMinigame\(\)](#)) or, for EndGame, marks [Game::_finished](#). The WaitingFor* states have no case here and instead advance only via the corresponding select/skip method.

Advances the state machine by one step according to the current state.

Note

Throws `std::runtime_error` if state holds a value not covered by the switch.

The documentation for this class was generated from the following files:

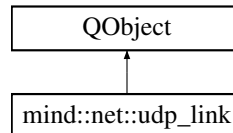
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.cpp](#)

10.89 mind::net::udp_link Class Reference

Thin UDP transport for loss-tolerant streams: sends command datagrams to a brick and receives heartbeats on a bound port. Decoding is the caller's job (mind::proto); this layer only moves raw bytes. Runs on its own QThread; send_command is self-marshalling, so callers on any thread queue onto this object's thread via QMetaObject::invokeMethod.

```
#include <udp_link.hpp>
```

Inheritance diagram for mind::net::udp_link:



Public Slots

- void [start](#) ()
Binds the listen socket and arms the readyRead notifier.
- void [send_command](#) (const QHostAddress &addr, quint16 port, const QByteArray &payload)
Fire-and-forget send of one UDP datagram.

Signals

- void [datagram_received](#) (QByteArray payload, QHostAddress sender)
Emitted once per received UDP datagram.
- void [command_sent](#) ()
Emitted after writeDatagram() returns on the worker thread. Used by the [latency_indicator](#) widget to flash on every desktop -> brick send so a high-FPS recording can frame-count desktop->wheel latency.

Public Member Functions

- [udp_link](#) (quint16 listen_port, QObject *parent=nullptr)
Constructs the endpoint and creates the underlying socket, without binding it.
- [~udp_link](#) () override
Destroys the [udp_link](#); the owned QUdpSocket is released via Qt parent-child ownership.
- [udp_link](#) (const udp_link &)=delete
- [udp_link](#) & [operator=](#) (const [udp_link](#) &)=delete

10.89.1 Detailed Description

Thin UDP transport for loss-tolerant streams: sends command datagrams to a brick and receives heartbeats on a bound port. Decoding is the caller's job (mind::proto); this layer only moves raw bytes. Runs on its own QThread; send_command is self-marshalling, so callers on any thread queue onto this object's thread via QMetaObject::invokeMethod.

10.89.2 Constructor & Destructor Documentation

10.89.2.1 udp_link()

```
mind::net::udp_link::udp_link (
    quint16 listen_port,
    QObject * parent = nullptr) [explicit]
```

Constructs the endpoint and creates the underlying socket, without binding it.

Parameters

<i>listen_port</i>	Local UDP port this endpoint will bind to once start() runs.
<i>parent</i>	Optional QObject parent for ownership/cleanup.

10.89.3 Member Function Documentation

10.89.3.1 datagram_received

```
void mind::net::udp_link::datagram_received (
    QByteArray payload,
    QHostAddress sender) [signal]
```

Emitted once per received UDP datagram.

Parameters

<i>payload</i>	Raw datagram bytes; decoding is left to the caller (mind::proto).
<i>sender</i>	Source address of the datagram - the desktop learns each brick's command IP from this, since no IP is carried in the payload itself.

10.89.3.2 send_command

```
void mind::net::udp_link::send_command (
    const QHostAddress & addr,
    quint16 port,
    const QByteArray & payload) [slot]
```

Fire-and-forget send of one UDP datagram.

Parameters

<i>addr</i>	Destination IP address.
<i>port</i>	Destination UDP port.
<i>payload</i>	Raw bytes to transmit; framing/encoding is the caller's responsibility.

Note

Safe to call from any thread; if called from a thread other than this object's own, the send is queued via QMetaObject::invokeMethod so the actual writeDatagram happens on this [udp_link](#)'s owning thread.

10.89.3.3 start

```
void mind::net::udp_link::start () [slot]
```

Binds the listen socket and arms the readyRead notifier.

Note

Must run on the owning thread: invoke after moveToThread + QThread::start via a queued connection, else the socket notifier arms on the wrong dispatcher and readyRead never fires.

The documentation for this class was generated from the following files:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/[udp_link.hpp](#)
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/[udp_link.cpp](#)

10.90 UdpClient Class Reference

Provides a UDP client interface for communicating with the server.

```
#include <UDP_Server.hpp>
```

Public Member Functions

- [UdpClient](#) (net::io_context &ioc, const std::string &host, const std::string &port)
Resolves the remote server's address and starts listening for its replies.
- void **send_to_server** (const std::string &message)
Sends a datagram to the server asynchronously.
- void [start_receive](#) ()
Initiates the receive loop to listen for server responses.

10.90.1 Detailed Description

Provides a UDP client interface for communicating with the server.

Resolves the server's address once at construction and exchanges datagrams with that single fixed endpoint.

10.90.2 Constructor & Destructor Documentation

10.90.2.1 UdpClient()

```
UdpClient::UdpClient (
    net::io_context & ioc,
    const std::string & host,
    const std::string & port)
```

Resolves the remote server's address and starts listening for its replies.

Parameters

<i>ioc</i>	The io_context used for async operations.
<i>host</i>	The hostname or IP address of the server.
<i>port</i>	The port to connect to.

Note

If address resolution fails, the error is logged and swallowed; the socket is left constructed but the receive loop is never started.

10.90.3 Member Function Documentation

10.90.3.1 start_receive()

```
void UdpClient::start_receive ()
```

Initiates the receive loop to listen for server responses.

Note

Already invoked once by the constructor; only needs to be called again internally to re-arm the loop after each datagram is handled.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.cpp](#)

10.91 UdpReceiver Class Reference

Listens for UDP datagrams on a fixed port and reports each one via a callback. Binds to INADDR_ANY on construction and spins up a dedicated background thread that blocks on `recvfrom()` for the lifetime of the object.

```
#include <udp_reciever.hpp>
```

Public Member Functions

- [UdpReceiver](#) (int port, std::function< void(const std::string &, const std::string &)> onMessage)
Binds a UDP socket on port and starts the background receive thread.
- [~UdpReceiver](#) ()
Stops the receive loop, closes the socket, and joins the background thread.

10.91.1 Detailed Description

Listens for UDP datagrams on a fixed port and reports each one via a callback. Binds to INADDR_ANY on construction and spins up a dedicated background thread that blocks on recvfrom() for the lifetime of the object.

10.91.2 Constructor & Destructor Documentation

10.91.2.1 UdpReceiver()

```
UdpReceiver::UdpReceiver (
    int port,
    std::function< void(const std::string &, const std::string &)> onMessage) [inline]
```

Binds a UDP socket on port and starts the background receive thread.

Parameters

<i>port</i>	Local UDP port to listen on (all interfaces).
<i>onMessage</i>	Callback invoked from the background thread for every packet received; receives the payload and the sender's IP (dotted-decimal string) so callers can reply to the source without hardcoding its address.

The documentation for this class was generated from the following file:

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/[udp_reciever.hpp](#)

10.92 UdpSender Class Reference

Fire-and-forget UDP sender for one fixed destination (ip:port). Create one instance per target and reuse it for every [send\(\)](#) call; each instance owns a single UDP socket for its lifetime.

```
#include <udp_sender.hpp>
```

Public Member Functions

- [UdpSender](#) (std::string_view ip, int port)
Opens a UDP socket and stores the destination address for later sends. On Windows, lazily initializes Winsock via a process-wide reference count shared across all [UdpSender](#) instances (see `wsaCount_`).
- [~UdpSender](#) ()
Closes the socket, if any, and releases the Winsock reference on Windows.
- [UdpSender](#) (const [UdpSender](#) &)=delete
Deleted: copying is disallowed since each instance uniquely owns a socket.
- [UdpSender](#) & **operator=** (const [UdpSender](#) &)=delete
- [UdpSender](#) ([UdpSender](#) &&other) noexcept
Transfers ownership of the socket and destination address from another instance. Leaves the source object holding `INVALID_SOCKET` so its destructor becomes a no-op.
- void [send](#) (std::string_view payload) const
Sends a datagram containing payload to the configured destination. Does nothing if the socket failed to initialize or the sender was moved-from.

10.92.1 Detailed Description

Fire-and-forget UDP sender for one fixed destination (ip:port). Create one instance per target and reuse it for every [send\(\)](#) call; each instance owns a single UDP socket for its lifetime.

10.92.2 Constructor & Destructor Documentation

10.92.2.1 UdpSender() [1/2]

```
UdpSender::UdpSender (
    std::string_view ip,
    int port) [inline]
```

Opens a UDP socket and stores the destination address for later sends. On Windows, lazily initializes Winsock via a process-wide reference count shared across all [UdpSender](#) instances (see `wsaCount_`).

Parameters

<i>ip</i>	Destination IPv4 address in dotted-decimal notation.
<i>port</i>	Destination UDP port.

10.92.2.2 UdpSender() [2/2]

```
UdpSender::UdpSender (
    UdpSender && other) [inline], [noexcept]
```

Transfers ownership of the socket and destination address from another instance. Leaves the source object holding `INVALID_SOCKET` so its destructor becomes a no-op.

Parameters

<i>other</i>	The sender to move from.
--------------	--------------------------

10.92.3 Member Function Documentation**10.92.3.1 send()**

```
void UdpSender::send (
    std::string_view payload) const [inline]
```

Sends a datagram containing payload to the configured destination. Does nothing if the socket failed to initialize or the sender was moved-from.

Parameters

<i>payload</i>	The raw bytes to send as the UDP packet body.
----------------	---

The documentation for this class was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/udp_sender.hpp](#)

10.93 UdpServer Class Reference

Manages a UDP server that tracks multiple connected clients.

```
#include <UDP_Server.hpp>
```

Public Member Functions

- [UdpServer](#) (net::io_context &ioc, uint16_t port, std::shared_ptr< [SessionRegistry](#) > registry)
Binds to the specified port and immediately starts listening for incoming packets.
- void [start_receive](#) ()
Initiates the asynchronous receive loop.
- void [send_to_user](#) (ConnectionID user_id, const std::string &message)
Sends a string message to a specific connected user.
- void [broadcast_to_all](#) (const std::string &message)
Sends a message to all currently connected users.
- void [network_broadcast](#) (uint16_t port, const std::string &message)

10.93.1 Detailed Description

Manages a UDP server that tracks multiple connected clients.

Since UDP does not maintain state, this class maps remote endpoints to ConnectionIDs to simulate a session-based communication.

10.93.2 Constructor & Destructor Documentation

10.93.2.1 UdpServer()

```
UdpServer::UdpServer (
    net::io_context & ioc,
    uint16_t port,
    std::shared_ptr< SessionRegistry > registry)
```

Binds to the specified port and immediately starts listening for incoming packets.

Binds to the specified port and immediately starts listening for incoming packets. It is important to note that the socket is already opened after the constructor call.

Parameters

<i>ioc</i>	The io_context used for async operations.
<i>port</i>	The port to listen on.
<i>registry</i>	shared instance of all connections.

10.93.3 Member Function Documentation

10.93.3.1 broadcast_to_all()

```
void UdpServer::broadcast_to_all (
    const std::string & message)
```

Sends a message to all currently connected users.

Parameters

<i>message</i>	The payload to broadcast.
----------------	---------------------------

Note

Each send is dispatched independently and asynchronously; a failure for one recipient is logged but does not affect delivery to the others.

10.93.3.2 send_to_user()

```
void UdpServer::send_to_user (
    ConnectionID user_id,
    const std::string & message)
```

Sends a string message to a specific connected user.

Parameters

<i>user_id</i>	The ID of the target user.
<i>message</i>	The payload to transmit.

Note

If `user_id` does not match a registered connection, the call is a no-op and a warning is logged.

10.93.3.3 start_receive()

```
void UdpServer::start_receive ()
```

Initiates the asynchronous receive loop.

Note

Already invoked once by the constructor; only needs to be called again internally to re-arm the loop after each datagram is handled.

The documentation for this class was generated from the following files:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.hpp](#)
- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.cpp](#)

10.94 Vec3< T > Struct Template Reference

Generic 3-component vector; used here to store a player's world-space (x, y, z) position.

```
#include <player.hpp>
```

Public Attributes

- `T x`
- `T y`
- `T z`

10.94.1 Detailed Description

```
template<typename T>  
struct Vec3< T >
```

Generic 3-component vector; used here to store a player's world-space (x, y, z) position.

The documentation for this struct was generated from the following file:

- [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.hpp](#)

Chapter 11

File Documentation

11.1 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/↵ latency_indicator.cpp File Reference

Implementation of the latency measurement indicator widget.

```
#include "latency_indicator.hpp"  
#include <QColor>  
#include <QPalette>  
#include <QTimer>
```

11.1.1 Detailed Description

Implementation of the latency measurement indicator widget.

11.2 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/↵ latency_indicator.hpp File Reference

Declares a small widget that visually flashes on each command sent, for latency measurement.

```
#include <QWidget>  
#include "utilities/Logger.hpp"
```

Classes

- class [latency_indicator](#)
Visual indicator for measuring round-trip latency.

11.2.1 Detailed Description

Declares a small widget that visually flashes on each command sent, for latency measurement.

11.3 latency_indicator.hpp

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006
00007 #include <QWidget>
00008 #include "utilities/Logger.hpp"
00009
00010 class QTimer;
00011
00021 class latency_indicator : public QWidget {
00022     Q_OBJECT
00023
00024 public:
00029     explicit latency_indicator(QWidget* parent = nullptr);
00030
00031 public slots:
00041     void flash();
00042
00043 private slots:
00049     void revert();
00050
00051 private:
00056     void set_color_window(const QColor& c);
00057
00058     QTimer* revert_timer_ = nullptr;
00059     bool    lit_          = false;
00060 };

```

11.4 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/main.cpp File Reference

Application entry point: builds the Qt windows and game-logic objects, wires the fleet into the map, and runs the TCP/JSON server for the AR app on its own thread alongside the Qt event loop.

```

#include <thread>
#include <boost/asio/io_context.hpp>
#include <nlohmann/json.hpp>
#include <QApplication>
#include "map/board.hpp"
#include "map/mapwindow.hpp"
#include "communication/session_registry.hpp"
#include "communication/handler_template.hpp"
#include "communication/TCP_Server.hpp"
#include "communication/UDP_Server.hpp"
#include "qtpanels/control_window.hpp"
#include "qtpanels/main_window.hpp"
#include "qtpanels/shop_test_window.hpp"
#include "utilities/Logger.hpp"
#include "items/item.hpp"
#include "items/shop.hpp"
#include "map/roulette.hpp"
#include "gameplay/turnManager.hpp"
#include "utilities/enum_tools.hpp"

```

Functions

- void **register_lobby_handlers** ([Server](#) &tcp_server, [UdpServer](#) &udp_server, [Shop](#) &shop)
- int **main** (int argc, char *argv[])

11.4.1 Detailed Description

Application entry point: builds the Qt windows and game-logic objects, wires the fleet into the map, and runs the TCP/JSON server for the AR app on its own thread alongside the Qt event loop.

11.5 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/mock_camera.cpp File Reference

Standalone tool that simulates the tracking camera by publishing synthetic anchor/robot/board-marker positions over MQTT, so downstream consumers (e.g. the Unity AR app) can be developed and tested without real camera hardware.

```
#include <array>
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>
#include <thread>
#include <nlohmann/json.hpp>
#include <mosquitto.h>
#include <QtCore/QCoreApplication>
#include <QtCore/Qtimer>
#include <QtCore/QElapsedTimer>
#include "mock_camera.moc"
```

Classes

- class [MockCamera](#)

Fakes the tracking camera pipeline: on a timer, publishes a fixed calibration anchor plus procedurally-animated robot and board-marker positions to MQTT, in the same JSON shape and on the same topics the real camera ([main_window](#)) uses.

Functions

- int [main](#) (int argc, char *argv[])

Entry point: connects a [MockCamera](#) to the MQTT broker and drives its update() slot on a fixed ~30 FPS timer for the lifetime of the Qt event loop.

11.5.1 Detailed Description

Standalone tool that simulates the tracking camera by publishing synthetic anchor/robot/board-marker positions over MQTT, so downstream consumers (e.g. the Unity AR app) can be developed and tested without real camera hardware.

11.5.2 Function Documentation

11.5.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Entry point: connects a [MockCamera](#) to the MQTT broker and drives its update() slot on a fixed ~30 FPS timer for the lifetime of the Qt event loop.

Parameters

<i>argc</i>	Argument count.
<i>argv</i>	Argument vector; argv[1], if present, overrides the broker address (defaults to "127.0.0.1").

Returns

Qt application exit code from `QCoreApplication::exec()`.

11.6 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/↵ camera_calibration.cpp File Reference

Implementation of [CameraCalibration](#): camera capture loop and solvePnP-based extrinsic calibration.

```
#include "camera_calibration.hpp"
#include <iostream>
```

11.6.1 Detailed Description

Implementation of [CameraCalibration](#): camera capture loop and solvePnP-based extrinsic calibration.

11.7 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/↵ camera_calibration.hpp File Reference

Declares [CameraCalibration](#), a live camera capture loop plus OpenCV solvePnP-based extrinsic pose calibration.

```
#include <QObject>
#include <QTimer>
#include <QImage>
#include <vector>
#include <opencv2/opencv.hpp>
#include "utilities/Logger.hpp"
```

Classes

- class [CameraCalibration](#)

Captures frames from a webcam and computes the camera's extrinsic pose relative to a known 3D field layout. Owns a QTimer-driven capture loop that emits each frame as a QImage for display, and wraps OpenCV's solvePnP to derive rotation/translation vectors once intrinsic parameters and a set of matching 3D/2D point correspondences (e.g. detected field markers) are supplied.

11.7.1 Detailed Description

Declares [CameraCalibration](#), a live camera capture loop plus OpenCV solvePnP-based extrinsic pose calibration.

11.8 camera_calibration.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <QObject>
00009 #include <QTimer>
00010 #include <QImage>
00011 #include <vector>
00012 #include <opencv2/opencv.hpp>
00013 #include "utilities/Logger.hpp"
00014
00021 class CameraCalibration : public QObject {
00022     Q_OBJECT
00023
00024 public:
00029     explicit CameraCalibration(QObject* parent = nullptr);
00030
00032     ~CameraCalibration();
00033
00039     bool start_camera(int device_id = 0);
00040
00042     void stop_camera();
00043
00049     void set_intrinsics(const cv::Mat& camera_matrix, const cv::Mat& dist_coeffs);
00050
00061     bool calibrate_extrinsics(const std::vector<cv::Point3f>& object_points,
00062                             const std::vector<cv::Point2f>& image_points);
00063
00069     bool is_calibrated() const;
00070
00075     const cv::Mat& get_rvec() const { return rvec_; }
00076
00081     const cv::Mat& get_tvec() const { return tvec_; }
00082
00083 signals:
00088     void frame_ready(const QImage& image);
00089
00090 private slots:
00095     void update_frame();
00096
00097 private:
00098     cv::VideoCapture capture_;
00099     QTimer* timer_ = nullptr;
00100
00101     cv::Mat camera_matrix_;
00102     cv::Mat dist_coeffs_;
00103
00104     cv::Mat rvec_;
00105     cv::Mat tvec_;
00106     bool is_calibrated_ = false;
00107
00115     QImage cv_mat_to_qimage(const cv::Mat& mat);
00116 };

```

11.9 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_↵ _avoidance/collision_planner.cpp File Reference

Implementation of collision_planner: free-space and graph path planning, corner rounding, per-pair proximity hysteresis, and CTFS (Component-Token Freeze-Senior Step-Off) conflict resolution.

```
#include "collision_avoidance/collision_planner.hpp"
#include <algorithm>
#include <cmath>
#include <utility>
```

Functions

- int `collision::default_priority` (const `robot_pose` &a, const `robot_pose` &b)
Default priority rule: the higher robot id yields.

11.9.1 Detailed Description

Implementation of collision_planner: free-space and graph path planning, corner rounding, per-pair proximity hysteresis, and CTFS (Component-Token Freeze-Senior Step-Off) conflict resolution.

11.9.2 Function Documentation

11.9.2.1 default_priority()

```
int collision::default_priority (
    const robot_pose & a,
    const robot_pose & b)
```

Default priority rule: the higher robot id yields.

Parameters

<i>a</i>	One of the two conflicting robots.
<i>b</i>	The other conflicting robot.

Returns

The id of whichever of a/b is numerically higher.

11.10 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.hpp File Reference

Declares `collision_planner`, the multi-robot collision-avoidance and routing coordinator (CTFS: Component-Token Freeze-Senior Step-Off) for the playboard.

```
#include <stdint>
#include <functional>
#include <map>
#include <mutex>
#include <unordered_map>
#include <utility>
#include <vector>
#include "pathfinding/gridmap.hpp"
#include "pathfinding/path_planner.hpp"
#include "collision_avoidance/path_update.hpp"
#include "collision_avoidance/playboard_graph.hpp"
```

Classes

- struct `collision::robot_pose`
Live pose of one robot in the board frame (centimetres), fed to the planner each tick. `moving` distinguishes an actively-driving robot (a collision partner, watched pairwise) from a parked one (a static obstacle for others).
- class `collision::collision_planner`
Central, single-instance collision-avoidance coordinator (plain C++/STL + OpenCV, no Qt). Owns the board frame, the stateless `PathPlanner`, every robot's nominal + effective path, and per-pair proximity state; driven by `update()`, publishes versioned `path_updates` through a sink. Implements CTFS (Component-Token Freeze-Senior Step-Off): on every tick, connected components of currently conflicting robots are resolved by letting the lowest-id member of each component keep moving while every other member is frozen, stepped off the lane, or held.

Typedefs

- using `collision::priority_rule` = `std::function<int(const robot_pose&, const robot_pose&)>`
Function type that decides which of two conflicting robots yields (maneuvers).

Enumerations

- enum class `collision::route_result` { `routed` , `waiting_for_blocker` , `no_route` , `unknown_pose` }
Outcome of `request_route_to` (move along the playboard graph).
- enum class `collision::shove_result` { `shoved` , `no_spot` , `unreachable` , `unknown_pose` }
Outcome of `shove_blocker` (nudge a parked robot off a node).

Functions

- int `collision::default_priority` (const `robot_pose` &a, const `robot_pose` &b)
Default priority rule: the higher robot id yields.

11.10.1 Detailed Description

Declares `collision_planner`, the multi-robot collision-avoidance and routing coordinator (CTFS: Component-Token Freeze-Senior Step-Off) for the playboard.

11.10.2 Typedef Documentation

11.10.2.1 `priority_rule`

```
using collision::priority_rule = std::function<int(const robot_pose&, const robot_pose&)>
```

Function type that decides which of two conflicting robots yields (maneuvers).

Returns

The yielding robot's id.

Note

The default rule (`default_priority`) has the higher id yield, i.e. lowest id = highest priority - deterministic and deadlock-free.

11.10.3 Enumeration Type Documentation

11.10.3.1 `route_result`

```
enum class collision::route_result [strong]
```

Outcome of `request_route_to` (move along the playboard graph).

Enumerator

<code>routed</code>	A drivable edge-following path was published.
<code>waiting_for_blocker</code>	Reserved outcome for a shove-then-wait flow; the current <code>request_route_to</code> never returns it - an occupied target node is instead parked beside (first-come-first-served), see <code>request_route_to</code> .
<code>no_route</code>	Target unreachable, unknown node, or no graph injected.
<code>unknown_pose</code>	The robot's pose has not been fed via <code>update()</code> yet.

11.10.3.2 shove_result

```
enum class collision::shove_result [strong]
```

Outcome of shove_blocker (nudge a parked robot off a node).

Enumerator

shoved	A park spot was found and the blocker's move was published.
no_spot	No valid off-edge, node-clear spot exists near the node.
unreachable	A spot was found but no collision-free path leads to it.
unknown_pose	The blocker's pose has not been fed via update() yet.

11.10.4 Function Documentation

11.10.4.1 default_priority()

```
int collision::default_priority (
    const robot_pose & a,
    const robot_pose & b)
```

Default priority rule: the higher robot id yields.

Parameters

<i>a</i>	One of the two conflicting robots.
<i>b</i>	The other conflicting robot.

Returns

The id of whichever of a/b is numerically higher.

11.11 collision_planner.hpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 #pragma once
00009
00010 #include <stdint>
00011 #include <functional>
00012 #include <map>
00013 #include <mutex>
00014 #include <unordered_map>
00015 #include <utility>
00016 #include <vector>
00017
00018
00019
00020 #include "pathfinding/gridmap.hpp"
00021 #include "pathfinding/path_planner.hpp"
00022 #include "collision_avoidance/path_update.hpp"
00023 #include "collision_avoidance/playboard_graph.hpp"
00024
00025 namespace collision {
00026
```

```

00033 struct robot_pose {
00034     int robot_id;
00035     cv::Point2f position_cm;
00036     float heading_deg;
00037     bool moving;
00038 };
00039
00046 using priority_rule = std::function<int(const robot_pose&, const robot_pose&)>;
00047
00054 int default_priority(const robot_pose& a, const robot_pose& b);
00055
00059 enum class route_result {
00060     routed,
00061     waiting_for_blocker,
00065     no_route,
00066     unknown_pose,
00067 };
00068
00072 enum class shove_result {
00073     shoved,
00074     no_spot,
00075     unreachable,
00076     unknown_pose,
00077 };
00078
00094 class collision_planner {
00095 public:
00102     explicit collision_planner(float board_width_cm = 300.0f,
00103                             float board_height_cm = 185.0f,
00104                             float resolution_cm = 2.0f);
00105
00110     void set_path_sink(path_sink sink);
00111
00118     void set_priority_rule(priority_rule rule);
00119
00125     void set_graph(playboard_graph graph);
00126
00138     bool request_path_to(int robot_id, cv::Point2f goal_cm);
00139
00155     route_result request_route_to(int robot_id, int target_node);
00156
00171     shove_result shove_blocker(int blocker_id, int clear_node);
00172
00181     void clear_path(int robot_id);
00182
00191     void forget(int robot_id);
00192
00204     void update(const std::vector<robot_pose>& poses);
00205
00212     path_update effective_path(int robot_id) const;
00213
00221     bool is_yielding(int robot_id) const;
00222
00223     // --- Introspection for the visualization overlay + tests ---
00224
00226     std::vector<cv::Point2f> nominal_path(int robot_id) const;
00228     std::vector<std::pair<int, int>> watched_pairs() const;
00230     std::vector<int> route_nodes(int robot_id) const;
00231
00239     pathfinding::GridMap debug_obstacle_grid(int self_id, int block_id = -1) const;
00240
00241 private:
00243     enum class pair_phase { monitoring, avoiding };
00244
00252     enum class plan_mode { free, graph };
00253
00255     struct robot_record {
00256         plan_mode mode = plan_mode::free;
00257         cv::Point2f goal_cm{0.f, 0.f};
00258         bool has_goal = false;
00259         std::vector<cv::Point2f> nominal;
00260         std::vector<cv::Point2f> effective;
00261         std::uint64_t version = 0;
00262         follow_state state = follow_state::nominal;
00263         bool active = false;
00264
00265         // Graph-mode (request_route_to) state.
00266         std::vector<int> route;
00267         int target_node = -1;
00268         bool pending = false;
00272         bool self_shove = false;
00273
00274         // Self-shove progress watchdog: a planner-nudged blocker may have no
00275         // driver, so if it makes no progress toward its spot within a timeout,
00276         // revert it to a static obstacle so it stops masking the node to clear.
00277         std::uint64_t shove_tick = 0;
00278         float shove_start_dist = -1.0f;

```

```

00279
00280     // Concurrent-movement (CTFS) step-off: a yielding non-mover drives off
00281     // the lane to step_spot and holds until its mover (step_token) passes,
00282     // then resumes its graph route; route/target_node are preserved so the
00283     // resume rebuilds the real edge-route.
00284     bool stepping = false;
00285     bool stepped = false;
00286     cv::Point2f step_spot{0.f, 0.f};
00287     int step_token = -1;
00288
00289     // Same-target loser: it raced another robot to the SAME tile and lost, so
00290     // it is redirected once to PARK BESIDE that tile as its FINAL spot (it does
00291     // not resume to the taken node). Distinct from step-off, which is temporary.
00292     bool aside = false;
00293 };
00294
00295 struct pair_record {
00296     pair_phase phase = pair_phase::monitoring;
00297     int yielder = -1;
00298     float last_dist_cm = 1e9f;
00300     bool holder_held = false;
00301     std::uint64_t retry_tick = 0;
00302 };
00303
00304 pathfinding::GridMap make_grid(int self_id, int block_id,
00305                                int extra_exclude = -1,
00306                                float park_radius_cm = -1.0f) const;
00307
00308 std::vector<cv::Point2f> plan_path(int self_id, cv::Point2f start,
00309                                    cv::Point2f goal, int block_id) const;
00310
00311 std::vector<cv::Point2f> plan_route_polyline(int self_id, cv::Point2f start,
00312                                              const std::vector<int>& nodes) const;
00313
00314 std::vector<cv::Point2f> round_corners(const pathfinding::GridMap& grid,
00315                                       const std::vector<cv::Point2f>& path) const;
00316
00317 std::vector<int> compute_route(cv::Point2f from, int target) const;
00318
00319 void trim_reached_nodes(std::vector<int>& route, cv::Point2f pos,
00320                        float final_tol) const;
00321
00322 void clip_route_start(std::vector<int>& route, cv::Point2f pose) const;
00323
00324 cv::Point2f nearest_free(const pathfinding::GridMap& grid, cv::Point2f p,
00325                          float max_radius_cm = -1.0f) const;
00326
00327 void publish_route(robot_record& rec, int robot_id, std::vector<path_update>& out);
00328
00329 void revalidate_active_paths(std::vector<path_update>& out);
00330
00331 bool is_executing(int robot_id) const;
00332
00333 bool is_holding(int robot_id) const;
00334
00335 bool target_occupied(int node, int exclude_id, int& blocker_out) const;
00336
00337 bool node_physically_occupied(int node, int exclude_id) const;
00338
00339 cv::Point2f node_pos(int node_id) const;
00340 cv::Point2f clamp_to_board(cv::Point2f p) const;
00341
00342 bool find_park_spot(int blocker_id, int clear_node, cv::Point2f& spot_out,
00343                    int arriver_id = -1) const;
00344
00345 shove_result shove_blocker_locked(int blocker_id, int clear_node,
00346                                   std::vector<path_update>& out,
00347                                   int arriver_id = -1);
00348
00349 void enter_avoidance(int yielder, int holder, std::vector<path_update>& out);
00350
00351 void resolve_components(std::vector<path_update>& out);
00352
00353 bool try_step_off(int blocker, int mover, std::vector<path_update>& out);
00354
00355 void resume_if_free(int robot_id, std::vector<path_update>& out);
00356 void resume_nominal(int robot_id, std::vector<path_update>& out);
00357
00358 void publish(robot_record& rec, int robot_id, follow_state state,
00359             std::vector<cv::Point2f> path, std::vector<path_update>& out);
00360
00361 const robot_pose* pose_of(int robot_id) const;
00362
00363 static std::pair<int, int> pair_key(int a, int b) {
00364     return a < b ? std::make_pair(a, b) : std::make_pair(b, a);
00365 }
00366

```

```

00564     float board_width_cm_ = 0.0f;
00565     float board_height_cm_ = 0.0f;
00566     float resolution_cm_ = 0.0f;
00567     int grid_w_ = 0;
00568     int grid_h_ = 0;
00569
00570     path_sink sink_;
00571     priority_rule priority_;
00572     playboard_graph graph_;
00573
00574     std::unordered_map<int, robot_pose> poses_;
00575     std::unordered_map<int, robot_record> robots_;
00576     std::map<std::pair<int, int>, pair_record> pairs_;
00577
00578     std::uint64_t tick_ = 0;
00579
00580     mutable std::mutex mu_;
00581 };
00582
00583 } // namespace collision

```

11.12 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/path_update.hpp File Reference

Output contract (path_update, follow_state, path_sink) between the collision planner and the path follower / PID driver.

```

#include <cstdint>
#include <functional>
#include <vector>
#include <opencv2/core.hpp>

```

Classes

- struct `collision::path_update`

One immutable snapshot of a single robot's effective path. The planner publishes a fresh snapshot (never edits one in place) whenever the path changes; consumers keep only the highest version seen and follow it.

Typedefs

- using `collision::path_sink` = `std::function<void(const path_update&)>`

Callback type for the push-model seam between planner and follower. The follower registers one of these once; the planner calls it on every path change (initial plan, each avoidance replan, resume). A follower that runs its own control loop can instead pull `collision_planner::effective_path(robot_id)` at its own cadence.

Enumerations

- enum class `collision::follow_state` { `nominal` , `avoiding` , `holding` }

Why a robot's current path looks the way it does. Optional for the follower to act on, but lets it distinguish "drive normally" from "this is a detour" from "hold position".

11.12.1 Detailed Description

Output contract (path_update, follow_state, path_sink) between the collision planner and the path follower / PID driver.

11.12.2 Enumeration Type Documentation

11.12.2.1 follow_state

```
enum class collision::follow_state [strong]
```

Why a robot's current path looks the way it does. Optional for the follower to act on, but lets it distinguish "drive normally" from "this is a detour" from "hold position".

Enumerator

nominal	the robot's plain plan-once path to its goal
avoiding	a one-shot detour around another robot
holding	wait in place (e.g. narrow-corridor yield); waypoints is empty

11.13 path_update.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <stdint>
00010 #include <functional>
00011 #include <vector>
00012
00013 #include <opencv2/core.hpp>
00014
00015 // The output contract between the collision planner and the (teammate-owned)
00016 // path follower / PID driver. Plain C++ + OpenCV only - no Qt - so the planner
00017 // stays Qt-free and the contract is trivially unit-testable.
00018 namespace collision {
00019
00025 enum class follow_state {
00026     nominal,
00027     avoiding,
00028     holding
00029 };
00030
00036 struct path_update {
00037     int robot_id;
00038     std::uint64_t version;
00039     std::vector<cv::Point2f> waypoints;
00042     follow_state state;
00043 };
00044
00052 using path_sink = std::function<void(const path_update&);>;
00053
00054 } // namespace collision
```

11.14 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ collision_avoidance/playboard_graph.cpp File Reference

Implementation of the playboard node graph: lookups, nearest-node search, Dijkstra routing, and edge/node distance queries.

```
#include "collision_avoidance/playboard_graph.hpp"
#include <algorithm>
#include <cmath>
#include <queue>
#include <utility>
```

11.14.1 Detailed Description

Implementation of the playboard node graph: lookups, nearest-node search, Dijkstra routing, and edge/node distance queries.

11.15 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.hpp File Reference

Board-frame node graph used for edge-following routes and park-spot placement, with Dijkstra shortest-path routing.

```
#include <functional>
#include <vector>
#include <opencv2/core.hpp>
```

Classes

- struct [collision::graph_node](#)

One node of the playboard graph in the planner's board frame (centimetres). This is the Qt-free, unit-consistent mirror of [map/board.hpp](#)'s [Node](#): the caller ([MapWindow](#)) converts each [Node](#)'s screen pixels to cm once and injects the result via [collision_planner::set_graph](#).

- class [collision::playboard_graph](#)

The playboard's node graph, used for edge-following routes ([move_playboard](#)) and for placing a shoved robot off the lanes. Plain C++/STL + OpenCV, no Qt. Routing is exact shortest path (Dijkstra by Euclidean edge length); the graph is small (~21 nodes) so this is trivially cheap.

11.15.1 Detailed Description

Board-frame node graph used for edge-following routes and park-spot placement, with Dijkstra shortest-path routing.

11.16 playboard_graph.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <functional>
00010 #include <vector>
00011
00012 #include <opencv2/core.hpp>
00013
00014 namespace collision {
00015
00022 struct graph_node {
00023     int id;
00024     cv::Point2f pos_cm;
00025     std::vector<int> neighbors;
00026 };
00027
00035 class playboard_graph {
00036 public:
00042     void set_nodes(std::vector<graph_node> nodes);
```



```

00043
00045     bool empty() const noexcept { return nodes_.empty(); }
00047     bool has_node(int id) const noexcept { return index_of(id) >= 0; }
00053     [[nodiscard]] const graph_node* node(int id) const;
00055     const std::vector<graph_node>& nodes() const noexcept { return nodes_; }
00056
00063     [[nodiscard]] int nearest_node(cv::Point2f p) const;
00064
00074     [[nodiscard]] std::vector<int> route(int from, int to,
00075                                         const std::function<bool(int, int)>& edge_blocked = {}) const;
00076
00083     float min_edge_distance(cv::Point2f p) const;
00084
00092     float min_node_distance(cv::Point2f p, int except_id = -1) const;
00093
00094 private:
00096     int index_of(int id) const;
00097
00098     std::vector<graph_node> nodes_;
00099 };
00100
00101 } // namespace collision

```

11.17 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/handler_template.cpp File Reference

Implementation of the AR app -> desktop TCP/JSON message handling template.

```

#include "handler_template.hpp"
#include <cstdint>
#include <nlohmann/json.hpp>
#include "message_processing.hpp"
#include "TCP_Server.hpp"
#include "utilities/Logger.hpp"

```

Functions

- void `register_handlers` (`Server` &server, `MessageTarget` &target)
Registers every handler this feature needs with the message dispatcher.
- void `broadcast_update` (`Server` &server, `std::uint8_t` player_index)
Sends a message to every connected client, unprompted.

11.17.1 Detailed Description

Implementation of the AR app -> desktop TCP/JSON message handling template.

11.17.2 Function Documentation

11.17.2.1 broadcast_update()

```

void broadcast_update (
    Server & server,
    std::uint8_t player_index)

```

Sends a message to every connected client, unprompted.

Sends a message to every connected client, unprompted. Call from wherever the event originates and holds a [Server](#); this is not a reply to an inbound message.

Sends a message to every connected client, unprompted.

Parameters

<i>server</i>	The server used to broadcast the JSON payload.
<i>player_index</i>	Placeholder payload identifying which player's state changed.

11.17.2.2 register_handlers()

```
void register_handlers (
    Server & server,
    MessageTarget & target)
```

Registers every handler this feature needs with the message dispatcher.

Parameters

<i>server</i>	Lets a registered handler reply to the sender.
<i>target</i>	The object handlers call into; must outlive the server.

11.18 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ communication/handler_template.hpp File Reference

Template for AR app -> desktop TCP/JSON message handling.

```
#include <cstdint>
#include <nlohmann/json.hpp>
#include "TCP_Server.hpp"
```

Classes

- class [MessageTarget](#)

Placeholder for the class that owns the logic a message should trigger. In real use this is something like [GameManager](#) or [Board](#). Passed into [register_handlers\(\)](#) by reference and must outlive the server.

Functions

- void [register_handlers](#) ([Server](#) &server, [MessageTarget](#) &target)
Registers every handler this feature needs with the message dispatcher.
- void [broadcast_update](#) ([Server](#) &server, std::uint8_t player_index)
Sends a message to every connected client, unprompted. Call from wherever the event originates and holds a [Server](#); this is not a reply to an inbound message.

11.18.1 Detailed Description

Template for AR app -> desktop TCP/JSON message handling.

Inbound frames are dispatched by their "type" field: `process()` (`message_processing.cpp`) parses each frame and calls the handler registered for that type. To handle a new message kind you register a handler for its type and call into the class that owns the logic.

Wire format: "type" selects the handler, the remaining fields are its arguments, e.g.

```
{"type": "example_type", "player": 0, "value": 0}
```

Replace the placeholders below (`MessageTarget`, the method, the message type and its fields) with the real class and feature. The file is kept compilable on its own so it can be used as a starting point.

Register handlers once at startup, before the `io_context` is run. The registry is then read-only while messages dispatch, so no locking is needed.

11.18.2 Function Documentation

11.18.2.1 `broadcast_update()`

```
void broadcast_update (
    Server & server,
    std::uint8_t player_index)
```

Sends a message to every connected client, unprompted. Call from wherever the event originates and holds a `Server`; this is not a reply to an inbound message.

Sends a message to every connected client, unprompted.

Parameters

<i>server</i>	The server used to broadcast the JSON payload.
<i>player_index</i>	Placeholder payload identifying which player's state changed.

Sends a message to every connected client, unprompted. Call from wherever the event originates and holds a `Server`; this is not a reply to an inbound message.

11.18.2.2 `register_handlers()`

```
void register_handlers (
    Server & server,
    MessageTarget & target)
```

Registers every handler this feature needs with the message dispatcher.

Parameters

<i>server</i>	Lets a registered handler reply to the sender.
---------------	--

<i>target</i>	The object handlers call into; must outlive the server.
---------------	---

11.19 handler_template.hpp

[Go to the documentation of this file.](#)

```

00001
00022
00023 #pragma once
00024
00025 #include <cstdint>
00026
00027 #include <nlohmann/json.hpp>
00028
00029 #include "TCP_Server.hpp"
00030
00036 class MessageTarget {
00037 public:
00045     void handle(std::uint8_t player_index, int value);
00046 };
00047
00053 void register_handlers(Server& server, MessageTarget& target);
00054
00062 void broadcast_update(Server& server, std::uint8_t player_index);

```

11.20 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.cpp File Reference

Implementation of the JSON message dispatch registry declared in [message_processing.hpp](#).

```

#include "communication/message_processing.hpp"
#include "communication/session_registry.hpp"
#include <unordered_map>
#include <utility>
#include "utilities/Logger.hpp"

```

Functions

- void [register_message_handler](#) (std::string type, [message_handler](#) handler)
Registers the handler invoked for messages whose "type" field equals type.
- void [process](#) (const std::string &input_string, std::uint8_t from_id)
Parses one received frame as JSON and dispatches it to the handler registered for its "type" field.

11.20.1 Detailed Description

Implementation of the JSON message dispatch registry declared in [message_processing.hpp](#).

11.20.2 Function Documentation

11.20.2.1 process()

```
void process (
    const std::string & input_string,
    std::uint8_t from_id)
```

Parses one received frame as JSON and dispatches it to the handler registered for its "type" field.

Never throws - this runs inside an asio read handler. Malformed JSON, a message missing a string "type" field, or a "type" with no registered handler are all logged and the frame is dropped.

Parameters

<i>input_string</i>	Raw message text received from the socket (expected to be one JSON object).
<i>from_id</i>	ConnectionID of the sender, forwarded unchanged to the handler.

11.20.2.2 register_message_handler()

```
void register_message_handler (
    std::string type,
    message_handler handler)
```

Registers the handler invoked for messages whose "type" field equals type.

Registering the same type twice replaces the first handler. Not safe against concurrent dispatch - call only before the server starts accepting connections.

Parameters

<i>type</i>	The "type" field value that selects this handler.
<i>handler</i>	Callback to invoke with the parsed message and sender ID.

11.21 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.hpp File Reference

JSON message dispatch for the AR app <-> desktop TCP channel: socket.cpp hands each received frame to [process\(\)](#), which parses it and routes it by its "type" field to the handler registered for that type via [register_message_handler\(\)](#). Add a new message kind with a single [register_message_handler\(\)](#) call. EV3 traffic is a separate binary protocol, not this.

```
#include <stdint>
#include <functional>
#include <string>
#include <nlohmann/json.hpp>
```

Typedefs

- using [message_handler](#) = std::function<void(const nlohmann::json& message, std::uint8_t from_id)>
Callback type invoked by [process\(\)](#) for a message whose "type" field matches the registered key.

Functions

- void [register_message_handler](#) (std::string type, [message_handler](#) handler)
Registers the handler invoked for messages whose "type" field equals type.
- void [process](#) (const std::string &input_string, std::uint8_t from_id)
Parses one received frame as JSON and dispatches it to the handler registered for its "type" field.

11.21.1 Detailed Description

JSON message dispatch for the AR app <-> desktop TCP channel: socket.cpp hands each received frame to [process\(\)](#), which parses it and routes it by its "type" field to the handler registered for that type via [register_message_handler\(\)](#). Add a new message kind with a single [register_message_handler\(\)](#) call. EV3 traffic is a separate binary protocol, not this.

Threading: handlers run on the asio thread. Register all handlers at startup, before the server accepts connections, so the registry stays read-only during dispatch (no locking). A handler touching Qt must marshal onto the Qt thread itself.

11.21.2 Typedef Documentation

11.21.2.1 message_handler

```
using message_handler = std::function<void(const nlohmann::json& message, std::uint8_t from_id)>
```

Callback type invoked by [process\(\)](#) for a message whose "type" field matches the registered key.

Parameters

<i>message</i>	Parsed JSON payload of the incoming frame.
<i>from_id</i>	Sender's ConnectionID (see TCP_Server.hpp), kept as a plain integer so this header does not depend on TCP_Server.hpp .

11.21.3 Function Documentation

11.21.3.1 process()

```
void process (
    const std::string & input_string,
    std::uint8_t from_id)
```

Parses one received frame as JSON and dispatches it to the handler registered for its "type" field.

Never throws - this runs inside an asio read handler. Malformed JSON, a message missing a string "type" field, or a "type" with no registered handler are all logged and the frame is dropped.

Parameters

<i>input_string</i>	Raw message text received from the socket (expected to be one JSON object).
<i>from_id</i>	ConnectionID of the sender, forwarded unchanged to the handler.

11.21.3.2 register_message_handler()

```
void register_message_handler (
    std::string type,
    message_handler handler)
```

Registers the handler invoked for messages whose "type" field equals type.

Registering the same type twice replaces the first handler. Not safe against concurrent dispatch - call only before the server starts accepting connections.

Parameters

<i>type</i>	The "type" field value that selects this handler.
<i>handler</i>	Callback to invoke with the parsed message and sender ID.

11.22 message_processing.hpp

[Go to the documentation of this file.](#)

```
00001
00012
00013 #pragma once
00014
00015 #include <cstdint>
00016 #include <functional>
00017 #include <string>
00018
00019 #include <nlohmann/json.hpp>
00020
00028 using message_handler = std::function<void(const nlohmann::json& message, std::uint8_t from_id)>;
00029
00037 void register_message_handler(std::string type, message_handler handler);
00038
00048 void process(const std::string& input_string, std::uint8_t from_id);
```

11.23 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ communication/network_messages.hpp File Reference

JSON-serializable payload structs (DTOs) for the shop and roulette network events exchanged with the AR/Unity client, matched by [message_processing.cpp](#)'s "type" field dispatch.

```
#include "map/roulette.hpp"
#include <nlohmann/json.hpp>
```

Classes

- struct [ns::OpenShopEvent](#)
Payload of the "open_shop" message: identifies which player opened the shop. Parsed in [shop.cpp](#)'s "open_shop" handler and used to build a temporary [Player](#) before calling [Shop::openShop\(\)](#).
- struct [ns::SelectedItemEvent](#)
Payload describing a player's item selection from the shop UI.
- struct [ns::CloseShopEvent](#)
Payload of the "close_shop" message: identifies which player closed the shop. Parsed in [shop.cpp](#)'s "close_shop" handler and used to build a temporary [Player](#) before calling [Shop::close_shop\(\)](#).
- struct [ns::StartRouletteEvent](#)
Payload of the "roulette_start" message: identifies which player is starting a roulette round. Parsed in [roulette.cpp](#)'s "roulette_start" handler and used to build a temporary [Player](#) before calling [Roulette::start_roulette\(\)](#).
- struct [ns::GetPlayerBetEvent](#)
Payload of the "make_bet" message: the player's roulette wager. Parsed in [roulette.cpp](#)'s "make_bet" handler and forwarded to [Roulette::setBet\(\)](#), which wakes any thread blocked in [Roulette::choose_bet_type\(\)](#).
- struct [ns::EndRouletteEvent](#)
Payload of the "end_roulette" message: identifies which player's roulette round is ending. Parsed in [roulette.cpp](#)'s "end_roulette" handler and used to build a temporary [Player](#) before calling [Roulette::end_roulette\(\)](#).

Namespaces

- namespace [ns](#)
Network message payload types shared between the desktop app and the AR/Unity client. Each struct uses `NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE` to generate `to_json/from_json`, so a handler registered via [register_message_handler\(\)](#) can deserialize the incoming JSON with `message.get<ns::SomeEvent>()`.

11.23.1 Detailed Description

JSON-serializable payload structs (DTOs) for the shop and roulette network events exchanged with the AR/Unity client, matched by [message_processing.cpp](#)'s "type" field dispatch.

11.24 network_messages.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007
00008
00009 #include "map/roulette.hpp"
00010 #include <nlohmann/json.hpp>
00011
00012
00019 namespace ns {
00025     struct OpenShopEvent {
00026         int playerId;
00027     };
00028     NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(OpenShopEvent, playerId)
00029
00030
00036     struct SelectedItemEvent {
00037         int playerId, itemID;
00038     };
00039     NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(SelectedItemEvent, playerId, itemID)
00040
00041
00042
00043     struct CloseShopEvent {
00044         int playerId;
00045     };
00046     NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(CloseShopEvent, playerId)
00047
00048
00049     struct StartRouletteEvent {
00050         int playerId;
00051     };
00052     NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(StartRouletteEvent, playerId)
00053
00054
00055     struct GetPlayerBetEvent {
00056         int playerId, amount, choice;
00057         Bet_type bet_type;
00058     };
00059     NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(GetPlayerBetEvent, playerId, amount, bet_type, choice)
00060
00061
00062     struct EndRouletteEvent {
00063         int playerId;
00064     };
00065     NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(EndRouletteEvent, playerId)
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076
00077
00078
00079
00080
00081
00082
00083
00084
00085
00086
00087 }

```

11.25 session_registry.hpp

```

00001 #pragma once
00002
00003 #include <string>
00004 #include <string_view>
00005 #include <unordered_map>
00006 #include <memory>
00007 #include <shared_mutex>
00008 #include <chrono>
00009 #include <atomic>
00010 #include <cstdint>
00011 #include <span>
00012 #include <boost/asio/ip/udp.hpp>
00013 #include "communication/TCP_Server.hpp"
00014
00015 using ConnectionID = uint8_t;
00016
00017 class TcpConnection;
00018
00019 struct PlayerSession
00020 {
00021     ConnectionID id{};
00022     std::string device_name{};
00023     std::shared_ptr<TcpConnection> tcp_conn{};

```

```

00024     boost::asio::ip::udp::endpoint udp_endpoint{};
00025     std::chrono::steady_clock::time_point last_seen{};
00026     std::uint64_t bytes_tx = 0;
00027     std::uint64_t bytes_rx = 0;
00028
00030     std::unordered_map<std::string, std::pair<std::uint64_t, std::uint64_t> typed_traffic{};
00031 };
00032
00033 class SessionRegistry
00034 {
00035 private:
00036     mutable std::shared_mutex rw_mutex_{};
00037     std::unordered_map<ConnectionID, PlayerSession> sessions_{};
00038     std::unordered_map<std::string, ConnectionID> name_map_{};
00039     std::atomic<uint8_t> next_id_{1};
00040
00041 public:
00042     [[nodiscard]] ConnectionID create_session(std::string_view name, std::shared_ptr<TcpConnection>
conn);
00043
00044     void update_udp_endpoint(ConnectionID id, const boost::asio::ip::udp::endpoint& ep);
00045
00046     [[nodiscard]] boost::asio::ip::udp::endpoint get_udp_endpoint(ConnectionID id) const;
00047
00048     void remove_session(ConnectionID id);
00049
00050     void touch_session(ConnectionID id);
00051
00052     void record_traffic(ConnectionID id, std::uint64_t tx, std::uint64_t rx);
00053
00054     void record_traffic(ConnectionID id, std::uint64_t tx, std::uint64_t rx,
std::string_view category);
00055
00056     void sweep_stale_sessions(std::chrono::seconds timeout);
00057
00058     [[nodiscard]] ConnectionID get_id_by_name(std::string_view name) const;
00059
00060     [[nodiscard]] PlayerSession get_session(ConnectionID id) const;
00061
00062     [[nodiscard]] std::unordered_map<ConnectionID, boost::asio::ip::udp::endpoint>
get_all_udp_endpoints() const;
00063
00064     [[nodiscard]] std::unordered_map<ConnectionID, PlayerSession> get_all_sessions() const;
00065
00066 };

```

11.26 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.hpp File Reference

Definiert die TCP-Netzwerkschnittstellen für Client- und Server-Komponenten.

```

#include <boost/asio/buffer.hpp>
#include <boost/asio/io_context.hpp>
#include <boost/asio/ip/tcp.hpp>
#include <boost/asio/write.hpp>
#include <boost/asio/read.hpp>
#include <boost/asio.hpp>
#include <boost/beast.hpp>
#include <boost/system/detail/error_code.hpp>
#include <memory>
#include <string>
#include <string_view>
#include <vector>
#include <iostream>
#include <nlohmann/json.hpp>
#include "utilities/Logger.hpp"
#include "message_processing.hpp"
#include "communication/session_registry.hpp"

```

Classes

- class [TcpConnection](#)
Repräsentiert eine einzelne aktive TCP-Verbindung zu einem Client (Unity / AR-App / Gerät).
- class [Server](#)
Verwaltet den TCP-Netzwerk-Server und die Verteilung von Nachrichten an verbundene Clients.

Typedefs

- using **tcp** = net::ip::tcp
- using **error_code** = boost::system::error_code

Variables

- constexpr char **message_delimiter** = '\n'

11.26.1 Detailed Description

Definiert die TCP-Netzwerkschnittstellen für Client- und Server-Komponenten.

Verwalte TCP-Verbindungen, asynchrone Datenübertragung (Boost.Asio) und Serialisierung von JSON-Nachrichten für das Mindstorms-Party Spiel.

11.27 TCP_Server.hpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 #pragma once
00009
00010 #include <boost/asio/buffer.hpp>
00011 #include <boost/asio/io_context.hpp>
00012 #include <boost/asio/ip/tcp.hpp>
00013 #include <boost/asio/write.hpp>
00014 #include <boost/asio/read.hpp>
00015 #include <boost/asio.hpp>
00016 #include <boost/beast.hpp>
00017 #include <boost/system/detail/error_code.hpp>
00018
00019 #include <memory>
00020 #include <string>
00021 #include <string_view>
00022 #include <vector>
00023 #include <iostream>
00024 #include <nlohmann/json.hpp>
00025 #include "utilities/Logger.hpp"
00026 #include "message_processing.hpp"
00027 #include "communication/session_registry.hpp"
00028
00029 namespace net = boost::asio;
00030 using tcp = net::ip::tcp;
00031 using error_code = boost::system::error_code;
00032
00033 // Delimiter für getrennte JSON-Nachrichten im TCP-Stream
00034 inline constexpr char message_delimiter = '\n';
00035 using ConnectionID = uint8_t;
00036 class SessionRegistry;
00037
00038 class Server;
00039
00043 class TcpConnection : public std::enable_shared_from_this<TcpConnection>
00044 {
00045 public:

```

```

00046     ConnectionID ID{1};
00047     tcp::socket socket;
00048     tcp::resolver resolver;
00049     std::shared_ptr<SessionRegistry> session_registry_;
00050     net::streambuf read_buffer_{};
00051
00052     TcpConnection(net::io_context& ioc, ConnectionID id, std::shared_ptr<SessionRegistry> registry);
00053
00055     auto connect_to_server(net::io_context& ioc, const std::string& host, const std::string& port) ->
void;
00056
00058     auto send_to_server(const std::string& message) -> void;
00059
00061     auto start_read() -> void;
00062 };
00063
00067 class Server
00068 {
00069 private:
00070     boost::asio::io_context& io_context_;
00071     tcp::acceptor acceptor_;
00072     std::shared_ptr<SessionRegistry> session_registry_{};
00073     std::unique_ptr<net::steady_timer> heartbeat_timer_;
00074     void on_heartbeat_timer(boost::system::error_code ec);
00075 public:
00076     friend class TcpConnection;
00077
00078     Server(const Server&) = delete;
00079     Server& operator=(const Server&) = delete;
00080     Server(Server&&) = delete;
00081     Server& operator=(Server&&) = delete;
00082
00089     Server(net::io_context& ioc, int port, std::shared_ptr<SessionRegistry> registry);
00090
00092     auto start_accept() -> void;
00093
00100     auto send_to_user(ConnectionID user_id, std::string_view message,
std::optional<LogLevel> LogLevel = LogLevel::Info,
std::string_view category = "game") const -> void;
00103
00109     auto broadcast_to_all(std::string_view message,
std::optional<LogLevel> LogLevel = LogLevel::Info,
std::string_view category = "game") const -> void;
00112
00113     void start_heartbeat();
00114
00115     auto send_json(ConnectionID user_id, const nlohmann::json& message,
std::optional<LogLevel> LogLevel = LogLevel::Info,
std::string_view category = "game") const -> void;
00118
00124     auto broadcast_json(const nlohmann::json& message,
std::optional<LogLevel> LogLevel = LogLevel::Info,
std::string_view category = "game") const -> void;
00127
00129     [[nodiscard]] std::shared_ptr<SessionRegistry> get_registry() const { return session_registry_; }
00130 };

```

11.28 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.cpp File Reference

Implementation of UDP communication interfaces.

```

#include "UDP_Server.hpp"
#include "utilities/Logger.hpp"
#include "utilities/enum_tools.hpp"
#include <stdint>

```

11.28.1 Detailed Description

Implementation of UDP communication interfaces.

Handles asynchronous datagram transmission and maps remote UDP endpoints to unique session IDs.

11.29 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.hpp File Reference

Defines UDP networking interfaces for server and client nodes.

```
#include <boost/asio.hpp>
#include <map>
#include <string>
#include <array>
#include <stdint>
#include <nlohmann/json.hpp>
#include "utilities/Logger.hpp"
#include "message_processing.hpp"
#include "communication/session_registry.hpp"
```

Classes

- class [UdpServer](#)
Manages a UDP server that tracks multiple connected clients.
- class [UdpClient](#)
Provides a UDP client interface for communicating with the server.

Typedefs

- using **udp** = net::ip::udp

11.29.1 Detailed Description

Defines UDP networking interfaces for server and client nodes.

Because UDP is connectionless, these classes handle the mapping of remote endpoints to unique IDs and manage asynchronous receive buffers. Each datagram is treated as one complete message; unlike the TCP transport in [TCP_Server.hpp](#), no message_delimiter framing is needed or applied.

11.30 UDP_Server.hpp

[Go to the documentation of this file.](#)

```
00001
00009
00010 #pragma once
00011
00012 #include <boost/asio.hpp>
00013 #include <map>
00014 #include <string>
00015 #include <array>
00016 #include <stdint>
00017 #include <nlohmann/json.hpp>
00018 #include "utilities/Logger.hpp"
00019 #include "message_processing.hpp"
00020 #include "communication/session_registry.hpp"
00021 namespace net = boost::asio;
00022 using udp = net::ip::udp;
00023 using ConnectionID = uint8_t;
00024
```

```

00030 class UdpServer
00031 {
00032 private:
00033     net::io_context& io_context_;
00034     udp::socket socket_;
00035     std::array<char, 1024> recv_buffer_{};
00036     std::shared_ptr<SessionRegistry> session_registry_{};
00037     udp::endpoint remote_endpoint_{};
00038
00039 public:
00040     UdpServer(net::io_context& ioc, uint16_t port, std::shared_ptr<SessionRegistry> registry);
00052     void start_receive();
00053
00061     void send_to_user(ConnectionID user_id, const std::string& message);
00062
00069     void broadcast_to_all(const std::string& message);
00070
00071     void network_broadcast(uint16_t port, const std::string& message);
00072
00073 private:
00074     void handle_receive(const boost::system::error_code& error, std::size_t bytes_transferred);
00089 };
00090
00096 class UdpClient
00097 {
00098 private:
00099     net::io_context& io_context_;
00100     udp::socket socket_;
00101     udp::endpoint server_endpoint_;
00102     std::array<char, 1024> recv_buffer_{};
00103     udp::endpoint sender_endpoint_;
00104
00105 public:
00114     UdpClient(net::io_context& ioc, const std::string& host, const std::string& port);
00115
00117     void send_to_server(const std::string& message);
00118
00124     void start_receive();
00125
00126 private:
00139     void handle_receive(const boost::system::error_code& error, std::size_t bytes_transferred);
00140 };

```

11.31 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ gameplay/bonus.cpp File Reference

Implementation of the end-of-game bonus-star scoring phase.

```

#include "bonus.hpp"
#include "game.hpp"
#include "../player/player.hpp"
#include "../utilities/Logger.hpp"
#include <algorithm>
#include <array>
#include <unordered_map>
#include <utility>

```

11.31.1 Detailed Description

Implementation of the end-of-game bonus-star scoring phase.

11.32 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ gameplay/bonus.hpp File Reference

Declares [BonusPhase](#), which awards end-of-game bonus stars based on per-player session statistics.

```
#include "utilities/Logger.hpp"
#include <string>
#include <vector>
```

Classes

- struct [BonusAward](#)
- struct [PlayerStanding](#)
- struct [BonusResult](#)
- class [BonusPhase](#)

Stateless end-of-game scoring phase. Scans all players in a [Game](#), awards bonus stars, and returns the complete result needed by presentation and networking layers.

Enumerations

- enum class [BonusCategory](#) { [Movement](#) , [Items](#) , [Coins](#) }

11.32.1 Detailed Description

Declares [BonusPhase](#), which awards end-of-game bonus stars based on per-player session statistics.

11.33 bonus.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include "utilities/Logger.hpp"
00010
00011 #include <string>
00012 #include <vector>
00013
00014
00015 class Game;
00016
00017 enum class BonusCategory
00018 {
00019     Movement,
00020     Items,
00021     Coins
00022 };
00023
00024 struct BonusAward
00025 {
00026     BonusCategory category;
00027     int value;
00028     std::vector<int> winnerIds;
00029 };
00030
00031 struct PlayerStanding
00032 {
00033     int rank;
```

```

00034     int playerId;
00035     std::string playerName;
00036     int starsBeforeBonus;
00037     int starsAfterBonus;
00038     int coins;
00039 };
00040
00041 struct BonusResult
00042 {
00043     std::vector<BonusAward> awards;
00044     std::vector<PlayerStanding> standings;
00045     std::vector<int> winnerIds;
00046
00047     bool isTie() const
00048     {
00049         return winnerIds.size() > 1;
00050     }
00051 };
00052
00053 class BonusPhase
00054 {
00055 public:
00056     static BonusResult execute(Game& game);
00057 };

```

11.34 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ gameplay/bonus_json.cpp File Reference

JSON serialization and TCP broadcast for the end-of-game result.

```

#include "bonus_json.hpp"
#include "communication/TCP_Server.hpp"
#include "utilities/Logger.hpp"
#include <nlohmann/json.hpp>

```

Functions

- nlohmann::json [makeGameResultsMessage](#) (const [BonusResult](#) &result)
- void [broadcastGameResults](#) (const [BonusResult](#) &result, [Server](#) &server)

11.34.1 Detailed Description

JSON serialization and TCP broadcast for the end-of-game result.

11.34.2 Function Documentation

11.34.2.1 broadcastGameResults()

```

void broadcastGameResults (
    const BonusResult & result,
    Server & server)

```

Broadcasts a `game_results` payload to every connected Unity client.

11.34.2.2 makeGameResultsMessage()

```
nlohmann::json makeGameResultsMessage (
    const BonusResult & result)
```

Builds the complete `game_results` payload without sending it.

11.35 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.hpp File Reference ↩

Converts end-of-game bonus results into the JSON message consumed by Unity.

```
#include "bonus.hpp"
#include <nlohmann/json_fwd.hpp>
```

Functions

- nlohmann::json `makeGameResultsMessage` (const `BonusResult` &result)
- void `broadcastGameResults` (const `BonusResult` &result, `Server` &server)

11.35.1 Detailed Description

Converts end-of-game bonus results into the JSON message consumed by Unity.

11.35.2 Function Documentation

11.35.2.1 broadcastGameResults()

```
void broadcastGameResults (
    const BonusResult & result,
    Server & server)
```

Broadcasts a `game_results` payload to every connected Unity client.

11.35.2.2 makeGameResultsMessage()

```
nlohmann::json makeGameResultsMessage (
    const BonusResult & result)
```

Builds the complete `game_results` payload without sending it.

11.36 bonus_json.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "bonus.hpp"
00009
00010 #include <nlohmann/json_fwd.hpp>
00011
00012 class Server;
00013
00015 nlohmann::json makeGameResultsMessage(const BonusResult& result);
00016
00018 void broadcastGameResults(const BonusResult& result, Server& server);
```

11.37 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↔ gameplay/game.cpp File Reference

Implementation of the [Game](#) class: turn/round bookkeeping and dice-roll-based turn order determination.

```
#include "game.hpp"
#include <algorithm>
#include <iostream>
```

11.37.1 Detailed Description

Implementation of the [Game](#) class: turn/round bookkeeping and dice-roll-based turn order determination.

11.38 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↔ gameplay/game.hpp File Reference

Declares the [Game](#) class, which owns the board and player list for a match and tracks whose turn it is, the current round, and match completion.

```
#include "utilities/Logger.hpp"
#include <vector>
#include <memory>
#include "../map/board.hpp"
#include "../player/player.hpp"
```

Classes

- class [Game](#)

Top-level game session state: the board, the participating players, and whose turn/round it currently is. Turn order and round advancement are driven externally (e.g. by a turn manager) via [nextPlayer\(\)](#)/[nextRound\(\)](#)/[determineTurnOrder\(\)](#); [Game](#) itself does not run a game loop.

11.38.1 Detailed Description

Declares the `Game` class, which owns the board and player list for a match and tracks whose turn it is, the current round, and match completion.

11.39 game.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009
00010
00011 #include "utilities/Logger.hpp"
00012
00013 #include <vector>
00014 #include <memory>
00015 #include "../map/board.hpp"
00016 #include "../player/player.hpp"
00017
00018
00026 class Game
00027 {
00028
00029
00030
00031 public:
00032
00038     Game(std::vector<std::shared_ptr<Player>> players, int maxRounds = 10);
00039     Board _board;
00040     std::vector<std::shared_ptr<Player>> _players;
00041
00042
00043
00049     Player& getCurrentPlayer();
00050
00051     int _currentPlayerIndex = 0;
00052     int _currentRound = 1;
00053     int _maxRounds = 10;
00054     bool _finished = false;
00055
00056
00057
00058
00059
00060
00064     void nextPlayer();
00065
00070     void nextRound();
00071
00076     bool isLastPlayer() const;
00077
00086     void determineTurnOrder();
00087 };
```

11.40 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ gameplay/game_manager.cpp File Reference

Implementation of `GameManager`: player ownership and mini-game orchestration.

```
#include <algorithm>
#include <iostream>
#include <mutex>
#include "game_manager.hpp"
```

11.40.1 Detailed Description

Implementation of [GameManager](#): player ownership and mini-game orchestration.

11.41 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ gameplay/game_manager.hpp File Reference

Defines the [GameManager](#) class, which owns all players and orchestrates running mini-games between them.

```
#include <mutex>
#include <vector>
#include "../map/board.hpp"
#include "../mini_games/minigame.hpp"
#include "../mini_games/minigame_participant.hpp"
#include "../mini_games/minigame_result.hpp"
#include "../player/player.hpp"
#include "../utilities/Logger.hpp"
```

Classes

- class [GameManager](#)

Owns all Players and orchestrates running mini-games. Converts Players to [MiniGameParticipant](#) DTOs before a mini-game starts, runs the mini-game to completion, and reconciles the returned [MiniGameResult](#) back into the real [Player](#) objects (currently: coin balance only; stars are awarded elsewhere).

11.41.1 Detailed Description

Defines the [GameManager](#) class, which owns all players and orchestrates running mini-games between them.

11.42 game_manager.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <mutex>
00010 #include <vector>
00011
00012 #include "../map/board.hpp"
00013 #include "../mini_games/minigame.hpp"
00014 #include "../mini_games/minigame_participant.hpp"
00015 #include "../mini_games/minigame_result.hpp"
00016 #include "../player/player.hpp"
00017 #include "../utilities/Logger.hpp"
00018
00019 // ===== GAME MANAGER =====
00020
00027 class GameManager {
00028 public:
00029
00034     GameManager(const std::vector<Player>& players);
00035
00036     // ===== MINIGAME FLOW =====
00037
```

```

00047     void runMiniGame(MiniGame& game);
00048
00060     void notifyRobotPosition(int index, double x_cm, double y_cm);
00061
00062     // ===== PLAYERS =====
00063
00067     const std::vector<Player>& getPlayers() const;
00068
00075     void syncPlayers(const std::vector<std::shared_ptr<Player>>& players);
00076
00077 private:
00078
00079     std::vector<Player> players_;
00080
00081     MiniGame* activeGame_ = nullptr;
00082     mutable std::mutex activeGameMutex_;
00083
00084     // ===== CONVERSION =====
00085
00092     static MiniGameParticipant toParticipant(const Player& player);
00093
00103     void applyResult(const MiniGameResult& result);
00104 };

```

11.43 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/ gameplay/lobby.cpp File Reference

Handles lobby related UDP and TCP messages.

```

#include "communication/UDP_Server.hpp"
#include "communication/TCP_Server.hpp"
#include "communication/message_processing.hpp"
#include "gameplay/start_game.hpp"
#include "utilities/Logger.hpp"
#include <nlohmann/json.hpp>
#include "communication/session_registry.hpp"
#include <algorithm>
#include <cctype>
#include <vector>
#include <string>
#include "items/shop.hpp"

```

Functions

- void **register_lobby_handlers** ([Server](#) &tcp_server, [UdpServer](#) &udp_server, [Shop](#) &shop)

11.43.1 Detailed Description

Handles lobby related UDP and TCP messages.

11.44 network_utils.hpp

```

00001 #pragma once
00002 #include <string>
00003 #include <stdint>
00004
00005 namespace RobotGame {
00006
00007     // Sends a one-off UDP broadcast on the given port. Opens its own short-lived
00008     // socket each call (no shared state), so it's safe to call from any thread,
00009     // including TurnManager's minigame background thread.
00010     void broadcastUdpMessage(uint16_t port, const std::string& message);
00011
00012 } // namespace RobotGame

```

11.45 start_game.hpp

```

00001 #pragma once
00002
00003 #include <memory>
00004 #include <string>
00005 #include <vector>
00006
00007 #include "../player/player.hpp"
00008 #include "game.hpp"
00009 #include "turnManager.hpp"
00010
00011 class GameManager;
00012 class Server;
00013 class Shop;
00014
00015 class start_game {
00016
00017 public:
00018     static void startGame(int playerCount, int roundCount, GameManager* gm, Server& server, Shop& shop);
00019     static void setPlayerNames(std::vector<std::string> names);
00020
00021     static void setGameManager(GameManager* gm);
00022
00023     static void setFleet(mind::fleet::robot_fleet* fleet);
00024
00025     static void update();
00026
00027     static void selectItem(int itemIndex);
00028
00029     static void skipItemSelection();
00030
00031     static void selectPath(int nodeId);
00032
00033     static void selectTargetPlayer(int playerId);
00034
00035     static TurnManager* getTurnManager();
00036
00037 private:
00038     static std::unique_ptr<Game> game;
00039
00040     static std::unique_ptr<TurnManager> turnManager;
00041     static std::vector<std::string> playerNames;
00042 };

```

11.46 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/← gameplay/turnManager.cpp File Reference

Implementation of [TurnManager](#), the per-turn state machine.

```

#include <memory>
#include <stdexcept>
#include "../mini_games/splatoon.hpp"
#include "../player/boardEffects.hpp"
#include "../player/dice.hpp"
#include "../player/player.hpp"
#include "../items/shop.hpp"
#include "bonus.hpp"
#include "bonus_json.hpp"
#include "game.hpp"
#include "game_manager.hpp"
#include "turnManager.hpp"
#include "network_utils.hpp"
#include "communication/TCP_Server.hpp"
#include "communication/message_processing.hpp"
#include "start_game.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "robot_controller/robot_controller.hpp"
#include "utilities/Logger.hpp"

```

Functions

- void `register_handlers_turnmanager` (`Server` &server, `TurnManager` &target)
Registers network message handlers for `TurnManager` ("roll_dice", "path_chosen", etc.). Wires message events to the given target `TurnManager` instance.
- void `register_handlers_turnmanager` (`Server` &server)
Registers network message handlers for the active `start_game` `TurnManager` session.

11.46.1 Detailed Description

Implementation of `TurnManager`, the per-turn state machine.

11.46.2 Function Documentation

11.46.2.1 `register_handlers_turnmanager()` [1/2]

```
void register_handlers_turnmanager (
    Server & server)
```

Registers network message handlers for the active `start_game` `TurnManager` session.

Parameters

<code>server</code>	The TCP server to register message handlers with.
---------------------	---

11.46.2.2 `register_handlers_turnmanager()` [2/2]

```
void register_handlers_turnmanager (
    Server & server,
    TurnManager & target)
```

Registers network message handlers for `TurnManager` ("roll_dice", "path_chosen", etc.). Wires message events to the given target `TurnManager` instance.

Parameters

<code>server</code>	The TCP server to register message handlers with.
<code>target</code>	The <code>TurnManager</code> instance to dispatch messages to.

11.47 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ gameplay/turnManager.hpp File Reference

Declares `TurnManager`, the per-turn state machine driving a player through item selection, dice rolling, movement, tile effects, and minigames.

```
#include <atomic>
#include <thread>
#include "game.hpp"
#include "utilities/Logger.hpp"
```

Classes

- class [TurnManager](#)

Per-game state machine that drives a single player through one turn: item usage, dice roll, movement, tile effects, and (at the end of a round) the minigame phase. Owns no gameplay data itself; each handler mutates the associated Game/Player and advances state. Progression through most states is driven by [update\(\)](#), but the `WaitingFor*` states pause until the matching select/skip method is called (e.g. from network/UI input).

Functions

- void [register_handlers_turnmanager](#) ([Server](#) &server, [TurnManager](#) &target)
Registers network message handlers for [TurnManager](#) ("roll_dice", "path_chosen", etc.). Wires message events to the given target [TurnManager](#) instance.
- void [register_handlers_turnmanager](#) ([Server](#) &server)
Registers network message handlers for the active [start_game TurnManager](#) session.

11.47.1 Detailed Description

Declares [TurnManager](#), the per-turn state machine driving a player through item selection, dice rolling, movement, tile effects, and minigames.

11.47.2 Function Documentation

11.47.2.1 [register_handlers_turnmanager\(\)](#) [1/2]

```
void register_handlers_turnmanager (
    Server & server)
```

Registers network message handlers for the active [start_game TurnManager](#) session.

Parameters

<i>server</i>	The TCP server to register message handlers with.
---------------	---

11.47.2.2 [register_handlers_turnmanager\(\)](#) [2/2]

```
void register_handlers_turnmanager (
    Server & server,
    TurnManager & target)
```

Registers network message handlers for [TurnManager](#) ("roll_dice", "path_chosen", etc.). Wires message events to the given target [TurnManager](#) instance.

Parameters

<i>server</i>	The TCP server to register message handlers with.
<i>target</i>	The TurnManager instance to dispatch messages to.

11.48 turnManager.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include <atomic>
00010 #include <thread>
00011
00012 #include "game.hpp"
00013 #include "utilities/Logger.hpp"
00014
00015 // Forward declarations -- avoids pulling heavy headers into every file that
00016 // includes turnManager.hpp. Full definitions are only needed in .cpp.
00017 namespace mind::fleet {
00018     class robot_fleet;
00019 }
00020
00021 class GameManager;
00022 class Server;
00023 class Shop;
00024
00025
00026 class TurnManager
00027 {
00028 public:
00029     enum TurnState
00030     {
00031         StartRound,
00032
00033         StartTurn,
00034         WaitingForItemChoice,
00035         WaitingForTargetChoice,
00036
00037         Rolling,
00038         Moving,
00039         WaitingForPathChoice,
00040
00041         ApplyingTile,
00042
00043         Minigame,
00044
00045         EndTurn,
00046         EndRound,
00047         EndGame
00048     };
00049
00050     Game& game;
00051     TurnState state;
00052
00053     Player& currentPlayer();
00054
00055     TurnManager(Game& game, Server& server, Shop& shop);
00056
00057     ~TurnManager();
00058 private:
00059     int _remainingSteps = 0;
00060     int _lastRoll = 0;
00061     int _previousNodeId = 0;
00062
00063     // --- Network wiring ---
00064     Server& server_;
00065
00066     // --- Game wiring ---
00067     GameManager* gameManager_ = nullptr;
00068     Shop& shop_;
00069     std::atomic<bool> minigameRunning_{false};
00070     std::thread minigameThread_;
00071     int _selectedItemIndex = 0;
00072     int _selectedTargetPlayer = 0;
00073     bool _playerMoving = false;
00074
00075     // --- Fleet wiring ---
00076     mind::fleet::robot_fleet* fleet_ = nullptr;
00077
00078     void driveRobotToNode(int fromNodeId, int toNodeId);
00079 public:
00080     void setGameManager(GameManager* gm);

```

```

00114
00119     void setFleet(mind::fleet::robot_fleet* fleet);
00120
00121 public:
00122
00127     TurnState getState() const;
00128
00130     int lastRoll() const { return _lastRoll; }
00131
00141     void update();
00142
00148     void startTurn();
00149
00150     //Items
00155     void selectItem(int inventoryIndex);
00156
00160     void skipItemSelection();
00161
00171     void executeSelectedItem();
00172
00173     //TargetPlayer
00184     void selectTargetPlayer(int playerId, int robotId);
00185
00186     //rollDice
00197     void rollDice();
00198
00199     //Movement
00211     void movePlayer();
00212
00220     void selectPath(int nodeId);
00221
00222     //Finish
00228     void applyTile();
00229
00236     void finishTurn();
00237
00244     void finishRound();
00245
00246     //Minigame
00253     void startMinigame();
00254
00255 };
00256
00263 void register_handlers_turnmanager(Server& server, TurnManager& target);
00264
00269 void register_handlers_turnmanager(Server& server);

```

11.49 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/↵ item.cpp File Reference

Implementation of the [Item](#) class: dice-roll overrides and targeted effects applied when a player uses an item.

```

#include "item.hpp"
#include "player/player.hpp"
#include "map/board.hpp"
#include "player/dice.hpp"

```

11.49.1 Detailed Description

Implementation of the [Item](#) class: dice-roll overrides and targeted effects applied when a player uses an item.

11.50 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/↵ item.hpp File Reference

Declares [ItemType](#), [ItemData](#), and the [Item](#) class: items that can override a player's dice roll and/or apply a targeted or self effect (position swap, control inversion, dice theft, coin bonus) during a turn.

```
#include <string>
#include "utilities/Logger.hpp"
#include <optional>
```

Classes

- struct [ItemData](#)

Static catalog entry describing a purchasable item: display name, effect type, catalog id, and shop price.

- class [Item](#)

A single item instance identified by its [ItemType](#). Depending on the type, using it either overrides the current player's dice roll ([getDiceOverride\(\)](#)) or applies an effect to the current and/or a target player ([applyItemEffects\(\)](#)).

Enumerations

- enum class [ItemType](#) {
DoubleDice , **TripleDice** , **SwapPosition** , **InvertedControls** ,
StealDice , **DoubleCoins** , **CasinoDice** }

Identifies the effect an [Item](#) represents; drives the switches in [Item::isDiceItem\(\)](#), [Item::requiresTarget\(\)](#), [Item::getDiceOverride\(\)](#), and [Item::applyItemEffects\(\)](#).

11.50.1 Detailed Description

Declares [ItemType](#), [ItemData](#), and the [Item](#) class: items that can override a player's dice roll and/or apply a targeted or self effect (position swap, control inversion, dice theft, coin bonus) during a turn.

11.51 item.hpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 #pragma once
00009
00010 #include <string>
00011 #include "utilities/Logger.hpp"
00012 #include <optional>
00013
00014 class Player;
00015 class Board;
00016
00022 enum class ItemType
00023 {
00024     DoubleDice,
00025     TripleDice,
00026     SwapPosition,
00027     InvertedControls,
00028     StealDice,
00029     DoubleCoins,
00030     CasinoDice
00031 };
00032
00037 struct ItemData
00038 {
00039     std::string name;
00040     ItemType type{};
00041     int id = 0;
00042     int price = 0;
00043 };
00044
00051 class Item
00052 {
```

```

00053 public:
00054
00058     Item() = default;
00059
00064     Item(ItemType type)
00065         : type(type)
00066     {
00067     }
00068
00072     ItemType getItemType() const;
00073
00078     bool isDiceItem() const;
00079
00084     bool requiresTarget() const;
00085
00090     int rollSingle();
00091
00097     int rollSum(int count);
00098
00099
00100
00108     [[nodiscard]] std::optional<int> getDiceOverride(Player& currentPlayer);
00109
00118     void applyItemEffects(
00119         Player& currentPlayer,
00120         Player* targetPlayer,
00121         const Board& board);
00122
00123
00124
00125
00126 private:
00127
00128     ItemType type{};
00129
00130
00131
00132
00133
00143     void swapPosition(
00144         Player& currentPlayer,
00145         Player& targetPlayer,
00146         const Board& board);
00147
00152     void invertedControls(Player& targetPlayer);
00153
00162     void stealDice(
00163         Player& currentPlayer,
00164         Player& targetPlayer);
00165
00169     void doubleCoins(Player& currentPlayer);
00170
00171
00172 };

```

11.52 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.cpp File Reference

Implementation of [Shop](#)'s purchase flow and the network handlers that open/close the shop and broadcast its inventory to connected clients.

```

#include "shop.hpp"
#include "item.hpp"
#include "../player/player.hpp"
#include "../player/inventory.hpp"
#include "../communication/message_processing.hpp"
#include "../communication/network_messages.hpp"

```

Functions

- void **register_handlers_shop** ([Server](#) &server, [Shop](#) &target)

Registers the shop's network message handlers.

- void **broadcast_update_shop** ([Shop](#) &shop, [Server](#) &server, std::uint8_t player_index)

Builds and broadcasts a "shop_open" message containing the full shop inventory.

11.52.1 Detailed Description

Implementation of [Shop](#)'s purchase flow and the network handlers that open/close the shop and broadcast its inventory to connected clients.

11.53 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.hpp File Reference

Declares the [Shop](#) class (fixed item catalog and purchase flow) and the free functions that register its network message handlers and broadcast shop state to connected clients.

```
#include "item.hpp"
#include <vector>
#include "../player/player.hpp"
#include <cstdint>
#include "../communication/TCP_Server.hpp"
#include "utilities/Logger.hpp"
```

Classes

- class [Shop](#)

The in-game item shop: a fixed catalog of purchasable items, opened/closed via the "open_shop"/"close_shop" network messages (see [openShop\(\)](#), [close_shop\(\)](#)). An item selection made on the AR/Unity client is intended to arrive asynchronously via [setSelectedItem\(\)](#) and be consumed by [selectItem\(\)](#), which blocks the calling thread until a selection is made, after which [purchaseItem\(\)](#) charges the player; neither pathway is currently wired to a network message handler in this codebase.

Functions

- void **register_handlers_shop** ([Server](#) &server, [Shop](#) &target)
Registers the shop's network message handlers.
- void **broadcast_update_shop** ([Shop](#) &shop, [Server](#) &server, std::uint8_t player_index)
Builds and broadcasts a "shop_open" message containing the full shop inventory.

11.53.1 Detailed Description

Declares the [Shop](#) class (fixed item catalog and purchase flow) and the free functions that register its network message handlers and broadcast shop state to connected clients.

11.54 shop.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008 #include "item.hpp"
00009 #include <vector>
00010 #include "../player/player.hpp"
00011 #include <cstdint>
00012
00013
00014 #include "../communication/TCP_Server.hpp"
00015
00016 #include "utilities/Logger.hpp"
00017
00026 class Shop {
00027 public:
00028     explicit Shop(Server& server) : server_(server) {}
00029
00030     void openShop(Player& player);
00031     int getItemPrice(ItemType type);
00032     void close_shop(Player& player);
00033     Item selectItem();
00034     void purchaseItem(Player& player);
00035     void setSelectedItem(int item_id);
00036     const std::vector<ItemData>& getShopInventory() const { return ShopInventory; }
00037
00038 private:
00039     std::mutex shop_mutex_;
00040     std::condition_variable shop_cv_;
00041     Server& server_;
00042     int selected_item_id_ = -1;
00043
00044     std::vector<ItemData> ShopInventory = {
00045         { "Double Dice", ItemType::DoubleDice, 1, 5 },
00046         { "TripleDice", ItemType::TripleDice, 2, 10 },
00047         { "Warp Dice", ItemType::SwapPosition, 3, 5 },
00048         { "Poison Dice", ItemType::InvertedControls, 4, 5 },
00049         { "Thief Dice", ItemType::StealDice, 5, 5 },
00050         { "Greedy Dice", ItemType::DoubleCoins, 6, 10 },
00051         { "Casino Dice", ItemType::CasinoDice, 7, 7 }
00052     };
00053 };
00054
00055 void register_handlers_shop(Server& server, Shop& target);
00056
00060 void broadcast_update_shop(Shop& shop, Server& server, std::uint8_t player_index);

```

11.55 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ mainpage.md File Reference

Mindstorms Party Desktop Application Documentation *.

11.55.1 Detailed Description

Mindstorms Party Desktop Application Documentation *.

- * Welcome to the API documentation for the Mindstorms Party desktop application.
-
- ## Getting Started
-
- The desktop application is a Qt6/C++23 application that serves as the central hub

- of the Mindstorms Kart project. It bridges physical LEGO Mindstorms EV3 robots
- with an AR mobile app, providing real-time computer vision tracking, autonomous
- path planning with collision avoidance, and a Mario-Kart-style board game.
-
- **### Key Modules**
- - [robot_controller](#) - Per-robot MQTT/UDP command facade
- - [robot_fleet](#) - Auto-discovery via UDP heartbeats
- - [protocol](#) - Binary wire codec for desktop<->EV3 communication
- - [pathfinding](#) - A* grid path planner
- - [collision_planner](#) - Multi-robot CTFS avoidance
- - [path_follower](#) - Pure pursuit + heading PID driver
- - [tracker](#) - ArUco marker detection pipeline
- - [Logger](#) - Thread-safe asynchronous logging system
-
- **### Building**
- * `cmake --preset <preset>`
- * `cmake --build build`
- *
-
- **### Documentation**
- - System architecture, network protocols, and logger design: see the PDF manual at `build/docs_out/↵`
`manual.pdf`
- - This HTML site: low-level API reference for all C++ classes and functions */

11.56 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/↵ board.cpp File Reference

Implementation of [Board](#): the hard-coded node graph, node lookups, and TCP broadcast of board data.

```
#include "board.hpp"
#include "../communication/TCP_Server.hpp"
#include "../communication/message_processing.hpp"
```

Functions

- void [register_handlers_board](#) (Server &server, [Board](#) &board)
Registers the "board_load" TCP message handler and associates the board with the server.

11.56.1 Detailed Description

Implementation of [Board](#): the hard-coded node graph, node lookups, and TCP broadcast of board data.

11.56.2 Function Documentation

11.56.2.1 register_handlers_board()

```
void register_handlers_board (
    Server & server,
    Board & board)
```

Registers the "board_load" TCP message handler and associates the board with the server.

Registers the "board_load" TCP message handler that broadcasts the board graph on request. Declared as a free function for registration consistency with the other subsystems.

Registers the "board_load" TCP message handler and associates the board with the server.

Parameters

<i>server</i>	The TCP server to register the handler on.
<i>board</i>	The board instance whose graph is broadcast when a "board_load" message arrives.

11.57 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/↵ board.hpp File Reference

Declares the static game-board node graph ([Board](#), [Node](#), [TileEffect](#)) and the TCP registration for board-load requests.

```
#include <vector>
#include <stdexcept>
#include <cstdint>
#include <iostream>
#include "utilities/Logger.hpp"
```

Classes

- struct [TileEffect](#)
One effect entry attached to a board node. The actual gameplay logic for each type (coin gain/loss, opening the shop) is implemented in [applyTileEffect\(\)](#) ([player/boardEffects.cpp](#)), not here - this struct only carries the data.
- class [Node](#)
One position ("tile") in the board's node graph. neighbors form an undirected adjacency list used for robot movement/pathfinding.
- class [Board](#)
Owns the static node graph for the game board and can broadcast it to clients over TCP. The graph is hard-coded in the constructor; nothing in this class mutates it afterward.

Enumerations

- enum class [TileType](#) { **Plus** , **Minus** , **Shop** , **Hazard** }
Categories of gameplay effect a board node can apply to a player that lands on it.

Functions

- void [register_handlers_board](#) ([Server](#) &server, [Board](#) &board)

Registers the "board_load" TCP message handler that broadcasts the board graph on request. Declared as a free function for registration consistency with the other subsystems.

11.57.1 Detailed Description

Declares the static game-board node graph ([Board](#), [Node](#), [TileEffect](#)) and the TCP registration for board-load requests.

11.57.2 Function Documentation

11.57.2.1 register_handlers_board()

```
void register_handlers_board (
    Server & server,
    Board & board)
```

Registers the "board_load" TCP message handler that broadcasts the board graph on request. Declared as a free function for registration consistency with the other subsystems.

Registers the "board_load" TCP message handler and associates the board with the server.

Parameters

<i>server</i>	The TCP server to register the handler on.
<i>board</i>	The board instance whose graph is broadcast when a "board_load" message arrives.

Registers the "board_load" TCP message handler that broadcasts the board graph on request. Declared as a free function for registration consistency with the other subsystems.

11.58 board.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <vector>
00010 #include <stdexcept>
00011 #include <stdint>
00012 #include <iostream>
00013 #include "utilities/Logger.hpp"
00014
00015 // Forward declaration to avoid pulling Boost / JSON headers into files including board.hpp
00016 class Server;
00017
00018 // =====
00019 // Tile effect types
00020 // =====
00021
00025 enum class TileType {
00026     Plus,
00027     Minus,
00028     Shop,
```

```

00029     Hazard
00030 };
00031
00032 // =====
00033 // Effect applied when a robot lands on a tile
00034 // =====
00035
00041 struct TileEffect {
00042     TileType type;
00043     int power;
00044 };
00045
00046 // =====
00047 // Graph node - one position on the board
00048 // =====
00049
00054 struct Node {
00055     int id;
00056     int x;
00057     int y;
00058     std::vector<int> neighbors;
00059     std::vector<TileEffect> effects;
00060 };
00061
00062 // =====
00063 // Board - holds the full node graph
00064 // =====
00065
00070 class Board {
00071 private:
00072     std::vector<Node> nodes;
00073
00074 public:
00081     Board();
00082
00089     const Node& getNode(int id) const;
00090
00095     const std::vector<Node>& getNodes() const;
00096
00103     void broadcast_board(Server& server);
00104
00110     std::string tileTypeToString(TileType type);
00111
00117     bool isValidNode(int id) const;
00118 };
00119
00126 void register_handlers_board(Server& server, Board& board);

```

11.59 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.cpp File Reference

Implementation of the path-follower tuning panel: builds the spin-box grid and handles JSON persistence.

```

#include "map/follower_tuning_panel.hpp"
#include <QCheckBox>
#include <QDoubleSpinBox>
#include <QFile>
#include <QGridLayout>
#include <QJsonDocument>
#include <QJsonObject>
#include <QLabel>
#include <QPushButton>

```

11.59.1 Detailed Description

Implementation of the path-follower tuning panel: builds the spin-box grid and handles JSON persistence.

11.60 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.hpp File Reference

Qt widget for live-tuning the path-follower's motion/PID gains, with JSON save/load and a panic-stop button.

```
#include <vector>
#include <QString>
#include <QWidget>
#include "path_follower/path_follower.hpp"
```

Classes

- class [follower_tuning_panel](#)

Live tuning panel for the path-following driver. A thin Qt view over the plain `follower_params` struct: editing any gain emits `changed()` so the owner can push it to every running follower (taking effect on the next 33 ms tick). Save/Load persist a tune to JSON so it survives a restart; PANIC STOP halts all robots.

11.60.1 Detailed Description

Qt widget for live-tuning the path-follower's motion/PID gains, with JSON save/load and a panic-stop button.

11.61 follower_tuning_panel.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <vector>
00009
00010 #include <QString>
00011 #include <QWidget>
00012
00013 #include "path_follower/path_follower.hpp"
00014
00015 class QDoubleSpinBox;
00016 class QCheckBox;
00017
00025 class follower_tuning_panel : public QWidget {
00026     Q_OBJECT
00027
00028 public:
00035     explicit follower_tuning_panel(follower::follower_params init,
00036                                   QWidget* parent = nullptr);
00037
00042     follower::follower_params params() const { return current_; }
00043
00049     void setParams(const follower::follower_params& p);
00050
00051 signals:
00056     void changed(follower::follower_params p);
00058     void panicStop();
00059
00060 private:
00062     void emitChanged();
00064     void saveJson();
00066     void loadJson();
00067
00069     struct row {
00070         QDoubleSpinBox* box;
00071         float follower::follower_params:: field;
00072         QString key;
00073     };
00074     std::vector<row> rows_;
00075     QCheckBox* flip_turn_ = nullptr;
00076     follower::follower_params current_;
00077 };
```

11.62 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/↵ mapwindow.cpp File Reference

Implementation of [MapWindow](#): rendering, camera tracking, collision-aware robot driving (real + offline sim), arrival detection, and auto-calibration. [Node](#) graph data lives in [Board](#) ([board.cpp](#)).

```
#include <algorithm>
#include <climits>
#include <cmath>
#include <stdexcept>
#include <vector>
#include <QAction>
#include <QLineEdit>
#include <QBrush>
#include <QDateTime>
#include <QFile>
#include <QJsonDocument>
#include <QColor>
#include <QPushButton>
#include <QComboBox>
#include <QJsonObject>
#include <QMouseEvent>
#include <QPaintEvent>
#include <QPainter>
#include <QPainterPath>
#include <QPen>
#include <QResizeEvent>
#include <QToolBar>
#include <QWidget>
#include "communication/UDP_Server.hpp"
#include "mapwindow.hpp"
#include "communication/TCP_Server.hpp"
#include "map/follower_tuning_panel.hpp"
#include "qtpanel/fleet_dashboard.hpp"
#include "qtpanel/network_monitor.hpp"
#include "qtpanel/path_debug.hpp"
#include "robot_controller/robot_controller.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "utilities/Logger.hpp"
#include "utilities/enum_tools.hpp"
```

11.62.1 Detailed Description

Implementation of [MapWindow](#): rendering, camera tracking, collision-aware robot driving (real + offline sim), arrival detection, and auto-calibration. [Node](#) graph data lives in [Board](#) ([board.cpp](#)).

11.63 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/↵ mapwindow.hpp File Reference

Declares [MapWindow](#), the live tracking + driving Qt window: renders the board and robots, consumes the shared camera/offline-sim tick loop, and drives each robot via a [path_follower](#).

```

#include <array>
#include <vector>
#include <opencv2/opencv.hpp>
#include <QColor>
#include <QImage>
#include <QMainWindow>
#include <QPixmap>
#include <QPoint>
#include <QString>
#include <QTimer>
#include "board.hpp"
#include "communication/UDP_Server.hpp"
#include "communication/TCP_Server.hpp"
#include "robot_tracker/tracker.hpp"
#include "collision_avoidance/collision_planner.hpp"
#include "path_follower/path_follower.hpp"

```

Classes

- struct [Robot](#)

Live state of one tracked/simulated robot slot (0-3), used for rendering and hit-testing. The authoritative pose for driving is `follower::pose2d` (see `MapWindow::poseOf`); `x/y/heading_rad` here are the on-screen projection of that pose, refreshed every tick.

- class [MapWindow](#)

Live tracking + driving Qt window for up to 4 robots. Owns the board node graph, the ArUco marker_tracker, the collision_planner, and one path_follower per robot slot. Runs in either an offline kinematic simulation (default, no camera required) or LIVE camera mode (toggled via the toolbar); a separate ARM DRIVE toggle independently gates whether commands reach real EV3 bricks over the injected robot_fleet. Ticks every 33 ms via an internal QTimer (see `onTrackerTick`).

11.63.1 Detailed Description

Declares [MapWindow](#), the live tracking + driving Qt window: renders the board and robots, consumes the shared camera/offline-sim tick loop, and drives each robot via a path_follower.

11.64 mapwindow.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 // =====
00008 // mapwindow.h
00009 // Qt window - handles rendering, shared camera
00010 // tracking, robot movement, and arrival detection
00011 //
00012 // Uses Board for node graph data.
00013 // Game logic uses Board for movement decisions
00014 // and MapWindow for sending commands to robots.
00015 // =====
00016
00017 #ifndef MAPWINDOW_H
00018 #define MAPWINDOW_H
00019
00020 #include <array>
00021 #include <vector>
00022
00023 #include <opencv2/opencv.hpp>
00024

```

```

00025 #include <QColor>
00026 #include <QImage>
00027 #include <QMainWindow>
00028 #include <QPixmap>
00029 #include <QPoint>
00030 #include <QString>
00031 #include <QTimer>
00032
00033 #include "board.hpp"
00034 #include "communication/UDP_Server.hpp"
00035 #include "communication/TCP_Server.hpp"
00036 #include "robot_tracker/tracker.hpp"
00037 #include "collision_avoidance/collision_planner.hpp"
00038 #include "path_follower/path_follower.hpp"
00039
00040 class QPainter;
00041 class QMouseEvent;
00042 class QAction;
00043 class follower_tuning_panel;
00044 class FleetDashboard;
00045 class NetworkMonitor;
00046 class PathDebug;
00047
00048 namespace mind::fleet {
00049 class robot_fleet;
00050 }
00051
00052 QT_BEGIN_NAMESPACE
00053 namespace Ui {
00054 class MapWindow;
00055 }
00056 QT_END_NAMESPACE
00057
00058 // =====
00059 // Robot state
00060 //
00061 // One instance per physical robot (max 4). id is the robot SLOT (0-3); the
00062 // ArUco marker that tracks each slot is bound to the physical marker table in
00063 // marker_tracker (marker IDs 0-3 map directly to slots 0-3).
00064 // =====
00065
00071 struct Robot {
00072     int id;
00073     int x;
00074     int y;
00075     int current_node = -1;
00076     QColor color;
00077     float angle = 0.0f;
00078     float heading_rad = 0.0f;
00079 };
00080
00081 // =====
00082 // Main application window
00083 //
00084 // Responsibilities:
00085 // - draws the board, nodes, and robots
00086 // - reads camera every 33ms via QTimer
00087 // - converts camera pixel -> physical cm -> screen pixel
00088 // - detects which node each robot is near
00089 // - sends movement commands to EV3 via sendRobotToNode
00090 // - fires robotArrived signal when a robot reaches its target
00091 // =====
00092
00104 class MapWindow : public QMainWindow
00105 {
00106     Q_OBJECT
00107
00108 public:
00109
00117     explicit MapWindow(UdpServer& udp_server, Server& tcp_server, QWidget *parent = nullptr);
00119     virtual ~MapWindow() override ;
00120
00125     bool initializeCamera();
00126
00135     void updateRobotPosition(int robot_id, cv::Point2f camera_pixel, float marker_angle);
00136
00145     void sendRobotToNode(int robot_id, int target_node_id);
00146
00148     const Board& getBoard() const { return board_; }
00149
00155     int getRobotNode(int robot_id) const;
00156
00164     void planPathFor(int robot_id, int target_node_id);
00165
00174     void movePlayboard(int robot_id, int target_node_id);
00175
00180     collision::collision_planner& collisionPlanner() { return planner_; }

```

```

00181
00186     void setFleet(mind::fleet::robot_fleet* fleet) { fleet_ = fleet; }
00187
00189     void invalidateBoardCache() { board_cache_valid_ = false; }
00190
00195     void setTrackingMarkers(const std::vector<marker_tracker::marker_position>& markers);
00196
00201     void setTrackingState(bool active);
00202
00209     void paintBoard(QPainter& painter);
00216     void boardClicked(QPoint pos, Qt::MouseButton button);
00217
00224     void startCalibration();
00225
00227     follower::follower_params followerParams() const noexcept { return follower_params_; }
00228
00229     // --- Public accessors for debug panels ---
00230     bool isSimMode() const noexcept { return sim_mode_; }
00231     bool isDrivingEnabled() const noexcept { return driving_enabled_; }
00232     bool isCameraReady() const noexcept { return camera_ready_; }
00233     bool isHomographyReady() const noexcept { return homography_ready_; }
00234     int getTargetNode(int id) const noexcept { return (id >= 0 && id < 4) ?
target_node_[static_cast<size_t>(id)] : -1; }
00235     const std::vector<Robot>& getRobots() const noexcept { return robots; }
00236     const std::array<follower::path_follower, 4>& getFollowers() const noexcept { return followers_; }
00237     mind::fleet::robot_fleet* getFleet() const noexcept { return fleet_; }
00238
00239 public slots:
00240
00241 signals:
00242
00249     void robotArrived(int robot_id, int node_id);
00250
00251 private slots:
00252
00258     void onTrackerTick();
00267     void checkArrival(int robot_id);
00268
00270     void openTuning();
00272     void openFleetDashboard();
00274     void openNetworkMonitor();
00276     void openPathDebug();
00278     void panicStopAll();
00279
00280     // --- Internal helpers (moved out of private for clarity) ---
00281
00288     void tryBuildHomography(const std::vector<marker_tracker::marker_position>& markers);
00289
00295     void buildCollisionGraph();
00296
00303     void feedCollisionPlanner();
00304
00310     void drawCollisionOverlay(QPainter& painter);
00311
00318     int robotSlotForMarker(int marker_id) const;
00319
00325     void driveRobots();
00326
00333     follower::pose2d poseOf(int robot_id) const;
00334
00342     bool isPoseStale(int robot_id) const;
00343
00345     void applyFollowerParams(const follower::follower_params& p);
00346
00351     void saveFollowerParams() const;
00352
00354     void loadFollowerParams();
00355
00360     void snapCurrentNode(int robot_id);
00361
00363     void repaintBoard();
00364
00366     void rebuildBoardCache();
00367
00375     void runCalibrationStep(int robot_id, float dt, qint64 now);
00376
00386     void simIntegrate(int robot_id, float w_dps, float v_mm_s, float dt);
00387
00388 #pragma region private members
00389 private:
00390     static constexpr int UNITY_BROADCAST_INTERVAL{10};
00391
00392     // --- Core ---
00393     Ui::MapWindow *ui = nullptr;
00394     UdpServer& udp_server_;
00395     Server& tcp_server_;
00396     Board board_;

```

```

00397     std::vector<Robot> robots;
00398     QTimer* timer_ = nullptr;
00399     QTimer* unity_broadcast_timer_{nullptr};
00400
00401     // --- Camera / homography ---
00402     bool camera_ready_{false};
00403     cv::Mat homography_;
00404     bool homography_ready_{false};
00405
00406     // --- Scale factors: physical cm -> screen pixels (Board 300x185 cm, window 1400x860 px) ---
00407     double scale_x_{1400.0 / 300.0};
00408     double scale_y_{860.0 / 185.0};
00409
00410     // --- Navigation ---
00411     std::array<int, 4> target_node_{-1, -1, -1, -1};
00412     collision::collision_planner planner_;
00413     std::array<follower::path_follower, 4> followers_;
00414     follower::follower_params follower_params_;
00415     mind::fleet::robot_fleet* fleet_{nullptr};
00416     std::array<QString, 4> marker_to_ev3_;
00417
00418     // --- Tracking ---
00419     std::vector<marker_tracker::marker_position> latest_markers_;
00420     bool tracking_markers_pending_{false};
00421
00422     // --- UI state ---
00423     bool driving_enabled_{false};
00424     bool sim_mode_{true};
00425     int selected_robot_{0};
00426     bool flip_heading_{false};
00427     bool show_rl_only_{false};
00428     QAction* arm_action_{nullptr};
00429     QWidget* canvas_{nullptr};
00430     std::array<bool, 4> warned_no_ctrl_{};
00431
00432     // --- Simulation ---
00433     std::array<follower::pose2d, 4> sim_pose_;
00434     quint64 last_drive_ms_{4}{0, 0, 0, 0};
00435     quint64 last_seen_ms_{4}{0, 0, 0, 0};
00436     int arrive_ticks_{4}{0, 0, 0, 0};
00437     cv::Point2f arrive_last_pos_{4}{};
00438     cv::Point2f goal_point_{4}{};
00439
00440     // --- Lazy panels ---
00441     follower_tuning_panel* tuning_panel_{nullptr};
00442     FleetDashboard* fleet_dashboard_{nullptr};
00443     NetworkMonitor* network_monitor_{nullptr};
00444     PathDebug* path_debug_{nullptr};
00445
00446     // --- Board cache ---
00447     QPixmap board_cache_;
00448     QPixmap board_nodes_cache_;
00449     bool board_cache_valid_{false};
00450     quint64 last_unity_broadcast_ms_{0};
00451
00452     // --- Auto-calibration ---
00453     enum class CalPhase { idle, turn, drive };
00454     CalPhase cal_phase_{CalPhase::idle};
00455     int cal_robot_{-1};
00456     int cal_pass_{0};
00457     int cal_rejects_{0};
00458     std::vector<float> cal_gains_{};
00459     bool cal_settling_{false};
00460     quint64 cal_phase_start_ms_{0};
00461     quint64 cal_settle_start_ms_{0};
00462     follower::pose2d cal_start_pose_{};
00463
00464 };
00465
00466 #endif // MAP_H

```

11.65 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.cpp File Reference

Implementation of the [Generator](#) roulette-wheel random number generator.

```
#include "rng.hpp"
```


Variables

- constexpr int **MIN** = 0
Lowest valid index into `m_numbers` (first wheel position).
- constexpr int **MAX** = 36
Highest valid index into `m_numbers` (last wheel position).

11.65.1 Detailed Description

Implementation of the [Generator](#) roulette-wheel random number generator.

11.66 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.hpp File Reference

Declares the [Generator](#) class, a roulette-wheel style random number generator.

```
#include <iostream>
#include <random>
```

Classes

- class [Generator](#)
Simulates drawing a number from a European roulette wheel. Picks a uniformly random position around the wheel and maps it to the pocket number printed at that position, so results follow the physical wheel layout rather than a plain sequential 0-36 order.

11.66.1 Detailed Description

Declares the [Generator](#) class, a roulette-wheel style random number generator.

11.67 rng.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007 #include <iostream>
00008 #include <random>
00009
00016 class Generator {
00017 public:
00024     int roll_ball();
00025
00030     int get_last_number();
00031
00032 protected:
00038     int random_number();
00039
00040     int m_last_number = 0;
00041     int m_numbers[37] =
00042     {
00043         0, 32, 15, 19, 4, 21, 2, 25, 17, 34, 6, 27, 13,
00044         36, 11, 30, 8, 23, 10, 5, 24, 16, 33, 1, 20, 14,
00045         31, 9, 22, 18, 29, 7, 28, 12, 35, 3, 26
00046     };
00047 };
00048
```

11.68 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/↵ roulette.cpp File Reference

Implementation of the [Roulette](#) betting game and its network message handlers.

```
#include "roulette.hpp"
#include "../communication/message_processing.hpp"
#include "../communication/network_messages.hpp"
#include <iostream>
```

Functions

- void [register_handlers_roulette](#) ([Server](#) &server, [Roulette](#) &target)
Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers.
- void [broadcast_update_roulette](#) ([Server](#) &server, std::uint8_t player_index)
Broadcasts a "roulette_start" message announcing which player is taking their turn.

11.68.1 Detailed Description

Implementation of the [Roulette](#) betting game and its network message handlers.

11.68.2 Function Documentation

11.68.2.1 broadcast_update_roulette()

```
void broadcast_update_roulette (
    Server & server,
    std::uint8_t player_index)
```

Broadcasts a "roulette_start" message announcing which player is taking their turn.

Parameters

<i>server</i>	The TCP server to broadcast through.
<i>player_index</i>	ID of the player whose roulette round is starting.

11.68.2.2 register_handlers_roulette()

```
void register_handlers_roulette (
    Server & server,
    Roulette & target)
```

Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers.

Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers. Wires each parsed JSON event to the corresponding [Roulette](#) method.

Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers.

Parameters

<i>server</i>	The TCP server to register handlers on.
<i>target</i>	The Roulette instance the handlers dispatch into.

11.69 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/↵ roulette.hpp File Reference

Declares the [Roulette](#) betting game: bet types, round state, and its network message wiring.

```
#include <stdint>
#include "../communication/TCP_Server.hpp"
#include "../player/player.hpp"
#include "utilities/Logger.hpp"
#include "rng.hpp"
#include <mutex>
#include <condition_variable>
```

Classes

- class [Roulette](#)

Runs one roulette betting round for a single player, driven by network messages from the AR/Unity client. Owns the roll-to-result logic (via [Generator](#)) and blocks the calling thread in [choose_bet_type\(\)](#) until the client's bet arrives over the network, synchronizing between the asio message-handler thread ([setBet\(\)](#)) and whichever thread drives the game turn ([give_take_coins\(\)](#)).

Enumerations

- enum class [Bet_type](#) { [Odd_even](#) , [Red_black](#) }

Identifies which category of roulette bet a player has placed. Only two of the three classic European roulette bet categories are implemented; a single-number ("straight") bet is sketched out below as a comment but not available.

Functions

- void [register_handlers_roulette](#) ([Server](#) &server, [Roulette](#) &target)
Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers. Wires each parsed JSON event to the corresponding [Roulette](#) method.
- void [broadcast_update_roulette](#) ([Server](#) &server, std::uint8_t player_index)
Broadcasts a "roulette_start" message announcing which player is taking their turn.

11.69.1 Detailed Description

Declares the [Roulette](#) betting game: bet types, round state, and its network message wiring.

11.69.2 Function Documentation

11.69.2.1 broadcast_update_roulette()

```
void broadcast_update_roulette (
    Server & server,
    std::uint8_t player_index)
```

Broadcasts a "roulette_start" message announcing which player is taking their turn.

Parameters

<i>server</i>	The TCP server to broadcast through.
<i>player_index</i>	ID of the player whose roulette round is starting.

11.69.2.2 register_handlers_roulette()

```
void register_handlers_roulette (
    Server & server,
    Roulette & target)
```

Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers. Wires each parsed JSON event to the corresponding [Roulette](#) method.

Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers.

Parameters

<i>server</i>	The TCP server to register handlers on.
<i>target</i>	The Roulette instance the handlers dispatch into.

Registers the "roulette_start", "make_bet" and "end_roulette" network message handlers. Wires each parsed JSON event to the corresponding [Roulette](#) method.

11.70 roulette.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007 #include <stdint>
00008 #include "../communication/TCP_Server.hpp"
00009 #include "../player/player.hpp"
00010 #include "utilities/Logger.hpp"
00011 #include "rng.hpp"
00012
```

```

00013 #include <mutex>
00014 #include <condition_variable>
00015
00021 enum class Bet_type{
00022     Odd_even,
00023     Red_black,
00024     //Straight, //single number
00025 };
00032 class Roulette {
00033 public:
00034     explicit Roulette(Server& server) : server_(server) {}
00035
00036     Bet_type bet_type;
00037     int player_choice;
00038     int bet_amount;
00039
00040     void start_roulette(Player& player);
00041     Bet_type choose_bet_type(Player& player);
00042     int make_bet(Player& player);
00043     bool roulette_odd_even();
00044     bool roulette_red_black();
00045     bool calculate_results(Bet_type type, Player& player, int player_choice);
00046     void give_take_coins(Player& player);
00047     void end_roulette(Player& player);
00048     void setBet(Bet_type type, int amount, int choice);
00049
00050 private:
00051     Server& server_;
00052     std::mutex roulette_mutex_;
00053     std::condition_variable roulette_cv_;
00054     bool bet_received_ = false;
00055     int m_table_result = 0;
00056 };
00057
00064 void register_handlers_roulette(Server& server, Roulette& target);
00065
00071 void broadcast_update_roulette(Server& server, std::uint8_t player_index);
00072

```

11.71 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame.hpp File Reference

Defines the abstract [MiniGame](#) interface implemented by every playable mini-game.

```

#include <string>
#include <vector>
#include "minigame_participant.hpp"
#include "minigame_result.hpp"

```

Classes

- class [MiniGame](#)

Abstract base class for all mini-games. Operates only on [MiniGameParticipant](#) - no dependency on [Player](#), [Board](#), or inventory. Results are returned via [MiniGameResult](#) and applied to real Players by [GameManager](#).

11.71.1 Detailed Description

Defines the abstract [MiniGame](#) interface implemented by every playable mini-game.

11.72 minigame.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <string>
00009 #include <vector>
00010
00011 #include "minigame_participant.hpp"
00012 #include "minigame_result.hpp"
00013
00014 // ===== MINIGAME =====
00015
00021 class MiniGame {
00022 public:
00024     virtual ~MiniGame() = default;
00025
00033     virtual MiniGameResult play(std::vector<MiniGameParticipant>& participants) = 0;
00034
00036     virtual std::string getName() const = 0;
00037
00048     virtual void onRobotMoved(int index, double x_cm, double y_cm) {}
00049 };

```

11.73 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_participant.hpp File Reference

Defines the plain data-transfer object mini-games use to represent players.

```
#include <string>
```

Classes

- struct [MiniGameParticipant](#)

Data-transfer object representing a player during a mini-game. Created from a real [Player](#) before the mini-game starts. Mini-games read and mutate this copy in isolation from Player/Board/inventory; [GameManager](#) reconciles the (possibly changed) fields back into the real [Player](#) once play() returns.

11.73.1 Detailed Description

Defines the plain data-transfer object mini-games use to represent players.

11.74 minigame_participant.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <string>
00009
00010 // ===== MINIGAME PARTICIPANT =====
00011
00018 struct MiniGameParticipant {
00019     int         playerId = -1;
00020     int         robotId  = -1;
00021     std::string name;
00022     int         coins    = 0;
00023
00024 };
00025
00026

```

11.75 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_result.hpp File Reference

Defines the outcome type returned by [MiniGame::play\(\)](#).

```
#include <vector>
#include "minigame_participant.hpp"
```

Classes

- struct [MiniGameResult](#)

Returned by every mini-game after play() finishes. The board-game layer ([GameManager](#)) reads this and applies rewards to the real [Player](#) objects.

11.75.1 Detailed Description

Defines the outcome type returned by [MiniGame::play\(\)](#).

11.76 minigame_result.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <vector>
00009
00010 #include "minigame_participant.hpp"
00011
00012 // ===== MINIGAME RESULT =====
00013
00018 struct MiniGameResult {
00019     std::vector<MiniGameParticipant> participants;
00020     int winnerId = -1;
00021 };
```

11.77 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatton.cpp File Reference

Implementation of the Splatton mini-game: camera-driven grid painting, UDP sync with each player's phone, and end-of-round scoring.

```
#include <algorithm>
#include <chrono>
#include <cstdio>
#include <iostream>
#include <string>
#include <thread>
#include "utilities/Logger.hpp"
#include "splatton.hpp"
```

11.77.1 Detailed Description

Implementation of the Splatoon mini-game: camera-driven grid painting, UDP sync with each player's phone, and end-of-round scoring.

11.78 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_↵ games/splatoon.hpp File Reference

Splatoon mini-game: robots paint cells of a grid by driving over them, tracked via camera position and mirrored to each player's phone over UDP.

```
#include <map>
#include <memory>
#include <mutex>
#include <string>
#include <vector>
#include <chrono>
#include "minigame.hpp"
#include "splatoon_grid.hpp"
#include "udp_sender.hpp"
#include "udp_reciever.hpp"
```

Classes

- class [RobotGame::Splatoon](#)

Mini-game where each robot paints cells of a 9x15 grid by driving over them. Converts camera-tracked robot positions into grid cells, mirrors the board state and countdown to every player's phone over UDP, and scores by painted-cell count once the fixed-length round (DURATION_SECONDS) ends.

11.78.1 Detailed Description

Splatoon mini-game: robots paint cells of a grid by driving over them, tracked via camera position and mirrored to each player's phone over UDP.

11.79 splatoon.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <map>
00010 #include <memory>
00011 #include <mutex>
00012 #include <string>
00013 #include <vector>
00014
00015
00016 #include <chrono>
00017
00018 #include "minigame.hpp"
00019 #include "splatoon_grid.hpp"
00020
```



```

00021 #include "udp_sender.hpp"
00022
00023 #include "udp_reciever.hpp" // For Joystick input from phones
00024
00025 namespace RobotGame {
00026
00027     // ===== SPLATOON =====
00028
00035     class Splatoon : public MiniGame {
00036     public:
00042         Splatoon();
00043
00060         MiniGameResult play(std::vector<MiniGameParticipant>& participants) override;
00061
00063         std::string getName() const override;
00064
00078         void updateRobotPosition(int participantIndex, double x_cm, double y_cm);
00079
00088         void onRobotMoved(int index, double x_cm, double y_cm) override;
00089
00090
00091     private:
00092         static constexpr int DURATION_SECONDS = 60;
00093         static constexpr int COINS_FIRST = 100;
00094         static constexpr int COINS_SECOND = 50;
00095
00096         std::chrono::steady_clock::time_point startTime_;
00097
00102         int remainingSeconds() const;
00103
00104         // Grid calculation constants to avoid magic numbers
00105         static constexpr double Y_OFFSET_CM = 2.5;
00106         static constexpr double CELL_SIZE_CM = 20.0;
00107
00108         // Mutex to protect shared data structures from concurrent access (OpenCV tracking vs. MQTT
00109         vs. Game loop)
00110         mutable std::mutex gameMutex_;
00111
00112         SplatoonGrid grid_;
00113
00114         // UDP senders, one per player's phone, keyed by playerId (1..4) and
00115         // learned at runtime from the source IP of each phone's claim packets,
00116         // so no phone IP is hardcoded. phoneSenders_ carries the grid (port
00117         // 12345); robotPosSenders_ carries the camera-tracked robot position
00118         // (port 12347). phoneIps_ holds the last IP seen per player so a sender
00119         // is rebuilt only when a phone first appears or its address changes.
00120         std::map<int, UdpSender> phoneSenders_;
00121         std::map<int, UdpSender> robotPosSenders_;
00122         std::map<int, std::string> phoneIps_;
00123
00124         std::vector<int> positions_;
00125
00126         std::unique_ptr<UdpReceiver> claimReceiver_;
00127
00139         void handleCellClaim(const std::string& msg, const std::string& fromIp);
00140
00141         // ===== HELPERS =====
00142
00144         void initializeGrid();
00145
00151         void prepareParticipants(const std::vector<MiniGameParticipant>& participants);
00152
00158         void claimCell(int cellIndex, int participantIndex);
00159
00165         std::vector<std::pair<int, int> > calculateRanking(const std::vector<MiniGameParticipant>&
00166         participants) const;
00173
00174         void applyRewards(std::vector<MiniGameParticipant>& participants,
00175         const std::vector<std::pair<int, int> >& ranking) const;
00176
00181         static int indexToPlayerValue(int index);
00182
00187         void publishGrid() const;
00188
00197         void sendStartCell(int participantIndex, int row, int col) const;
00198
00204         void sendWinner(int playerId) const;
00205     };
00206
00207 }

```

11.80 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_↵ games/splattoon_grid.cpp File Reference

Implementation of [SplattoonGrid](#), the flat paint-ownership grid used by the Splattoon mini-game.

```
#include "splattoon_grid.hpp"  
#include "utilities/Logger.hpp"  
#include <algorithm>  
#include <stdexcept>
```

11.80.1 Detailed Description

Implementation of [SplattoonGrid](#), the flat paint-ownership grid used by the Splattoon mini-game.

11.81 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_↵ games/splattoon_grid.hpp File Reference

Fixed-size grid of per-cell paint ownership used by the Splattoon mini-game.

```
#include <array>  
#include <cstddef>
```

Classes

- class [SplattoonGrid](#)

*9x15 grid modeling the physical board area painted during the Splattoon mini-game. Each cell holds 0 (neutral/↵ unpainted) or 1-4 (painted by that player). Row 0 is the top row, column 0 is the left column, and cells are stored row-major in a flat array (index = row * COLS + col). Pure data class - no Qt, no rendering.*

11.81.1 Detailed Description

Fixed-size grid of per-cell paint ownership used by the Splattoon mini-game.

11.82 splatoon_grid.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <array>
00009 #include <cstdint>
00010
00017 class SplatoonGrid {
00018 public:
00019
00020     static constexpr int ROWS = 9;
00021     static constexpr int COLS = 15;
00022     static constexpr int SIZE = ROWS * COLS;
00023
00025     SplatoonGrid();
00026
00034     [[nodiscard]] int get(int row, int col) const;
00035
00043     void set(int row, int col, int value);
00044
00049     void clear(int value = 0);
00050
00056     [[nodiscard]] int countValue(int value) const noexcept;
00057
00062     [[nodiscard]] const std::array<int, SIZE>& cells() const noexcept;
00063
00064 private:
00065
00073     static std::size_t toIndex(int row, int col);
00074
00075     std::array<int, SIZE> cells_{};
00076 };

```

11.83 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_↵ games/udp_reciever.hpp File Reference

Cross-platform (Windows/POSIX) UDP receiver that dispatches each datagram to a callback on a background thread.

```

#include <string>
#include <functional>
#include <thread>
#include <atomic>
#include <cstdint>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>
#include <cstring>

```

Classes

- class [UdpReceiver](#)

Listens for UDP datagrams on a fixed port and reports each one via a callback. Binds to INADDR_ANY on construction and spins up a dedicated background thread that blocks on recvfrom() for the lifetime of the object.

Typedefs

- using **rsocket_t** = int

11.83.1 Detailed Description

Cross-platform (Windows/POSIX) UDP receiver that dispatches each datagram to a callback on a background thread.

11.84 udp_reciever.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007 #include <string>
00008 #include <functional>
00009 #include <thread>
00010 #include <atomic>
00011 #include <cstdint>
00012
00013 #if defined(_WIN32)
00014     #include <winsock2.h>
00015     #include <ws2tcpip.h>
00016     #pragma comment(lib, "ws2_32.lib")
00017     using rsocket_t = SOCKET;
00018     static constexpr rsocket_t R_INVALID = INVALID_SOCKET;
00019 #else
00020     #include <arpa/inet.h>
00021     #include <sys/socket.h>
00022     #include <unistd.h>
00023     #include <cstring>
00024     using rsocket_t = int;
00025     static constexpr rsocket_t R_INVALID = -1;
00026 #endif
00027
00033 class UdpReceiver
00034 {
00035 public:
00043     UdpReceiver(int port, std::function<void(const std::string&, const std::string&)> onMessage)
00044         : onMessage_(std::move(onMessage))
00045     {
00046     #if defined(_WIN32)
00047         WSADATA wsa; WSStartup(MAKEWORD(2,2), &wsa);
00048     #endif
00049         sockfd_ = ::socket(AF_INET, SOCK_DGRAM, 0);
00050
00051         sockaddr_in addr{};
00052         std::memset(&addr, 0, sizeof(addr));
00053         addr.sin_family = AF_INET;
00054         addr.sin_addr.s_addr = INADDR_ANY; // listen on all interfaces
00055         addr.sin_port = htons(static_cast<uint16_t>(port));
00056         ::bind(sockfd_, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
00057
00058         running_ = true;
00059         thread_ = std::thread(&UdpReceiver::loop, this);
00060     }
00061
00065     ~UdpReceiver()
00066     {
00067         running_ = false;
00068         if (sockfd_ != R_INVALID) {
00069     #if defined(_WIN32)
00070             ::closesocket(sockfd_);
00071     #else
00072             ::close(sockfd_);
00073     #endif
00074         }
00075         if (thread_.joinable()) thread_.join();
00076     #if defined(_WIN32)
00077         WSACleanup();
00078     #endif
00079     }
00080
00081 private:
00088     void loop()
00089     {
00090         char buf[1024];
00091         while (running_)
00092         {
00093             sockaddr_in from{};
00094     #if defined(_WIN32)

```

```

00095         int fromLen = sizeof(from);
00096     #else
00097         socklen_t fromLen = sizeof(from);
00098     #endif
00099     int n = ::recvfrom(sockfd_, buf, sizeof(buf) - 1, 0,
00100                      reinterpret_cast<sockaddr*>(&from), &fromLen);
00101     if (n > 0)
00102     {
00103         buf[n] = '\0';
00104         char ip[INET_ADDRSTRLEN] = {0};
00105         ::inet_ntop(AF_INET, &from.sin_addr, ip, sizeof(ip));
00106         if (onMessage_) onMessage_(std::string(buf, n), std::string(ip));
00107     }
00108     else if (n <= 0)
00109     {
00110         if (!running_) break;    // socket closed on shutdown
00111     }
00112 }
00113 }
00114
00115 rsocket_t sockfd_ = R_INVALID;
00116 std::function<void(const std::string&, const std::string&)> onMessage_;
00117 std::thread thread_;
00118 std::atomic<bool> running_{false};
00119 };

```

11.85 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/udp_sender.hpp File Reference

Cross-platform (Windows/POSIX) UDP datagram sender bound to a single destination.

```

#include <cstring>
#include <cstdint>
#include <string>
#include <string_view>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>

```

Classes

- class [UdpSender](#)

Fire-and-forget UDP sender for one fixed destination (ip:port). Create one instance per target and reuse it for every [send\(\)](#) call; each instance owns a single UDP socket for its lifetime.

Typedefs

- using `socket_t` = int

11.85.1 Detailed Description

Cross-platform (Windows/POSIX) UDP datagram sender bound to a single destination.

11.86 udp_sender.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007 #include <cstring>
00008 #include <cstdint>
00009 #include <string>
00010 #include <string_view>
00011
00012 #if defined(_WIN32)
00013     #include <winsock2.h>
00014     #include <ws2tcpip.h>
00015     #pragma comment(lib, "ws2_32.lib")    // auto-link Winsock (MSVC)
00016     using socket_t = SOCKET;
00017     static constexpr socket_t INVALID_SOCKET = INVALID_SOCKET;
00018 #else
00019     #include <arpa/inet.h>
00020     #include <sys/socket.h>
00021     #include <unistd.h>
00022     using socket_t = int;
00023     static constexpr socket_t INVALID_SOCKET = -1;
00024 #endif
00025
00031 class UdpSender
00032 {
00033 public:
00041     UdpSender(std::string_view ip, int port)
00042     {
00043         #if defined(_WIN32)
00044             // Winsock must be initialised once per process. Ref-count it.
00045             if (wsaCount_++ == 0) {
00046                 WSADATA wsa;
00047                 WSStartup(MAKEWORD(2, 2), &wsa);
00048             }
00049         #endif
00050         sockfd_ = ::socket(AF_INET, SOCK_DGRAM, 0);
00051
00052         std::memset(&dest_, 0, sizeof(dest_));
00053         dest_.sin_family = AF_INET;
00054         dest_.sin_port = htons(static_cast<uint16_t>(port));
00055         ::inet_pton(AF_INET, std::string(ip).c_str(), &dest_.sin_addr);
00056     }
00057
00061     ~UdpSender()
00062     {
00063         if (sockfd_ != INVALID_SOCKET) {
00064             #if defined(_WIN32)
00065                 ::closesocket(sockfd_);
00066             #else
00067                 ::close(sockfd_);
00068             #endif
00069         }
00070         #if defined(_WIN32)
00071         if (--wsaCount_ == 0) {
00072             WSACleanup();
00073         }
00074         #endif
00075     }
00076
00080     UdpSender(const UdpSender&) = delete;
00081     UdpSender& operator=(const UdpSender&) = delete;
00082
00088     UdpSender(UdpSender&& other) noexcept
00089     {
00090         sockfd_ = other.sockfd_;
00091         dest_ = other.dest_;
00092         other.sockfd_ = INVALID_SOCKET;
00093         #if defined(_WIN32)
00094         wsaCount_++;    // this object now also holds a Winsock ref
00095         #endif
00096     }
00097
00103     void send(std::string_view payload) const
00104     {
00105         if (sockfd_ == INVALID_SOCKET) return;
00106         ::sendto(sockfd_, payload.data(),
00107             #if defined(_WIN32)
00108             static_cast<int>(payload.size()),
00109             #else
00110             payload.size(),
00111             #endif
00112             0, reinterpret_cast<const sockaddr*>(&dest_), sizeof(dest_));
00113     }

```

```

00114
00115 private:
00116     socket_t     sockfd_ = INVALID_SOCKET;
00117     sockaddr_in dest_{};
00118
00119 #if defined(_WIN32)
00120     static inline int wsaCount_ = 0;
00121 #endif
00122 };

```

11.87 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.cpp File Reference

Implementation of the Qt/libmosquitto MQTT client wrapper.

```

#include "mqtt_client/client.hpp"
#include <atomic>
#include <iostream>
#include <string>
#include <QCoreApplication>
#include <mosquitto.h>
#include "utilities/Logger.hpp"
#include "utilities/enum_tools.hpp"

```

11.87.1 Detailed Description

Implementation of the Qt/libmosquitto MQTT client wrapper.

11.88 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.hpp File Reference

Declares the Qt wrapper around libmosquitto for async MQTT connectivity.

```

#include <QByteArray>
#include <QHash>
#include <QObject>
#include <QString>

```

Classes

- class [mind::mqtt::client](#)

Asynchronous MQTT client, wrapping libmosquitto behind Qt signals.

11.88.1 Detailed Description

Declares the Qt wrapper around libmosquitto for async MQTT connectivity.

11.89 client.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <QByteArray>
00009 #include <QHash>
00010 #include <QObject>
00011 #include <QString>
00012
00013 struct mosquitto;
00014 struct mosquitto_message;
00015
00016 namespace mind::mqtt {
00017
00028 class client : public QObject {
00029     Q_OBJECT
00030
00031 public:
00041     explicit client(QObject* parent = nullptr);
00047     ~client() override;
00048
00049     client(const client&) = delete;
00050     client& operator=(const client&) = delete;
00051
00062     void connect_to(QString host, int port);
00063
00068     void disconnect_from_broker();
00069
00078     bool publish(QString topic, const QByteArray& payload, int qos = 1);
00079
00090     bool subscribe(QString topic, int qos = 1);
00091
00092 signals:
00094     void connected();
00096     void disconnected();
00102     void message_received(QString topic, QByteArray payload);
00103
00104 private:
00111     static void on_connect_trampoline(struct mosquitto* mosq, void* userdata, int rc);
00117     static void on_disconnect_trampoline(struct mosquitto* mosq, void* userdata, int rc);
00124     static void on_message_trampoline(struct mosquitto* mosq, void* userdata,
00125                                     const struct mosquitto_message* msg);
00126
00133     void handle_connected_on_main();
00137     void handle_disconnected_on_main(int rc);
00143     void handle_message_on_main(QString topic, QByteArray payload);
00144
00145     struct mosquitto* mosq_{nullptr};
00146     QString host_{};
00147     int port_{};
00148     bool is_connected_{false};
00149     bool loop_started_{false};
00150     int error_code_{};
00151
00152     QHash<QString, int> subscriptions_;
00153 };
00154
00155 } // namespace mind::mqtt

```

11.90 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_↵ follower/path_follower.cpp File Reference

Implementation of the pure-pursuit + heading-PID path follower: polyline projection, lookahead, rotate-in-place turn dead-reckoning with live turn-rate learning, and the drive_cmd -> brick percentage mapping.

```

#include "path_follower/path_follower.hpp"
#include <algorithm>
#include <cmath>
#include <limits>

```


Functions

- `drive_pct follower::to_percent` (const `drive_cmd` &cmd, const `follower_params` &p)

Maps a `drive_cmd` velocity setpoint to brick tank-drive percentages.

11.90.1 Detailed Description

Implementation of the pure-pursuit + heading-PID path follower: polyline projection, lookahead, rotate-in-place turn dead-reckoning with live turn-rate learning, and the `drive_cmd` -> brick percentage mapping.

11.90.2 Function Documentation

11.90.2.1 `to_percent()`

```
drive_pct follower::to_percent (
    const drive_cmd & cmd,
    const follower_params & p)
```

Maps a `drive_cmd` velocity setpoint to brick tank-drive percentages.

Maps a `drive_cmd` velocity setpoint to brick tank-drive percentages. Scales `v_mm_s` / `omega_dps` against the `follower_params` brick maxima and the learned `drive_scale` / `turn_scale` calibration, then clamps to [-100, 100]. Also applies the in-place-pivot and forward-drive deadband floors (`min_turn_pct` / `min_speed_pct`) so a floored command still overcomes the EV3's static friction.

Parameters

<code>cmd</code>	The velocity setpoint to convert; <code>cmd.stop</code> short-circuits to {0, 0}.
<code>p</code>	Follower parameters providing the brick maxima, calibration scales, and deadband floors.

Returns

The clamped speed/turn percentages to send to the brick.

11.91 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.hpp File Reference ↩

Per-robot path-following controller (pure pursuit + heading PID) that turns the collision planner's waypoint polyline and the robot's current pose into per-tick velocity/turn-rate drive commands.

```
#include <cstdint>
#include <vector>
#include <opencv2/core.hpp>
#include "collision_avoidance/path_update.hpp"
```

Classes

- struct `follower::pose2d`
Robot pose in the board frame. Position in centimetres, heading in radians using atan2(dy, dx) (0 = +x), the SAME convention as the planner and the kinematic sim. The board cm frame is y-down (left-handed); that is fine as long as everything (planner, follower, sim) shares it - the only place the real-world chirality matters is the hardware steering sign (params.flip_turn).
- struct `follower::drive_cmd`
Controller output: a velocity setpoint for the drive base.
- struct `follower::follower_params`
All tunable gains and limits for the path follower, in one plain, copyable, JSON-friendly struct. A Qt tuning panel can read/write these live and save/load them. The `path_follower` re-reads this struct fresh on every step(), so `set_params()` takes effect on the very next tick.
- struct `follower::drive_pct`
Brick tank-drive percentages, each clamped to [-100, 100].
- class `follower::path_follower`
Per-robot path-following controller: pure pursuit + heading PID. Combines lookahead pursuit on the planner's waypoint polyline with a heading PID, curvature/goal speed-scaling, a rotate-in-place phase for large heading errors, and arrival/stop detection. Consumes the planner's already-rounded polyline, so it never cuts corners itself. Plain C++/↔ STL + OpenCV only, no Qt, no network - unit-testable in a kinematic sim.

Functions

- `drive_pct follower::to_percent` (const `drive_cmd` &cmd, const `follower_params` &p)
Maps a `drive_cmd` velocity setpoint to brick tank-drive percentages.

11.91.1 Detailed Description

Per-robot path-following controller (pure pursuit + heading PID) that turns the collision planner's waypoint polyline and the robot's current pose into per-tick velocity/turn-rate drive commands.

11.91.2 Function Documentation

11.91.2.1 to_percent()

```
drive_pct follower::to_percent (
    const drive_cmd & cmd,
    const follower_params & p)
```

Maps a `drive_cmd` velocity setpoint to brick tank-drive percentages.

Maps a `drive_cmd` velocity setpoint to brick tank-drive percentages. Scales `v_mm_s` / `omega_dps` against the `follower_params` brick maxima and the learned `drive_scale` / `turn_scale` calibration, then clamps to [-100, 100]. Also applies the in-place-pivot and forward-drive deadband floors (`min_turn_pct` / `min_speed_pct`) so a floored command still overcomes the EV3's static friction.

Parameters

<code>cmd</code>	The velocity setpoint to convert; <code>cmd.stop</code> short-circuits to {0, 0}.
<code>p</code>	Follower parameters providing the brick maxima, calibration scales, and deadband floors.

Returns

The clamped speed/turn percentages to send to the brick.

11.92 path_follower.hpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 #pragma once
00009
00010 #include <cstdint>
00011 #include <vector>
00012
00013 #include <opencv2/core.hpp>
00014
00015 #include "collision_avoidance/path_update.hpp"
00016
00017 namespace follower {
00018
00027 struct pose2d {
00028     float x_cm = 0.0f;
00029     float y_cm = 0.0f;
00030     float heading_rad = 0.0f;
00031 };
00032
00039 struct drive_cmd {
00040     float v_mm_s = 0.0f;
00041     float omega_dps = 0.0f;
00042     bool stop = true;
00043 };
00044
00052 struct follower_params {
00053     float v_max_mm_s = 150.0f;
00054     float v_min_mm_s = 40.0f;
00055     float lookahead_cm = 20.0f;
00056     float omega_cap_dps = 120.0f;
00057     float omega_rotate_dps = 90.0f;
00058     float min_turn_pct = 40.0f;
00063     float min_speed_pct = 0.0f;
00069     float kp = 1.6f;
00070     float ki = 0.0f;
00071     float kd = 0.20f;
00072     float i_max_dps = 30.0f;
00073     float rotate_enter_deg = 40.0f;
00074     float rotate_exit_deg = 25.0f;
00079     float decel_radius_cm = 30.0f;
00080     float arrive_radius_cm = 1.5f;
00081     float creep_mm_s = 25.0f;
00082     float brick_max_mm_s = 450.0f;
00083     float brick_max_dps = 250.0f;
00084     bool flip_turn = false;
00085     // Learned actuator-calibration corrections (auto-tuned from camera-measured
00086     // commanded-vs-actual motion). 1.0 = perfect. <1 commands less (the robot
00087     // over-shoots/over-turns); >1 commands more (it under-shoots).
00088     float drive_scale = 1.0f;
00089     float turn_scale = 1.0f;
00090 };
00091
00095 struct drive_pct {
00096     int speed_pct = 0;
00097     int turn_pct = 0;
00098 };
00099
00112 drive_pct to_percent(const drive_cmd& cmd, const follower_params& p);
00113
00122 class path_follower {
00123 public:
00131     enum class mode { idle, rotate_in_place, pursue, arrive_creep, stopped };
00132
00138     explicit path_follower(follower_params params = {}) : params_(params) {}
00139
00155     drive_cmd step(const pose2d& pose, const collision::path_update& path, float dt_s);
00156
00162     void reset();
00163
00165     void set_params(const follower_params& p) { params_ = p; }
00167     const follower_params& params() const noexcept { return params_; }
00168
00169     // Introspection for the overlay + sim assertions.
00171     cv::Point2f lookahead_cm() const noexcept { return lookahead_; }
00173     mode current_mode() const noexcept { return mode_; }
00175     float cross_track_cm() const noexcept { return cross_track_; }
00181     float turn_corr() const noexcept { return turn_corr_; }
00182
00183 private:
00191     float pid(float bearing_rad, float dt_s);
00202     float effective_turn_dps() const;
00203

```

```

00204     follower_params params_;
00205
00206     // heading PID state
00207     float i_term_ = 0.0f;
00208     float prev_err_ = 0.0f;
00209     bool have_prev_ = false;
00210
00211     // progress / hysteresis state
00212     std::size_t seg_hint_ = 0;
00213     std::uint64_t last_version_ = 0;
00214     bool in_rotate_ = false;
00215     float rotate_timer_ms_ = 0.0f;
00216     bool rotate_settling_ = false;
00217     float rotate_remaining_rad_ = 0.0f;
00218     int rotate_passes_ = 0;
00220
00230     float turn_corr_ = 1.0f;
00231     float turn_start_head_ = 0.0f;
00232     float turn_target_rad_ = 0.0f;
00233     bool turn_measured_ = false;
00234     bool turn_dirty_ = false;
00235     bool has_pursued_ = false;
00236
00237     mode mode_ = mode::idle;
00238     cv::Point2f lookahead_{0.0f, 0.0f};
00239     float cross_track_ = 0.0f;
00240 };
00241
00242 } // namespace follower

```

11.93 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ pathfinding/gridmap.cpp File Reference

Implementation of the 2D occupancy grid map used for path planning.

```

#include "gridmap.hpp"
#include "utilities/Logger.hpp"

```

11.93.1 Detailed Description

Implementation of the 2D occupancy grid map used for path planning.

11.94 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ pathfinding/gridmap.hpp File Reference

Defines the 2D occupancy grid map used for path planning.

```

#include <opencv2/opencv.hpp>
#include <vector>

```

Classes

- class [pathfinding::GridMap](#)

2D occupancy grid over the physical play area, used for path planning. Stores one walkable/blocked flag per cell and provides obstacle marking, world<->grid coordinate conversion, and a line-of-sight test consumed by the pathfinder.

11.94.1 Detailed Description

Defines the 2D occupancy grid map used for path planning.

11.95 gridmap.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <opencv2/opencv.hpp>
00009 #include <vector>
00010
00011 namespace pathfinding {
00012
00019 class GridMap {
00020 public:
00027     GridMap(int width, int height, float resolution);
00028
00036     void addObstacle(cv::Point2f world_pos, float radius_mm);
00037
00044     void blockBorder(int thickness);
00045
00049     void clear();
00050
00057     [[nodiscard]] bool isInside(int x, int y) const;
00058
00065     [[nodiscard]] bool isWalkable(int x, int y) const;
00066
00077     [[nodiscard]] bool hasLineOfSight(cv::Point2i p1, cv::Point2i p2) const;
00078
00079     // Coordinate conversions
00080
00086     [[nodiscard]] cv::Point2i worldToGrid(cv::Point2f world_pos) const;
00087
00093     [[nodiscard]] cv::Point2f gridToWorld(cv::Point2i grid_pos) const;
00094
00096     int getWidth() const noexcept { return width_; }
00098     int getHeight() const noexcept { return height_; }
00100     float getResolution() const noexcept { return resolution_; }
00101
00102 private:
00103     int width_ = 0;
00104     int height_ = 0;
00105     float resolution_ = 0.0f;
00106     std::vector<std::vector<bool>> grid_;
00107 };
00108
00109 } // namespace pathfinding

```

11.96 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ pathfinding/path_planner.cpp File Reference

Implementation of the A* path planner declared in [path_planner.hpp](#).

```

#include "path_planner.hpp"
#include <queue>
#include <unordered_map>
#include <cmath>
#include <algorithm>
#include "utilities/Logger.hpp"

```

Classes

- struct [pathfinding::NodeCompare](#)

Min-heap comparator for the A open list, ordering [Node](#) pointers so that `std::priority_queue` pops the node with the lowest f-cost first.*

11.96.1 Detailed Description

Implementation of the A* path planner declared in [path_planner.hpp](#).

11.97 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ pathfinding/path_planner.hpp File Reference

Declares the A* grid path planner used to route between two world-space points on a GridMap.

```
#include <opencv2/opencv.hpp>
#include <vector>
#include <memory>
#include "gridmap.hpp"
```

Classes

- struct [pathfinding::Node](#)

A single cell visited during A search, linked to its predecessor for path retracing. Coordinates (x, y) are grid cell indices (not world/mm coordinates); g/h/f follow the standard A* convention where g is the accumulated cost from the start, h is the heuristic estimate to the goal, and f = g + h is the priority used to order the open list.*

- class [pathfinding::PathPlanner](#)

Grid-based A path planner that finds a walkable route between two world-space points. Queries a [GridMap](#) for walkability and coordinate conversion, then post-processes the raw grid-aligned path with line-of-sight smoothing before returning it.*

11.97.1 Detailed Description

Declares the A* grid path planner used to route between two world-space points on a GridMap.

11.98 path_planner.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <opencv2/opencv.hpp>
00009 #include <vector>
00010 #include <memory>
00011 #include "gridmap.hpp"
00012
00013 namespace pathfinding {
00014
00021 struct Node {
```

```

00022     int x, y;
00023     float g, h, f;
00024     Node* parent = nullptr;
00025
00029     Node(int x, int y) : x(x), y(y), g(0), h(0), f(0) {}
00030
00035     bool operator>(const Node& other) const {
00036         return f > other.f;
00037     }
00038 };
00039
00045 class PathPlanner {
00046 public:
00048     PathPlanner() = default;
00049
00060     std::vector<cv::Point2f> plan(const GridMap& map, cv::Point2f start, cv::Point2f goal);
00061
00062 private:
00070     std::vector<cv::Point2f> smoothPath(const GridMap& map, const std::vector<cv::Point2f>& path);
00071
00075     float heuristic(int x1, int y1, int x2, int y2);
00076 };
00077
00078 } // namespace pathfinding

```

11.99 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/↵ boardEffects.cpp File Reference

Implementation of tile-effect application (coin changes, shop, hazards) for the board.

```

#include "boardEffects.hpp"
#include "player.hpp"
#include "items/shop.hpp"

```

Functions

- void `applyTileEffect` (const `Node` &node, `Player` &player, `Shop` &shop)

Applies every effect attached to a board node to the given player.

11.99.1 Detailed Description

Implementation of tile-effect application (coin changes, shop, hazards) for the board.

11.99.2 Function Documentation

11.99.2.1 `applyTileEffect()`

```

void applyTileEffect (
    const Node & node,
    Player & player,
    Shop & shop)

```

Applies every effect attached to a board node to the given player.

Parameters

<i>node</i>	The board node (tile) the player landed on.
<i>player</i>	The player who lands on the node and receives the effects.
<i>shop</i>	The shop instance to open when a Shop tile is landed on.

11.100 [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/](#) **boardEffects.hpp File Reference**

Declares the function that applies a board tile's effects to a player.

```
#include "map/board.hpp"
#include "utilities/Logger.hpp"
```

Functions

- void [applyTileEffect](#) (const [Node](#) &node, [Player](#) &player, [Shop](#) &shop)
Applies every effect attached to a board node to the given player.

11.100.1 Detailed Description

Declares the function that applies a board tile's effects to a player.

11.100.2 Function Documentation

11.100.2.1 [applyTileEffect\(\)](#)

```
void applyTileEffect (
    const Node & node,
    Player & player,
    Shop & shop)
```

Applies every effect attached to a board node to the given player.

Parameters

<i>node</i>	The board node (tile) the player landed on.
<i>player</i>	The player who lands on the node and receives the effects.
<i>shop</i>	The shop instance to open when a Shop tile is landed on.

11.101 **boardEffects.hpp**

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "map/board.hpp"
00009 #include "utilities/Logger.hpp"
00010
00011 class Player;
00012 class Shop;
00013
00020 void applyTileEffect (const Node& node, Player& player, Shop& shop);
```


11.102 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.cpp File Reference

Implementation of the [Dice](#) class's random-roll generation.

```
#include "dice.hpp"
#include <random>
```

11.102.1 Detailed Description

Implementation of the [Dice](#) class's random-roll generation.

11.103 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.hpp File Reference

Declares the [Dice](#) class used to roll random dice values for a player's turn.

```
#include "utilities/Logger.hpp"
```

Classes

- class [Dice](#)

Represents a die that produces a random roll in [1, 10]. Instances are typically constructed as throwaway objects purely to invoke [throwDice\(\)](#); the "dice" parameter accepted by the member functions below is currently unused.

11.103.1 Detailed Description

Declares the [Dice](#) class used to roll random dice values for a player's turn.

11.104 dice.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007 #include "utilities/Logger.hpp"
00008
00014 class Dice {
00015     public:
00022     int throwDice();
00023 };
```

11.105 [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/↔](#) inventory.cpp File Reference

Implementation of the [Inventory](#) class for managing a player's held items.

```
#include "inventory.hpp"
```

11.105.1 Detailed Description

Implementation of the [Inventory](#) class for managing a player's held items.

11.106 [/Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/↔](#) inventory.hpp File Reference

Defines the [Inventory](#) class, a bounded collection of items held by a player.

```
#include <vector>
#include "items/item.hpp"
#include "utilities/Logger.hpp"
```

Classes

- class [Inventory](#)
Holds a bounded collection of items owned by a player. Enforces a maximum capacity (MAX_INVENTORY_SIZE) and provides index-based lookup, addition, and removal of items in insertion order.

Variables

- constexpr int **MAX_INVENTORY_SIZE** = 3

11.106.1 Detailed Description

Defines the [Inventory](#) class, a bounded collection of items held by a player.

11.107 inventory.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007 #include <vector>
00008 #include "items/item.hpp"
00009 #include "utilities/Logger.hpp"
00010
00011
00012 constexpr int MAX_INVENTORY_SIZE = 3;
00013
00019 class Inventory
00020 {
00021 private:
00022     std::vector<Item> items;
00023
00024 public:
00030     bool addItem(const Item& item);
00031
00037     bool removeItem(int index);
00038
00044     bool hasItem(ItemType type) const;
00045
00050     const std::vector<Item>& getItems() const;
00051
00056     int getSize() const;
00057
00062     bool isFull() const;
00063
00069     Item* getItem(int index);
00070 };

```

11.108 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/↵ player.cpp File Reference

Implementation of the [Player](#) and [Effect](#) classes: turn state, board position, status effects, and coin/star/inventory bookkeeping.

```

#include "player.hpp"
#include "map/board.hpp"
#include "items/item.hpp"
#include <algorithm>
#include <collision_avoidance/collision_planner.hpp>
#include "utilities/Logger.hpp"

```

Variables

- [collision::collision_planner](#) **collision_planner**

Process-wide collision planner instance used to route a player's robot when its board node changes (see [Player::setCurrentNode](#)).

11.108.1 Detailed Description

Implementation of the [Player](#) and [Effect](#) classes: turn state, board position, status effects, and coin/star/inventory bookkeeping.

11.109 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/↵ player.hpp File Reference

Defines the [Player](#) class (turn state, board position, coins/stars, inventory, and status effects) and the [Effect](#) base class used to model timed buffs/debuffs applied to a player.

```
#include "map/board.hpp"
#include <string>
#include <vector>
#include <memory>
#include "dice.hpp"
#include "inventory.hpp"
#include "items/item.hpp"
#include "utilities/Logger.hpp"
```

Classes

- struct [Vec3< T >](#)
Generic 3-component vector; used here to store a player's world-space (x, y, z) position.
- struct [PlayerStats](#)
Aggregate per-player session statistics (turns played, items used, effects received, fields moved, coins earned).
- class [Effect](#)
Base class for timed status effects attached to a [Player](#). Concrete subclasses implement [applyStartOfTurn\(\)](#)/[applyEndOfTurn\(\)](#) to modify player state; the effect expires once its duration counts down to zero (see [tick\(\)](#)/[isExpired\(\)](#)).
- class [Player](#)
Represents one game participant: identity, board position, coins/stars, active status effects, inventory, and per-turn state. A player is paired with a physical robot (robotID); moving the player's logical node (setCurrentNode) also requests the collision planner to route that robot to the new node.

Enumerations

- enum class [EffectType](#) { **Positive** , **Negative** }
Classifies an [Effect](#) as beneficial or detrimental.

11.109.1 Detailed Description

Defines the [Player](#) class (turn state, board position, coins/stars, inventory, and status effects) and the [Effect](#) base class used to model timed buffs/debuffs applied to a player.

11.110 player.hpp

[Go to the documentation of this file.](#)

```

00001
00007
00008 #pragma once
00009
00010 #include "map/board.hpp"
00011
00012 #include <string>
00013 #include <vector>
00014 #include <memory>
00015 #include "dice.hpp"
00016
00017 #include "inventory.hpp"
00018 #include "items/item.hpp"
00019 #include "utilities/Logger.hpp"
00020
00021
00022
00023 // Forward declarations
00024 class Board;
00025 class Player;
00026
00031 template<typename T>
00032 struct Vec3
00033 {
00034     T x;
00035     T y;
00036     T z;
00037 };
00038
00039 // Effekte
00043 enum class EffectType {
00044     Positive,
00045     Negative
00046 };
00047
00048
00053 struct PlayerStats {
00054     int totalFieldsMoved = 0;
00055     int turnsPlayed = 0;
00056     int itemsUsed = 0;
00057     int effectsReceived = 0;
00058     int totalCoinsEarned = 0;
00059 };
00060
00061
00062 // ===== EFFECT =====
00063
00070 class Effect {
00071 protected:
00072     int duration = 0;
00073     EffectType type = EffectType::Positive;
00074
00075 public:
00081     Effect(int duration, EffectType type);
00082
00083     virtual ~Effect() = default;
00084
00088     virtual std::string getName() const = 0;
00089
00094     virtual void applyStartOfTurn(Player& player) = 0;
00095
00100     virtual void applyEndOfTurn(Player& player) = 0;
00101
00105     bool isExpired() const;
00106
00110     void tick();
00111
00115     EffectType getType() const;
00116 };
00117
00118 // ===== PLAYER =====
00119
00127 class Player {
00128 protected:
00129     int playerId = 0;
00130     std::string name;
00131
00132     int stars = 0;
00133     int coins = 0;
00134
00135     int currentNodeId = 0;
00136

```

```

00137     bool isCurrentPlayer = false;
00138
00139     PlayerStats stats;
00140     Inventory inventory;
00141
00142     std::vector<std::shared_ptr<Effect>> effects;
00143     //std::vector<Effect*> effects;
00144
00145     bool doubleCoinsActive = false;
00146     bool invertedControlsActive = false;
00147     int nextDiceMultiplier = 1;
00148     int diceBonusSteps = 0;
00149     int robotID = 0;
00150
00151
00152     Vec3<float> worldPosition{};
00153
00154
00155 public:
00163     Player(int id, int robotID, const std::string& name, int startNode);
00164
00165     virtual ~Player() = default;
00166
00172     bool canAfford(int price) const noexcept;
00173
00177     [[nodiscard]] int getRobotId() const noexcept;
00178
00179
00180
00181     //virtual void applyBoardEffect() = 0;    //eig nicht
00182
00183     // Turn
00190     void startTurn();
00191
00195     void endTurn();
00196
00201     bool getCurrentPlayer() const noexcept;
00202
00207     [[nodiscard]] int getCurrentPlayerID() const noexcept; //Turn logic
00208
00213     void applyEndOfTurnEffects();
00214
00220     void checkNegativeEffects();
00221
00225     void removeExpiredEffects();
00226
00227
00228     // Position
00232     [[nodiscard]] int getCurrentNode() const noexcept;
00233
00239     const Vec3<float> &getWorldPosition() const;
00240
00247     void setWorldPosition(const Vec3<float>& pos);
00248
00256     void setCurrentNode(int nodeId, int robotId);
00257
00258     // Identity
00262     [[nodiscard]] const std::string& getName() const noexcept;
00263
00268     [[nodiscard]] int getId() const noexcept;
00269
00270     // Effects
00275     void addEffect(std::shared_ptr<Effect> effect);
00276
00277     // Coins / Stars
00283     void addCoins(int amount);
00284
00288     void addStar();
00289
00293     [[nodiscard]] int getStars() const noexcept;
00294
00298     [[nodiscard]] int getCoins() const noexcept;
00299
00300     //Inventory
00304     [[nodiscard]] Inventory& getInventory() noexcept;
00305
00309     [[nodiscard]] const Inventory& getInventory() const noexcept;
00310
00311     // Items
00319     void addItem(const Item& item);
00320
00321
00322     // Dice
00327     void setDiceMultiplier(int mult);
00328
00332     [[nodiscard]] int getDiceMultiplier() const noexcept;
00333

```

```

00337     void resetDiceMultiplier();
00338
00339     // Double Coins
00343     void activateDoubleCoins();
00344
00348     [[nodiscard]] bool hasDoubleCoins() const noexcept;
00349
00353     void clearDoubleCoins();
00354
00355     // Inverted Controls
00359     void activateInvertedControls();
00360
00364     [[nodiscard]] bool hasInvertedControls() const noexcept;
00365
00369     void clearInvertedControls();
00370
00371     // Swap Position
00379     void swapPosition(Player& other, const Board& board);
00380
00381     // Steal Coins
00387     void stealDiceSteps(Player& other, int amount);
00388
00389     // Stats
00393     [[nodiscard]] const PlayerStats& getStats() const noexcept;
00394
00398     void addMovedField();
00399
00403     void addUsedItem();
00404
00405 };

```

11.111 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.cpp File Reference ↵

Implementation of the binary wire codec for the desktop <-> EV3 contract.

```
#include "protocol/codec.hpp"
```

Functions

- QByteArray `mind::proto::encode_drive` (std::uint8_t nn, std::int8_t speed_pct, std::int8_t turn_pct)
Encodes a continuous tank-drive UDP command.
- QByteArray `mind::proto::encode_wheel_drive` (std::uint8_t nn, std::int8_t left_wheel, std::int8_t right_wheel)
Encodes a continuous independent-wheel-drive UDP command.
- QByteArray `mind::proto::encode_drive_for` (std::uint16_t id, std::int32_t distance_mm, std::int16_t speed_mm_s) ↵
Encodes a single-shot straight-line move command.
- QByteArray `mind::proto::encode_turn` (std::uint16_t id, std::int16_t angle_deg, std::int16_t rate_dps)
Encodes a single-shot in-place rotation command.
- QByteArray `mind::proto::encode_stop` (std::uint16_t id, bool brake)
Encodes an immediate-stop command.
- QByteArray `mind::proto::encode_beep` (std::uint16_t id, std::uint16_t duration_ms, std::uint16_t frequency_hz) ↵
Encodes a short-tone command.
- QByteArray `mind::proto::encode_ping` (std::uint16_t id)
Encodes a latency/liveness probe command.
- std::optional< status_type > `mind::proto::peek_status_type` (const QByteArray &payload)
Reads just the leading type byte of a status payload without fully decoding it.
- std::optional< ack_msg > `mind::proto::decode_ack` (const QByteArray &payload)
Decodes an ack status payload.
- std::optional< heartbeat_msg > `mind::proto::decode_heartbeat` (const QByteArray &payload)
Decodes a heartbeat status payload.

11.111.1 Detailed Description

Implementation of the binary wire codec for the desktop <-> EV3 contract.

11.111.2 Function Documentation

11.111.2.1 `decode_ack()`

```
std::optional< ack_msg > mind::proto::decode_ack (  
    const QByteArray & payload) [nodiscard]
```

Decodes an `ack` status payload.

Parameters

<i>payload</i>	Raw payload; must be exactly 7 bytes: [0x10][cmd_id:u16][state:u8][battery_mv:u16][err_code:u8].
----------------	---

Returns

The decoded `ack_msg`, or `nullopt` if the length, type byte, state, or `err_code` is invalid.

11.111.2.2 `decode_heartbeat()`

```
std::optional< heartbeat_msg > mind::proto::decode_heartbeat (  
    const QByteArray & payload) [nodiscard]
```

Decodes a `heartbeat` status payload.

Parameters

<i>payload</i>	Raw payload; must be exactly 5 bytes: [0x11][nn:u8][state:u8][battery_mv:u16].
----------------	---

Returns

The decoded `heartbeat_msg`, or `nullopt` if the length, type byte, or state is invalid.

11.111.2.3 encode_beep()

```
QByteArray mind::proto::encode_beep (
    std::uint16_t id,
    std::uint16_t duration_ms,
    std::uint16_t frequency_hz)
```

Encodes a short-tone command.

Parameters

<i>id</i>	Command id; echoed back as cmd_id in the completion ack.
<i>duration_ms</i>	Tone duration, in milliseconds.
<i>frequency_hz</i>	Tone frequency, in hertz.

Returns

Raw little-endian payload ready to publish over MQTT.

11.111.2.4 encode_drive()

```
QByteArray mind::proto::encode_drive (
    std::uint8_t nn,
    std::int8_t speed_pct,
    std::int8_t turn_pct)
```

Encodes a continuous tank-drive UDP command.

Parameters

<i>nn</i>	Robot index (the NN of ev3_NN).
<i>speed_pct</i>	Forward/back speed as a percent of WHEEL_MAX_MM_S, -100..100.
<i>turn_pct</i>	Turn rate as a percent of DRIVE_MAX_TURN_DPS, -100..100.

Returns

Raw little-endian payload ready to send as a UDP datagram.

11.111.2.5 encode_drive_for()

```
QByteArray mind::proto::encode_drive_for (
    std::uint16_t id,
    std::int32_t distance_mm,
    std::int16_t speed_mm_s)
```

Encodes a single-shot straight-line move command.

Parameters

<i>id</i>	Command id; echoed back as cmd_id in the completion ack.
<i>distance_mm</i>	Distance to travel, in millimeters (real units, not a calibration percent).
<i>speed_mm↔_s</i>	Target speed, in millimeters per second.

Returns

Raw little-endian payload ready to publish over MQTT.

11.111.2.6 encode_ping()

```
QByteArray mind::proto::encode_ping (
    std::uint16_t id)
```

Encodes a latency/liveness probe command.

Parameters

<i>id</i>	Command id; echoed back as cmd_id in the reply ack.
-----------	---

Returns

Raw little-endian payload ready to publish over MQTT.

11.111.2.7 encode_stop()

```
QByteArray mind::proto::encode_stop (
    std::uint16_t id,
    bool brake)
```

Encodes an immediate-stop command.

Parameters

<i>id</i>	Command id; echoed back as cmd_id in the completion ack.
<i>brake</i>	If true, shorts the motor coils for a fast stiff stop; if false, coasts to a halt.

Returns

Raw little-endian payload ready to publish over MQTT.

11.111.2.8 encode_turn()

```
QByteArray mind::proto::encode_turn (
    std::uint16_t id,
    std::int16_t angle_deg,
    std::int16_t rate_dps)
```

Encodes a single-shot in-place rotation command.

Parameters

<i>id</i>	Command id; echoed back as <i>cmd_id</i> in the completion ack.
<i>angle_deg</i>	Angle to rotate, in degrees (real units, not a calibration percent).
<i>rate_dps</i>	Rotation rate, in degrees per second.

Returns

Raw little-endian payload ready to publish over MQTT.

11.111.2.9 encode_wheel_drive()

```
QByteArray mind::proto::encode_wheel_drive (
    std::uint8_t nn,
    std::int8_t left_wheel,
    std::int8_t right_wheel)
```

Encodes a continuous independent-wheel-drive UDP command.

Parameters

<i>nn</i>	Robot index (the NN of ev3_NN).
<i>left_wheel</i>	Left wheel speed as a percent of WHEEL_MAX_MM_S, -100..100.
<i>right_wheel</i>	Right wheel speed as a percent of WHEEL_MAX_MM_S, -100..100.

Returns

Raw little-endian payload ready to send as a UDP datagram.

11.111.2.10 peek_status_type()

```
std::optional< status_type > mind::proto::peek_status_type (
    const QByteArray & payload) [nodiscard]
```

Reads just the leading type byte of a status payload without fully decoding it.

Reads just the leading type byte of a status payload without fully decoding it. Used to decide which of `decode_ack` / `decode_heartbeat` to call next.

Parameters

<i>payload</i>	Raw status payload as received from the wire.
----------------	---

Returns

The `status_type` found in byte 0, or `nullopt` if the payload is empty or its type byte does not match a known `status_type`.

11.112 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codec.hpp File Reference

Binary wire codec for the desktop <-> EV3 contract (shared/protocol.md, Goal C).

```
#include <cstdint>
#include <optional>
#include <QByteArray>
```

Classes

- struct `mind::proto::ack_msg`
Decoded *ack* status payload (EV3 -> desktop, MQTT). Echoes the command that triggered it (or *no_cmd_id*) plus the robot's current state, battery level, and error detail.
- struct `mind::proto::heartbeat_msg`
Decoded *heartbeat* status payload (EV3 -> desktop, UDP). Published periodically regardless of state; the datagram's source address doubles as the desktop's discovery mechanism for the robot.

Enumerations

- enum class `mind::proto::opcode` : `std::uint8_t` {
`drive` = 0x01 , `wheel_drive` = 0x02 , `drive_for` = 0x03 , `turn` = 0x04 ,
`stop` = 0x05 , `beep` = 0x06 , `ping` = 0x07 }
Command/status discriminator written as the leading type byte of a wire payload. *drive* and *wheel_drive* travel over UDP (robot index *nn* in byte 0, this opcode in byte 1 of the datagram); the rest travel over MQTT (this opcode in byte 0, the robot id is carried in the topic instead).
- enum class `mind::proto::status_type` : `std::uint8_t` { `ack` = 0x10 , `heartbeat` = 0x11 }
Status payload discriminator (EV3 -> desktop), read from byte 0 of the payload.
- enum class `mind::proto::robot_state` : `std::uint8_t` { `idle` = 0 , `driving` = 1 , `moving` = 2 , `error` = 3 }
EV3 robot state as reported in *ack* and *heartbeat* payloads.
- enum class `mind::proto::err_code` : `std::uint8_t` {
`none` = 0 , `unknown_opcode` = 1 , `bad_length` = 2 , `bad_param` = 3 ,
`motor_fault` = 4 , `internal` = 5 }
Error detail carried in an *ack*. Meaningful only when the accompanying *robot_state* is *robot_state::error*; otherwise treat as *err_code::none*.

Functions

- QByteArray `mind::proto::encode_drive` (std::uint8_t nn, std::int8_t speed_pct, std::int8_t turn_pct)
Encodes a continuous tank-drive UDP command.
- QByteArray `mind::proto::encode_wheel_drive` (std::uint8_t nn, std::int8_t left_wheel, std::int8_t right_wheel)
Encodes a continuous independent-wheel-drive UDP command.
- QByteArray `mind::proto::encode_drive_for` (std::uint16_t id, std::int32_t distance_mm, std::int16_t speed_mm_s)
Encodes a single-shot straight-line move command.
- QByteArray `mind::proto::encode_turn` (std::uint16_t id, std::int16_t angle_deg, std::int16_t rate_dps)
Encodes a single-shot in-place rotation command.
- QByteArray `mind::proto::encode_stop` (std::uint16_t id, bool brake)
Encodes an immediate-stop command.
- QByteArray `mind::proto::encode_beep` (std::uint16_t id, std::uint16_t duration_ms, std::uint16_t frequency_hz)
Encodes a short-tone command.
- QByteArray `mind::proto::encode_ping` (std::uint16_t id)
Encodes a latency/liveness probe command.
- std::optional< status_type > `mind::proto::peek_status_type` (const QByteArray &payload)
Reads just the leading type byte of a status payload without fully decoding it.
- std::optional< ack_msg > `mind::proto::decode_ack` (const QByteArray &payload)
Decodes an ack status payload.
- std::optional< heartbeat_msg > `mind::proto::decode_heartbeat` (const QByteArray &payload)
Decodes a heartbeat status payload.

Variables

- constexpr std::uint16_t `mind::proto::no_cmd_id` = 0xFFFF
Sentinel cmd_id used by acks that are not tied to a specific command (e.g. a watchdog-induced stop of drive / wheel_drive).

11.112.1 Detailed Description

Binary wire codec for the desktop <-> EV3 contract (shared/protocol.md, Goal C).

All multi-byte integers are little-endian. Encoders build the command payloads (desktop -> EV3); decoders parse the status payloads (EV3 -> desktop) and return nullopt on malformed input.

11.112.2 Enumeration Type Documentation

11.112.2.1 opcode

```
enum class mind::proto::opcode : std::uint8_t [strong]
```

Command/status discriminator written as the leading type byte of a wire payload. `drive` and `wheel_drive` travel over UDP (robot index nn in byte 0, this opcode in byte 1 of the datagram); the rest travel over MQTT (this opcode in byte 0, the robot id is carried in the topic instead).

Enumerator

drive	Continuous tank drive (speed_pct, turn_pct). UDP, republished at PUBLISH_MS while active.
wheel_drive	Continuous independent-wheel drive (left/right percent). UDP, republished at PUBLISH_MS while active.
drive_for	Single-shot straight-line move by distance/speed (real units). MQTT.
turn	Single-shot in-place rotation by angle/rate (real units). MQTT.
stop	Immediate halt; interrupts any in-flight drive_for/turn poll loop. MQTT.
beep	Short tone. MQTT.
ping	Latency/liveness probe; EV3 replies with an ack echoing the same id. MQTT.

11.112.2.2 robot_state

```
enum class mind::proto::robot_state : std::uint8_t [strong]
```

EV3 robot state as reported in ack and heartbeat payloads.

Enumerator

idle	No command in flight.
driving	Executing a continuous drive / wheel_drive command.
moving	Executing a single-shot drive_for / turn command.
error	Handler failure; see the accompanying err_code .

11.112.2.3 status_type

```
enum class mind::proto::status_type : std::uint8_t [strong]
```

Status payload discriminator (EV3 -> desktop), read from byte 0 of the payload.

Enumerator

ack	Command acknowledgement. MQTT.
heartbeat	Periodic liveness/battery report. UDP.

11.112.3 Function Documentation**11.112.3.1 decode_ack()**

```
std::optional< ack_msg > mind::proto::decode_ack (
    const QByteArray & payload) [nodiscard]
```

Decodes an ack status payload.

Parameters

<i>payload</i>	Raw payload; must be exactly 7 bytes: [0x10][cmd_id:u16][state:u8][battery_mv:u16][err_code:u8].
----------------	---

Returns

The decoded ack_msg, or nullptr if the length, type byte, state, or err_code is invalid.

11.112.3.2 decode_heartbeat()

```
std::optional< heartbeat_msg > mind::proto::decode_heartbeat (
    const QByteArray & payload) [nodiscard]
```

Decodes a heartbeat status payload.

Parameters

<i>payload</i>	Raw payload; must be exactly 5 bytes: [0x11][nn:u8][state:u8][battery_mv:u16].
----------------	---

Returns

The decoded heartbeat_msg, or nullptr if the length, type byte, or state is invalid.

11.112.3.3 encode_beep()

```
QByteArray mind::proto::encode_beep (
    std::uint16_t id,
    std::uint16_t duration_ms,
    std::uint16_t frequency_hz)
```

Encodes a short-tone command.

Parameters

<i>id</i>	Command id; echoed back as cmd_id in the completion ack.
<i>duration_ms</i>	Tone duration, in milliseconds.
<i>frequency_hz</i>	Tone frequency, in hertz.

Returns

Raw little-endian payload ready to publish over MQTT.

11.112.3.4 encode_drive()

```
QByteArray mind::proto::encode_drive (
    std::uint8_t nn,
    std::int8_t speed_pct,
    std::int8_t turn_pct)
```

Encodes a continuous tank-drive UDP command.

Parameters

<i>nn</i>	Robot index (the NN of ev3_NN).
<i>speed_pct</i>	Forward/back speed as a percent of WHEEL_MAX_MM_S, -100..100.
<i>turn_pct</i>	Turn rate as a percent of DRIVE_MAX_TURN_DPS, -100..100.

Returns

Raw little-endian payload ready to send as a UDP datagram.

11.112.3.5 encode_drive_for()

```
QByteArray mind::proto::encode_drive_for (
    std::uint16_t id,
    std::int32_t distance_mm,
    std::int16_t speed_mm_s)
```

Encodes a single-shot straight-line move command.

Parameters

<i>id</i>	Command id; echoed back as cmd_id in the completion ack.
<i>distance_mm</i>	Distance to travel, in millimeters (real units, not a calibration percent).
<i>speed_mm_s</i>	Target speed, in millimeters per second.

Returns

Raw little-endian payload ready to publish over MQTT.

11.112.3.6 encode_ping()

```
QByteArray mind::proto::encode_ping (
    std::uint16_t id)
```

Encodes a latency/liveness probe command.

Parameters

<i>id</i>	Command id; echoed back as <code>cmd_id</code> in the reply ack.
-----------	--

Returns

Raw little-endian payload ready to publish over MQTT.

11.112.3.7 encode_stop()

```
QByteArray mind::proto::encode_stop (
    std::uint16_t id,
    bool brake)
```

Encodes an immediate-stop command.

Parameters

<i>id</i>	Command id; echoed back as <code>cmd_id</code> in the completion ack.
<i>brake</i>	If true, shorts the motor coils for a fast stiff stop; if false, coasts to a halt.

Returns

Raw little-endian payload ready to publish over MQTT.

11.112.3.8 encode_turn()

```
QByteArray mind::proto::encode_turn (
    std::uint16_t id,
    std::int16_t angle_deg,
    std::int16_t rate_dps)
```

Encodes a single-shot in-place rotation command.

Parameters

<i>id</i>	Command id; echoed back as <code>cmd_id</code> in the completion ack.
<i>angle_deg</i>	Angle to rotate, in degrees (real units, not a calibration percent).
<i>rate_dps</i>	Rotation rate, in degrees per second.

Returns

Raw little-endian payload ready to publish over MQTT.

11.112.3.9 encode_wheel_drive()

```
QByteArray mind::proto::encode_wheel_drive (
    std::uint8_t nn,
    std::int8_t left_wheel,
    std::int8_t right_wheel)
```

Encodes a continuous independent-wheel-drive UDP command.

Parameters

<i>nn</i>	Robot index (the NN of ev3_NN).
<i>left_wheel</i>	Left wheel speed as a percent of WHEEL_MAX_MM_S, -100..100.
<i>right_wheel</i>	Right wheel speed as a percent of WHEEL_MAX_MM_S, -100..100.

Returns

Raw little-endian payload ready to send as a UDP datagram.

11.112.3.10 peek_status_type()

```
std::optional< status_type > mind::proto::peek_status_type (
    const QByteArray & payload) [nodiscard]
```

Reads just the leading type byte of a status payload without fully decoding it.

Reads just the leading type byte of a status payload without fully decoding it. Used to decide which of decode_ack / decode_heartbeat to call next.

Parameters

<i>payload</i>	Raw status payload as received from the wire.
----------------	---

Returns

The status_type found in byte 0, or nullopt if the payload is empty or its type byte does not match a known status_type.

11.113 codec.hpp

[Go to the documentation of this file.](#)

```
00001
00009
00010 #pragma once
00011
00012 #include <stdint>
00013 #include <optional>
00014
00015 #include <QByteArray>
00016
00017 namespace mind::proto {
00018
00025 enum class opcode : std::uint8_t {
```

```

00026     drive      = 0x01,
00027     wheel_drive = 0x02,
00028     drive_for   = 0x03,
00029     turn        = 0x04,
00030     stop        = 0x05,
00031     beep        = 0x06,
00032     ping        = 0x07,
00033 };
00034
00038 enum class status_type : std::uint8_t {
00039     ack      = 0x10,
00040     heartbeat = 0x11,
00041 };
00042
00046 enum class robot_state : std::uint8_t {
00047     idle    = 0,
00048     driving = 1,
00049     moving  = 2,
00050     error   = 3,
00051 };
00052
00058 enum class err_code : std::uint8_t {
00059     none          = 0,
00060     unknown_opcode = 1,
00061     bad_length    = 2,
00062     bad_param     = 3,
00063     motor_fault   = 4,
00064     internal      = 5,
00065 };
00066
00071 constexpr std::uint16_t no_cmd_id = 0xFFFF;
00072
00078 struct ack_msg {
00079     std::uint16_t cmd_id;
00080     robot_state state;
00081     std::uint16_t battery_mv;
00082     err_code err;
00083 };
00084
00090 struct heartbeat_msg {
00091     std::uint8_t nn;
00092     robot_state state;
00093     std::uint16_t battery_mv;
00094 };
00095
00096 // Command encoders (desktop -> EV3).
00097 // UDP datagrams carry the robot index nn in byte 0, opcode in byte 1.
00098
00106 QByteArray encode_drive(std::uint8_t nn, std::int8_t speed_pct, std::int8_t turn_pct);
00107
00115 QByteArray encode_wheel_drive(std::uint8_t nn, std::int8_t left_wheel, std::int8_t right_wheel);
00116
00117 // MQTT payloads carry the opcode in byte 0 (robot_id is in the topic).
00118
00126 QByteArray encode_drive_for(std::uint16_t id, std::int32_t distance_mm, std::int16_t speed_mm_s);
00127
00135 QByteArray encode_turn(std::uint16_t id, std::int16_t angle_deg, std::int16_t rate_dps);
00136
00143 QByteArray encode_stop(std::uint16_t id, bool brake);
00144
00152 QByteArray encode_beep(std::uint16_t id, std::uint16_t duration_ms, std::uint16_t frequency_hz);
00153
00159 QByteArray encode_ping(std::uint16_t id);
00160
00161 // Status decoders (EV3 -> desktop). Return nullopt on a bad length, an
00162 // unexpected type byte, or an out-of-range enum.
00163
00171 [[nodiscard]] std::optional<status_type> peek_status_type(const QByteArray& payload);
00172
00180 [[nodiscard]] std::optional<ack_msg> decode_ack(const QByteArray& payload);
00181
00189 [[nodiscard]] std::optional<heartbeat_msg> decode_heartbeat(const QByteArray& payload);
00190
00191 } // namespace mind::proto

```

11.114 background_thread.hpp

```

00001 #pragma once
00002 #include <QObject>
00003 #include <QTimer>
00004 #include <QImage>
00005 #include <memory>
00006 #include <vector>

```

```

00007 #include <opencv2/opencv.hpp>
00008 #include "robot_tracker/tracker.hpp"
00009 #include "camera/camera_calibration.hpp"
00010
00011 struct TrackedRobot {
00012     int slot_id;
00013     int marker_id;
00014     double x_m;
00015     double y_m;
00016 };
00017
00018 class ArUcoWorker : public QObject {
00019     Q_OBJECT
00020 public:
00021     explicit ArUcoWorker(QObject* parent = nullptr);
00022     ~ArUcoWorker();
00023
00024 public slots:
00025     void start_tracking();
00026     void stop_tracking();
00027     void calibrate_camera();
00028     void process_frame();
00029
00030 signals:
00031     void frame_ready(const QImage& frame);
00032     void robots_updated(const std::vector<TrackedRobot>& robots);
00033     void status_message(const QString& message);
00034     void anchor_calibrated(double tx, double ty, double tz, double rx, double ry, double rz);
00035
00036 private:
00037     bool pixel_to_floor_xy(const cv::Point2f& pixel_uv, cv::Point2f& out_xy_m, double z_plane_m)
00038     const;
00039     QImage cv_mat_to_qimage_nocopy(const cv::Mat& mat);
00040
00041     std::unique_ptr<marker_tracker::marker_tracker> tracker_;
00042     std::unique_ptr<CameraCalibration> calibrator_;
00043     QTimer* timer_ = nullptr;
00044
00045     cv::Mat camera_matrix_, dist_coeffs_, rvec_, tvec_;
00046     cv::Mat persistent_rgb_frame_; // Memory buffer to prevent runtime allocations
00047     bool has_extrinsics_ = false;
00048 };

```

11.115 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.cpp File Reference

Implementation of [control_window](#): MQTT/UDP setup, robot fleet wiring, and UI layout for the desktop control window.

```

#include "control_window.hpp"
#include <QCoreApplication>
#include <QThread>
#include <QTimer>
#include <QVBoxLayout>
#include <QWidget>
#include "fleet_panel.hpp"
#include "app/latency_indicator.hpp"
#include "test_panel.hpp"
#include "gameplay/network_utils.hpp"
#include "mqtt_client/client.hpp"
#include "robot_controller/robot_controller.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "udp_link/udp_link.hpp"
#include "utilities/Logger.hpp"

```

11.115.1 Detailed Description

Implementation of [control_window](#): MQTT/UDP setup, robot fleet wiring, and UI layout for the desktop control window.

11.116 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.hpp File Reference

Top-level Qt window that wires up MQTT, UDP, and robot fleet discovery for the desktop control UI.

```
#include <memory>
#include <QMainWindow>
```

Classes

- class [control_window](#)

Standalone window for robot control. Owns the MQTT client, the robot_fleet auto-discovery, and a [fleet_panel](#) that shows one row per discovered robot. Connects to the broker on construction (host + port from env, defaults to localhost:1883). [main.cpp](#) just opens this.

11.116.1 Detailed Description

Top-level Qt window that wires up MQTT, UDP, and robot fleet discovery for the desktop control UI.

11.117 control_window.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <memory>
00010
00011 #include <QMainWindow>
00012
00013 class QThread;
00014
00015 namespace mind::mqtt {
00016 class client;
00017 }
00018
00019 namespace mind::net {
00020 class udp_link;
00021 }
00022
00023 namespace mind::fleet {
00024 class robot_fleet;
00025 }
00026
00027 class fleet_panel;
00028 class test_panel;
00029 class latency_indicator;
00030
00031 class control_window : public QMainWindow {
00032     Q_OBJECT
00040
00041 public:
00052     explicit control_window(QWidget* parent = nullptr);
00053
00059     ~control_window() override;
00060
00062     mind::fleet::robot_fleet* getFleet() const { return fleet_.get(); }
00063
00064 private:
00065     std::unique_ptr<mind::mqtt::client> mqtt_;
00066     std::unique_ptr<mind::net::udp_link> udp_;
00067     // Dedicated thread that owns udp_link's QUdpSocket I/O. Isolates
00068     // writeDatagram and readyRead from the Qt main thread so UI load
00069     // cannot delay outbound drive packets or inbound heartbeats.
00070     std::unique_ptr<QThread> udp_thread_;
00071     std::unique_ptr<mind::fleet::robot_fleet> fleet_;
00072     fleet_panel* fleet_panel_ = nullptr;
00073     test_panel* test_panel_ = nullptr;
00074     latency_indicator* latency_indicator_ = nullptr;
00075     bool drive_test_fired_ = false;
00076 };
```

11.118 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ qtpanels/fleet_dashboard.cpp File Reference

Casino-themed Fleet Dashboard: live per-robot status overview.

```
#include "fleet_dashboard.hpp"
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QGridLayout>
#include <QFrame>
#include <QLabel>
#include <QStyle>
#include <QTimer>
#include <QScrollArea>
#include "map/mapwindow.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "robot_controller/robot_controller.hpp"
#include "path_follower/path_follower.hpp"
```

11.118.1 Detailed Description

Casino-themed Fleet Dashboard: live per-robot status overview.

11.119 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ qtpanels/fleet_dashboard.hpp File Reference

Standalone debug window: per-robot fleet overview with live battery, connection, follower state, and calibration data in the casino theme.

```
#include <array>
#include <QFrame>
#include <QLabel>
#include <QTimer>
#include <QWidget>
```

Classes

- class [FleetDashboard](#)

Casino-themed Fleet Dashboard showing all 4 robot slots with live connection, battery, EV3 state, follower mode, target node, cross-track error, and calibration scales. Refreshes at 4 Hz via an internal QTimer.

11.119.1 Detailed Description

Standalone debug window: per-robot fleet overview with live battery, connection, follower state, and calibration data in the casino theme.

11.120 fleet_dashboard.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include <array>
00010 #include <QFrame>
00011 #include <QLabel>
00012 #include <QTimer>
00013 #include <QWidget>
00014
00015 class QVBoxLayout;
00016
00017 class MapWindow;
00018
00019 namespace mind::fleet {
00020 class robot_fleet;
00021 }
00022
00028 class FleetDashboard : public QWidget {
00029     Q_OBJECT
00030
00031 public:
00032     explicit FleetDashboard(MapWindow& mapwindow, QWidget* parent = nullptr);
00033
00034 private slots:
00035     void refresh();
00036
00037 private:
00038     struct RobotCard {
00039         QFrame* frame = nullptr;
00040         QLabel* dot = nullptr;
00041         QLabel* id_label = nullptr;
00042         QLabel* connection_label = nullptr;
00043         QLabel* battery_label = nullptr;
00044         QLabel* battery_bar = nullptr;
00045         QLabel* state_label = nullptr;
00046         QLabel* follower_label = nullptr;
00047         QLabel* target_label = nullptr;
00048         QLabel* cross_track_label = nullptr;
00049         QLabel* turn_corr_label = nullptr;
00050         QLabel* scale_label = nullptr;
00051     };
00052
00053     MapWindow& mapwindow_;
00054     QTimer* timer_ = nullptr;
00055     std::array<RobotCard, 4> cards_;
00056
00057     void buildUi();
00058     QFrame* createCard(int index, QVBoxLayout* parent_layout);
00059     static QString modeToFollowerString(int mode);
00060     static QString stateToColor(const QString& state);
00061     static QString batteryColor(int mv);
00062 };

```

11.121 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.cpp File Reference

Implementation of the fleet panel's row creation and live-update wiring.

```

#include "fleet_panel.hpp"
#include <QHBoxLayout>
#include <QLabel>
#include <QPushButton>
#include <QTimer>
#include <QVBoxLayout>
#include "robot_controller/robot_controller.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "utilities/Logger.hpp"

```

11.121.1 Detailed Description

Implementation of the fleet panel's row creation and live-update wiring.

11.122 `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.hpp` File Reference

Widget that lists all discovered robots with live status and quick-test controls.

```
#include <QString>
#include <QWidget>
```

Classes

- class `fleet_panel`

Vertical stack of one row per discovered robot. Listens to `robot_fleet` for appear events and to each `robot_controller` for state, battery, and connection updates. Rows stay around when a robot goes offline (the state label flips to "offline") - the controller itself stays alive for state continuity if the brick returns.

11.122.1 Detailed Description

Widget that lists all discovered robots with live status and quick-test controls.

11.123 `fleet_panel.hpp`

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <QString>
00009 #include <QWidget>
00010
00011 namespace mind::fleet {
00012 class robot_controller;
00013 class robot_fleet;
00014 }
00015
00016 class QVBoxLayout;
00017
00025 class fleet_panel : public QWidget {
00026     Q_OBJECT
00027
00028 public:
00034     explicit fleet_panel(mind::fleet::robot_fleet* fleet, QWidget* parent = nullptr);
00035
00036 private slots:
00046     void on_robot_appeared(QString robot_id, mind::fleet::robot_controller* controller);
00047
00048 private:
00049     mind::fleet::robot_fleet* fleet_ = nullptr;
00050     QVBoxLayout* layout_ = nullptr;
00051 };
```


11.124 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.cpp File Reference

Implementation of [main_window](#): widget setup, the ArUco tracking loop, camera extrinsic calibration, MQTT/TCP position publishing, and the Splatoon test harness.

```
#include "main_window.hpp"
#include "gameplay/network_utils.hpp"
#include "gameplay/start_game.hpp"
#include <iostream>
#include <vector>
#include <mosquitto.h>
#include <nlohmann/json.hpp>
#include <QCoreApplication>
#include <QImage>
#include <QLabel>
#include <QPixmap>
#include <QPushButton>
#include <QString>
#include <QTimer>
#include <QVBoxLayout>
#include <QWidget>
#include <opencv2/opencv.hpp>
#include "camera/camera_calibration.hpp"
#include "gameplay/game_manager.hpp"
#include "communication/TCP_Server.hpp"
#include "mini_games/splatoon.hpp"
#include "player/player.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "robot_tracker/tracker.hpp"
#include "utilities/Logger.hpp"
```

11.124.1 Detailed Description

Implementation of [main_window](#): widget setup, the ArUco tracking loop, camera extrinsic calibration, MQTT/TCP position publishing, and the Splatoon test harness.

11.125 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.hpp File Reference

Declares [main_window](#), the race-monitor window that runs ArUco marker tracking, camera extrinsic calibration, and MQTT/TCP position publishing, plus a standalone Splatoon mini-game test harness.

```
#include <memory>
#include <thread>
#include <vector>
#include <QMainWindow>
#include <opencv2/opencv.hpp>
#include "../gameplay/game_manager.hpp"
#include "../robot_tracker/tracker.hpp"
```

Classes

- class [main_window](#)

Main race-monitor window: owns the camera marker_tracker, runs the ArUco detection loop on a timer, performs extrinsic calibration via solvePnP, and publishes anchor + robot positions to MQTT and to any connected TCP clients.

11.125.1 Detailed Description

Declares [main_window](#), the race-monitor window that runs ArUco marker tracking, camera extrinsic calibration, and MQTT/TCP position publishing, plus a standalone Splatoon mini-game test harness.

11.126 main_window.hpp

[Go to the documentation of this file.](#)

```
00001
00002
00003 #pragma once
00004
00005 #include <memory>
00006 #include <thread>
00007 #include <vector>
00008
00009 #include <QMainWindow>
00010 #include <opencv2/opencv.hpp>
00011
00012 #include "../gameplay/game_manager.hpp"
00013 #include "../robot_tracker/tracker.hpp"
00014
00015 class Server;
00016
00017 struct mosquito;
00018 class QWidget;
00019 class QVBoxLayout;
00020 class QLabel;
00021 class QPushButton;
00022 class QTimer;
00023 class QImage;
00024 class CameraCalibration;
00025
00026 namespace mind::fleet {
00027 class robot_fleet;
00028 }
00029
00030 class main_window : public QMainWindow {
00031     Q_OBJECT
00032
00033 public:
00034     explicit main_window(Server& server, QWidget* parent = nullptr);
00035     virtual ~main_window() override;
00036     void set_fleet(mind::fleet::robot_fleet* fleet);
00037
00038 signals:
00039     void marker_positions_updated(const std::vector<marker_tracker::marker_position>& markers);
00040     void tracking_state_changed(bool active);
00041     void frame_updated(const QImage& img);
00042
00043 private slots:
00044     void toggle_tracking();
00045     void start_tracking();
00046     void stop_tracking();
00047     void calibrate_camera();
00048     void update_tracking();
00049     void start_splatoon();
00050 }
```

```

00133 private:
00145     bool pixel_to_floor_xy(const cv::Point2f& pixel_uv, cv::Point2f& out_xy_m,
00146                          double z_plane_m = 0.0) const;
00147
00155     QImage cv_mat_to_qimage_internal(const cv::Mat& mat);
00156
00157     mosquito* mosq = nullptr;
00158     QWidget* central_widget_ = nullptr;
00159     QVBoxLayout* layout_ = nullptr;
00160     QLabel* status_label_ = nullptr;
00161     QLabel* video_screen_ = nullptr;
00162     QPushButton* start_button_ = nullptr;
00163     QPushButton* calibrate_button_ = nullptr;
00164     QPushButton* splatoon_button_ = nullptr;
00165     QTimer* update_timer_ = nullptr;
00166     bool is_tracking_ = false;
00167     std::unique_ptr<marker_tracker::marker_tracker> tracker_;
00168     std::unique_ptr<CameraCalibration> calibrator_;
00169
00170     bool has_extrinsics_ = false;
00171     cv::Mat camera_matrix_;
00172     cv::Mat dist_coeffs_;
00173     cv::Mat rvec_;
00174     cv::Mat tvec_;
00175
00176     std::unique_ptr<GameManager> gameManager_;
00177
00178     mind::fleet::robot_fleet* fleet_ = nullptr;
00179     Server& tcp_server_;
00180
00181     // Splatoon test harness state: the game loop runs on splatoon_thread_ (below).
00182     bool splatoon_running_ = false;
00183     std::thread splatoon_thread_;
00184 };

```

11.127 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.cpp File Reference

Casino-themed Network Monitor: live connectivity and session overview.

```

#include "network_monitor.hpp"
#include <algorithm>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QGridLayout>
#include <QFrame>
#include <QLabel>
#include <QTimer>
#include "map/mapwindow.hpp"
#include "communication/session_registry.hpp"
#include "communication/TCP_Server.hpp"
#include "communication/UDP_Server.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "robot_controller/robot_controller.hpp"
#include "qtpanels/network_topology_widget.hpp"

```

11.127.1 Detailed Description

Casino-themed Network Monitor: live connectivity and session overview.

11.128 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.hpp File Reference

Standalone debug window: network connectivity, TCP sessions, and tracking status in the casino theme.

```
#include <cstdint>
#include <memory>
#include <unordered_map>
#include <QFrame>
#include <QLabel>
#include <QPushButton>
#include <QTimer>
#include <QWidget>
```

Classes

- class [NetworkMonitor](#)

Casino-themed Network Monitor showing TCP server sessions, tracking camera state, homography state, and per-robot fleet connectivity. Refreshes at 4 Hz via an internal QTimer.

11.128.1 Detailed Description

Standalone debug window: network connectivity, TCP sessions, and tracking status in the casino theme.

11.129 network_monitor.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <cstdint>
00010 #include <memory>
00011 #include <unordered_map>
00012 #include <QFrame>
00013 #include <QLabel>
00014 #include <QPushButton>
00015 #include <QTimer>
00016 #include <QWidget>
00017
00018 class SessionRegistry;
00019 class MapWindow;
00020 class NetworkTopologyWidget;
00021 class UdpServer;
00022
00028 class NetworkMonitor : public QWidget {
00029     Q_OBJECT
00030
00031 public:
00032     explicit NetworkMonitor(MapWindow& mapwindow,
00033                             UdpServer& udp_server,
00034                             std::shared_ptr<SessionRegistry> registry,
00035                             QWidget* parent = nullptr);
00036
00037 private slots:
00038     void refresh();
00039
00040 private:
00041     void buildUi();
00042
00043     MapWindow& mapwindow_;
```

```
00044     UdpServer& udp_server_;
00045     std::shared_ptr<SessionRegistry> registry_;
00046     QTimer* timer_ = nullptr;
00047
00048     // Status indicators
00049     QLabel* tracking_dot_ = nullptr;
00050     QLabel* tracking_value_ = nullptr;
00051     QLabel* homography_dot_ = nullptr;
00052     QLabel* homography_value_ = nullptr;
00053     QLabel* sim_mode_value_ = nullptr;
00054     QLabel* driving_value_ = nullptr;
00055     QLabel* session_count_value_ = nullptr;
00056
00057     // Session list labels (up to 8 slots)
00058     static constexpr int kMaxSessions = 8;
00059     std::array<QLabel*, kMaxSessions> session_labels_{};
00060     std::array<QLabel*, kMaxSessions> session_id_labels_{};
00061     std::array<QLabel*, kMaxSessions> session_seen_labels_{};
00062
00063     // Fleet connectivity
00064     std::array<QLabel*, 4> fleet_dots_{};
00065     std::array<QLabel*, 4> fleet_labels_{};
00066     QLabel* fleet_online_value_ = nullptr;
00067
00068     // Network topology diagram
00069     NetworkTopologyWidget* topology_ = nullptr;
00070
00071     // Connect buttons (moved from MapWindow toolbar)
00072     QPushButton* connect_unity_btn_ = nullptr;
00073     QLabel* connect_unity_status_ = nullptr;
00074     QPushButton* connect_robot_btn_ = nullptr;
00075     QLabel* connect_robot_status_ = nullptr;
00076
00077     // Broadcast timers for Connect Unity/Robot
00078     QTimer* unity_broadcast_timer_ = nullptr;
00079     QTimer* robot_broadcast_timer_ = nullptr;
00080
00081     // Topology toggle buttons
00082     QPushButton* toggle_game_ = nullptr;
00083     QPushButton* toggle_response_ = nullptr;
00084     QPushButton* toggle_heartbeat_ = nullptr;
00085     QPushButton* toggle_robot_ = nullptr;
00086     QPushButton* toggle_pairing_ = nullptr;
00087
00088     // Traffic rate tracking (per-session per-category previous byte counts)
00089     std::unordered_map<uint8_t, std::unordered_map<std::string, std::pair<uint64_t, uint64_t>>
prev_typed_{};
00090 };
```

11.130 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanel/network_topology_widget.cpp File Reference

Compiles the MOC output for the header-only [NetworkTopologyWidget](#).

```
#include "qtpanel/network_topology_widget.hpp"
#include "moc_network_topology_widget.cpp"
```

11.130.1 Detailed Description

Compiles the MOC output for the header-only [NetworkTopologyWidget](#).

11.131 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanel/network_topology_widget.hpp File Reference

Custom QPainter-drawn network topology diagram: 3-column layout (Unity clients | [Server](#) | [Robot](#) clients) with animated traffic pulses color-coded by category and pairing links between matched Unity-Robot pairs.

```
#include <algorithm>
#include <cmath>
#include <random>
#include <unordered_map>
#include <vector>
#include <QFont>
#include <QFontMetrics>
#include <QPainter>
#include <QPainterPath>
#include <QPoint>
#include <QRect>
#include <QString>
#include <QTimer>
#include <QWidget>
```

Classes

- struct [TopoClient](#)
One client entry for the topology diagram.
- struct [TopoPulse](#)
A single animated pulse traveling along a link arrow.
- class [NetworkTopologyWidget](#)
Network topology widget with 3-column layout: Unity TCP clients (left) | SERVER (center) | Robot UDP clients (right).
- struct [NetworkTopologyWidget::PairingSlot](#)

Namespaces

- namespace [TopoCategory](#)
Traffic category identifiers used throughout the topology.

Variables

- constexpr char [TopoCategory::Game](#) [] = "game"
- constexpr char [TopoCategory::Heartbeat](#) [] = "heartbeat"
- constexpr char [TopoCategory::Response](#) [] = "response"
- constexpr char [TopoCategory::Robot](#) [] = "robot"
- constexpr char [TopoCategory::Pairing](#) [] = "pairing"

11.131.1 Detailed Description

Custom QPainter-drawn network topology diagram: 3-column layout (Unity clients | [Server](#) | [Robot](#) clients) with animated traffic pulses color-coded by category and pairing links between matched Unity-Robot pairs.

Each connection shows two separate arrows (bidirectional):

- OUT line (offset -7): server -> client (game data, commands, heartbeat pings)
- IN line (offset +7): client -> server (responses, telemetry, heartbeat pongs)

11.132 network_topology_widget.hpp

[Go to the documentation of this file.](#)

```

00001
00011
00012 #pragma once
00013
00014 #include <algorithm>
00015 #include <cmath>
00016 #include <random>
00017 #include <unordered_map>
00018 #include <vector>
00019
00020 #include <QFont>
00021 #include <QFontMetrics>
00022 #include <QPainter>
00023 #include <QPainterPath>
00024 #include <QPoint>
00025 #include <QRect>
00026 #include <QString>
00027 #include <QTimer>
00028 #include <QWidget>
00029
00031 namespace TopoCategory {
00032     inline constexpr char Game[] = "game";
00033     inline constexpr char Heartbeat[] = "heartbeat";
00034     inline constexpr char Response[] = "response";
00035     inline constexpr char Robot[] = "robot";
00036     inline constexpr char Pairing[] = "pairing";
00037 } // namespace TopoCategory
00038
00042 struct TopoClient {
00043     QString label;
00044     bool tcp_online = false;
00045     bool udp_online = false;
00046     int game_bps = 0;
00047     int response_bps = 0;
00048     int heartbeat_bps = 0;
00049     int robot_bps = 0;
00050     bool is_robot = false;
00051     int pairing_id = -1;
00052 };
00053
00057 struct TopoPulse {
00058     float pos{0.0f};
00059     float speed{0.0f};
00060     float alpha{0.0f};
00061     float radius{0.0f};
00062 };
00063
00075 class NetworkTopologyWidget : public QWidget {
00076     Q_OBJECT
00077
00078 public:
00079     explicit NetworkTopologyWidget(QWidget* parent = nullptr)
00080         : QWidget(parent)
00081     {
00082         setMinimumHeight(400);
00083         setMinimumWidth(320);
00084
00085         anim_timer_.setInterval(33); // ~30 fps
00086         connect(&anim_timer_, &QTimer::timeout, this, [this]() {
00087             tickAnimation();
00088             update();
00089         });
00090         anim_timer_.start();
00091     }
00092
00094     void setClients(const std::vector<TopoClient>& clients)
00095     {
00096         clients_ = clients;
00097
00098         // Rebuild index arrays
00099         unity_indices_.clear();
00100         robot_indices_.clear();
00101         for (int i = 0; i < static_cast<int>(clients_.size()); ++i) {
00102             if (clients_[static_cast<size_t>(i)].is_robot)
00103                 robot_indices_.push_back(i);
00104             else
00105                 unity_indices_.push_back(i);
00106         }
00107
00108         // Rebuild pairing map
00109         pairing_map_.clear();
00110         for (int i = 0; i < static_cast<int>(clients_.size()); ++i) {

```

```

00111         int pid = clients_[static_cast<size_t>(i)].pairing_id;
00112         if (pid < 0) continue;
00113         auto& slot = pairing_map_[pid];
00114         if (clients_[static_cast<size_t>(i)].is_robot)
00115             slot.robot = i;
00116         else
00117             slot.unity = i;
00118     }
00119
00120     // Resize pulse arrays
00121     const int n = static_cast<int>(clients_.size());
00122     auto ensureSize = [n](std::vector<std::vector<TopoPulse>& v) {
00123         v.resize(static_cast<size_t>(std::max(static_cast<int>(v.size()), n)));
00124     };
00125     ensureSize(game_pulses_);
00126     ensureSize(heartbeat_pulses_);
00127     ensureSize(response_pulses_);
00128     ensureSize(robot_pulses_);
00129     ensureSize(pairing_pulses_);
00130
00131     pairing_bps_.resize(static_cast<size_t>(n), 0);
00132 }
00133
00134 void setPairingBps(int client_idx, int bps)
00135 {
00136     if (client_idx >= 0 && client_idx < static_cast<int>(pairing_bps_.size()))
00137         pairing_bps_[static_cast<size_t>(client_idx)] = bps;
00138 }
00139
00140 void setCategoryVisible(const char* category, bool visible)
00141 {
00142     visibility_[category] = visible;
00143 }
00144
00145 [[nodiscard]] bool isCategoryVisible(const char* category) const
00146 {
00147     auto it = visibility_.find(category);
00148     return it != visibility_.end() ? it->second : true;
00149 }
00150
00151 [[nodiscard]] const std::vector<TopoClient>& clients() const { return clients_; }
00152 [[nodiscard]] const std::vector<int>& unityIndices() const { return unity_indices_; }
00153 [[nodiscard]] const std::vector<int>& robotIndices() const { return robot_indices_; }
00154
00155 struct PairingSlot { int unity = -1; int robot = -1; };
00156 [[nodiscard]] const std::unordered_map<int, PairingSlot>& pairingMap() const { return
00157 pairing_map_; }
00158
00159 QSize sizeHint() const override { return QSize(500, 500); }
00160
00161 protected:
00162 void paintEvent(QPaintEvent*) override
00163 {
00164     QPainter p(this);
00165     p.setRenderHint(QPainter::Antialiasing, true);
00166
00167     const int W = width();
00168     const int H = height();
00169
00170     const int legend_h = 24;
00171     const int area_h = H - legend_h;
00172
00173     // Background
00174     p.fillRect(rect(), QColor(0x0B, 0x19, 0x29));
00175
00176     // Subtle grid
00177     p.setPen(QPen(QColor(0x15, 0x25, 0x35), 1));
00178     for (int x = 0; x < W; x += 30)
00179         p.drawLine(x, 0, x, area_h);
00180     for (int y = 0; y < area_h; y += 30)
00181         p.drawLine(0, y, W, y);
00182
00183     // --- Layout ---
00184     const int node_w = 100;
00185     const int node_h = 32;
00186     const int row_h = node_h + 14;
00187
00188     // 3-column x positions
00189     const int unity_x = 12;
00190     const int server_x = W / 2 - node_w / 2;
00191     const int robot_x = W - node_w - 12;
00192
00193     const int server_y = area_h / 2 - node_h / 2;
00194
00195     const int n_unity = static_cast<int>(unity_indices_.size());
00196     const int n_robot = static_cast<int>(robot_indices_.size());
00197
00198     auto columnStartY = [&](int count) {

```



```

00200         int total = count * node_h + qMax(0, count - 1) * (row_h - node_h);
00201         return area_h / 2 - total / 2;
00202     };
00203
00204     // Compute per-client screen positions
00205     std::vector<QPoint> client_pos(clients_.size());
00206
00207     int uy = columnStartY(n_unity);
00208     for (int idx : unity_indices_) {
00209         client_pos[static_cast<size_t>(idx)] = QPoint(unity_x, uy);
00210         uy += row_h;
00211     }
00212
00213     int ry = columnStartY(n_robot);
00214     for (int idx : robot_indices_) {
00215         client_pos[static_cast<size_t>(idx)] = QPoint(robot_x, ry);
00216         ry += row_h;
00217     }
00218
00219     // Arrow vertical offset from center of each connection
00220     const int out_offset = -7; // OUT line (server -> client)
00221     const int in_offset = +7; // IN line (client -> server)
00222
00223     // --- Draw bidirectional links and pulses (behind nodes) ---
00224
00225     // For each Unity client, draw TWO TCP lines to server
00226     for (int idx : unity_indices_) {
00227         const auto& cl = clients_[static_cast<size_t>(idx)];
00228         // Unity right edge -> server left edge
00229         QPoint ul(client_pos[static_cast<size_t>(idx)].x() + node_w,
00230 client_pos[static_cast<size_t>(idx)].y() + node_h / 2);
00231         QPoint sl(server_x, server_y + node_h / 2);
00232
00233         bool tcp_active = cl.tcp_online;
00234         QColor tcp_col = tcp_active ? QColor(0x00, 0xCC, 0x66) : QColor(0x33, 0x33, 0x33);
00235
00236         // OUT line: server -> client (game data, heartbeat pings)
00237         drawLink(p, sl, ul, out_offset, tcp_col, tcp_active);
00238         // IN line: client -> server (responses, heartbeat pongs)
00239         drawLink(p, ul, sl, in_offset, tcp_col, tcp_active);
00240
00241         if (tcp_active) {
00242             // Game pulses travel OUT (server -> client)
00243             if (isCategoryVisible(TopoCategory::Game) && idx <
00244 static_cast<int>(game_pulses_.size()))
00245                 drawPulses(p, sl, ul, out_offset, game_pulses_[static_cast<size_t>(idx)],
00246 QColor(0x00, 0xFF, 0x88));
00247             // Heartbeat pings travel OUT (server -> client)
00248             if (isCategoryVisible(TopoCategory::Heartbeat) && idx <
00249 static_cast<int>(heartbeat_pulses_.size()))
00250                 drawPulses(p, sl, ul, out_offset, heartbeat_pulses_[static_cast<size_t>(idx)],
00251 QColor(0xFF, 0xCC, 0x00));
00252             // Response travels IN (client -> server): pong, game input, telemetry
00253             if (isCategoryVisible(TopoCategory::Response) && idx <
00254 static_cast<int>(response_pulses_.size()))
00255                 drawPulses(p, ul, sl, in_offset, response_pulses_[static_cast<size_t>(idx)],
00256 QColor(0x00, 0xDD, 0xCC));
00257         }
00258     }
00259
00260     // For each Robot client, draw TWO UDP lines to server
00261     for (int idx : robot_indices_) {
00262         const auto& cl = clients_[static_cast<size_t>(idx)];
00263         // Robot left edge -> server right edge
00264         QPoint rl(robot_x, client_pos[static_cast<size_t>(idx)].y() + node_h / 2);
00265         QPoint sr(server_x + node_w, server_y + node_h / 2);
00266
00267         bool udp_active = cl.udp_online;
00268         QColor udp_col = udp_active ? QColor(0x00, 0x99, 0xFF) : QColor(0x33, 0x33, 0x33);
00269
00270         // OUT line: server -> robot (commands)
00271         drawLink(p, sr, rl, out_offset, udp_col, udp_active);
00272         // IN line: robot -> server (telemetry)
00273         drawLink(p, rl, sr, in_offset, udp_col, udp_active);
00274
00275         if (udp_active && isCategoryVisible(TopoCategory::Robot)
00276             && idx < static_cast<int>(robot_pulses_.size()))
00277         {
00278             // Robot telemetry travels IN (robot -> server)
00279             drawPulses(p, rl, sr, in_offset, robot_pulses_[static_cast<size_t>(idx)], QColor(0x33,
00280 0xBB, 0xFF));
00281         }
00282     }
00283
00284     // Draw pairing links
00285     for (const auto& [pid, slot] : pairing_map_) {
00286         if (slot.unity < 0 || slot.robot < 0) continue;
00287     }

```

```

00279
00280     bool unity_on = clients_[static_cast<size_t>(slot.unity)].tcp_online;
00281     bool robot_on = clients_[static_cast<size_t>(slot.robot)].udp_online;
00282     bool active = unity_on && robot_on;
00283
00284     QPoint ul(client_pos[static_cast<size_t>(slot.unity)].x() + node_w,
00285               client_pos[static_cast<size_t>(slot.unity)].y() + node_h / 2);
00286     QPoint rl(client_pos[static_cast<size_t>(slot.robot)].x(),
00287               client_pos[static_cast<size_t>(slot.robot)].y() + node_h / 2);
00288
00289     drawPairingLink(p, ul, rl, active);
00290
00291     if (active && isCategoryVisible(TopoCategory::Pairing)
00292         && slot.unity < static_cast<int>(pairing_pulses_.size()))
00293     {
00294         drawPulses(p, ul, rl, 0, pairing_pulses_[static_cast<size_t>(slot.unity)],
00295                   QColor(0xD4, 0xAF, 0x37));
00296         drawPulses(p, rl, ul, 0, pairing_pulses_[static_cast<size_t>(slot.robot)],
00297                   QColor(0xD4, 0xAF, 0x37));
00298     }
00299 }
00300
00301 // Placeholder
00302 if (clients_.empty()) {
00303     p.setPen(QColor(0x55, 0x55, 0x55));
00304     p.setFont(QFont("Consolas", 11));
00305     p.drawText(QRect(0, 0, W, area_h), Qt::AlignCenter, "No clients connected");
00306 }
00307
00308 // --- Draw nodes (on top of links) ---
00309 drawNode(p, server_x, server_y, node_w, node_h,
00310         "SERVER", QColor(0xD4, 0xAF, 0x37), true);
00311
00312 for (int idx : unity_indices_) {
00313     const auto& cl = clients_[static_cast<size_t>(idx)];
00314     bool online = cl.tcp_online;
00315     QColor border = online ? QColor(0x00, 0xCC, 0x66) : QColor(0x33, 0x33, 0x33);
00316     drawNode(p, client_pos[static_cast<size_t>(idx)].x(),
00317             client_pos[static_cast<size_t>(idx)].y(), node_w, node_h,
00318             cl.label, border, false);
00319 }
00320
00321 for (int idx : robot_indices_) {
00322     const auto& cl = clients_[static_cast<size_t>(idx)];
00323     bool online = cl.udp_online;
00324     QColor border = online ? QColor(0x00, 0x99, 0xFF) : QColor(0x33, 0x33, 0x33);
00325     drawNode(p, client_pos[static_cast<size_t>(idx)].x(),
00326             client_pos[static_cast<size_t>(idx)].y(), node_w, node_h,
00327             cl.label, border, false);
00328 }
00329
00330 // --- Legend ---
00331 drawLegend(p, W, legend_h, area_h);
00332 }
00333
00334 private:
00335 void tickAnimation()
00336 {
00337     const int n = static_cast<int>(clients_.size());
00338
00339     for (int i = 0; i < n; ++i) {
00340         const auto& cl = clients_[static_cast<size_t>(i)];
00341
00342         if (i < static_cast<int>(game_pulses_.size()))
00343             advancePulses(game_pulses_[static_cast<size_t>(i)], cl.game_bps);
00344         if (i < static_cast<int>(heartbeat_pulses_.size()))
00345             advancePulses(heartbeat_pulses_[static_cast<size_t>(i)], cl.heartbeat_bps);
00346         if (i < static_cast<int>(response_pulses_.size()))
00347             advancePulses(response_pulses_[static_cast<size_t>(i)], cl.response_bps);
00348         if (i < static_cast<int>(robot_pulses_.size()))
00349             advancePulses(robot_pulses_[static_cast<size_t>(i)], cl.robot_bps);
00350         if (i < static_cast<int>(pairing_pulses_.size()) && i <
00351             static_cast<int>(pairing_bps_.size()))
00352             advancePulses(pairing_pulses_[static_cast<size_t>(i)],
00353                 pairing_bps_[static_cast<size_t>(i)]);
00354     }
00355 }
00356
00357 void advancePulses(std::vector<TopoPulse>& pulses, int bytes_per_sec)
00358 {
00359     for (auto& pulse : pulses)
00360         pulse.pos += pulse.speed;
00361
00362     pulses.erase(
00363         std::remove_if(pulses.begin(), pulses.end(),
00364             [](const TopoPulse& p) { return p.pos >= 1.0f; }),

```

```

00362         pulses.end());
00363
00364         if (bytes_per_sec > 0) {
00365             float rate = static_cast<float>(bytes_per_sec) / 5000.0f;
00366             //the idea is that the more messages we send over a channel, the higher likely it is a
            pulse will spawn
00367             float probab = qMin(rate, 8.0f) * 0.15f;
00368             std::bernoulli_distribution spawn(static_cast<double>(probab));
00369             if (spawn(rng_)) {
00370                 TopoPulse p;
00371                 p.pos = 0.0f;
00372                 p.speed = 0.015f + 0.015f * qMin(rate / 4.0f, 1.0f);
00373                 p.alpha = 0.5f + 0.5f * qMin(rate / 6.0f, 1.0f);
00374                 p.radius = 3.0f + 2.0f * qMin(rate / 4.0f, 1.0f);
00375                 pulses.push_back(std::move(p));
00376             }
00377         }
00378     }
00379
00380     // ---- drawing -----
00381
00382     void drawPulses(QPainter& p, QPoint from, QPoint to,
00383                     int y_offset, const std::vector<TopoPulse>& pulses,
00384                     QColor base_color) const
00385     {
00386         // Shrink line inward by arrow_gap so pulses stay visible between nodes
00387         double dx = to.x() - from.x();
00388         double dy = to.y() - from.y();
00389         double len = std::sqrt(dx * dx + dy * dy);
00390         double ux = (len > 1.0) ? dx / len : 1.0;
00391         double uy = (len > 1.0) ? dy / len : 0.0;
00392         int gap = 14;
00393
00394         QPoint a(from.x() + static_cast<int>(ux * gap), from.y() + y_offset + static_cast<int>(uy *
            gap));
00395         QPoint b(to.x() - static_cast<int>(ux * gap), to.y() + y_offset - static_cast<int>(uy * gap));
00396
00397         for (const auto& pulse : pulses) {
00398             float x = static_cast<float>(a.x()) + (static_cast<float>(b.x()) -
                static_cast<float>(a.x())) * pulse.pos;
00399             float y = static_cast<float>(a.y()) + (static_cast<float>(b.y()) -
                static_cast<float>(a.y())) * pulse.pos;
00400
00401             // Fade in near source, fade out near destination
00402             float edge_fade = 1.0f;
00403             if (pulse.pos < 0.15f)
00404                 edge_fade = pulse.pos / 0.15f;
00405             else if (pulse.pos > 0.85f)
00406                 edge_fade = (1.0f - pulse.pos) / 0.15f;
00407             float a_mod = pulse.alpha * edge_fade;
00408             if (a_mod < 0.01f) continue;
00409
00410             // Outer glow
00411             QColor glow = base_color;
00412             glow.setAlpha(static_cast<int>(a_mod * 40));
00413             p.setPen(Qt::NoPen);
00414             p.setBrush(glow);
00415             p.drawEllipse(QPointF(static_cast<qreal>(x), static_cast<qreal>(y)),
                static_cast<qreal>(pulse.radius) * 2.5, static_cast<qreal>(pulse.radius) *
00416                 2.5);
00417
00418             // Core
00419             QColor core = base_color;
00420             core.setAlpha(static_cast<int>(a_mod * 255));
00421             p.setBrush(core);
00422             p.drawEllipse(QPointF(static_cast<qreal>(x), static_cast<qreal>(y)),
                static_cast<qreal>(pulse.radius), static_cast<qreal>(pulse.radius));
00423
00424             // Bright center
00425             QColor bright = Qt::white;
00426             bright.setAlpha(static_cast<int>(a_mod * 180));
00427             p.setBrush(bright);
00428             p.drawEllipse(QPointF(static_cast<qreal>(x), static_cast<qreal>(y)),
                static_cast<qreal>(pulse.radius) * 0.35, static_cast<qreal>(pulse.radius) *
00429                 0.35);
00430         }
00431     }
00432 }
00433
00434 void drawNode(QPainter& p, int x, int y, int w, int h,
00435               const QString& label, QColor border, bool is_server)
00436 {
00437     QRect rect(x, y, w, h);
00438
00439     p.setPen(Qt::NoPen);
00440     p.setBrush(is_server ? QColor(0x0D, 0x1A, 0x0A) : QColor(0x0D, 0x14, 0x20));
00441     p.drawRoundedRect(rect, 6, 6);
00442 }

```

```

00443     for (int i = 3; i >= 1; --i) {
00444         QColor glow = border;
00445         glow.setAlpha(30 * i);
00446         p.setPen(Qt::NoPen);
00447         p.setBrush(glow);
00448         p.drawRoundedRect(rect.adjusted(-i, -i, i, i), 8, 8);
00449     }
00450
00451     QRadialGradient grad(rect.center(), rect.width() / 2.0);
00452     if (is_server) {
00453         grad.setColorAt(0.0, QColor(0x1A, 0x2A, 0x15));
00454         grad.setColorAt(1.0, QColor(0x0D, 0x1A, 0x0A));
00455     } else {
00456         grad.setColorAt(0.0, QColor(0x14, 0x1E, 0x28));
00457         grad.setColorAt(1.0, QColor(0x0D, 0x14, 0x20));
00458     }
00459     p.setPen(QPen(border, 2));
00460     p.setBrush(grad);
00461     p.drawRoundedRect(rect, 6, 6);
00462
00463     p.setPen(QColor(0xFF, 0xF8, 0xD6));
00464     QFont f("Georgia", is_server ? 10 : 8, QFont::Bold);
00465     p.setFont(f);
00466     p.drawText(rect, Qt::AlignCenter, label);
00467 }
00468
00469 void drawLink(QPainter& p, QPoint from, QPoint to,
00470             int y_offset, QColor color, bool active)
00471 {
00472     // Shrink endpoints inward by arrow_gap so arrows don't overlap nodes
00473     double dx = to.x() - from.x();
00474     double dy = to.y() - from.y();
00475     double len = std::sqrt(dx * dx + dy * dy);
00476     double ux = (len > 1.0) ? dx / len : 1.0;
00477     double uy = (len > 1.0) ? dy / len : 0.0;
00478     int gap = 14;
00479
00480     QPoint a(from.x() + static_cast<int>(ux * gap),
00481             from.y() + y_offset + static_cast<int>(uy * gap));
00482     QPoint b(to.x() - static_cast<int>(ux * gap),
00483             to.y() + y_offset - static_cast<int>(uy * gap));
00484
00485     QPen line_pen(color, active ? 2 : 1);
00486     if (!active)
00487         line_pen.setStyle(Qt::DashLine);
00488     p.setPen(line_pen);
00489     p.setBrush(Qt::NoBrush);
00490     p.drawLine(a, b);
00491
00492     // Arrowhead at destination end
00493     if (active) {
00494         double adx = b.x() - a.x();
00495         double ady = b.y() - a.y();
00496         double alen = std::sqrt(adx * adx + ady * ady);
00497         if (alen > 1.0) {
00498             double aux = adx / alen;
00499             double auy = ady / alen;
00500             double tip_x = b.x();
00501             double tip_y = b.y();
00502             double ax = tip_x - aux * 8 - auy * 4;
00503             double ay = tip_y - auy * 8 + aux * 4;
00504             double bx2 = tip_x - aux * 8 + auy * 4;
00505             double by2 = tip_y - auy * 8 - aux * 4;
00506
00507             QPainterPath arrow;
00508             arrow.moveTo(tip_x, tip_y);
00509             arrow.lineTo(ax, ay);
00510             arrow.lineTo(bx2, by2);
00511             arrow.closeSubpath();
00512
00513             p.setPen(Qt::NoPen);
00514             p.setBrush(color);
00515             p.drawPath(arrow);
00516         }
00517     }
00518 }
00519
00520 void drawPairingLink(QPainter& p, QPoint from, QPoint to, bool active)
00521 {
00522     QColor color(0xD4, 0xAF, 0x37); // Gold
00523
00524     int mid_x = (from.x() + to.x()) / 2;
00525     int mid_y = (from.y() + to.y()) / 2;
00526     int curve_offset = (to.y() > from.y()) ? 30 : -30;
00527
00528     QPainterPath path;
00529     path.moveTo(from);

```

```

00530         path.cubicTo(QPoint(mid_x, from.y() + curve_offset),
00531                     QPoint(mid_x, to.y() + curve_offset),
00532                     to);
00533
00534         QPen pen(color, active ? 2 : 1);
00535         if (!active) pen.setStyle(Qt::DashLine);
00536         p.setPen(pen);
00537         p.setBrush(Qt::NoBrush);
00538         p.drawPath(path);
00539
00540         // Pairing badge
00541         QFont badge_font("Consolas", 7, QFont::Bold);
00542         p.setFont(badge_font);
00543         QFontMetrics fm(badge_font);
00544         QRect text_rect = fm.boundingRect("PAIR");
00545         int badge_w = text_rect.width() + 8;
00546         int badge_h = text_rect.height() + 3;
00547
00548         QRect badge_rect(mid_x - badge_w / 2, mid_y - badge_h / 2, badge_w, badge_h);
00549         QColor bg = active ? QColor(0x1A, 0x1A, 0x0D) : QColor(0x15, 0x15, 0x15);
00550         p.setPen(QPen(color, 1));
00551         p.setBrush(bg);
00552         p.drawRoundedRect(badge_rect, 3, 3);
00553
00554         p.setPen(active ? color : QColor(0x55, 0x55, 0x55));
00555         p.drawText(badge_rect, Qt::AlignCenter, "PAIR");
00556     }
00557
00558 void drawLegend(QPainter& p, int W, int legend_h, int y_start)
00559 {
00560     p.setPen(QPen(QColor(0x25, 0x35, 0x45), 1));
00561     p.drawLine(8, y_start + 1, W - 8, y_start + 1);
00562
00563     struct LegendEntry { const char* label; QColor color; const char* key; };
00564     const LegendEntry entries[] = {
00565         { "Game",          QColor(0x00, 0xFF, 0x88), TopoCategory::Game },
00566         { "Response",      QColor(0x00, 0xDD, 0xCC), TopoCategory::Response },
00567         { "Heartbeat",     QColor(0xFF, 0xCC, 0x00), TopoCategory::Heartbeat },
00568         { "Robot UDP",     QColor(0x33, 0xBB, 0xFF), TopoCategory::Robot },
00569         { "Pairing",       QColor(0xD4, 0xAF, 0x37), TopoCategory::Pairing },
00570     };
00571
00572     QFont lf("Consolas", 8);
00573     p.setFont(lf);
00574     QFontMetrics fm(lf);
00575
00576     int x = 16;
00577     int dot_size = 8;
00578     int text_h = fm.height();
00579
00580     for (const auto& e : entries) {
00581         bool vis = isCategoryVisible(e.key);
00582         QColor c = vis ? e.color : QColor(0x33, 0x33, 0x33);
00583         p.setPen(Qt::NoPen);
00584         p.setBrush(c);
00585         p.drawRoundedRect(x, y_start + (legend_h - dot_size) / 2,
00586                         dot_size, dot_size, 2, 2);
00587         x += dot_size + 4;
00588
00589         p.setPen(vis ? QColor(0xCC, 0xCC, 0xCC) : QColor(0x55, 0x55, 0x55));
00590         p.drawText(x, y_start + (legend_h + text_h) / 2 - 1, e.label);
00591         x += fm.horizontalAdvance(e.label) + 16;
00592     }
00593 }
00594
00595 // ---- data -----
00596
00597 std::vector<TopoClient> clients_;
00598 std::vector<int> unity_indices_;
00599 std::vector<int> robot_indices_;
00600
00601 std::unordered_map<int, PairingSlot> pairing_map_;
00602
00603 std::vector<std::vector<TopoPulse>> game_pulses_;
00604 std::vector<std::vector<TopoPulse>> heartbeat_pulses_;
00605 std::vector<std::vector<TopoPulse>> response_pulses_;
00606 std::vector<std::vector<TopoPulse>> robot_pulses_;
00607 std::vector<std::vector<TopoPulse>> pairing_pulses_;
00608
00609 std::vector<int> pairing_bps_;
00610
00611 std::unordered_map<std::string, bool> visibility_ = {
00612     { TopoCategory::Game,      true },
00613     { TopoCategory::Heartbeat, true },
00614     { TopoCategory::Response,  true },
00615     { TopoCategory::Robot,     true },
00616     { TopoCategory::Pairing,   true },

```

```

00617     };
00618
00619     QTimer anim_timer_;
00620     std::mt19937 rng_{std::random_device{}}();
00621 };

```

11.133 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/path_debug.cpp File Reference

Casino-themed Path Debug: collision planner and follower state overview.

```

#include "path_debug.hpp"
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QGridLayout>
#include <QFrame>
#include <QLabel>
#include <QStyle>
#include <QTimer>
#include "map/mapwindow.hpp"
#include "collision_avoidance/collision_planner.hpp"
#include "path_follower/path_follower.hpp"

```

11.133.1 Detailed Description

Casino-themed Path Debug: collision planner and follower state overview.

11.134 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/path_debug.hpp File Reference

Standalone debug window: collision planner paths, routes, and follower state in the casino theme.

```

#include <array>
#include <QFrame>
#include <QLabel>
#include <QTimer>
#include <QWidget>

```

Classes

- class [PathDebug](#)

Casino-themed Path Debug panel showing per-robot collision planner state (effective path, nominal path, route nodes, yielding status, watched pairs) and follower phase/cross-track data. Refreshes at 4 Hz.

11.134.1 Detailed Description

Standalone debug window: collision planner paths, routes, and follower state in the casino theme.

11.135 path_debug.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include <array>
00010 #include <QFrame>
00011 #include <QLabel>
00012 #include <QTimer>
00013 #include <QWidget>
00014
00015 class MapWindow;
00016
00022 class PathDebug : public QWidget {
00023     Q_OBJECT
00024
00025 public:
00026     explicit PathDebug(MapWindow& mapwindow, QWidget* parent = nullptr);
00027
00028 private slots:
00029     void refresh();
00030
00031 private:
00032     void buildUi();
00033
00034     MapWindow& mapwindow_;
00035     QTimer* timer_ = nullptr;
00036
00037     struct RobotPathCard {
00038         QFrame* frame = nullptr;
00039         QLabel* id_label = nullptr;
00040         QLabel* route_label = nullptr;
00041         QLabel* path_state_label = nullptr;
00042         QLabel* path_wps_label = nullptr;
00043         QLabel* yielding_label = nullptr;
00044         QLabel* follower_mode_label = nullptr;
00045         QLabel* cross_track_label = nullptr;
00046         QLabel* lookahead_label = nullptr;
00047     };
00048
00049     std::array<RobotPathCard, 4> cards_;
00050
00051     // Watched pairs
00052     QLabel* pairs_value_ = nullptr;
00053     QLabel* tick_value_ = nullptr;
00054
00055     static QString followStateString(int state);
00056     static QString followerModeString(int mode);
00057 };

```

11.136 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.cpp File Reference

Implementation of [ShopTestWindow](#): builds the test panel UI and forwards button clicks to [Shop::openShop\(\)](#) / [Roulette::start_roulette\(\)](#) for a synthetic test player.

```

#include "shop_test_window.hpp"
#include "items/shop.hpp"
#include "player/player.hpp"
#include "utilities/Logger.hpp"

```

11.136.1 Detailed Description

Implementation of [ShopTestWindow](#): builds the test panel UI and forwards button clicks to [Shop::openShop\(\)](#) / [Roulette::start_roulette\(\)](#) for a synthetic test player.

11.137 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.hpp File Reference

Declares [ShopTestWindow](#), a manual test panel for triggering shop and roulette network broadcasts to the AR/Unity client.

```
#include "items/shop.hpp"
#include "map/roulette.hpp"
#include <QWidget>
#include <QSpinBox>
#include <QPushButton>
#include <QLabel>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFrame>
#include <memory>
```

Classes

- class [ShopTestWindow](#)

Manual test/debug panel with buttons to trigger a shop-open or roulette-start broadcast for a chosen player id, without going through the actual game flow. Constructs (and reuses) a throwaway [Player](#) instance purely to satisfy the Shop/Roulette APIs, which take a [Player&](#) but only read its current player id.

11.137.1 Detailed Description

Declares [ShopTestWindow](#), a manual test panel for triggering shop and roulette network broadcasts to the AR/Unity client.

11.138 shop_test_window.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008 #include "items/shop.hpp"
00009 #include "map/roulette.hpp"
00010 #include <QWidget>
00011 #include <QSpinBox>
00012 #include <QPushButton>
00013 #include <QLabel>
00014 #include <QVBoxLayout>
00015 #include <QHBoxLayout>
00016 #include <QFrame>
00017
00018 // Forward declaration of Shop
00019 class Shop;
00020 class Roulette;
00021
00022
00023 #include <memory>
00024 class Player;
00025
00032 class ShopTestWindow : public QWidget {
00033     Q_OBJECT
00034
00035 public:
00043     explicit ShopTestWindow(Shop& shop, Roulette& roulette, QWidget* parent = nullptr);
00044
```



```

00048     ~ShopTestWindow() override = default;
00049
00050 private slots:
00057     void handleOpenShopClicked();
00058
00066     void handleStartRouletteClicked();
00067
00068 private:
00069     Shop& shop_;
00070     Roulette& roulette_;
00071     QSpinBox* playerSpinBox_ = nullptr;
00072     QPushButton* openShopButton_ = nullptr;
00073     QPushButton* startRouletteButton_ = nullptr;
00074     std::unique_ptr<Player> testPlayer_;
00075 };

```

11.139 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.cpp File Reference

Implementation of the manual robot-drive test panel.

```

#include "test_panel.hpp"
#include <QGridLayout>
#include <QLabel>
#include <QPushButton>
#include <QTimer>
#include <QVBoxLayout>
#include "robot_controller/robot_controller.hpp"
#include "robot_controller/robot_fleet.hpp"
#include "utilities/Logger.hpp"

```

11.139.1 Detailed Description

Implementation of the manual robot-drive test panel.

11.140 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.hpp File Reference

Defines a manual test panel widget for exercising the active robot's drive API.

```

#include <QWidget>

```

Classes

- class [test_panel](#)

Six-button panel for manually exercising the active robot's drive API. Listens to robot_fleet::active_changed to retarget itself. Buttons cover the Goal D behaviours: continuous drive (hot path + brick brake stop), precision drive_for / turn (calibration verify), stop interrupting a move mid-flight (drive_for + stop, turn + stop), and an emergency stop. Per-test QTimer's are stop()'d before each press so back-to-back clicks do not chop each other.

11.140.1 Detailed Description

Defines a manual test panel widget for exercising the active robot's drive API.

11.141 test_panel.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <QWidget>
00009
00010 namespace mind::fleet {
00011     class robot_controller;
00012     class robot_fleet;
00013 }
00014
00015 class QLabel;
00016 class QPushButton;
00017 class QTimer;
00018
00027 class test_panel : public QWidget {
00028     Q_OBJECT
00029
00030 public:
00037     explicit test_panel(mind::fleet::robot_fleet* fleet, QWidget* parent = nullptr);
00038
00039 private slots:
00046     void on_active_changed(mind::fleet::robot_controller* active);
00047
00048 private:
00053     void set_buttons_enabled(bool enabled);
00054
00055     mind::fleet::robot_fleet* fleet_ = nullptr;
00056     mind::fleet::robot_controller* active_ = nullptr;
00057     QLabel* active_label_ = nullptr;
00058
00059     QPushButton* drive_2s_btn_ = nullptr;
00060     QPushButton* drive_for_btn_ = nullptr;
00061     QPushButton* turn_btn_ = nullptr;
00062     QPushButton* drive_stop_btn_ = nullptr;
00063     QPushButton* turn_stop_btn_ = nullptr;
00064     QPushButton* stop_btn_ = nullptr;
00065
00066     // One QTimer per timed test, restarted on each press so a stale earlier
00067     // timer cannot cut a later press short.
00068     QTimer* drive_2s_timer_ = nullptr;
00069     QTimer* drive_stop_timer_ = nullptr;
00070     QTimer* turn_stop_timer_ = nullptr;
00071 };

```

11.142 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.cpp File Reference

Implementation of robot_controller: MQTT/UDP command publishing, the continuous-drive republish loop, and the move_relative state machine.

```

#include "robot_controller/robot_controller.hpp"
#include <algorithm>
#include <cmath>
#include <optional>
#include <utility>
#include <QDateTime>
#include "mqtt_client/client.hpp"
#include "protocol/codec.hpp"
#include "udp_link/udp_link.hpp"
#include "utilities/Logger.hpp"

```

11.142.1 Detailed Description

Implementation of robot_controller: MQTT/UDP command publishing, the continuous-drive republish loop, and the move_relative state machine.

11.143 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.hpp File Reference

Declares robot_controller, the per-robot facade that turns method calls into MQTT/UDP commands and turns EV3 status messages into Qt signals.

```
#include <cstdint>
#include <QByteArray>
#include <QHostAddress>
#include <QObject>
#include <QString>
#include <QTimer>
```

Classes

- class `mind::fleet::robot_controller`

Per-robot facade owned by `robot_fleet`: translates method calls into MQTT/UDP commands and translates incoming EV3 status into Qt signals. Owns its own timers and runs the move_relative turn -> drive_for state machine internally; one instance exists per tracked robot.

11.143.1 Detailed Description

Declares robot_controller, the per-robot facade that turns method calls into MQTT/UDP commands and turns EV3 status messages into Qt signals.

11.144 robot_controller.hpp

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include <cstdint>
00010
00011 #include <QByteArray>
00012 #include <QHostAddress>
00013 #include <QObject>
00014 #include <QString>
00015 #include <QTimer>
00016
00017 namespace mind::mqtt {
00018     class client;
00019 }
00020
00021 namespace mind::net {
00022     class udp_link;
00023 }
00024
00025 namespace mind::fleet {
00026
```

```

00033 class robot_controller : public QObject {
00034     Q_OBJECT
00035
00036 public:
00048     robot_controller(QString robot_id, mind::mqtt::client* mqtt,
00049                     mind::net::udp_link* udp, QObject* parent = nullptr);
00050
00051     robot_controller(const robot_controller&) = delete;
00052     robot_controller& operator=(const robot_controller&) = delete;
00053
00055     QString robot_id() const noexcept { return robot_id_; }
00057     bool is_online() const noexcept { return online_; }
00059     QString state() const noexcept { return state_; }
00061     int battery_mv() const noexcept { return battery_mv_; }
00062
00074     void set_drive(int speed_pct, int turn_pct);
00075
00084     void set_wheel_drive(int left_wheel, int right_wheel);
00085
00093     void stop();
00094
00100     void beep(int duration_ms, int frequency_hz);
00101
00112     void move_relative(double forward_cm, double right_cm);
00113
00120     void drive_for(int distance_mm, int speed_mm_s);
00121
00128     void turn(int angle_deg, int rate_dps);
00129
00138     void handle_status(const QByteArray& status);
00139
00146     void set_brick_address(QHostAddress addr);
00147
00148 public slots:
00158     void on_pose_updated(double x_mm, double y_mm, double theta_rad,
00159                         qint64 ts_ms);
00160
00161 signals:
00163     void connection_changed(bool online);
00165     void state_changed(QString state);
00167     void battery_changed(int millivolts);
00168
00169 private slots:
00176     void on_republish_tick();
00177
00182     void on_offline_timer_fired();
00183
00189     void on_move_timeout();
00190
00191 private:
00193     enum class drive_mode { none, tank, wheel };
00195     enum class move_state { idle, awaiting_turn_ack, awaiting_drive_ack };
00196
00202     void publish_drive(int speed_pct, int turn_pct);
00203
00209     void publish_wheel_drive(int left_wheel, int right_wheel);
00210
00212     void publish_stop();
00213
00215     void publish_beep(int duration_ms, int frequency_hz);
00216
00222     int publish_turn(int angle_deg, int rate_dps);
00223
00229     int publish_drive_for(int distance_mm, int speed_mm_s);
00230
00237     void advance_move_relative();
00238
00239     mind::mqtt::client* mqtt_ = nullptr;
00240     mind::net::udp_link* udp_ = nullptr;
00241     QString robot_id_;
00242     QString cmd_topic_;
00243     std::uint8_t nn_ = 0;
00244     QHostAddress brick_addr_;
00245     QString state_ = "idle";
00246     bool online_ = false;
00247     int battery_mv_ = 0;
00248     int next_id_ = 1;
00249
00250     // Continuous-drive state.
00251     drive_mode drive_mode_ = drive_mode::none;
00252     int tank_speed_pct_ = 0;
00253     int tank_turn_pct_ = 0;
00254     int wheel_left_pct_ = 0;
00255     int wheel_right_pct_ = 0;
00256     qint64 zero_since_ms_ = 0;
00257     QTimer* republish_timer_ = nullptr;
00258

```

```

00259 // move_relative state machine.
00260 move_state move_state_ = move_state::idle;
00261 int move_pending_cmd_id_ = 0;
00262 int move_drive_distance_mm_ = 0;
00263 QTimer* move_timeout_timer_ = nullptr;
00264
00265 // Per-controller offline detector; restarts on each handle_status.
00266 QTimer* offline_timer_ = nullptr;
00267
00268 // Last pose received via on_pose_updated (closed-loop seam).
00269 double last_pose_x_mm_ = 0.0;
00270 double last_pose_y_mm_ = 0.0;
00271 double last_pose_theta_rad_ = 0.0;
00272 qint64 last_pose_ts_ms_ = 0;
00273 };
00274
00275 } // namespace mind::fleet

```

11.145 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.cpp File Reference ↩

Implementation of robot_fleet: UDP-heartbeat-driven robot discovery and MQTT status ack routing.

```

#include "robot_controller/robot_fleet.hpp"
#include <QStringList>
#include "mqtt_client/client.hpp"
#include "protocol/codec.hpp"
#include "robot_controller/robot_controller.hpp"
#include "udp_link/udp_link.hpp"
#include "utilities/Logger.hpp"

```

11.145.1 Detailed Description

Implementation of robot_fleet: UDP-heartbeat-driven robot discovery and MQTT status ack routing.

11.146 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.hpp File Reference ↩

Declares robot_fleet, which discovers EV3 robots via UDP heartbeats, routes MQTT status acks to them, and owns their robot_controller instances.

```

#include <QByteArray>
#include <QHash>
#include <QHostAddress>
#include <QObject>
#include <QString>

```

Classes

- class `mind::fleet::robot_fleet`

Auto-discovers EV3 robots and owns one [robot_controller](#) per robot. Robots are registered on their first UDP heartbeat (see `on_datagram_received`); MQTT status acks on `mind/+status` are then routed to the matching already-discovered controller (see `on_status_message`). Also tracks which robot is currently "active" (see [select\(\)](#)/`active()`).

11.146.1 Detailed Description

Declares `robot_fleet`, which discovers EV3 robots via UDP heartbeats, routes MQTT status acks to them, and owns their `robot_controller` instances.

11.147 `robot_fleet.hpp`

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include <QByteArray>
00010 #include <QHash>
00011 #include <QHostAddress>
00012 #include <QObject>
00013 #include <QString>
00014
00015 namespace mind::mqtt {
00016     class client;
00017 }
00018
00019 namespace mind::net {
00020     class udp_link;
00021 }
00022
00023 namespace mind::fleet {
00024
00025     class robot_controller;
00026
00034     class robot_fleet : public QObject {
00035         Q_OBJECT
00036
00037     public:
00047         robot_fleet(mind::mqtt::client* mqtt, mind::net::udp_link* udp,
00048                     QObject* parent = nullptr);
00049
00050         robot_fleet(const robot_fleet&) = delete;
00051         robot_fleet& operator=(const robot_fleet&) = delete;
00052
00054         robot_controller* active() const { return active_; }
00055
00064         robot_controller* controller(QString robot_id) const;
00065
00066     public slots:
00073         void select(QString robot_id);
00074
00075     signals:
00078         void robot_appeared(QString robot_id, robot_controller* controller);
00079
00082         void robot_disappeared(QString robot_id);
00083
00085         void active_changed(robot_controller* active);
00086
00089         void robot_ip_discovered(quint8 nn, QString ip_address);
00090
00091     private slots:
00099         void on_status_message(QString topic, QByteArray payload);
00100
00112         void on_datagram_received(QByteArray payload, QHostAddress sender);
00113
00114     private:
00115         mind::mqtt::client* mqtt_ = nullptr;
00116         mind::net::udp_link* udp_ = nullptr;
00117         QHash<QString, robot_controller*> controllers_;
00118         robot_controller* active_ = nullptr;
00119     };
00120
00121 } // namespace mind::fleet

```

11.148 `/Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.cpp` File Reference

Implementation of the ArUco marker detection camera pipeline ([marker_tracker::marker_tracker](#)).

```
#include <algorithm>
#include <array>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <string>
#include "tracker.hpp"
#include "utilities/Logger.hpp"
```

Variables

- constexpr int **marker_tracker::CAMERA_WIDTH** = 1920
- constexpr int **marker_tracker::CAMERA_HEIGHT** = 1080
- constexpr int **marker_tracker::CAMERA_FPS** = 60
- constexpr double **marker_tracker::CAMERA_EXPOSURE** = -6.0
- constexpr double **marker_tracker::CAMERA_BRIGHTNESS** = 100.0
- constexpr int **marker_tracker::CAMERA_FOCUS** = 0
- constexpr int **marker_tracker::ARUCO_MARKER_COUNT** = 12
- constexpr int **marker_tracker::ARUCO_MARKER_BITS** = 3
- constexpr std::array< int, 4 > **marker_tracker::robot_marker_ids** = {0, 1, 2, 3}

11.148.1 Detailed Description

Implementation of the ArUco marker detection camera pipeline ([marker_tracker::marker_tracker](#)).

11.149 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.hpp File Reference ↔

Declares the ArUco marker detection camera pipeline ([marker_tracker::marker_tracker](#)) and the per-marker detection result ([marker_tracker::marker_position](#)).

```
#include <string>
#include <vector>
#include <opencv2/opencv.hpp>
#include <opencv2/aruco.hpp>
#include <opencv2/core.hpp>
#include <opencv2/imgcodecs.hpp>
```

Classes

- struct [marker_tracker::marker_position](#)
Result of one ArUco marker detection: identifier plus its pose in the current camera frame. Position and angle are derived directly from the raw detected corners in camera pixel space; converting to real-world floor coordinates (e.g. via homography/extrinsics) is left to the caller.
- class [marker_tracker::marker_tracker](#)
Owns one camera device and the ArUco detection pipeline for that camera. Wraps a cv::VideoCapture plus a custom 12-marker ArUco dictionary/detector, and exposes the camera intrinsic calibration (shared via static members) used to interpret detected marker poses.

11.149.1 Detailed Description

Declares the ArUco marker detection camera pipeline (`marker_tracker::marker_tracker`) and the per-marker detection result (`marker_tracker::marker_position`).

11.150 tracker.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #ifndef MARKER_TRACKER_TRACKER_H
00008 #define MARKER_TRACKER_TRACKER_H
00009
00010 #include <string>
00011 #include <vector>
00012
00013 #include <opencv2/opencv.hpp>
00014 #include <opencv2/aruco.hpp>
00015 #include <opencv2/core.hpp>
00016 #include <opencv2/imgcodecs.hpp>
00017
00018 namespace marker_tracker {
00019
00026     struct marker_position {
00027         int marker_id;
00028         bool detected;
00029         cv::Point2f world_position;
00030         double angle;
00031         cv::Point2f front_px{0.f, 0.f};
00032         cv::Point2f back_px{0.f, 0.f};
00033     };
00036
00043     class marker_tracker {
00044     public:
00045
00052         marker_tracker();
00053
00057         ~marker_tracker();
00058
00063         static std::vector<int> get_valid_robot_marker_ids();
00064
00071         static int robot_slot_for_marker(int marker_id);
00072
00085         bool initialize_camera(int camera_index = 0);
00086
00094         bool initialize_camera_auto(const std::string& name_substr = "EOS Webcam Utility",
00095                                     int fallback_index = 0);
00096
00104         static int find_camera_index_by_name(const std::string& name_substr);
00105
00111         bool reinitialize(int camera_index);
00112
00117         void shutdown_camera();
00118
00126         std::vector<marker_position> detect_markers();
00127
00136         cv::Mat get_latest_frame();
00137
00145         void load_calibration(const cv::Mat& camera_matrix, const cv::Mat& dist_coeffs);
00146
00151         int get_camera_index() const { return camera_index_; }
00152
00158         const cv::Mat& get_current_frame() const { return current_frame_; }
00159
00165         static const cv::Mat& get_camera_matrix() { return camera_matrix_; }
00166
00172         static const cv::Mat& get_dist_coeffs() { return dist_coeffs_; }
00173
00174     private:
00175         int camera_index_ = 0;
00176         cv::Mat current_frame_;
00177         cv::VideoCapture camera_;
00178         bool is_initialized_ = false;
00179
00180         cv::aruco::Dictionary aruco_dictionary_;
00181         cv::aruco::DetectorParameters detector_params_;
00182         cv::Ptr<cv::aruco::ArucoDetector> aruco_detector_;
00183

```



```

00184         // Camera calibration data
00185         static cv::Mat camera_matrix_;
00186         static cv::Mat dist_coeffs_;
00187         bool is_calibrated_ = false;
00188
00196         void initialize_aruco_detector();
00197
00207         std::vector<marker_position> process_aruco_markers(
00208             const std::vector<int>& marker_ids,
00209             const std::vector<std::vector<cv::Point2f>& marker_corners);
00210
00211     };
00212
00213 } // namespace marker_tracker
00214
00215 #endif // MARKER_TRACKER_TRACKER_H

```

11.151 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/udp_link.cpp File Reference

Implementation of the UDP transport for command and heartbeat datagrams.

```

#include "udp_link/udp_link.hpp"
#include <QNetworkDatagram>
#include <QThread>
#include <QUdpSocket>
#include "utilities/Logger.hpp"

```

11.151.1 Detailed Description

Implementation of the UDP transport for command and heartbeat datagrams.

11.152 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/udp_link.hpp File Reference

Defines a thin, self-marshalling UDP transport for exchanging command and heartbeat datagrams with a brick.

```

#include <QByteArray>
#include <QHostAddress>
#include <QObject>
#include <QtGlobal>

```

Classes

- class `mind::net::udp_link`

Thin UDP transport for loss-tolerant streams: sends command datagrams to a brick and receives heartbeats on a bound port. Decoding is the caller's job (mind::proto); this layer only moves raw bytes. Runs on its own QThread; send_command is self-marshalling, so callers on any thread queue onto this object's thread via QMetaObject::invokeMethod.

Variables

- constexpr quint16 **mind::net::udp_cmd_port** = 7778
Port the brick binds; desktop -> brick commands.
- constexpr quint16 **mind::net::udp_status_port** = 7779
Port the desktop binds; brick -> desktop heartbeats.

11.152.1 Detailed Description

Defines a thin, self-marshalling UDP transport for exchanging command and heartbeat datagrams with a brick.

11.153 udp_link.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <QByteArray>
00009 #include <QHostAddress>
00010 #include <QObject>
00011 #include <QtGlobal>
00012
00013 class QUdpSocket;
00014
00015 namespace mind::net {
00016
00017 // UDP ports per shared/protocol.md.
00018 constexpr quint16 udp_cmd_port = 7778;
00019 constexpr quint16 udp_status_port = 7779;
00020
00021 class udp_link : public QObject {
00022     Q_OBJECT
00023
00024 public:
00025     explicit udp_link(quint16 listen_port, QObject* parent = nullptr);
00026     ~udp_link() override;
00027
00028     udp_link(const udp_link&) = delete;
00029     udp_link& operator=(const udp_link&) = delete;
00030
00031 public slots:
00032     void start();
00033
00034     void send_command(const QHostAddress& addr, quint16 port, const QByteArray& payload);
00035
00036 signals:
00037     void datagram_received(QByteArray payload, QHostAddress sender);
00038     void command_sent();
00039
00040 private slots:
00041     void on_ready_read();
00042
00043 private:
00044     QUdpSocket* socket_ = nullptr;
00045     quint16 listen_port_ = 0;
00046 };
00047
00048 } // namespace mind::net
```

11.154 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/↵ mapLogic.cpp File Reference

Standalone prototype of the board graph model (nodes, edges, node types) for the Mario Party style map.

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include "utilities/Logger.hpp"
```

Classes

- class [Node](#)
One position ("tile") in the board's node graph. neighbors form an undirected adjacency list used for robot movement/pathfinding.
- class [Board](#)
Owns the static node graph for the game board and can broadcast it to clients over TCP. The graph is hard-coded in the constructor; nothing in this class mutates it afterward.

Enumerations

- enum class [NodeType](#) {
 START , **NORMAL** , **MINIGAME** , **TRAP** ,
 SHOP , **FINISH** }
Categorizes the gameplay role of a board node (e.g. start/finish tile, shop, minigame trigger, trap).

Functions

- string [nodeTypeToString](#) ([NodeType](#) type)
Converts a [NodeType](#) value to its human-readable name, for logging/debug output.
- int [main](#) ()
Demo entry point: builds a small sample board graph and prints it. Hardcodes a fixed layout of 7 nodes (start, normal tiles, a minigame, a shop, a trap, and a finish) and their connections, then dumps the resulting graph via [Board::printGraph](#).

11.154.1 Detailed Description

Standalone prototype of the board graph model (nodes, edges, node types) for the Mario Party style map.

11.154.2 Function Documentation

11.154.2.1 [main\(\)](#)

```
int main ()
```

Demo entry point: builds a small sample board graph and prints it. Hardcodes a fixed layout of 7 nodes (start, normal tiles, a minigame, a shop, a trap, and a finish) and their connections, then dumps the resulting graph via [Board::printGraph](#).

Returns

Always 0.

11.154.2.2 nodeTypeToString()

```
string nodeTypeToString (
    NodeType type)
```

Converts a [NodeType](#) value to its human-readable name, for logging/debug output.

Parameters

<i>type</i>	The node type to convert.
-------------	---------------------------

Returns

The matching uppercase name (e.g. "START"), or "UNKNOWN" if the value does not match any enumerator.

11.155 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ utilities/enum_tools.hpp File Reference

Utilities for converting enums to string representations. Also defines the [LogLevel](#) enum type used elsewhere in the codebase.

Enumerations

- enum [LogLevel](#) {
 Trace , **Debug** , **Error** , **Warn** ,
 Info }
 Log severity levels for the [Logger](#) system.

Functions

- template<typename E>
 auto [EnumToString](#) (E) noexcept -> const char *
 Generic template to convert an enum to a string.
- template<> auto [EnumToString](#) ([LogLevel](#) enum_) noexcept -> const char *
 Specialization of EnumToString for [LogLevel](#).

11.155.1 Detailed Description

Utilities for converting enums to string representations. Also defines the [LogLevel](#) enum type used elsewhere in the codebase.

11.155.2 Enumeration Type Documentation

11.155.2.1 LogLevel

```
enum LogLevel
```

Log severity levels for the [Logger](#) system.

Note

Declaration order does not represent a severity ranking (e.g. Error precedes Warn), so do not compare [LogLevel](#) values with relational operators.

11.155.3 Function Documentation

11.155.3.1 EnumToString() [1/2]

```
template<typename E>
auto EnumToString (
    E ) -> const char * [inline], [noexcept]
```

Generic template to convert an enum to a string.

Note

This should be specialized for every enum type used.

Template Parameters

<i>E</i>	The enum type.
----------	----------------

Parameters

<i>enum</i> ↔ —	The enum value to convert.
--------------------	----------------------------

Returns

const char* A string representation of the enum value.

11.155.3.2 EnumToString() [2/2]

```
template<>
auto EnumToString (
    LogLevel enum_ ) -> const char * [inline], [noexcept]
```

Specialization of EnumToString for [LogLevel](#).

Parameters

<i>enum</i> ↔ —	The LogLevel value to convert.
--------------------	--

Returns

const char* String constant representing the log level.

11.156 enum_tools.hpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00016 template<typename E>
00017 inline auto EnumToString(E) noexcept -> const char* {
00018     return "UNKNOWN ENUM";
00019 };
00020
00026 enum LogLevel
00027 {
00028     Trace,
00029     Debug,
00030     Error,
00031     Warn,
00032     Info
00033 };
00034
00040 template<>
00041 inline auto EnumToString(LogLevel enum_) noexcept -> const char*
00042 {
00043     const char* final_enum = [&]() {
00044         {
00045             switch(enum_)
00046             {
00047                 case Trace : return "Trace";
00048                 case Debug : return "Debug";
00049                 case Error : return "Error";
00050                 case Warn:  return "Warn";
00051                 case Info:  return "Info";
00052                 default:    return "UNKNOWN ENUM TO STRING CALL";
00053             }
00054         }();
00055     return final_enum;
00056 }
00057

```

11.157 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ utilities/Logger.cpp File Reference

Implementation of the [Logger](#) and RAII helper classes.

```

#include "Logger.hpp"
#include "utilities/enum_tools.hpp"

```

11.157.1 Detailed Description

Implementation of the [Logger](#) and RAII helper classes.

11.158 /Users/lukas/Desktop/mindstorms-party/desktop-app/src/↵ utilities/Logger.hpp File Reference

Thread-safe asynchronous logging system.

```

#include <atomic>
#include <chrono>
#include <condition_variable>

```

```

#include <mutex>
#include <queue>
#include <sstream>
#include <string>
#include <thread>
#include <ctime>
#include <type_traits>
#include <unordered_set>
#include <fstream>
#include <iostream>
#include "enum_tools.hpp"

```

Classes

- class [Logger](#)
Singleton that owns a background thread which asynchronously drains queued log messages.
- struct [LogStream](#)
RAII helper that accumulates a log message via operator<< and forwards the accumulated text to the [Logger](#) when it goes out of scope.
- struct [LogEntryExit](#)
RAII helper that records a start timestamp on construction and emits a Trace-level "EXIT - <seconds>s" log message (via a temporary [LogStream](#)) on destruction.

Macros

- #define **CURRENT_FUNCTION** __PRETTY_FUNCTION__
*Expands to the compiler's function-signature macro: **FUNCSIG** on MSVC, **PrettyFunction** elsewhere.*
- #define [LOG\(LogLevel\)](#)
Macro to initiate a log message, e.g. [LOG\(LogLevel::Info\)](#) << "message";.
- #define [SET_LOG_LEVELS\(...\)](#)
Configures the active logging filters by specifying which severity levels are enabled.
- #define [LOG_ENTRY_EXIT](#)
Macro to trace function entry and duration.
- #define [LOG_ENTRY_EXIT_THROTTLED](#)(ms_interval)
Throttled macro to trace function entry and duration with rate limiting.
- #define [EXPECT_NEQ](#)(action, expected, message)
Asserts that action is not equal to expected; logs an Error-level message if it is (i.e. the assertion is violated).
- #define [EXPECT_EQ](#)(action, expected, message)
Asserts that action is equal to expected; logs an Error-level message if it is not (i.e. the assertion is violated).

11.158.1 Detailed Description

Thread-safe asynchronous logging system.

- Provides a singleton [Logger](#) instance, streaming log interfaces, and RAII-based function entry/exit tracking.

11.158.2 Macro Definition Documentation

11.158.2.1 EXPECT_EQ

```
#define EXPECT_EQ(  
    action,  
    expected,  
    message)
```

Value:

```
do {  
    if ((action) != (expected)) {  
        LOG(LogLevel::Error) « message;  
    }  
} while (0)
```

\\
\\
\\

Asserts that *action* is equal to *expected*; logs an Error-level message if it is not (i.e. the assertion is violated).

Parameters

<i>action</i>	Expression to check (evaluated once).
<i>expected</i>	Value that <i>action</i> is expected to equal.
<i>message</i>	Text streamed via LOG(LogLevel::Error) when the check fails.

11.158.2.2 EXPECT_NEQ

```
#define EXPECT_NEQ(  
    action,  
    expected,  
    message)
```

Value:

```
do {  
    if ((action) == (expected)) {  
        LOG(LogLevel::Error) « message;  
    }  
} while (0)
```

\\
\\
\\

Asserts that *action* is not equal to *expected*; logs an Error-level message if it is (i.e. the assertion is violated).

Parameters

<i>action</i>	Expression to check (evaluated once).
<i>expected</i>	Value that <i>action</i> is expected to differ from.
<i>message</i>	Text streamed via LOG(LogLevel::Error) when the check fails.

11.158.2.3 LOG

```
#define LOG(  
    LogLevel)
```

Value:

```
LogStream(LogLevel, std::this_thread::get_id(), CURRENT_FUNCTION)
```

Macro to initiate a log message, e.g. `LOG(LogLevel::Info) << "message";`.

Parameters

<i>LogLevel</i>	The severity level of the log.
-----------------	--------------------------------

Note

The severity value is currently stored on the resulting `LogStream` but does not appear in the emitted text (see `LogStream`'s class-level note).

11.158.2.4 LOG_ENTRY_EXIT

```
#define LOG_ENTRY_EXIT
```

Value:

```
do { \
    LogStream(LogLevel::Trace, std::this_thread::get_id(), CURRENT_FUNCTION) << "ENTRY"; \
} while(0); \
LogEntryExit instance_##__LINE__(CURRENT_FUNCTION);
```

Macro to trace function entry and duration.

Immediately logs a Trace-level "ENTRY" message, then declares a local `LogEntryExit` object whose destructor logs the elapsed time when the enclosing scope ends.

11.158.2.5 LOG_ENTRY_EXIT_THROTTLED

```
#define LOG_ENTRY_EXIT_THROTTLED(  
    ms_interval)
```

Value:

```
static std::chrono::steady_clock::time_point s_last_log_time{}; \
auto _logger_now = std::chrono::steady_clock::now(); \
bool _logger_should_log = std::chrono::duration_cast<std::chrono::milliseconds>(_logger_now - \
    s_last_log_time).count() >= (ms_interval); \
if (_logger_should_log) s_last_log_time = _logger_now; \
LogEntryExit _scoped_log(__func__, _logger_should_log)
```

Throttled macro to trace function entry and duration with rate limiting.

Operates similarly to `LOG_ENTRY_EXIT`, but uses a static timer to suppress logs if they occur more frequently than the specified interval.

Parameters

<i>ms_interval</i>	The minimum time (in milliseconds) that must pass between consecutive log entries for this specific function scope.
--------------------	---

11.158.2.6 SET_LOG_LEVELS

```
#define SET_LOG_LEVELS(
    ...)
```

Value:

```
Logger::getInstance().setLoggingLevel(__VA_ARGS__)
```

Configures the active logging filters by specifying which severity levels are enabled.

This variadic macro accepts an arbitrary number of `LogLevel` arguments. It clears any previously configured filters and registers the new set globally. Any subsequent log messages whose level is not included in this set will be silently dropped.

-

Parameters

...	A comma-separated list of <code>LogLevel</code> enums to enable (e.g., <code>LogLevel::Info</code> , <code>LogLevel::Error</code>).
-----	--

Note

- This provides a clean interface to the underlying variadic template function on the `Logger` singleton.

See also

[Logger::setLoggingLevel](#)

11.159 Logger.hpp

[Go to the documentation of this file.](#)

```
00001
00007
00008 #pragma once
00009
00010 #if defined(_MSC_VER) && !defined(__PRETTY_FUNCTION__)
00011 #define __PRETTY_FUNCTION__ __FUNCSIG__
00012 #endif
00013
00014 #include <atomic>
00015 #include <chrono>
00016 #include <condition_variable>
00017 #include <mutex>
00018 #include <queue>
00019 #include <sstream>
00020 #include <string>
00021 #include <thread>
00022 #include <ctime>
00023 #include <type_traits>
00024 #include <unordered_set>
00025 #include <fstream>
00026 #include <iostream>
00027 #include <mutex>
00028 #include <thread>
00029 #include "enum_tools.hpp"
00030
```

```

00039 class Logger
00040 {
00041 private:
00042     std::queue<std::string> queue_{};
00043     mutable std::mutex m_{};
00044     std::condition_variable cv_;
00045     std::atomic<bool> stop_{false};
00046     std::thread thread_;
00047     std::unordered_set<LogLevel> logged_levels_{};
00048
00053     Logger();
00054
00064     void log();
00065
00066 public:
00067     std::atomic<bool> is_logging_enabled_{true};
00068     Logger(const Logger&) = delete;
00069     Logger& operator=(const Logger&) = delete;
00070     Logger(Logger&&) = delete;
00071     Logger& operator=(Logger&&) = delete;
00072
00077     inline static Logger& getInstance()
00078     {
00079         static Logger instance;
00080         return instance;
00081     }
00082
00094     template<typename... Levels>
00095     requires (std::is_same_v<Levels, LogLevel> && ...)
00096     void setLoggingLevel(Levels... to_log)
00097     {
00098         std::lock_guard<std::mutex> lk(m_);
00099         logged_levels_.clear();
00100         (logged_levels_.insert(to_log), ...);
00101     }
00102
00112     bool isLevelEnabled(LogLevel level) const ;
00113
00114
00120     void push(std::string&& msg);
00121
00125     ~Logger();
00126 };
00127
00135 struct LogStream
00136 {
00137 private:
00138     LogLevel log_level_{};
00139     const char* function_name_{};
00140     std::thread::id thread_id_{};
00141     std::stringstream stream_{};
00142
00143
00144 public:
00149     std::stringstream& getStream();
00150
00157     LogStream(LogLevel log_level, std::thread::id thread_id, const char* function_name);
00158
00162     ~LogStream();
00163
00170     template <typename T>
00171     LogStream& operator<<(const T& value)
00172     {
00173         stream_ << value;
00174         return *this;
00175     }
00176 };
00177
00183 struct LogEntryExit
00184 {
00185 private:
00186     const char* function_name_ = nullptr;
00187     std::chrono::time_point<std::chrono::high_resolution_clock> t0;
00188     bool should_log_{true};
00189
00190 public:
00195     explicit LogEntryExit(const char* function_name, bool should_log = true);
00199     ~LogEntryExit();
00200 };
00201
00206 #if defined(_WIN32) || defined(_WIN64)
00207 #define CURRENT_FUNCTION __FUNCSIG__
00208 #else
00209 #define CURRENT_FUNCTION __PRETTY_FUNCTION__
00210 #endif
00211
00218 #define LOG(LogLevel) LogStream(LogLevel, std::this_thread::get_id(), CURRENT_FUNCTION)

```

```

00219
00229 #define SET_LOG_LEVELS(...) Logger::getInstance().setLoggingLevel(__VA_ARGS__)
00230
00237 #define LOG_ENTRY_EXIT \
00238     do { \
00239         LogStream(LogLevel::Trace, std::this_thread::get_id(), CURRENT_FUNCTION) « "ENTRY"; \
00240     } while(0); \
00241     LogEntryExit instance_##__LINE__(CURRENT_FUNCTION);
00242
00250 #define LOG_ENTRY_EXIT_THROTTLED(ms_interval) \
00251     static std::chrono::steady_clock::time_point s_last_log_time{}; \
00252     auto _logger_now = std::chrono::steady_clock::now(); \
00253     bool _logger_should_log = std::chrono::duration_cast<std::chrono::milliseconds>(_logger_now -
s_last_log_time).count() >= (ms_interval); \
00254     if (_logger_should_log) s_last_log_time = _logger_now; \
00255     LogEntryExit _scoped_log(__func__, _logger_should_log)
00263 #define EXPECT_NEQ(action, expected, message) \
00264     do { \
00265         if ((action) == (expected)) { \
00266             LOG(LogLevel::Error) « message; \
00267         } \
00268     } while (0)
00269
00277 #define EXPECT_EQ(action, expected, message) \
00278     do { \
00279         if ((action) != (expected)) { \
00280             LOG(LogLevel::Error) « message; \
00281         } \
00282     } while (0)
00283

```

Index

/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/latency_indicator.cpp, [185](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/latency_indicator.hpp, [185](#), [186](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/main.cpp, [186](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/app/mock_camera.cpp, [187](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/camera_calibration.cpp, [188](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/camera/camera_calibration.hpp, [188](#), [189](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.cpp, [190](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/collision_planner.hpp, [191](#), [193](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/path_update.hpp, [196](#), [197](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.cpp, [197](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/collision_avoidance/playboard_graph.hpp, [198](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/TCP_Server.hpp, [208](#), [209](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.cpp, [210](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/UDP_Server.hpp, [211](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/handler_template.cpp, [199](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/handler_template.hpp, [200](#), [202](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.cpp, [202](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/message_processing.hpp, [203](#), [205](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/network_messages.hpp, [206](#), [207](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/communication/session_registry.hpp, [207](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus.cpp, [212](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus.hpp, [213](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.cpp, [214](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/bonus_json.hpp, [215](#), [216](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.cpp, [216](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game.hpp, [216](#), [217](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.cpp, [217](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/game_manager.hpp, [218](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/lobby.cpp, [219](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/network_utils.hpp, [219](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/start_game.hpp, [220](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.cpp, [220](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/gameplay/turnManager.hpp, [221](#), [223](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.cpp, [224](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/item.hpp, [224](#), [225](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.cpp, [226](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/items/shop.hpp, [227](#), [228](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/mainpage.md, [228](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.cpp, [229](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/board.hpp, [230](#), [231](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.cpp, [232](#)
/Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/follower_tuning_panel.hpp, [233](#)

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/mapwindow.cpp, 234
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/mapwindow.hpp, 234, 235
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.cpp, 238
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/rng.hpp, 239
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/roulette.cpp, 240
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/map/roulette.hpp, 241, 242
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games Directory Reference, 27
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame.hpp, 243, 244
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_participant.hpp, 244
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/minigame_result.hpp, 245
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatton.cpp, 245
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatton.hpp, 246
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatton_grid.cpp, 248
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/splatton_grid.hpp, 248, 249
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/udp_reciever.hpp, 249, 250
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mini_games/udp_sender.hpp, 251, 252
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.cpp, 253
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/mqtt_client/client.hpp, 253, 254
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.cpp, 254
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/path_follower/path_follower.hpp, 255, 257
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/gridmap.cpp, 258
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/gridmap.hpp, 258, 259
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.cpp, 259
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/pathfinding/path_planner.hpp, 260
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/boardEffects.cpp, 261
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/boardEffects.hpp, 262
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.cpp, 263
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/dice.hpp, 263
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.cpp, 264
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/inventory.hpp, 264, 265
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.cpp, 265
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/player/player.hpp, 266, 267
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codecs.cpp, 269
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/protocol/codecs.hpp, 274, 280
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/background_thread.hpp, 281
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.cpp, 282
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/control_window.hpp, 283
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_dashboard.cpp, 284
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_dashboard.hpp, 284, 285
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.cpp, 285
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/fleet_panel.hpp, 286
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.cpp, 287
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/main_window.hpp, 287, 288
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.cpp, 289
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_monitor.hpp, 290
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.cpp, 291
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/network_topology_widget.hpp, 291, 293
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/path_debug.cpp, 300
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/path_debug.hpp, 300, 301
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.cpp, 301
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/shop_test_window.hpp, 302
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.cpp, 303

- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/qtpanels/test_panel.hpp, 303, 304
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.cpp, 304
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_controller.hpp, 305
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.cpp, 307
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_controller/robot_fleet.hpp, 307, 308
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.cpp, 308
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/robot_tracker/tracker.hpp, 309, 310
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/udp_link.cpp, 311
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/udp_link/udp_link.hpp, 311, 312
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/unity/mapLogic.cpp, 312
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/Logger.cpp, 316
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/Logger.hpp, 316, 320
- /Users/lukas/Desktop/mindstorms-party/desktop-app/src/utilities/enum_tools.hpp, 314, 316
- ~LogStream
 - LogStream, 96
- ~MapWindow
 - MapWindow, 100
- ~client
 - mind::mqtt::client, 52
- ~control_window
 - control_window, 61
- ack
 - codec.hpp, 276
- addCoins
 - Player, 134
- addEffect
 - Player, 134
- addItem
 - Inventory, 85
 - Player, 135
- addNeighbor
 - Node, 121
- addNode
 - Board, 42
- addObstacle
 - pathfinding::GridMap, 81
- applyEndOfTurn
 - Effect, 65
- ApplyingTile
 - TurnManager, 171
- applyItemEffects
 - Item, 88
- applyStartOfTurn
 - Effect, 65
- applyTile
 - TurnManager, 171
- applyTileEffect
 - boardEffects.cpp, 261
 - boardEffects.hpp, 262
- ArUcoWorker, 40
- avoiding
 - path_update.hpp, 197
- back_px
 - marker_tracker::marker_position, 106
- beep
 - codec.hpp, 276
 - mind::fleet::robot_controller, 143
- blockBorder
 - pathfinding::GridMap, 81
- Board, 41
 - addNode, 42
 - Board, 41
 - broadcast_board, 42
 - connectNodes, 42
 - getNode, 42
 - getNodes, 43
 - isValidNode, 43
 - tileTypeToString, 43
- board.cpp
 - register_handlers_board, 230
- board.hpp
 - register_handlers_board, 231
- boardClicked
 - MapWindow, 101
- boardEffects.cpp
 - applyTileEffect, 261
- boardEffects.hpp
 - applyTileEffect, 262
- BoardLoader, 44
- BoardLoader.BoardData, 44
- BoardLoader.EffectData, 65
- BoardLoader.NodeData, 123
- bonus_json.cpp
 - broadcastGameResults, 214
 - makeGameResultsMessage, 214
- bonus_json.hpp
 - broadcastGameResults, 215
 - makeGameResultsMessage, 215
- BonusAward, 45
- BonusPhase, 45
 - execute, 46
- BonusResult, 46
- broadcast_board
 - Board, 42
- broadcast_json
 - Server, 153
- broadcast_to_all
 - Server, 153
- UdpServer, 183
- broadcast_update

- handler_template.cpp, 199
 - handler_template.hpp, 201
- broadcast_update_roulette
 - roulette.cpp, 240
 - roulette.hpp, 242
- broadcastGameResults
 - bonus_json.cpp, 214
 - bonus_json.hpp, 215
- calibrate_extrinsics
 - CameraCalibration, 48
- CameraCalibration, 47
 - calibrate_extrinsics, 48
 - CameraCalibration, 48
 - frame_ready, 48
 - get_rvec, 49
 - get_tvec, 49
 - is_calibrated, 49
 - set_intrinsics, 49
 - start_camera, 50
- canAfford
 - Player, 135
- cells
 - SplattonGrid, 161
- changed
 - follower_tuning_panel, 71
- checkNegativeEffects
 - Player, 135
- clear
 - SplattonGrid, 161
- clear_path
 - collision::collision_planner, 56
- client
 - mind::mqtt::client, 51
- codec.cpp
 - decode_ack, 270
 - decode_heartbeat, 270
 - encode_beep, 270
 - encode_drive, 271
 - encode_drive_for, 271
 - encode_ping, 272
 - encode_stop, 272
 - encode_turn, 272
 - encode_wheel_drive, 273
 - peek_status_type, 273
- codec.hpp
 - ack, 276
 - beep, 276
 - decode_ack, 276
 - decode_heartbeat, 277
 - drive, 276
 - drive_for, 276
 - driving, 276
 - encode_beep, 277
 - encode_drive, 277
 - encode_drive_for, 278
 - encode_ping, 278
 - encode_stop, 279
 - encode_turn, 279
 - encode_wheel_drive, 279
 - error, 276
 - heartbeat, 276
 - idle, 276
 - moving, 276
 - opcode, 275
 - peek_status_type, 280
 - ping, 276
 - robot_state, 276
 - status_type, 276
 - stop, 276
 - turn, 276
 - wheel_drive, 276
- collision::collision_planner, 54
 - clear_path, 56
 - collision_planner, 56
 - debug_obstacle_grid, 56
 - effective_path, 56
 - forget, 57
 - is_yielding, 57
 - nominal_path, 57
 - request_path_to, 57
 - request_route_to, 58
 - route_nodes, 58
 - set_graph, 59
 - set_path_sink, 59
 - set_priority_rule, 59
 - shove_blocker, 59
 - update, 60
 - watched_pairs, 60
- collision::graph_node, 79
- collision::path_update, 126
 - waypoints, 126
- collision::playboard_graph, 128
 - min_edge_distance, 129
 - min_node_distance, 129
 - nearest_node, 130
 - node, 130
 - route, 130
 - set_nodes, 131
- collision::robot_pose, 149
- collision_planner
 - collision::collision_planner, 56
- collision_planner.cpp
 - default_priority, 190
- collision_planner.hpp
 - default_priority, 193
 - no_route, 192
 - no_spot, 193
 - priority_rule, 192
 - route_result, 192
 - routed, 192
 - shove_result, 192
 - shoved, 193
 - unknown_pose, 192, 193
 - unreachable, 193
 - waiting_for_blocker, 192
- collisionPlanner

- MapWindow, 101
- connect_to
 - mind::mqtt::client, 52
- connect_to_server
 - TcpConnection, 165
- connectNodes
 - Board, 42
- control_window, 61
 - ~control_window, 61
 - control_window, 61
- controller
 - mind::fleet::robot_fleet, 148
- countValue
 - SplatoonGrid, 161
- currentPlayer
 - TurnManager, 171
- datagram_received
 - mind::net::udp_link, 177
- debug_obstacle_grid
 - collision::collision_planner, 56
- decode_ack
 - codec.cpp, 270
 - codec.hpp, 276
- decode_heartbeat
 - codec.cpp, 270
 - codec.hpp, 277
- default_priority
 - collision_planner.cpp, 190
 - collision_planner.hpp, 193
- detect_markers
 - marker_tracker::marker_tracker, 107
- determineTurnOrder
 - Game, 73
- Dice, 62
 - throwDice, 62
- disconnect_from_broker
 - mind::mqtt::client, 52
- drive
 - codec.hpp, 276
- drive_for
 - codec.hpp, 276
 - mind::fleet::robot_controller, 143
- driving
 - codec.hpp, 276
- Effect, 64
 - applyEndOfTurn, 65
 - applyStartOfTurn, 65
 - Effect, 64
- effective_path
 - collision::collision_planner, 56
- encode_beep
 - codec.cpp, 270
 - codec.hpp, 277
- encode_drive
 - codec.cpp, 271
 - codec.hpp, 277
- encode_drive_for
 - codec.cpp, 271
 - codec.hpp, 278
- encode_ping
 - codec.cpp, 272
 - codec.hpp, 278
- encode_stop
 - codec.cpp, 272
 - codec.hpp, 279
- encode_turn
 - codec.cpp, 272
 - codec.hpp, 279
- encode_wheel_drive
 - codec.cpp, 273
 - codec.hpp, 279
- EndGame
 - TurnManager, 171
- EndRound
 - TurnManager, 171
- EndTurn
 - TurnManager, 171
- enum_tools.hpp
 - EnumToString, 315
 - LogLevel, 314
- EnumToString
 - enum_tools.hpp, 315
- error
 - codec.hpp, 276
- execute
 - BonusPhase, 46
- executeSelectedItem
 - TurnManager, 172
- EXPECT_EQ
 - Logger.hpp, 318
- EXPECT_NEQ
 - Logger.hpp, 318
- find_camera_index_by_name
 - marker_tracker::marker_tracker, 107
- finishRound
 - TurnManager, 172
- finishTurn
 - TurnManager, 172
- flash
 - latency_indicator, 91
- fleet_panel, 66
 - fleet_panel, 67
- FleetDashboard, 67
- follow_state
 - path_update.hpp, 197
- follower::drive_cmd, 63
- follower::drive_pct, 63
- follower::follower_params, 68
 - min_speed_pct, 69
 - min_turn_pct, 69
 - rotate_exit_deg, 69
- follower::path_follower, 124
 - path_follower, 125
 - reset, 125
 - step, 125

- follower::pose2d, [139](#)
- follower_tuning_panel, [69](#)
 - changed, [71](#)
 - follower_tuning_panel, [70](#)
 - params, [71](#)
 - setParams, [71](#)
- forget
 - collision::collision_planner, [57](#)
- frame_ready
 - CameraCalibration, [48](#)
- Game, [72](#)
 - determineTurnOrder, [73](#)
 - Game, [73](#)
 - getCurrentPlayer, [73](#)
 - isLastPlayer, [73](#)
 - nextRound, [73](#)
- GameManager, [74](#)
 - GameManager, [75](#)
 - notifyRobotPosition, [75](#)
 - runMiniGame, [75](#)
 - syncPlayers, [76](#)
- Generator, [76](#)
 - get_last_number, [77](#)
 - m_numbers, [78](#)
 - random_number, [77](#)
 - roll_ball, [77](#)
- get
 - SplattonGrid, [161](#)
- get_camera_index
 - marker_tracker::marker_tracker, [108](#)
- get_camera_matrix
 - marker_tracker::marker_tracker, [108](#)
- get_current_frame
 - marker_tracker::marker_tracker, [108](#)
- get_dist_coeffs
 - marker_tracker::marker_tracker, [108](#)
- get_last_number
 - Generator, [77](#)
- get_latest_frame
 - marker_tracker::marker_tracker, [109](#)
- get_rvec
 - CameraCalibration, [49](#)
- get_tvec
 - CameraCalibration, [49](#)
- get_valid_robot_marker_ids
 - marker_tracker::marker_tracker, [109](#)
- getCurrentPlayer
 - Game, [73](#)
 - Player, [135](#)
- getCurrentPlayerID
 - Player, [136](#)
- getDiceOverride
 - Item, [88](#)
- getId
 - Player, [136](#)
- getInstance
 - Logger, [93](#)
- getItem
 - Inventory, [85](#)
- getItems
 - Inventory, [85](#)
- getName
 - MiniGame, [113](#)
 - RobotGame::Splatton, [159](#)
- getNode
 - Board, [42](#)
- getNodes
 - Board, [43](#)
- getRobotNode
 - MapWindow, [101](#)
- getSize
 - Inventory, [85](#)
- getState
 - TurnManager, [172](#)
- getStream
 - LogStream, [96](#)
- getWorldPosition
 - Player, [136](#)
- GridMap
 - pathfinding::GridMap, [80](#)
- gridToWorld
 - pathfinding::GridMap, [81](#)
- handle
 - MessageTarget, [112](#)
- handle_status
 - mind::fleet::robot_controller, [143](#)
- handler_template.cpp
 - broadcast_update, [199](#)
 - register_handlers, [200](#)
- handler_template.hpp
 - broadcast_update, [201](#)
 - register_handlers, [201](#)
- hasItem
 - Inventory, [85](#)
- hasLineOfSight
 - pathfinding::GridMap, [82](#)
- heartbeat
 - codec.hpp, [276](#)
- holding
 - path_update.hpp, [197](#)
- idle
 - codec.hpp, [276](#)
- initialize_camera
 - marker_tracker::marker_tracker, [109](#)
- initialize_camera_auto
 - marker_tracker::marker_tracker, [110](#)
- initializeCamera
 - MapWindow, [101](#)
- Inventory, [84](#)
 - addItem, [85](#)
 - getItem, [85](#)
 - getItems, [85](#)
 - getSize, [85](#)
 - hasItem, [85](#)
 - isFull, [86](#)

- removeItem, 86
- is_calibrated
 - CameraCalibration, 49
- is_yielding
 - collision::collision_planner, 57
- isDiceItem
 - Item, 88
- isFull
 - Inventory, 86
- isInside
 - pathfinding::GridMap, 82
- isLastPlayer
 - Game, 73
- isLevelEnabled
 - Logger, 93
- isValidNode
 - Board, 43
- isWalkable
 - pathfinding::GridMap, 82
- Item, 87
 - applyItemEffects, 88
 - getDiceOverride, 88
 - isDiceItem, 88
 - Item, 87
 - requiresTarget, 88
 - rollSingle, 89
 - rollSum, 89
- ItemData, 89
- latency_indicator, 90
 - flash, 91
 - latency_indicator, 91
- load_calibration
 - marker_tracker::marker_tracker, 110
- LOG
 - Logger.hpp, 318
- LOG_ENTRY_EXIT
 - Logger.hpp, 319
- LOG_ENTRY_EXIT_THROTTLED
 - Logger.hpp, 319
- LogEntryExit, 91
 - LogEntryExit, 92
- Logger, 92
 - getInstance, 93
 - isLevelEnabled, 93
 - push, 93
 - setLoggingLevel, 94
- Logger.hpp
 - EXPECT_EQ, 318
 - EXPECT_NEQ, 318
 - LOG, 318
 - LOG_ENTRY_EXIT, 319
 - LOG_ENTRY_EXIT_THROTTLED, 319
 - SET_LOG_LEVELS, 320
- LogLevel
 - enum_tools.hpp, 314
- LogStream, 95
 - ~LogStream, 96
 - getStream, 96
- LogStream, 95
 - operator<<, 96
- m_numbers
 - Generator, 78
- main
 - mapLogic.cpp, 313
 - mock_camera.cpp, 188
- main_window, 97
 - main_window, 98
 - set_fleet, 98
- mainpage, 1
- makeGameResultsMessage
 - bonus_json.cpp, 214
 - bonus_json.hpp, 215
- mapLogic.cpp
 - main, 313
 - nodeTypeToString, 313
- MapWindow, 98
 - ~MapWindow, 100
 - boardClicked, 101
 - collisionPlanner, 101
 - getRobotNode, 101
 - initializeCamera, 101
 - MapWindow, 100
 - movePlayboard, 102
 - paintBoard, 102
 - planPathFor, 102
 - robotArrived, 103
 - sendRobotToNode, 103
 - setFleet, 103
 - setTrackingMarkers, 104
 - setTrackingState, 104
 - startCalibration, 104
 - updateRobotPosition, 104
- marker_tracker
 - marker_tracker::marker_tracker, 107
- marker_tracker::marker_position, 105
 - back_px, 106
- marker_tracker::marker_tracker, 106
 - detect_markers, 107
 - find_camera_index_by_name, 107
 - get_camera_index, 108
 - get_camera_matrix, 108
 - get_current_frame, 108
 - get_dist_coeffs, 108
 - get_latest_frame, 109
 - get_valid_robot_marker_ids, 109
 - initialize_camera, 109
 - initialize_camera_auto, 110
 - load_calibration, 110
 - marker_tracker, 107
 - reinitialize, 111
 - robot_slot_for_marker, 111
 - shutdown_camera, 111
- message_handler
 - message_processing.hpp, 204
- message_processing.cpp
 - process, 203

- register_message_handler, 203
- message_processing.hpp
 - message_handler, 204
 - process, 205
 - register_message_handler, 205
- message_received
 - mind::mqtt::client, 52
- MessageTarget, 112
 - handle, 112
- min_edge_distance
 - collision::playboard_graph, 129
- min_node_distance
 - collision::playboard_graph, 129
- min_speed_pct
 - follower::follower_params, 69
- min_turn_pct
 - follower::follower_params, 69
- mind::fleet::robot_controller, 141
 - beep, 143
 - drive_for, 143
 - handle_status, 143
 - move_relative, 144
 - on_pose_updated, 144
 - robot_controller, 142
 - set_brick_address, 145
 - set_drive, 145
 - set_wheel_drive, 145
 - stop, 146
 - turn, 146
- mind::fleet::robot_fleet, 147
 - controller, 148
 - robot_appeared, 148
 - robot_disappeared, 148
 - robot_fleet, 148
 - robot_ip_discovered, 149
 - select, 149
- mind::mqtt::client, 50
 - ~client, 52
 - client, 51
 - connect_to, 52
 - disconnect_from_broker, 52
 - message_received, 52
 - publish, 53
 - subscribe, 53
- mind::net::udp_link, 176
 - datagram_received, 177
 - send_command, 177
 - start, 177
 - udp_link, 177
- mind::proto::ack_msg, 39
- mind::proto::heartbeat_msg, 83
- MiniGame, 113
 - getName, 113
 - onRobotMoved, 113
 - play, 114
- Minigame
 - TurnManager, 171
- MiniGameParticipant, 114
- MiniGameResult, 115
- mock_camera.cpp
 - main, 188
- MockCamera, 116
 - MockCamera, 116
 - update, 117
- move_relative
 - mind::fleet::robot_controller, 144
- movePlayboard
 - MapWindow, 102
- movePlayer
 - TurnManager, 172
- Moving
 - TurnManager, 171
- moving
 - codec.hpp, 276
- MqttReceiver, 117
- MqttReceiver.AnchorData, 40
- MqttReceiver.RobotPositionData, 150
- nearest_node
 - collision::playboard_graph, 130
- NetworkMonitor, 118
- NetworkTopologyWidget, 118
- NetworkTopologyWidget::PairingSlot, 123
- nextRound
 - Game, 73
- no_route
 - collision_planner.hpp, 192
- no_spot
 - collision_planner.hpp, 193
- Node, 119
 - addNeighbor, 121
 - Node, 120
- node
 - collision::playboard_graph, 130
- nodeTypeToString
 - mapLogic.cpp, 313
- nominal
 - path_update.hpp, 197
- nominal_path
 - collision::collision_planner, 57
- notifyRobotPosition
 - GameManager, 75
- ns, 37
- ns::CloseShopEvent, 54
- ns::EndRouletteEvent, 66
- ns::GetPlayerBetEvent, 78
- ns::OpenShopEvent, 123
- ns::SelectedItemEvent, 151
- ns::StartRouletteEvent, 163
- on_pose_updated
 - mind::fleet::robot_controller, 144
- onRobotMoved
 - MiniGame, 113
 - RobotGame::Splatoon, 159
- opcode
 - codec.hpp, 275

- operator<<
 - LogStream, 96
- paintBoard
 - MapWindow, 102
- params
 - follower_tuning_panel, 71
- path_follower
 - follower::path_follower, 125
- path_follower.cpp
 - to_percent, 255
- path_follower.hpp
 - to_percent, 256
- path_update.hpp
 - avoiding, 197
 - follow_state, 197
 - holding, 197
 - nominal, 197
- PathDebug, 126
- pathfinding::GridMap, 79
 - addObstacle, 81
 - blockBorder, 81
 - GridMap, 80
 - gridToWorld, 81
 - hasLineOfSight, 82
 - isInside, 82
 - isWalkable, 82
 - worldToGrid, 83
- pathfinding::Node, 121
- pathfinding::NodeCompare, 122
- pathfinding::PathPlanner, 127
 - plan, 128
- peek_status_type
 - codec.cpp, 273
 - codec.hpp, 280
- ping
 - codec.hpp, 276
- plan
 - pathfinding::PathPlanner, 128
- planPathFor
 - MapWindow, 102
- play
 - MiniGame, 114
 - RobotGame::Splatoon, 159
- Player, 131
 - addCoins, 134
 - addEffect, 134
 - addItem, 135
 - canAfford, 135
 - checkNegativeEffects, 135
 - getCurrentPlayer, 135
 - getCurrentPlayerID, 136
 - getId, 136
 - getWorldPosition, 136
 - Player, 134
 - setCurrentNode, 136
 - setDiceMultiplier, 137
 - setWorldPosition, 137
 - startTurn, 137
 - stealDiceSteps, 137
 - swapPosition, 138
- PlayerSession, 138
- PlayerStanding, 139
- PlayerStats, 139
- priority_rule
 - collision_planner.hpp, 192
- process
 - message_processing.cpp, 203
 - message_processing.hpp, 205
- publish
 - mind::mqtt::client, 53
- push
 - Logger, 93
- random_number
 - Generator, 77
- register_handlers
 - handler_template.cpp, 200
 - handler_template.hpp, 201
- register_handlers_board
 - board.cpp, 230
 - board.hpp, 231
- register_handlers_roulette
 - roulette.cpp, 240
 - roulette.hpp, 242
- register_handlers_turnmanager
 - turnManager.cpp, 221
 - turnManager.hpp, 222
- register_message_handler
 - message_processing.cpp, 203
 - message_processing.hpp, 205
- reinitialize
 - marker_tracker::marker_tracker, 111
- removeItem
 - Inventory, 86
- request_path_to
 - collision::collision_planner, 57
- request_route_to
 - collision::collision_planner, 58
- requiresTarget
 - Item, 88
- reset
 - follower::path_follower, 125
- Robot, 140
- robot_appeared
 - mind::fleet::robot_fleet, 148
- robot_controller
 - mind::fleet::robot_controller, 142
- robot_disappeared
 - mind::fleet::robot_fleet, 148
- robot_fleet
 - mind::fleet::robot_fleet, 148
- robot_ip_discovered
 - mind::fleet::robot_fleet, 149
- robot_slot_for_marker
 - marker_tracker::marker_tracker, 111
- robot_state
 - codec.hpp, 276

- robotArrived
 - MapWindow, 103
- RobotGame::Splatoon, 157
 - getName, 159
 - onRobotMoved, 159
 - play, 159
 - Splatoon, 158
 - updateRobotPosition, 159
- roll_ball
 - Generator, 77
- rollDice
 - TurnManager, 173
- Rolling
 - TurnManager, 171
- rollSingle
 - Item, 89
- rollSum
 - Item, 89
- rotate_exit_deg
 - follower::follower_params, 69
- Roulette, 150
- roulette.cpp
 - broadcast_update_roulette, 240
 - register_handlers_roulette, 240
- roulette.hpp
 - broadcast_update_roulette, 242
 - register_handlers_roulette, 242
- route
 - collision::playboard_graph, 130
- route_nodes
 - collision::collision_planner, 58
- route_result
 - collision_planner.hpp, 192
- routed
 - collision_planner.hpp, 192
- runMiniGame
 - GameManager, 75
- select
 - mind::fleet::robot_fleet, 149
- selectItem
 - TurnManager, 173
- selectPath
 - TurnManager, 173
- selectTargetPlayer
 - TurnManager, 174
- send
 - UdpSender, 182
- send_command
 - mind::net::udp_link, 177
- send_to_server
 - TcpConnection, 165
- send_to_user
 - Server, 154
 - UdpServer, 183
- sendRobotToNode
 - MapWindow, 103
- Server, 152
 - broadcast_json, 153
 - broadcast_to_all, 153
 - send_to_user, 154
 - Server, 153
 - start_accept, 154
- SessionRegistry, 155
- set
 - SplatoonGrid, 162
- set_brick_address
 - mind::fleet::robot_controller, 145
- set_drive
 - mind::fleet::robot_controller, 145
- set_fleet
 - main_window, 98
- set_graph
 - collision::collision_planner, 59
- set_intrinsics
 - CameraCalibration, 49
- SET_LOG_LEVELS
 - Logger.hpp, 320
- set_nodes
 - collision::playboard_graph, 131
- set_path_sink
 - collision::collision_planner, 59
- set_priority_rule
 - collision::collision_planner, 59
- set_wheel_drive
 - mind::fleet::robot_controller, 145
- setCurrentNode
 - Player, 136
- setDiceMultiplier
 - Player, 137
- setFleet
 - MapWindow, 103
 - TurnManager, 174
- setLoggingLevel
 - Logger, 94
- setParams
 - follower_tuning_panel, 71
- setTrackingMarkers
 - MapWindow, 104
- setTrackingState
 - MapWindow, 104
- setWorldPosition
 - Player, 137
- Shop, 155
- ShopTestWindow, 156
 - ShopTestWindow, 157
- shove_blocker
 - collision::collision_planner, 59
- shove_result
 - collision_planner.hpp, 192
- shoved
 - collision_planner.hpp, 193
- shutdown_camera
 - marker_tracker::marker_tracker, 111
- Splatoon
 - RobotGame::Splatoon, 158
- SplatoonGrid, 160

- cells, 161
- clear, 161
- countValue, 161
- get, 161
- set, 162
- start
 - mind::net::udp_link, 177
- start_accept
 - Server, 154
- start_camera
 - CameraCalibration, 50
- start_game, 163
- start_read
 - TcpConnection, 165
- start_receive
 - UdpClient, 179
 - UdpServer, 184
- startCalibration
 - MapWindow, 104
- startMinigame
 - TurnManager, 175
- StartRound
 - TurnManager, 171
- StartTurn
 - TurnManager, 171
- startTurn
 - Player, 137
 - TurnManager, 175
- status_type
 - codec.hpp, 276
- stealDiceSteps
 - Player, 137
- step
 - follower::path_follower, 125
- stop
 - codec.hpp, 276
 - mind::fleet::robot_controller, 146
- subscribe
 - mind::mqtt::client, 53
- swapPosition
 - Player, 138
- syncPlayers
 - GameManager, 76
- TcpConnection, 163
 - connect_to_server, 165
 - send_to_server, 165
 - start_read, 165
 - TcpConnection, 164
- test_panel, 166
 - test_panel, 166
- throwDice
 - Dice, 62
- TileEffect, 167
- tileTypeToString
 - Board, 43
- to_percent
 - path_follower.cpp, 255
 - path_follower.hpp, 256
- Todo List, 3
- TopoCategory, 38
- TopoClient, 167
- TopoPulse, 168
- TrackedRobot, 168
- turn
 - codec.hpp, 276
 - mind::fleet::robot_controller, 146
- TurnManager, 169
 - ApplyingTile, 171
 - applyTile, 171
 - currentPlayer, 171
 - EndGame, 171
 - EndRound, 171
 - EndTurn, 171
 - executeSelectedItem, 172
 - finishRound, 172
 - finishTurn, 172
 - getState, 172
 - Minigame, 171
 - movePlayer, 172
 - Moving, 171
 - rollDice, 173
 - Rolling, 171
 - selectItem, 173
 - selectPath, 173
 - selectTargetPlayer, 174
 - setFleet, 174
 - startMinigame, 175
 - StartRound, 171
 - StartTurn, 171
 - startTurn, 175
 - TurnManager, 171
 - TurnState, 171
 - update, 175
 - WaitingForItemChoice, 171
 - WaitingForPathChoice, 171
 - WaitingForTargetChoice, 171
- turnManager.cpp
 - register_handlers_turnmanager, 221
- turnManager.hpp
 - register_handlers_turnmanager, 222
- TurnState
 - TurnManager, 171
- udp_link
 - mind::net::udp_link, 177
- UdpClient, 178
 - start_receive, 179
 - UdpClient, 179
- UdpReceiver, 179
 - UdpReceiver, 180
- UdpSender, 180
 - send, 182
 - UdpSender, 181
- UdpServer, 182
 - broadcast_to_all, 183
 - send_to_user, 183
 - start_receive, 184

- UdpServer, [183](#)
- unknown_pose
 - collision_planner.hpp, [192](#), [193](#)
- unreachable
 - collision_planner.hpp, [193](#)
- update
 - collision::collision_planner, [60](#)
 - MockCamera, [117](#)
 - TurnManager, [175](#)
- updateRobotPosition
 - MapWindow, [104](#)
 - RobotGame::Splatoon, [159](#)
- Vec3< T >, [184](#)
- waiting_for_blocker
 - collision_planner.hpp, [192](#)
- WaitingForItemChoice
 - TurnManager, [171](#)
- WaitingForPathChoice
 - TurnManager, [171](#)
- WaitingForTargetChoice
 - TurnManager, [171](#)
- watched_pairs
 - collision::collision_planner, [60](#)
- waypoints
 - collision::path_update, [126](#)
- wheel_drive
 - codec.hpp, [276](#)
- worldToGrid
 - pathfinding::GridMap, [83](#)